Copyrighted Material

C PROGRAMMING PATTERNS

Essential



MOHMAD YAKUB

Copyrighted Material

Everybody

in this country

should learn

how to program a computer,

because it teaches you

h o w to t h i n k.

— Steve Jobs

C

PROGRAMMING PATTERNS

By

MOHMAD | YAKUB

Technical Review : Sadique S. Naikwadi
Proof Reader : Mrs.Shabana Mursal

Editing
Mohmad Yakub

Copyright @ 2019 Mohmad Yakub

All rights reserved. This book or any portion thereof may not be reproduced or used in any manner whatsoever without the express written permission of the author except for the use of brief quotations in a book review.

The author has made every effort in the preparation of this book to ensure the accuracy of the information. However, information in this book is sold without warranty either expressed or implied. The author will not be held responsible for any damages caused or alleged to be caused either directly or indirectly by this book.

Contact Information: **I will appreciate if any queries raised or corrections needs to be applied. I do appreciate honest feedback. Readers of this book can reach me at the following mentioned email address.**

Email : programming.pattern@gmail.com

ISBN: 978 - 1 - 08 - 224430 - 8

I dedicate this book as a medium of hope for those;

- **Who are unaware of any approach to craft any programming logic?**
- **Who had a hard time learning programming?**
- **Who had some experience in programming and yet still unconfident.**
- **Who carries false notion that coding is only for super smart people.**
- **Who are looking for 1st solid move to become self-taught programmer.**
- **Who are victim of discouragement comments similar like the following;**

  - Actually, you aren't interested.
  - You lack patience and determination.
  - Your IQ is well below average.

Preface

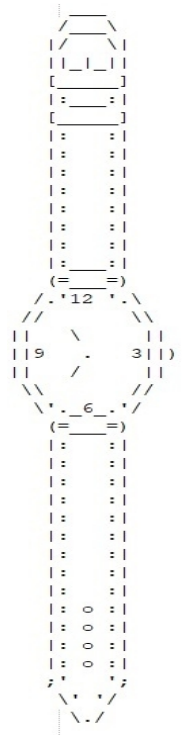
EVEN THE GREATEST WAS ONCE A BEGINNER.
DON' T BE AFRAID TO TAKE FIRST STEP.

- Muhammad Ali

Developing a fertile ground for **visualisation of programming logic** should be the prime focus for an absolute beginner. Unfortunately this perspective is almost alien not only to most of the beginners. You are invited to learn programming using more of common sense. It will going to start with something very simple so as to naturally adapt to any beginner. Leaving the basic programming part learner will get countable experience, how to deep dive to decode programming logic. In simple terms;

Programming = Programming Logic + features/API of programming language.

Skill of designing program's logic is required irrespective any programming language. Without this skill, a programmer can't be called a programmer. Programming logic is the core of a programming. If someone is really interested in coding, better first get improved in logic building skills. This will make learners more comfortable and interested to learn advanced topics of programming language. Solve pattern based problems because it will improve the visualization of logic. After some level of practice based on the formulated approach mentioned in this guide book, a **learner's mind will work like a mini-debugger.** A learner not only could able to visualize the flow of data, but also able to easily relate how different variables are taking values at different point of execution of code. If a problem asked in the interview or anywhere else, then we should able to get the logic correctly and in a single chance, instead of playing with guessing. For example consider the following case;

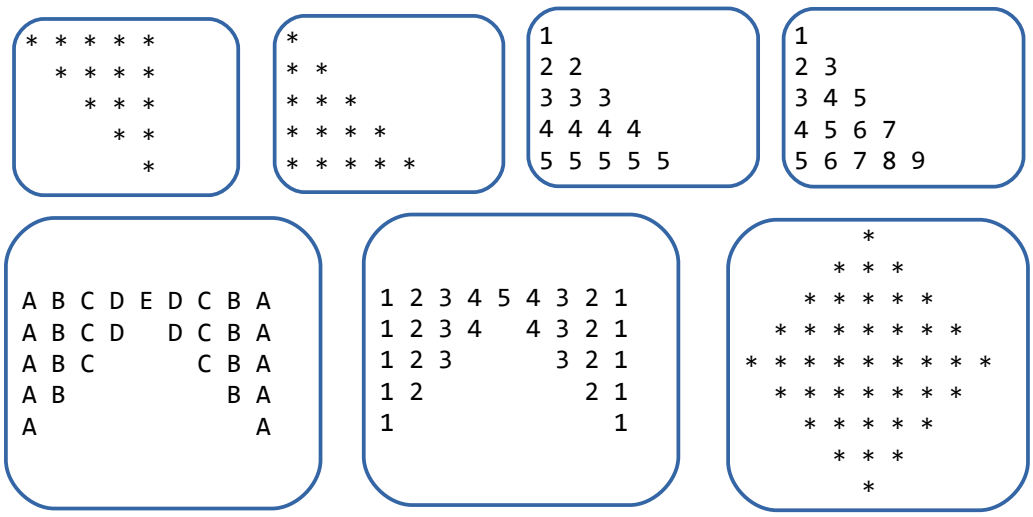


The programmer should able to write a generalized solution, so that based on input he/she able to generate a wrist-watch of customized sizes. A logic to output mentioned wrist-watch is full of twists and not an easy cake for even for 1-2 year experienced programmers. The fundamental intention is

that we should have a clear view how to architect a programming logic without entering into the amazon of guessing or rather involve in **playing a trial-error kung-fu**. Please check your ability solve this problem when you are done with this book. Do ask this problem in your study circle and see with what different ideas people try to solve this problem, do observe their approach. You may learn one or more new things. You may also able to coach one or more things. Some IT companies do see qualities of coaching in a candidate.

In the learning of any subject when the visualization and thought process becomes active then only it will turn out to be actually learning. Not only the person will able to understand the concept but also able to apply the concept naturally. Programming is not about downloading algorithms and memorizing programming logic in our mind. It is about applying the approach to solve the problems. Practising approach by solving exercises mentioned in this book, in general improves coding skills.

Patterns are 2-dimensional data output and whether a desktop/mobile or web based applications, there also mostly it deals with 2-dimensional data output. Practice with different patterns helps in developing visualization of logic. This way it builds good foundation and will surely help when a person actually work on real-time application. Following are just few of them and better to practice with many possibilities.



Learning programming with such simple picture based problems surely helps in improving coding skills. If the wrong logical condition is applied, so is the non-matching output. This is also a learning, since in different occasion of solving a problem we may need such type of logical condition. So, failures will become our lessons. Learning in this way makes learning interesting and force us to put efforts being fully focused. So, in this way it helps in logic building.

On the other hand, learning using mathematical based problems like prime numbers and other similar kind of problems doesn't create much of interest. Furthermore, on practical ground such mathematical based exercises are just boredom on steroids for most of the non-programmers or beginners.

In my opinion pattern-based problem gives lot many reasons to think logically in different directions and to work on it. The improvement in building logical skills coupled with interest is much more higher than any other approach. In general, it suits all learners whoever like to invest some consistent time. I had experimented this with lot many students and i found that this is, the most interesting way to learn programming quickly.

This approach is faster, better and easier way to improve coding skills if followed a proper approach and **not by applying dozens of guesswork to solve a particular problem**. Otherwise, a person will somewhat get improved in guessing and not in finding the approach to solve a particular problem, which is basically against the very definition of an algorithm.

*If pattern based problems are not handled properly it reveals logical weaknesses of a programmer and this the main reason whether at school level, beginners at college level or **junior level interviews**, the pattern based problems appear commonly. This book explains the approach to solve any pattern based problems and in general, it basically teaches **how to get the logic right.***

With insufficient logic building skills, the efforts on learning algorithms or advance topics of programming will be like actual talking to Gorilla, it will not be productive. This stuff is unique which I discovered accidentally during my teaching career. It is proven formula devised after experimenting with students. It formulated keeping in mind what suit best for students to learn quickly and correctly with zero reservation for memorizing program's logic. I was supposed to teach programming to complete beginners in a very less time of 2-3 months. I did some experiments and devised a plan and that plan was a great success. The results and response was amazingly nice. This book is the explanation of that approach which can definitely help a student, irrespective of their preferred learning style. This unique approach to learn programming logic, not only fast, but also with lots of enthusiasm. It layouts an approach based on common sense so that practically anyone can learn programming and interestingly from the very minimal technical topics. This book is a guide for teachers as well to hand over control on the programming to students with confidence and passionately. The approach is so adaptive that is well suited even for school going kids. So those parents who are struggling to teach programming to their kids can also refer this.

If you are struggling to learn or teach programming and you don't like to waste your time searching YouTube and other resources, here is the chance to learn quickly. Will you…?

After completing this book you will be in a better position to make good use of other resources.

Programming is a kind of mental aerobics which teaches us to keep focus, nourishes good amount of patience, discipline and forecasting. If you observe these are some of the main qualities of an achiever.

Programming is for <u>everybody</u> and not just an academic exercise.

Happy learning !!!

Table of Contents

Preface.....5

PART 1 : BASICS OF PROGRAMMING LOGIC.....15

Introduction.....16

The Right Attitude.....17
The Clarity Principles.....18
Any pre-requisite?.....19
About the book.....19

Kaizen Principles: To Improve Programming Logic.....20

Basic Format of C Program.....22

SampleFile.c.....22
Welcome.c.....22
Console Output.....23
PrintInSameLine.c.....24
PrintInEveryLine.c.....24
PrintInEveryLine1.c.....25
PrintInEveryLine2.c.....25
PrintInEveryLine3.c.....26
PrintInEveryLine4Ways.c.....26

Storing different data types.....27
Variable declaration.....27
Variable assignment.....28
Operators.....30
Arithmetic Operators.....30
Variable Declaration.....30
Memory Address.....30
Memory Content.....30
Variable Assignment.....30
Memory Address.....30
Memory Content.....30
ArithmeticOperationsInteger.c.....31
IncrementDecrementOperators.c.....33
Comparison Operators.....34
ComparisonOperators.c.....35
Logical Operators.....38

Art of Applying Logical Conditions.....40
Convert Logical Conditions to Simple Picture.....42
Convert Simple Picture to Logical Conditions.....48

Decision making.....70
a) if-statement.....70
StatementIf.c.....71
StatementIf1.c.....72
b) if-else statement.....73
StatementIfElse.c.....74
StatementIfElse1.c.....75
c) if-else if-else statement.....76
StatementIfElseIf.c.....77
StatementIfElseIf1.c.....78
StatementIfElseIf2.c.....79

d) switch-case....81
ConvertLadderIfElseToSwitchCase.c....82
My Notes : programming Logic Experiences....83

About for-loop....84

Basics....85
ForLoopIncrement.c....86
ForLoopIncrement1.c....86
ForLoopIncrement2.c....87
ForLoopDecrement.c....88
For-loop with {break} and {continue} keywords....89
ForLoopContinueNext.c....89

Nested for-loop/Inner for-loop....90
NestedForLoop.c....91

Nested Loop: Beginner Programmer's Best Friend....92
Why nested loops are such an important deal?....92
ForLoopEveryRowAllColumns.c....94
ForLoopEveryRowAllColumnsHorizontally.c....95
ForLoopCoordinateRepresentation.c....96
RowColumnStars.c....97

How to verify logical conditions?....99
VerifyAlphabetM.c....99
You can do it....100

Key summary points....100
Learning logic building skills by guessing – A tedious strategy....101

Logic trace table: Brain behind programming logic....103

PART 2 : PROGRAMMING IN ACTION....*106*

Outputting Simple Picture on Computer Screen....107
Cursor Movement....108

Pattern1A....109
Analysis of output....109
Pattern1A_1.c....111
Approach to solution....113
Pattern1A_2.c....115
Pattern1A_1.c....116

Pattern1B....117
Analysis of output....117
Approach to solution....120
PatternB1_1.c....122
PatternB1_1.c....123
Alternative solution2....124
PatternB2.c....125
PatternB2.c....126
Alternative solution3....127
PatternB3.c....129

Pattern1C **131**
Analysis of output **131**
Approach to solution **134**
PatternC1.c **136**
Alternative solution2 ..... **137**
PatternC2.c **139**
PatternC2.c **140**
Alternative solution3 ..... **141**
PatternC3.c **144**
Pattern1D **145**
Analysis of output **145**
PatternD1.c **149**
PatternD1.c **150**
Pattern1E **153**
Analysis of output **153**
PatternE1.c **159**
Alternative solution2 ..... **160**
PatternE2.c **163**
Pattern1F **164**
Analysis of output 164
PatternF1.c 169
Alternative solution2 170
Pattern1G **171**
Analysis of output **171**
Approach to solution **175**
PatternG1.c **179**
PatternG2.c **180**
Alternative solution2 ..... **181**
PatternG3.c **182**
PatternG3.c **183**
Pattern1H **184**
Analysis of output **184**
Approach to solution1 ..... **188**
PatternH1.c **192**
PatternH1_2.c **193**
Alternative solution2 ..... **194**
PatternH2_1.c **198**
PatternH2_2.c **201**
PatternH2_2.c **202**
Alternative solution3 ..... **203**
PatternH3.c **206**
PatternAA **208**
PatternCC **208**

Number Based Pattern.....212

NumberPattern1A.....212
Tabular representation.....212
Analysis of output.....213
NumberPattern1A.c.....216
NumberPattern1A.c.....217

NumberPattern1B.....218
Tabular representation.....218
Analysis of output.....219
Observations.....219
NumberPattern1B.c.....223
NumberPattern1B.c.....224
Alternative solution.....225
NumberPattern1B_2.c.....226
NumberPattern1B_2.c.....227

NumberPattern1C.....228
Tabular representation.....228
Analysis of output.....229
NumberPattern1C_1.c.....231
NumberPattern1C_2.c.....232
NumberPattern1C_3.c.....233
NumberPattern1C_3.c.....234
NumberPattern2C.c.....237
NumberPattern2C.c.....238

NumberPattern1D.....239
Tabular representation.....239
Analysis of output.....240
Observations.....240
NumberPattern1D.c.....243
NumberPattern1D.c.....244
Alternative solution.....245
NumberPattern2D.c.....245
Alternative solution.....246
NumberPattern3D.c.....248
NumberPattern3D.c.....249

NumberPattern1E.....250
Tabular representation.....250
Analysis of output.....251
NumberPattern1E.c.....254
NumberPattern1E.c.....255

NumberPattern1F.....256
Tabular representation.....256
Analysis of output.....257
NumberPattern1F.c.....260

Level – 2: Number Pattern.....261

NumberPatternAA.....261
Tabular representation.....261
Analysis of output.....262
Observations.....262
NumberPatternAA.c.....264
NumberPatternAA.c.....265

NumberPatternBB.....266
Tabular representation.....266
Analysis of output.....267
Observations.....267
NumberPatternBB.c.....270
NumberPatternBB.c.....271
Alternative solution.....272
NumberPatternBB2.c.....273

NumberPatternCC.....274
Tabular representation.....274
Analysis of output.....275
NumberPatternCC1.c.....277
NumberPatternCC2.c.....278
NumberPatternCC3.c.....280

NumberPatternDD.....281
Tabular representation.....281
Analysis of output.....282
NumberPatternDD1.c.....284
NumberPatternDD2.c.....285
NumberPatternDD3.c.....287

NumberPatternEE.....288
Tabular representation.....288
Analysis of output.....289
NumberPatternEE.c.....292

NumberPatternFF.....293
Tabular representation.....293
Analysis of output.....293
Alternate solution.....296
NumberPatternFF.c.....300
If-else Shortcut.....301
When to apply if-else shortcut.....301
NumberPatternFF2.c.....303

NumberPatternGG.....304
Tabular representation.....304
Analysis of output.....305

Writing our own functions.....307

NumberPatternHH.....310
Analysis of output.....310

Level– 3: Number pattern.....314

NumberPatternAAA.....314

Tabular representation.....314

Analysis of output.....315

Observations.....315

NumberPatternAAA.c.....317

NumberPatternBBB.....318

Tabular representation.....318

Observations.....319

NumberPatternBBB1.c.....322

Alternative solution.....323

NumberPatternBBB2.c.....323

NumberPatternCCC.....324

Tabular representation.....324

Analysis of output.....325

NumberPatternCCC1.c.....327

Alternative solution.....328

NumberPatternDDD.....332

Tabular representation.....332

Analysis of output.....333

Observations.....333

NumberPatternEEE.....338

Tabular representation.....338

Analysis of output.....339

Access the values of an array.....347

Accessing array elements using array's index value.....348

***PART 3*: POINTERS AND OTHER TOPICS.....348**

Pointers And Memory Matters.....349

Stack and function calls.....352

Call by Value.....357

Points to Remember.....358

Understanding the Concept of Pointers.....360

Points to Remember.....364

Accessing array elements using pointers.....366

String concatenation.....368

Programming Patterns Using String functions and Arrays.....369

Programming Pattern Using Pointer.....379

***Recursion:* Repeat without looping.....387**

Appendix-A: Installation of eclipse IDE and project set-up.....391

Final Exam – Prove Your Worth.....394

PART 1

BASICS

OF

PROGRAMMING LOGIC

Introduction

Without a doubt every subject has some key fundamental basis. If key concepts of any subject are truly internalize then the rest of the learning will be productive.

Many novice programmer and many frustrated programmers do ask similar kind of questions which are as follows;

“How to learn programming?”

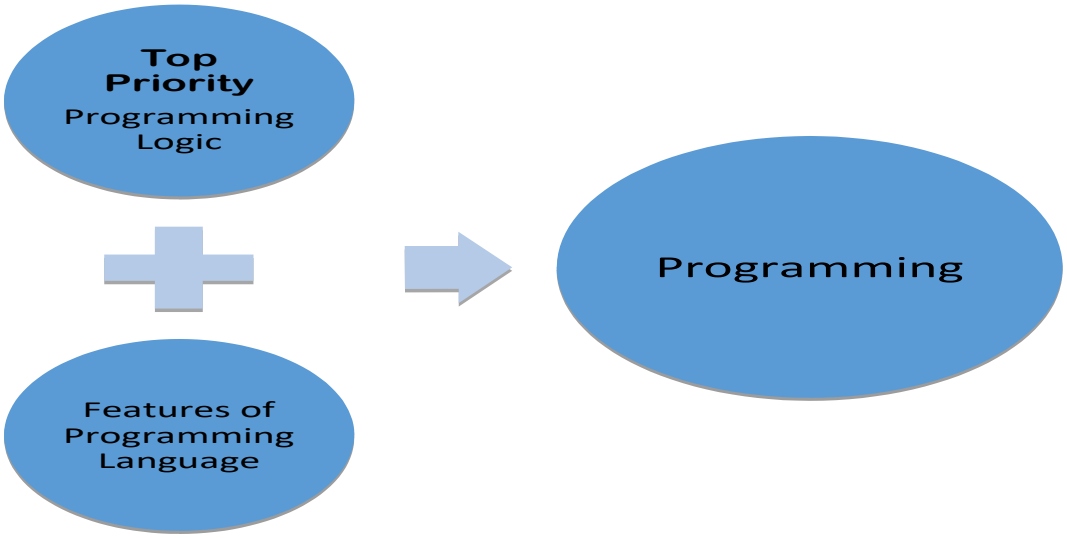
“How to develop logic building skill?”

“How to learn coding?”

“How to improve programming logic?”

Programming is an applied field. In my opinion if a programmer who is weak in programming logic is not a programmer even. We know that complaining is the easiest job and if someone is expert in that haven’t actually achieved anything big. Whenever there is a problem, we just need to think “**what could be the easiest possible way out**?”

In order to become confident in any programming language, the first step should be to gain some level of proficiency to self-hack programming logic. This technical manual is totally dedicated to a non-programmer/beginner or intermediate students who are just tired of hitting hard on many different places in order to become confident in programming. Additionally, if you are among those who got limited time to learn programming, this is the guide that can serve you well too. **So the rule of the thumb is, in order to learn programming language fast and properly, first learn the tactics of programming logic sufficiently**.



The most common mistake that many non-programmers/absolute beginners or trainers does is that they focus more on grammar/syntax and feature(s) of the programming language. Such shallow outlook about programming language doesn’t help beginners much. Computer science is an applied field to solve practical problems. It is foremost and essential need that a beginner should get trained sufficiently in area of programming logic skills at the first priority. How does an appropriate physical exercises and healthy food tones the body muscles, similar it goes with developing programming logical skills. It does need right kind of logic based programming exercises to groom visualization of programming logic.

It also happens that a beginner may know almost all the concepts of programming language but still feels something is missing. That missing element is due to insufficient logic building capabilities. Without a comfortable level of logic building skills, the beginner struck a lot and, on top of that, they face lot of confusions while developing programs.

The Right Attitude

> ***Failure taught me things about myself that I could have learned no other way.***
>
> **- J.K. Rowling**

By default children are not afraid to be wrong and so they are creative. It's well said **hiding mistakes is a mistake of higher order**. So it is always good to notice our mistakes as early as possible so that it shouldn't give a hard blow in the future during the much needed time.
Experimenting and learning from mistakes work like a some sort of body detoxification program. Initially, any detoxification program doesn't tastes nice but still person go for it because that person will love to see the way the body will going to respond after 2-3 weeks. Anything new that we do take some practice. Everything that is valuable has a cost associated with it. Failures are one such type of cost that are to be paid to learn and to remember lessons.
Getting accurate output after executing a program is just a by-product. So the total learning will be failures plus the success part. So we need to think in this way, more initial failures will going to help us to reduce the chances of failures in the future instead of thinking that we are not a good learner. When a learner knowingly or unknowingly applies mismatching logical conditions, may encounter unexpected output. This in turn creates a curiosity that which specific logical condition is impacting the output differently. It's easy for a brain to remember surprises. Moreover, picture helps brain to remember more than the flowing texts, let it be any fact or concept. So it's much useful if a learner experiment with a weird or with some typical logical conditions and try to find the reasons behind it. This will work like a self-motivating exercise and it simply creates urge to learn more.

We all have hidden potential and it is up to us to explore ourselves. You are the only person who can explore your hidden potential by taking challenges by experimenting it with your capabilities. Unless and until you take this journey of challenges, you will not know **actual YOU** or you will remain rather somebody else a common class. ***You, I and everybody*** *have to do this exercise of treasure hunt within ourselves.*

It is all about game of attitude and applying our actions in trying for excellence. We just need to bounce back repetitively with another honest attempt. One of the main aim of this guide is to ignite thought process and to develop a natural tendency to solve problems just by touching bare minimal technical details. We know that attitude measures altitude. A right intention gives lot of power to manage even tough task with confidence. It is all about the perspective, the way we think and approach the situation accordingly.

The Clarity Principles

Let help ourself by putting these principles as our reminders. We must give reminders to ourself, the key important values that helps to cleanse our subconscious mind. Definitely there are benefits in reminders, so give reminders to others or our-self.

> ***You can't manage what you can't measure.***
>
> **- Peter Drucker**

> ***Science is simply common sense at its best, that is, rigidly accurate in observation, and merciless to fallacy in logic.***
>
> **- Thomas Huxley**

> ***Most of man's problems upon this planet, in the long history of the race, have been met and solved either partially or as a whole by experiment based on common sense and carried out with courage.***
>
> **- Frances Perkins**

> ***I read, I study, I examine, I listen, I think, and out of all that I try to form an idea into which I put as much common sense as I can.***
>
> **- Marquis de Lafayette**

> ***Common sense is calculation applied to life.***
>
> **- Henri-Frédéric Amiel**

The most important aspect of calculation or measurement is that it gives the clarity. We will try to tap this "**clarity**" factor to get the hidden programming logic right using **logic trace table.**

Every learning covered in this guide is majorly revolved around the concept of **logic trace table** and it's practice is dealt in greater details in the section **Programming Patterns**.

We can't expect quality software without quality programs. Good programming logic is one of the essential quality to write error-free program and quality software.

Any pre-requisite?

No prior experience in any programming language is required. If you have any, it is good and this is optional as we are not dealing much of the syntactical technical matter. We only need very minimal language's features to practice programming exercises in order to cultivate interest and confidence in programming. We are majorly dealing with two main technical concept.

a) Conditional statements (If-statement, if-else statement)

b) Looping construct— just *for-loop*.

Programming syntax of conditional statements and ***for-loop*** of C resembles with many other programming languages like C, C#, C and C++. After learning from this book they can easily experiment in programming language of their interest.

One of the common quality of highly successful people are as soon as they acquire information, their speed of implementation is very fast. This quality helps them to be stay ahead in the competition. The best thing to **compete with ourself and try to** break our own record.

About the book

All the material presented here is only meant for action, a practical to do sessions. We are not covering C as a language but programming logic as a subject using C programming language's syntax. This book is not a product of guesswork. This book is outcome of experiments conducted among college students in order to improve their logic building skills or to learn programming from the scratch but in a quick way.

The learner has to practice the approach mentioned in the book for logic building skills related to 2-dimensional simple picture based exercises. Practice exercises are arranged in an increasing challenging way and motivates to refine programming skills continuously in a step by step manner. The solved exercises gives the complete idea and unsolved problems urges to apply logical concept. So, it is going to be **YOU versus YOU** in an interesting way.

Two major requirement are expected from every learner which are as follows:

a) Intention that you want learn programming

b) Need to spend some time consistently and it depends how much quickly you want to become a confident programmer. **Don't break the learning pattern at least for 3 months, spend at least minimum 20 - 30 minutes daily.** Just follow this course of action step by step, you will surely get substantial grip in the programming.

It's up to you afterwards to explore new horizons in programming or software development. I can only promise an excellent start in a short period of time regarding your journey of learning a programming logic. ***So let's start the journey of learning.***

Kaizen Principles: To Improve Programming Logic

The Kaizen principle is a Japanese philosophy that refers to the set of activities that promote **continuous improvement. Improvement is not an activity which is meant to be done once or few times and then leave it. A continuous improvement is a process which consists of simple steps that leads to improvements in various areas.**

Small improvements everyday

It focuses on incremental improvements as compared to finding exact solutions to the problems. Any improvement is better than ZERO improvement and 100% postponement. So make small simple targets and achieve it by putting efforts. Even 1% improvement is valuable and therefore small gains combined together in longer run yields big results. Basically don't break the learning pattern and it's better to learn something rather than postponing it 100%.This activity also help to break the habit of procrastination, if somebody who really find it hard to get rid of.

Measurement and feedback

The implementer of this principle has to be mentally prepared to find their own mistakes in logical expression and overall programming logic. Each mistake is treated as a lesson in the path of overall improvement. So total focus is on incremental improvements rather than on complete solutions to programming problems. The learning of process of programming logic should be based on pictorial based exercises rather than mathematical based problems. When a learner practice programming logic based on pictorial based exercises, the feedback on applied logic can be known easily and clearly. Not only limited to this, it will create lot of curiosity and interest to learn programming. Moreover, it also helps to eliminate one of the commonest issue of;

"**how to keep focus while learning programming?**".

Let go of assumptions

The usual and common way to learn programming is just by randomly experimenting it based on assumptions. This takes much more time to learn programming logic as it required hell lot of practice. Learning programming by assumptions is a switch off to many of them. Since it doesn't reflect any standard way of learning. Most of us can't able to make a good assumptions and in every subject area is highly impossible and there is no standard formula for assumptions that can work with anybody. If an approach for learning sparks visualisation and thought process, then it will be easy for anybody to learn it.

Speak with data, manage by facts

The programmer should device a solution based on the data and facts. The most important protocol which makes programming logic that works correctly is to understand the intelligence involved in the flow of data.

The trick to get this intelligence is to express flow of data in one or more logical expressions using our version of observation table known as **logic trace table**. The logical expression is made up of one or more variables and operators. Finally, programming logic is made up of one or more logical expressions. We can easily learn this art of making logical expressions using data and through appropriate practice mentioned in the **section-programming patterns**.

Implementing Kaizen model for logic building skills

- Start a topic and learn till you reach a point where you are ready to analyse a solved example. Concentrate on that example till you get a feel that you have understood everything about the program.

  Then try to implement that solved example without referring it. Make a strong will that once you start implementing already studied program, you are not going to refer it back that particular solved program. You can refer theoretical explanation but not program. There are lot of chances that you may get many error syntactically or logically as well. In that situation, you can refer theoretical explanation about the concept but do not allow yourself to see the solution. Try your best to resolve issues by yourself referring theoretical explanation.

- Please maintain an excel or spreadsheet file to note down the errors you encountered in a program. When you yourself able to resolve the errors, please note down the reasons also why you have adopted that approach. Also, what you thought previously and afterwards you faced that particular logical error.

- Every new day when you want to start your learning, please have a quick look to your error file that you were maintaining every time, you implement the solved program when encountered any error(s). Referring back to those errors while learning every new day will help to do better next time and it will also helps you to quickly revise the concept. We should maintain this practice to avoid common logical errors so those errors should not become part of our habit. So it is good to correct those loopholes in the beginning itself, since breaking a habit consumes sufficient efforts. What we practice, that will become our habit whether good or bad. Here, we are trying to **convert failures into lessons**. It is more rewarding when we get improved from mistakes as early as possible so that we can invest more time learning other important subjects in our life.

- If two times you are able to solve all the solved examples from the book back to back without referring to the solutions, then you will be in a better position to write program effectively.

- Hints are there for unsolved problems in the explanations of solved exercises. If you are unable to solve unsolved problem, just try 2-3 attempts and move forward to complete the book. Come back again and try to attempt the remaining ones.

- Two times back to back, if you are able to solve all the solved exercise without referring to the solutions and also unsolved problems, then logic wise you will be in a very comfortable position. You will get the courage to build programming logic for more difficult level and can be able to deal other heavier subjects of programming like data-structures and design of algorithms with confidence.

Repeat the above **kaizen(continuous improvement) model** to produce solutions, single attempts for all programs with

0 defect.

The GREAT aim of education is not knowledge but action.

- Herbert Spencer

Basic Format of C Program

The general format/template for our program will be something like as follows.

SampleFile.c

```
#include <stdio.h>

int main() {

	// Your program logic goes here

	return 0;

}
```

How we can write our programming logic using C file?

In order to check the correctness of our programming logic we are required to do two things:

1) From **SampleFile.c file** create your program file, give program's name.
2) Place all your programming code in between code block of main method.

```
int main() {

// Your program logic goes here

 return 0;

}
```

Code Block starts with **open curly bracket {** .
It ends with **closing curly bracket }**.

If suppose we want write a program and our name of program is Welcome, then the name of the program will be given by the **Welcome.c** program file.
For now just ignore **return 0, int main(),#include <stdio.h>** and other theoretical stuff.
Just reuse this **SampleFile.c** file whenever you want to write a new program.

The following **Welcome.c** program outputs the text **Welcome to C Programming.**

Welcome.c

```
/*
 * This is my first program.
 * Program to print welcome message.
 */
#include <stdio.h>
int main() {

	//program logic goes here

	printf("Welcome to C Programming.");

	return 0;

}
```

Output

```
Welcome to C Programming.
```

The multi-line comments are given by `/* multi-line...*/`
The single line comment is given by `// single-line...`

Comments are for our understanding/documentation purpose and not a part of our programming code. The C complier ignores it.

How to execute a C program in order to verify programming logic?

Please refer the following title at end of this book.

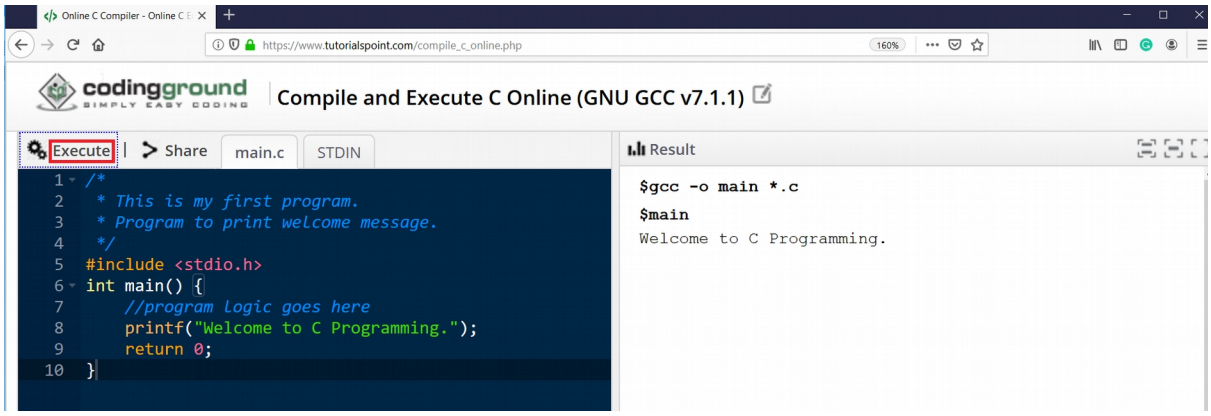
Appendix-A: Installation of IDE and project set-up

It shows step-by-step procedure to create a **C project** using eclipse IDE and further shows how to create and **execute a C program** with screen-shots. Eclipse IDE is for free and widely used in software industry also.

Alternatively, you can use one of the online editor shown as follows.

https://www.tutorialspoint.com/compile_c_online.php

Click the **Execute** button to run the program. By default, **Source File** tab will be opened to write the program. A separate Result window is shown, to display the output of the program.



Console Output

Let's try to use C programming language's inbuilt method **printf()** to output result in different ways. We can use these methods to generate 2-dimensional output to verify our understanding of logic.

printf("\ntext");	**Moves the cursor to the next line & prints the text to the console.**
printf("text**");**	**Print** text **to the console.**
printf("\n");	**Moves the cursor to the next line.**

Following programming statement(s) means the same. It outputs the given text and moves to the next line in the console outputting window.		
printf("text**");** **printf("\n");**	**printf("**text**\n**");	**printf("**text**");printf("\n");**

In order to output a 2-dimensional simple picture using a programming logic, it is required to know how to print a character horizontally and vertically. More is the complexity of the 2-dimensional picture, more challenging it will become to extract the programming logic and this is how it will force a learner to "**how to think like a programmer?**". You can expect learning will be in a thrilling way, just like a puzzle solving exercises. Prior to programming 2-dimensional pictures simple or complex, let do some basic required fundamental programming related to it. We can now start learning programming logic simply by trying to print a text repeatedly, a simple text **horizontally** in the same line or **vertically** in every line.

Exercise 1. Printing text "**Word:**" in the same line.

PrintInSameLine.c

```
#include <stdio.h>

int main() {

	printf("Word: ");

	printf("Word: ");

	printf("Word: ");

	return 0;

}
```

Output

```
Word: Word: Word:
```

Exercise 2. Let's try to output "**Word:**" three times in every line.

PrintInEveryLine.c

```
#include <stdio.h>

int main() {

	printf("Word: \n");

	printf("Word: \n");

	printf("Word: \n");

	return 0;

}
```

Output

```
Word:
Word:
Word:
```

`

The following exercises are showing **different ways of single line break** in the output.

Exercise 2.1. Printing "**Word:**" in every line and use of `printf("\n")` (single line break).

PrintInEveryLine1.c

```c
#include <stdio.h>
int main() {
	printf("Word: ");
	printf("\n");
	printf("Word: ");
	printf("\n");
	printf("Word: ");
	return 0;
}
```

Output

```
Word:
Word:
Word:
```

Exercise 2.2. Printing "**Word:**" in every line and use of "**\n**" (single line break character).

PrintInEveryLine2.c

```c
#include <stdio.h>
int main() {
	printf("Word: ");
	printf("\nWord:");
	printf("\nWord:");
	return 0;
}
```

Output

```
Word:
Word:
Word:
```

Exercise 2.3. Printing "**Word:**" in every line and use of "**\n**" (single line break character).

PrintInEveryLine3.c

```c
#include <stdio.h>

int main() {

	printf("Word:\n");

	printf("Word:\n");

	printf("Word: ");

	return 0;

}
```

Output

```
Word:
Word:
Word:
```

Exercise 2.4. Printing "**Word:**" in every line and in different ways.

PrintInEveryLine4Ways.c

```c
#include <stdio.h>

int main() {

	//1st way to print to console

	printf("Word:\n");

	//2nd way to print to console

	printf("Word:");printf("\n");

	//3rd way to print to console

	printf("\n");printf("Word: ");

	//4th way to print to console

	printf("\nWord: ");

	return 0;

}
```

Output

```
Word:
Word:

Word:
Word:
```

We will learn to sharpen our intelligence in programming logic by just using required only minimum technical knowledge. We are targeting to dive deep in the art of building programming logic rather than going deep in basic of grammar/syntax of a programming language. Let's check out some basic building blocks of programming logic.

Storing different data types

The intelligence of software is due to programming language(s).The processing of different types of data in some form, is something very core to almost every software. Facility to manipulate **different types of data correctly** should be a necessary feature of any programming language. Example of commonly used different data types.

General data type	Value examples
Integer	23, -4, 8908
Real Numbers	23.45, 3.1456, -45.27
Truth Values	true, false
text	"Just do it", "impossible"
Single Character	A, a, @, $, %,*, 9

Don't we use different variety of data in our daily life? We actually do, for example;

Name : ……..

Age : ……..

Weight(In Kgs) : ……..

Personal loan (true/false): ……

Gender(M/F) :……

We are sometimes required to fill correctly personal details in some way or the other way. Examples are many, like school/college admission process, flight booking or any online purchasing. We are intelligent enough to distinguish different types of data, whether it's numeric, textual or any other kind.

How can be a programming language made intelligent to distinguish different variety of data?

Yes, it is made possible through the facility of declaring variables.

Variable declaration

variable_name represents a memory address and also known as **lvalue**.

CONCEPTUAL LAYOUT OF A VARIABLE		
data-type	**variable_name**	Data within the range of **data-type**
		Memory Address

The format for variable declaration is as follows:

data-type variable_name;

int age; // age is **lvalue** and represents a memory location

Consider the following sample personal details needs to be filled in, for example, the variable declaration for the considered common personal details can be expressed as;

Equivalent variable declaration	Required to fill personal details in some real world situation
char[35] name; **int** age; **float** weightInKgs ; **boolean** personalLoan ; **char** gender;	**Name :** …….. **Age :** …… **Weight**(In Kgs) : …….. **Personal loan** (true/false): …… **Gender**(M/F) : ……..

For example, the data value 25 could be an age, price and there are many more possibilities. The name of variable should be appropriately chosen by giving full idea about the data it is going to refer. The chosen name of variable helps us to know meaning of stored value contained in it.

How to store data/value in a variable?

Yes, it is made possible through the facility of variable assignment using assignment operator (=)

Variable assignment

The format for variable assignment is as follows:

data_type variable_name = data; // assign only one data at a time

assignment operator

- variable_name represents a memory address and also known as **lvalue**.

- data value on the **right-hand side** of an assignment operator (=) is **rvalue**.

CONCEPTUAL LAYOUT OF A VARIABLE		
data-type	**variable_name =**	Data within the range of **data-type (rvalue)**
		***Memory Address* (lvalue)**

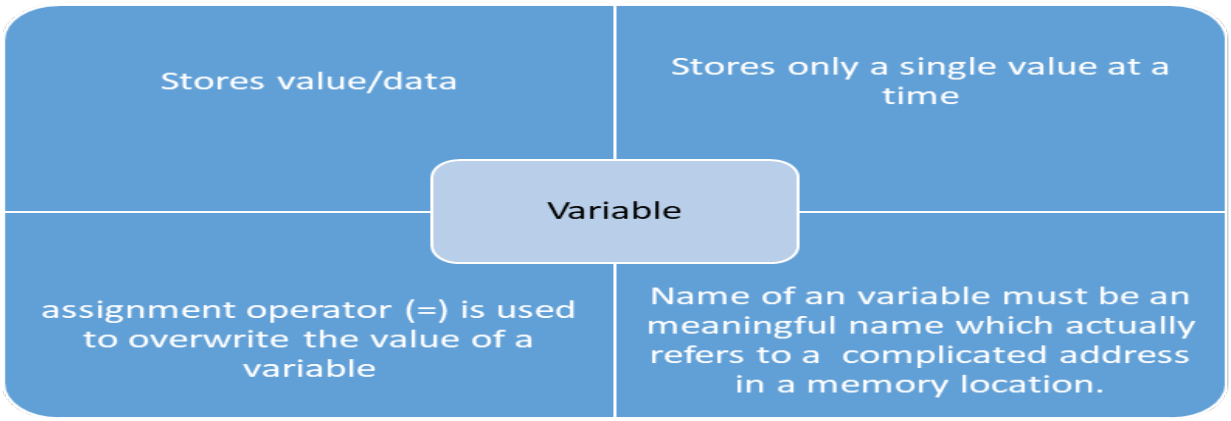
Any real time situation of manually filling data can be simulated using feature of a programming language known as variable assignment. Variable assignment is a simple act of storing an appropriate value in an appropriate variable. We can assign the value to the variable using assignment operator (=).The value of a variable can be over-written programmatically and that is why it's called variable. In a class room, the space of board are re-used by wiping out the details for every new class. In the similar way these variables can be over-written programmatically during the course of running/executing the program. **We can store or assign only one data at a time.**

Every single programming statement is separated by ";", similarly like how a full-stop (.) separates each sentence in a proper textual information.

Equivalent variable assignment	**Filled in personal details**
char[35] name = "James Arthur Gosling"; **int** age = 63 ; **float** weightInKgs = 53.6f; **boolean** personalLoan = false; **char** gender = 'M';	**Name** : James Arthur Gosling **Age**: 63 **Weight**(In Kgs) : 53.6 **Personal loan** (true/false): false **Gender** : M

The names of variables represents memory locations. It is very difficult to do programming using name of memory locations. Memory location address resembles not less than any secret key. The name of a variable that we choose should be a user-friendly name which represents a specific memory location allocated for the purpose to store data.

Please stick to the following points and do revise it before you start writing programs.



The space in memory location can be expressed in terms of byte units. So, the size of primitive data types too can be expressed in terms of bytes to store the value. (**Note: - 1 byte = 8 bits.**)

Data type	Size	Minimum Value	Maximum Value	Default Value
byte	1byte	-128	127	0
short	2bytes	-32768	32767	0
int	4bytes	-2,147,483,648	2,147,483,648	0
long	8bytes	-9,223,372,036,854,775,808	9,223,372,036,854,775,808	0
float	4bytes		upto 6 to 7 decimal digits	0.0f
double	8bytes		upto 15 decimal digits	0.0d
char	2bytes		65536 characters	'\u0000' or '' (single space)
boolean	1-bit	true or false (Only two possible values)		true or false

What is there in the variable's name?

We know that in geographical maps, the locations can be represented as combination of longitude & latitude. Think for a moment if our city, town, street places are replaced by longitude and latitude values, how much it will become difficult to use it in our normal communication. So the choice of variable should be a meaningful name and must indicate some purpose associate with it. It is not uncommon to find such programmers who has this special **SLVNS** sickness.

What is SLVNS sickness?

SLVNS stands for **Single Letter Variable Name Syndrome**. Many programmers has this special habit of creating suspense through name of a variable. They allocate just a single letter for variable's name in writing programs. When such program become lengthy and someone else has to fix a bug/error for that programming code, it will surely turned out to be irritating experience. Even such **SLVNS** programmers may find it difficult to get his/her own programming logic quickly on revisiting after a gap of time. If you find those programmers, please request them to leave this habit, it is an offence. Please prevent yourself from this sickness and **be patient to write meaningful variable name.**

Mind it, Better You Mind It.

Operators

Some special keyboard characters are used as operators in programming to denote frequent mathematical operations like addition, subtraction, comparing two numeric values and many such operations. We can categorize operators as per their purpose. In short, it's a simulation of simple school maths calculation activities but using programming. These simple mathematical operations are very much common in our daily life and plays a major role in dealing daily business or transactions. Such calculation based activities with accuracy and many such related activities are very much common in our daily routine.

Arithmetic Operators

Operator	**Arithmetic Operation Description**
+	Addition
-	Subtraction
*****	Multiplication
/	Division
%	Modulus (Remainder on division)
++	Increment
--	Decrement

For example, simple **addition operation** using "**+**" **operator** can be given as follows:

```
int intNum1 = 6;
int intNum2 = 4;
int intResult = intNum1 + intNum2;
```

We know that using assignment operator (=), at a time we can store single value in a variable. So in the considered example, **intNum1** will store the value 6 & **intNum2** will store the value 4.

Finally, **intResult** will store the result value of addition of **intNum1** and **intNum2** as addition operator (+) is applied between these two variables.

Variable Declaration	*Memory Address*	*Memory Content*
int intNum1;	1221	Nil
int intNum2;	1222	Nil
	1223	
int intResult;	1234	Nil
Variable Assignment	*Memory Address*	*Memory Content*
int intNum1 = 6;	1221	6
int intNum2 = 4;	1222	4
	1223	
int intResult = intNum1 + intNum2;	1234	10

Exercise 3. Demonstrate basic arithmetic operations using arithmetic operators.

ArithmeticOperationsInteger.c

```c
#include <stdio.h>
int main() {
	int intNum1 = 6, intNum2 = 4;
	int intResult = -999; // initialize with unexpected value

	printf("Integer numbers: ");
	printf("intNum1 = %d, intNum2 = %d", intNum1, intNum2 );

	printf("\n\n");
	printf("Operations | Operator | Expression | Result\n");
	printf("-----------------------------------------\n");

	printf("Add        | +        | ");
	intResult = intNum1 + intNum2;
	printf("%d  +  %d    |  %d\n", intNum1, intNum2, intResult );

	printf("Subtract   | -        | ");
	intResult = intNum1 - intNum2;
	printf("%d  -  %d    |  %d\n", intNum1, intNum2, intResult );

	printf("Multiply   | *        | ");
	intResult = intNum1 * intNum2;
	printf("%d  *  %d    |  %d\n", intNum1, intNum2, intResult );

	printf("Divide     | /        | ");
	intResult = intNum1 / intNum2;
	printf("%d  /  %d    |  %d\n", intNum1, intNum2, intResult );
```

```
        // Just use a second percent(%) sign to
       // output the percent(%) sign!!!
       printf("Remainder   | %%         | ");
       intResult = intNum1 % intNum2;
       printf("%d  %%  %d    |  %d\n", intNum1, intNum2, intResult );


       return 0;
}
```

Output

```
Integer numbers: intNum1 = 6, intNum2 = 4

Operations | Operator | Expression | Result
-----------------------------------------
Add        | +        | 6 + 4      | 10
Subtract   | -        | 6 - 4      | 2
Multiply   | *        | 6 * 4      | 24
Divide     | /        | 6 / 4      | 1
Remainder  | %        | 6 % 4      | 2
```

Exercise4. Write a program showing value changes using incrementing & decrementing operator.

IncrementDecrementOperators.c

```
#include <stdio.h>
int main() {
	int  value = 100;
	printf("Given values.\n");
	printf("value = %d\n\n", value );
	printf("Incrementing 3 times\n");
	value++;
	value++;
	value++;
	printf("Result = %d\n\n", value );
	printf("Decrementing 2 times\n");
	value--;
	value--;
	printf("Result = %d\n", value );
	return 0;
}
```

Output

```
Given values.
value = 100

Incrementing 3 times
Result = 103

Decrementing 2 times
Result = 101
```

Problem. Write a program named **ArithmeticOperatorsOnFloat.c** showing arithmetic operations Add, Subtract, Multiply, Divide & Remainder using the following variable names & values;

float floatVal1 = 2.55 ;

float floatVal2 = 1.77.;

Problem. Write a program to decrement the value of variable total 5 times and then increment it 5 times. The decrementing /incrementing counter value is 1.0

float total = 5.55;

Comparison Operators

The comparison result of two values either will be **true** or **false** and these values also known as **boolean values or truth values**. This feature is of very much importance since it will help to make decision in terms of true or false. The computer memory is based on 1(true) or 0(false) value called as **bit value**. Every data in computer is a combination of one's and zero's called bit pattern.

Different value or data means different bit pattern.

Operator	Comparison Operation Description
==	**Equal**
!=	**Not Equal**
<	**Less than**
<=	**Less than or equal**
>	**Greater than**
>=	**Greater than or equal**

Using comparison operators we can compare two numeric values. The following example shows the use of comparison operators using numeric values.

Operator	Operand1(Operator) Operand2	Result
==	4 == 4	true
	4 == 10	false
!=	6 != 4	true
	4 != 4	false
<	6 < 10	true
	6 < 6	false
<=	6 <= 6	true
	10 <= 6	false
>	6 > 6	false
	10 > 6	true
>=	4 >= 6	false
	10 >= 6	true

We can also test the above results by storing the input values in a variables and then applying comparison operators one by one.

Philosophically, true and false are the only two perfect answers in this world. While developing logic based on these perfect answers in a program we are required to do decision making. Specially, this feature allows flexibility in handling various different logical conditions in a program.

Sense and sensibility of this particular feature is a core of any good programmer.

So please, pay your special attention.

Let's verify the results using comparison operators on operands 4,6 and 10.

ComparisonOperators.C

```c
#include <stdio.h>
int main() {

	int four = 4;
	int six = 6;
	int ten = 10;
	// initialize with unexpected value to check if
	// comparison result is get executed or not.
	int isResult = -999;
	printf("*** Comparison Operators ***\n");
	printf("*** boolean result values[1 means true, 0 means false] ***");
	printf("\n\n");

	printf("equal ( == ) \n");
	isResult = ( four == four );
	printf("%d   ==   %d    |  %d\n", four, four, isResult );

	isResult = ( four == ten );
	printf("%d   ==   %d   |  %d\n", four, ten, isResult );
	printf("-----------------------------------------\n");

	printf("not equal ( != ) \n");
	isResult = ( four != four );
	printf("%d   !=   %d    |  %d\n", four, four, isResult );

	isResult = ( four != ten );
	printf("%d   !=   %d   |  %d\n", four, ten, isResult );
	printf("-----------------------------------------\n");
```

```
        printf("less than ( <  )\n");
        isResult = ( six < six );
        printf("%d   <   %d    |  %d\n", six, six, isResult );

        isResult = ( six < ten );
        printf("%d   <  %d    |  %d\n", six, ten, isResult );
        printf("-----------------------------------------\n");

        printf("less than or equal ( <= ) \n");
        isResult = ( six <= six );
        printf("%d   <=   %d    |  %d\n", six, six, isResult );

        isResult = ( ten <= six );
        printf("%d   <=  %d    |  %d\n", ten, six, isResult );
        printf("-----------------------------------------\n");

        printf("greater than ( > )\n ");
        isResult = ( six > six );
        printf("%d   >   %d   |  %d\n", six, six, isResult );

        isResult = ( ten > six );
        printf("%d   >  %d    |  %d\n", ten, six, isResult );
        printf("-----------------------------------------\n");

        printf("greater than or equal ( >= )\n");
        isResult = ( four >= six );
        printf("%d   >   %d    |  %d\n", six, six, isResult );

        isResult = ( ten > six );
        printf("%d   >  %d    |  %d\n", six, ten, isResult );

        return 0;
}
```

Output

```
*** Comparison Operators ***
*** boolean result values[1 means true, 0 means false] ***

equal ( == )
4   ==   4    |  1
4   ==   10   |  0
-----------------------------------------
not equal ( != )
4   !=   4    |  0
4   !=   10   |  1
-----------------------------------------
less than ( <  )
6   <   6    |  0
6   <  10    |  1
-----------------------------------------
less than or equal ( <= )
6   <=   6    |  1
10   <=  6    |  0
-----------------------------------------
greater than ( > )
 6   >   6   |  0
10   >  6    |  1
-----------------------------------------
greater than or equal ( >= )
6   >   6    |  0
6   >  10    |  1
```

Logical Operators

It also returns either **true or false** value like in the case of any comparison operator expression. We can combined many more such expressions which could be based on comparison operators. If there are many more conditions to be considered we usually use logical operators. Usage of these operators are quite common and plays a very important role in writing programs. Every programmer must have very much clear idea about usage of these operators. It will be worth if any new learner spent some extra time experimenting with these operators. Following are the different types of logical operators.

Operator	**Logical Operation Description**
&&	Logical AND
||	Logical OR
!	Logical NOT

Logical AND: Returns **true** only in a case when **compulsory both A and B are true** else false. More we add AND condition using && operator in a logical expression, less the size of data we get out of total data. It usually applied in a situation to get specific data or data of lesser range or size.

A	**B**	**A&&B**
true	***true***	***true***
true	false	false
false	true	false
false	false	false

Logical OR: Returns **false** only in a case when **compulsory both A and B are false** else true. More we add OR condition using || operator in a logical expression, more it increases the size of data as a output result. So it is used to get broader range of data.

A	**B**	**A ||B**
true	true	true
true	false	true
false	true	true
false	***false***	***false***

A & B are usually true/false value or comparison operator based expression(s) which

either returns result in true or false value. This is shown with the following example.

Logical NOT: Return true when A is false or just opposite, returns false when it is true.

A	**!A**	**Please note that true/false values are also known as boolean values or truth values.**
true	false	
false	true	

Let, there are two variables, a = 10 and b=5.Please observe the column "**Result**".

Logical operator	**Expression**	**Result**
&&	(a > 7) && (b < 7)	(true) && (true) is true
||	(a > 7) || (b < 3)	(true) || (false) is true
!	!(a == b)	!(false) is true

Art of Applying Logical Conditions

Logical conditions are the defence mechanism of working features of any software application. Most of the bugs/errors which occurred are due to the improper handling of logical condition(s). It mostly consists of **logical or comparison operators**. One of the most important element of programming logic is to apply logical conditions. It controls the flow of program similarly like how road traffic control system works. Road traffic control system logically distributes the crowd by conditionally allows or stops the portion of the crowd directing towards a particular road. Similarly, logical conditions in a program allow only certain set of programming statements to be get executed.

If that road side traffic control system doesn't work accordingly, then instead of maintain peaceful driving it may create traffic jams and road accidents. Similarly, wrong logical conditions gives unpredictable results if gone unnoticed.

Logical conditions are similar like road signs. Those road signs are got some meaning. They are been used based on some specific purpose and directly influences the traffic. In a similar way we will try to understand how logical conditions influences the output of the program which will help to improve our logical thinking capabilities.

We will use a simple version of scratch pad to verify our logical conditions manually. It may remind you of cross-words puzzle box. Let us take similar kind of box consists of 8 rows and 8 columns. For representing rows and columns we can take two variables named as **row** and **col** respectively.

Variable **row** → denotes values of row=1, 2,3,4,5,6,7,8.

Variable **col** → denotes values of col=1, 2,3,4,5,6,7,8.

Each one box is recognised by (row, col) value pair. So value pair (1, 1) means 1st row and 1st column.

	col=1	col=2	col=3	col=4	col=5	col=6	col=7	col=8
row=1	**1,1**	1,2	1,3	1,4	1,5	1,6	1,7	1,8
row=2	2,1	2,2	2,3	2,4	2,5	2,6	2,7	2,8
row=3	3,1	3,2	3,3	3,4	3,5	3,6	3,7	3,8
row=4	4,1	4,2	4,3	4,4	4,5	4,6	4,7	4,8
row=5	5,1	5,2	5,3	5,4	5,5	5,6	5,7	5,8
row=6	6,1	6,2	6,3	6,4	6,5	6,6	6,7	6,8
row=7	7,1	7,2	7,3	7,4	7,5	7,6	7,7	7,8
row=8	8,1	8,2	8,3	8,4	8,5	8,6	8,7	8,8

Rule of the game: *Where ever logical condition matches, need to put character * in a small box.*

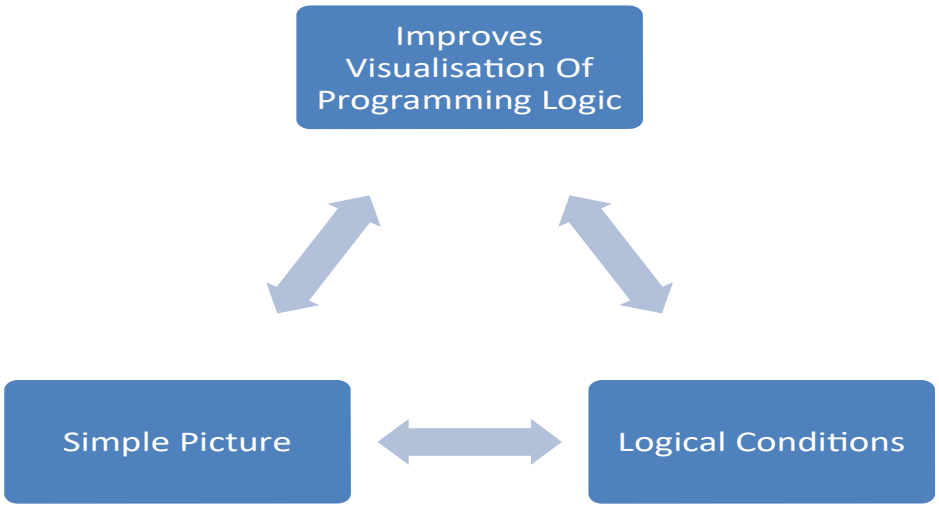
All (row, col) value pairs represent the total information or data. It has 8 rows and 8 columns which makes total 64 (row, col) value pairs. So, when logical conditions are mapped into this box we will select correct (row, col) value pairs from the total available 64(row, col) value pairs. Applying different logical conditions means manually selecting (row, col) value pairs matching each logical condition. This type of very simple observation based exercises will helps us to understand how logical condition works at the very core level.

We can practice to write logical conditions manually first using simple picture based exercises like alphabets of very simple structure.

Yes, of course we can verify all these logical conditions by writing programs also which will be our next level of learning. For your information, each logical condition can be verified easily by writing program using **nested loops**. Working with nested loops will be our next major step in learning. We will try to express action of writing of these simple structured alphabets into logical conditions. The countable experience we will get from such type of picture based exercises are that we can learn step by step how to apply logical conditions. The considered alphabetic pictures suggests to select only particular set of (row, col) value pairs and leave the others. It will force us to think what could be the possible accurate logical condition.

Since it's based on simple maths and common sense, so verifying manually these logical conditions won't be difficult.

When we do such exercises to do transformation from simple picture to logical conditions or do the reverse, it helps us to improve our visualization of programming logic.



Different alphabets means different pictures. So in totality it helps us in knowing how certain program works to give a targeted result. Additionally, we can verify all these logical conditions by placing it exactly in a program and executing it to verify the results. The right time has not yet come for novice /absolute beginner programmers to verify it through writing programs.

For those who had already been experienced in any programming language to the extent that they know conditional statements and nested loops, surely they can go ahead and test their logical conditions.

The first important lesson we should know how to give proper logical condition. We can clearly learn this important aspect of logic building by practising different alphabets of simple structure into equivalent logical conditions.

Convert Logical Conditions to Simple Picture

In this form of simple exercises, we need to express logical conditions by carefully selecting the appropriate boxes as per the logical condition and marking those boxes with * character. Each box is represented by unique combination of (row, col) value pair. Please check the given exercises to understand how the selection of boxes are done using the given conditions.

***ExerciseYB1*: Express Logical condition: (row == col) in the following 8X8 box.**

The condition **(row == col)** requests that please search those (row, col) value pairs in the given **8X8 box** where **row value** and **column value** will be equal. We can notice that on the diagonal of square the value of row and col is equal. **Just to get attention easily, boxes are marked with thick borders where row and col values are equal.**

	col=1	col=2	col=3	col=4	col=5	col=6	col=7	col=8
row=1	1,1	1,2	1,3	1,4	1,5	1,6	1,7	1,8
row=2	2,1	2,2	2,3	2,4	2,5	2,6	2,7	2,8
row=3	3,1	3,2	3,3	3,4	3,5	3,6	3,7	3,8
row=4	4,1	4,2	4,3	4,4	4,5	4,6	4,7	4,8
row=5	5,1	5,2	5,3	5,4	5,5	5,6	5,7	5,8
row=6	6,1	6,2	6,3	6,4	6,5	6,6	6,7	6,8
row=7	7,1	7,2	7,3	7,4	7,5	7,6	7,7	7,8
row=8	8,1	8,2	8,3	8,4	8,5	8,6	8,7	8,8

So as per rule of the game we should put * character on those boxes

where **row** and **col** values are equal.

	col=1	col=2	col=3	col=4	col=5	col=6	col=7	col=8
row=1	1,1*	1,2	1,3	1,4	1,5	1,6	1,7	1,8
row=2	2,1	2,2*	2,3	2,4	2,5	2,6	2,7	2,8
row=3	3,1	3,2	3,3*	3,4	3,5	3,6	3,7	3,8
row=4	4,1	4,2	4,3	4,4*	4,5	4,6	4,7	4,8
row=5	5,1	5,2	5,3	5,4	5,5*	5,6	5,7	5,8
row=6	6,1	6,2	6,3	6,4	6,5	6,6*	6,7	6,8
row=7	7,1	7,2	7,3	7,4	7,5	7,6	7,7*	7,8
row=8	8,1	8,2	8,3	8,4	8,5	8,6	8,7	8,8*

Removing other (row, col) value pairs where **row** and **col** values are not equal to indicate that considered logical condition doesn't selected them. It simply means that only those boxes were chosen and marked with * character that matches the given logical condition.

This way we can represent particular logical condition by filling small boxes with * character.

	col=1	col=2	col=3	col=4	col=5	col=6	col=7	col=8
row=1	1,1*							
row=2		2,2 *						
row=3			3,3*					
row=4				4,4*				
row=5					5,5*			
row=6						6,6*		
row=7							7,7*	
row=8								8,8*

So the meaning of logical condition now took the visual form. The expression of logical condition get transformed into the simple picture drawn using * characters. This help us to understand how filtering/selection of data takes place using logical condition.

ExerciseYB2. **Express Logical condition: (row == 8) || (col==1) in the following 8X8 box.**

Let's us take one part of logical condition **(row==8) ,** which means rows equal to 8.The boxes where value of **row** is 8 will be our selection of boxes irrespective of value of **col**. It can be brought into notice by highlighting border of the selected boxes.

	col=1	col=2	col=3	col=4	col=5	col=6	col=7	col=8
row=1	1,1	1,2	1,3	1,4	1,5	1,6	1,7	1,8
row=2	2,1	2,2	2,3	2,4	2,5	2,6	2,7	2,8
row=3	3,1	3,2	3,3	3,4	3,5	3,6	3,7	3,8
row=4	4,1	4,2	4,3	4,4	4,5	4,6	4,7	4,8
row=5	5,1	5,2	5,3	5,4	5,5	5,6	5,7	5,8
row=6	6,1	6,2	6,3	6,4	6,5	6,6	6,7	6,8
row=7	7,1	7,2	7,3	7,4	7,5	7,6	7,7	7,8
row=8	8,1*	8,2*	8,3*	8,4*	8,5*	8,6*	8,7*	8,8*

In the similar way, let's take the other part of logical condition which is **(col ==1).**

It means col value equals to 1**.** Let's select the boxes where value of col is 1 irrespective of value of row and are given by highlighting border of the selected boxes.

	col=1	col=2	col=3	col=4	col=5	col=6	col=7	col=8
row=1	1,1*	1,2	1,3	1,4	1,5	1,6	1,7	1,8
row=2	2,1*	2,2	2,3	2,4	2,5	2,6	2,7	2,8
row=3	3,1*	3,2	3,3	3,4	3,5	3,6	3,7	3,8
row=4	4,1*	4,2	4,3	4,4	4,5	4,6	4,7	4,8
row=5	5,1*	5,2	5,3	5,4	5,5	5,6	5,7	5,8
row=6	6,1*	6,2	6,3	6,4	6,5	6,6	6,7	6,8
row=7	7,1*	7,2	7,3	7,4	7,5	7,6	7,7	7,8
row=8	8,1*	8,2*	8,3*	8,4*	8,5*	8,6*	8,7*	8,8*

Removing (row, col) value pairs where character * is not present.

	col=1	col=2	col=3	col=4	col=5	col=6	col=7	col=8
row=1	1,1*							
row=2	2,1*							
row=3	3,1*							
row=4	4,1*							
row=5	5,1*							
row=6	6,1*							
row=7	7,1*							
row=8	8,1*	8,2*	8,3*	8,4*	8,5*	8,6*	8,7*	8,8*

By removing selected (row ,col) value pair details , we get.

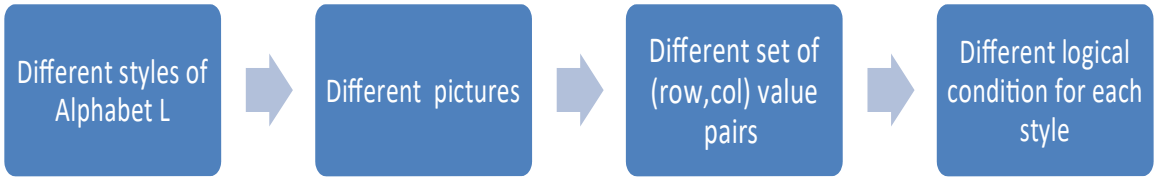
	*							
	*							
	*							
	*							
	*							
	*							
	*							
	*	*	*	*	*	*	*	*

This looks like similar to English alphabet L.

We can see few different styles of alphabet L.

L L L

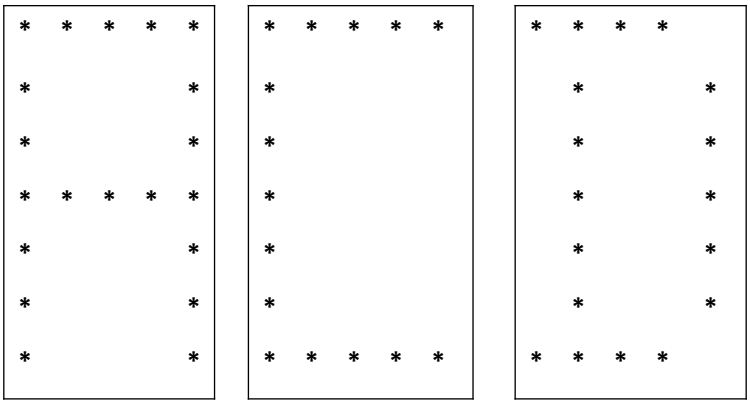
What possible thoughts shall cross our mind if we try to fit each different style of alphabet L in an 8 rows and 8 column box using * character. Different styles of alphabet means different pictures and each will have different set of (row, col) value pairs.



It's obvious that we can't get pixel perfect picture by using *character. The most important point we need to understand is that **different pictured output mean different logical conditions**. These set of pictures are somewhat advanced to start with it in order to learn to apply logical conditions.

We can surely start with something so simple that even school going kids can't make any excuse that they can't learn programming. We will pick something which will be so easy and familiar to most of us. We will consider English alphabets of very simplest structure shown in the following examples.

Drawn alphabets are A, C and D using * character.



Let's go back to old school days and try to recollect our learning experiences with the English alphabets. We were trained by our teachers, parents and others to write alphabets and they taught us in some logical way to write alphabets. The total logic applied to write one alphabet differs from another alphabet. Since the total structure of every alphabet is different we have chance to practice different kind of logical conditions.

This page intentionally left blank.

Convert Simple Picture to Logical Conditions

We will do some exercises by trying to get logical conditions from an English alphabets of very simple nature drawn using * character. We may consider 7 rows and 5 columns box. We have to select only those pair of (row, col) values which are helpful to make a required alphabet. We will be putting logical condition only for those (row, col) value pairs that are matching * characters.

***ExerciseYBA*: Alphabet A needs to be fit in 7X5 box.**

```
*   *   *   *   *
*               *
*               *
*   *   *   *   *
*               *
*               *
*               *
```

	col=1	col=2	col=3	col=4	col=5
row=1	1,1	1,2	1,3	1,4	1,5
row=2	2,1	2,2	2,3	2,4	2,5
row=3	3,1	3,2	3,3	3,4	3,5
row=4	4,1	4,2	4,3	4,4	4,5
row=5	5,1	5,2	5,3	5,4	5,5
row=6	6,1	6,2	6,3	6,4	6,5
row=7	7,1	7,2	7,3	7,4	7,5

In order to draw alphabet A let start it by putting * characters in the first row.

So logical condition will be **row==1** where == is equals comparator operator**.**

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2	2,3	2,4	2,5
row=3	3,1	3,2	3,3	3,4	3,5
row=4	4,1	4,2	4,3	4,4	4,5
row=5	5,1	5,2	5,3	5,4	5,5
row=6	6,1	6,2	6,3	6,4	6,5
row=7	7,1	7,2	7,3	7,4	7,5

We will use logical OR (||) conditional operator to add more data. In our case we need to add some more * characters to complete one more part of the alphabet A**.** Now we need to put * characters in the first column and the last column. So adding **(col==1)** and **(col==5)** logical conditions using **OR (||)** logical operator means more asterisk (*) characters are to be added in the following boxes**.**

The logical conditions updates are shown in the following

(row==1)||(col==1)

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1*	2,2	2,3	2,4	2,5
row=3	3,1*	3,2	3,3	3,4	3,5
row=4	4,1*	4,2	4,3	4,4	4,5
row=5	5,1*	5,2	5,3	5,4	5,5
row=6	6,1*	6,2	6,3	6,4	6,5
row=7	7,1*	7,2	7,3	7,4	7,5*

(row==1)||(col==1)||(col==5)

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1*	2,2	2,3	2,4	2,5*
row=3	3,1*	3,2	3,3	3,4	3,5*
row=4	4,1*	4,2	4,3	4,4	4,5*
row=5	5,1*	5,2	5,3	5,4	5,5*
row=6	6,1*	6,2	6,3	6,4	6,5*
row=7	7,1*	7,2	7,3	7,4	7,5*

Adding one more condition **(row == 4)** to the previous logical conditions to complete the middle part of the alphabet A**.** Now total * characters that displays alphabet A can be shown as follows with logical conditions

(row==1)||(col==1)||(col==5)||(row==4)

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1*	2,2	2,3	2,4	2,5*
row=3	3,1*	3,2	3,3	3,4	3,5*
row=4	4,1*	4,2*	4,3*	4,4*	4,5*
row=5	5,1*	5,2	5,3	5,4	5,5*
row=6	6,1*	6,2	6,3	6,4	6,5*
row=7	7,1*	7,2	7,3	7,4	7,5*

Now removing the (row, col) value pairs which doesn't have * characters.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1*				2,5*
row=3	3,1*				3,5*
row=4	4,1*	4,2*	4,3*	4,4*	4,5*
row=5	5,1*				5,5*
row=6	6,1*				6,5*
row=7	7,1*				7,5*

```
*   *   *   *   *
*                 *
*                 *
*   *   *   *   *
*                 *
*                 *
*                 *
```

Logical conditions: (row==1) || (col==1) ||(col==5)||(row==4)

***ExerciseYBB*: Alphabet B**

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2	2,3	2,4	2,5
row=3	3,1	3,2	3,3	3,4	3,5
row=4	4,1	4,2	4,3	4,4	4,5
row=5	5,1	5,2	5,3	5,4	5,5
row=6	6,1	6,2	6,3	6,4	6,5
row=7	7,1*	7,2*	7,3*	7,4*	7,5*

In order to draw alphabet B let start it by putting * characters in the first row and the last row. The equivalent logical conditions will be using logical condition **row==1** and **row ==7.**

The logical condition becomes

(row==1)|| (row==7)

Now we need to put * characters in the second column and the last column. So adding (col==2) and (col==5) logical conditions using OR (||) logical operator means more * characters are to be added in the following boxes**.** The updated logical conditions can be shown as the following:

(row==1)|| (row==7)|| (col==2)||(col==5)

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2*	2,3	2,4	2,5*
row=3	3,1	3,2*	3,3	3,4	3,5*
row=4	4,1	4,2*	4,3	4,4	4,5*
row=5	5,1	5,2*	5,3	5,4	5,5*
row=6	6,1	6,2*	6,3	6,4	6,5*
row=7	7,1*	7,2*	7,3*	7,4*	7,5*

Adding one more condition **(row ==4).** One issue with this logical condition is that

we don't want * character in the (row == 4) especially in the box with value pair (4, 1).

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2*	2,3	2,4	2,5*
row=3	3,1	3,2*	3,3	3,4	3,5*
row=4	4,1*	4,2*	4,3*	4,4*	4,5*
row=5	5,1	5,2*	5,3	5,4	5,5*
row=6	6,1	6,2*	6,3	6,4	6,5*
row=7	7,1*	7,2*	7,3*	7,4*	7,5*

We apply logical AND (&&) conditional operator to select less data. In our case we need to select one * character less in the **(row == 4**) to complete the middle part of the alphabet B.

The updated logical condition for middle part of the alphabet B will be **(row ==4 && col>=2).**

Now the logical condition becomes

(row==1)|| (row==7)|| (col==2)||(col==5)|| (row ==4 && col>=2)

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2*	2,3	2,4	2,5*
row=3	3,1	3,2*	3,3	3,4	3,5*
row=4	4,1	4,2*	4,3*	4,4*	4,5*
row=5	5,1	5,2*	5,3	5,4	5,5*
row=6	6,1	6,2*	6,3	6,4	6,5*
row=7	7,1*	7,2*	7,3*	7,4*	7,5*

Now removing the (row, col) value pairs which doesn't have * characters.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2		2,2*			2,5*
row=3		3,2*			3,5*
row=4		4,2*	4,3*	4,4*	4,5*
row=5		5,2*			5,5*
row=6		6,2*			6,5*
row=7	7,1*	7,2*	7,3*	7,4*	7,5*

OR

```
*   *   *   *   *
*           *   *
*           *   *
*   *   *   *   *
*           *   *
*           *   *
*   *   *   *   *
```

The complete logical condition becomes

(row==1)|| (row==7)|| (col==2)||(col==5)|| (row ==4 && col>=2)

***ExerciseYBC*: Alphabet C**

```
*     *     *     *     *
*
*
*
*
*
*
*     *     *     *     *
```

	col=1	col=2	col=3	col=4	col=5
row=1	1,1	1,2	1,3	1,4	1,5
row=2	2,1	2,2	2,3	2,4	2,5
row=3	3,1	3,2	3,3	3,4	3,5
row=4	4,1	4,2	4,3	4,4	4,5
row=5	5,1	5,2	5,3	5,4	5,5
row=6	6,1	6,2	6,3	6,4	6,5
row=7	7,1	7,2	7,3	7,4	7,5

On observing alphabet C and trying to fit into the box we can start putting * characters in the first row. So logical condition will be **row==1.**

Now we need to put * characters in the first column.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2	2,3	2,4	2,5
row=3	3,1	3,2	3,3	3,4	3,5
row=4	4,1	4,2	4,3	4,4	4,5
row=5	5,1	5,2	5,3	5,4	5,5
row=6	6,1	6,2	6,3	6,4	6,5
row=7	7,1	7,2	7,3	7,4	7,5

The equivalent logical condition for the first column will be **(col==1)** condition.

Using **logical OR (||) conditional operator** to combine logical conditions and it becomes

(row==1)||(col==1)

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1*	2,2	2,3	2,4	2,5
row=3	3,1*	3,2	3,3	3,4	3,5
row=4	4,1*	4,2	4,3	4,4	4,5
row=5	5,1*	5,2	5,3	5,4	5,5
row=6	6,1*	6,2	6,3	6,4	6,5
row=7	7,1*	7,2	7,3	7,4	7,5

Adding one more logical condition (row ==7) to the previous. Now the * character set resembling the alphabet C and complete logical conditions can be:**(row==1)||(col==1)||(row==7)**

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1*	2,2	2,3	2,4	2,5
row=3	3,1*	3,2	3,3	3,4	3,5
row=4	4,1*	4,2	4,3	4,4	4,5
row=5	5,1*	5,2	5,3	5,4	5,5
row=6	6,1*	6,2	6,3	6,4	6,5
row=7	7,1*	7,2*	7,3*	7,4*	7,5*

Now removing the (row, col) value pair which doesn't have * character.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1*				
row=3	3,1*				
row=4	4,1*				
row=5	5,1*				
row=6	6,1*				
row=7	7,1*	7,2*	7,3*	7,4*	7,5*

OR

* * * * *
*
*
*
*
*
* * * * *

Logical condition: (row==1) ||(col==1)||(row==7)

***ExerciseYBD*: Alphabet D**

We need to put * characters in the first row and the last row. So matching logical conditions will be (row==1) and (row==7)**.**

```
*     *     *     *     *
      *                 *
      *                 *
      *                 *
      *                 *
      *                 *
*     *     *     *     *
```

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2	2,3	2,4	2,5
row=3	3,1	3,2	3,3	3,4	3,5
row=4	4,1	4,2	4,3	4,4	4,5
row=5	5,1	5,2	5,3	5,4	5,5
row=6	6,1	6,2	6,3	6,4	6,5
row=7	7,1*	7,2*	7,3*	7,4*	7,5*

Now by putting * characters in the second column and the last column will complete the alphabet D. The logical conditions columns will be **(col==2) and (col==5).**Total logical conditions becomes **(row==1) || (row==7)|| (col==2)||(col==5)**

Removing all the (row, col) value pair that doesn't have * characters.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2*	2,3	2,4	2,5*
row=3	3,1	3,2*	3,3	3,4	3,5*
row=4	4,1	4,2*	4,3	4,4	4,5*
row=5	5,1	5,2*	5,3	5,4	5,5*
row=6	6,1	6,2*	6,3	6,4	6,5*
row=7	7,1*	7,2*	7,3*	7,4*	7,5*

OR

```
*     *     *     *     *
      *                 *
      *                 *
      *                 *
      *                 *
      *                 *
*     *     *     *     *
```

ProblemYBD: **Write the logical condition for the following alphabet.**

```
*     *     *     *
      *                 *
      *                 *
      *                 *
      *                 *
      *                 *
*     *     *     *
```

***ExerciseYBE*: Alphabet E**

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2	2,3	2,4	2,5
row=3	3,1	3,2	3,3	3,4	3,5
row=4	4,1	4,2	4,3	4,4	4,5
row=5	5,1	5,2	5,3	5,4	5,5
row=6	6,1	6,2	6,3	6,4	6,5
row=7	7,1*	7,2*	7,3*	7,4*	7,5*

We need to put * characters in the first row and the last row.
So matching logical conditions will be **(row==1)** and **(row==7).**

Also by putting * characters in the second column will complete the alphabet E.

On adding logical condition **(col==2),** the total logical conditions becomes

(row==1)|| (row==7)|| (col==2)

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2*	2,3	2,4	2,5
row=3	3,1	3,2*	3,3	3,4	3,5
row=4	4,1	4,2*	4,3	4,4	4,5
row=5	5,1	5,2*	5,3	5,4	5,5
row=6	6,1	6,2*	6,3	6,4	6,5
row=7	7,1*	7,2*	7,3*	7,4*	7,5*

Adding one more condition

(row==4 && col>=2) to the previous conditions.

The AND logical condition **(&& col>=2)** is because we don't want to put * character in the box with value pair (4, 1).

The AND logical condition is applied to provide restriction in selecting data and our data are (row, col) value pairs, the boxes in general**.**

The total logical conditions becomes **(row==1)|| (row==7)|| (col==2)|| (row ==4 && col>=2)**

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2*	2,3	2,4	2,5
row=3	3,1	3,2*	3,3	3,4	3,5
row=4	4,1	4,2*	4,3*	4,4*	4,5*
row=5	5,1	5,2*	5,3	5,4	5,5
row=6	6,1	6,2*	6,3	6,4	6,5
row=7	7,1*	7,2*	7,3*	7,4*	7,5*

OR

```
*   *   *   *
    *
    *
    *   *   *
    *
    *
*   *   *   *
```

***ExerciseYBF*: Alphabet F**

We need to put * characters in the first row and the second column.

So matching logical conditions will be (row==1) and (col==2)**.**

The logical conditions for now becomes **(row==1)|| (col==2).**

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2*	2,3	2,4	2,5
row=3	3,1	3,2*	3,3	3,4	3,5
row=4	4,1	4,2*	4,3	4,4	4,5
row=5	5,1	5,2*	5,3	5,4	5,5
row=6	6,1	6,2*	6,3	6,4	6,5
row=7	7,1	7,2*	7,3	7,4	7,5

Adding one more condition **(row ==4 && col>=2)** to the previous conditions. The AND logical condition **(&& col>=2)** is because we don't want to put * character in the box with **value pair (4, 1).** Now the collection of * characters that makes alphabet F can be given by the logical conditions which are as follows:

(row==1)|| (col==2)|| (row ==4 && col>=2).

Removing all the (row, col) value pair that doesn't have * characters.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2*	2,3	2,4	2,5
row=3	3,1	3,2*	3,3	3,4	3,5
row=4	4,1	4,2*	4,3*	4,4*	4,5*
row=5	5,1	5,2*	5,3	5,4	5,5
row=6	6,1	6,2*	6,3	6,4	6,5
row=7	7,1	7,2*	7,3	7,4	7,5

OR

```
*   *   *   *   *
    *
    *
    *   *   *   *
    *
    *
    *
```

***ExerciseYBG*: Alphabet G**

	col=1	col=2	col=3	col=4	col=5
row=1	**1,1***	**1,2***	**1,3***	**1,4***	**1,5***
row=2	2,1	2,2	2,3	2,4	2,5
row=3	3,1	3,2	3,3	3,4	3,5
row=4	4,1	4,2	4,3	4,4	4,5
row=5	5,1	5,2	5,3	5,4	5,5
row=6	6,1	6,2	6,3	6,4	6,5
row=7	**7,1***	**7,2***	**7,3***	**7,4***	**7,5***

We need to put * characters in the first row and the last row. So matching logical conditions will be **(row==1)** and **(row==7**).

Also we need to put * characters in the first column. So the matching logical condition will be **(col==1).**

The logical conditions becomes (row==1)|| (row==7)|| (col==1)

	col=1	col=2	col=3	col=4	col=5
row=1	**1,1***	**1,2***	**1,3***	**1,4***	**1,5***
row=2	**2,1***	2,2	2,3	2,4	2,5
row=3	**3,1***	3,2	3,3	3,4	3,5
row=4	**4,1***	4,2	4,3	4,4	4,5
row=5	**5,1***	5,2	5,3	5,4	5,5
row=6	**6,1***	6,2	6,3	6,4	6,5
row=7	**7,1***	**7,2***	**7,3***	**7,4***	**7,5***

Adding one more condition

(row == 4 && col>=3) to the previous condition, because we don't want * character in the box with **value pair (4, 2)**. Once again, adding one more condition **(col == 5 && row>=5**) to the previous condition, because we don't want * character in the box with **value pairs (2, 5) and (3, 5).**

Now the complete collection of * characters along with logical conditions can be shown as follows **(row==1) || (row==7)|| (col==1)|| (row ==4 && col>=3) || (col==5 && row>=5).**

Removing all the (row, col) value pair that doesn't have * characters.

	col=1	col=2	col=3	col=4	col=5
row=1	**1,1***	**1,2***	**1,3***	**1,4***	**1,5***
row=2	**2,1***	2,2	2,3	2,4	2,5
row=3	**3,1***	3,2	3,3	3,4	3,5
row=4	**4,1***	4,2	**4,3***	**4,4***	4,5*
row=5	**5,1***	5,2	5,3	5,4	**5,5***
row=6	**6,1***	6,2	6,3	6,4	**6,5***
row=7	**7,1***	**7,2***	**7,3***	**7,4***	**7,5***

OR

*	*	*	*	*	
*					
*					
*		*	*	*	
*				*	
*				*	
*	*	*	*	*	

***ExerciseYBJ*: Alphabet J**

Let fill * characters in the first row & in the middle column. So matching logical conditions will be **(row==1) || (col==3).**we have used logical OR (||) conditional operator to add more * characters.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2	2,3*	2,4	2,5
row=3	3,1	3,2	3,3*	3,4	3,5
row=4	4,1	4,2	4,3*	4,4	4,5
row=5	5,1	5,2	5,3*	5,4	5,5
row=6	6,1	6,2	6,3*	6,4	6,5
row=7	7,1	7,2	7,3*	7,4	7,5

Since we have to restrict column count to 3 at row=7, so we have to used logical AND

(&& col<=3) conditional operator. Now adding logical condition **(row==7 && col<=3)** to previous conditions using logical OR (||) conditional operator will give * character set can be shown in the box. The logical condition becomes

(row==1) || (col==3) || (row==7 && col<=3)

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2	2,3*	2,4	2,5
row=3	3,1	3,2	3,3*	3,4	3,5
row=4	4,1	4,2	4,3*	4,4	4,5
row=5	5,1	5,2	5,3*	5,4	5,5
row=6	6,1	6,2	6,3*	6,4	6,5
row=7	7,1*	7,2*	7,3*	7,4	7,5

Please Pay Attention

Using logical AND we have restricted or limited the placing of * character at row=7.The condition used is (row==7 && col<=3).

If condition were row==7 only, then the complete row=7 would have then got filled with * characters. The additional condition &&col<=3 in the expression (row==7 && col<=3) actually limited the * characters in the row=7, started from column 1 till column 3.

Again, adding condition to previous **(col==1 && row>=4).**

The selected * character set shown with thick border is representing condition **(col==1 && row>=4)**.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2	2,3*	2,4	2,5
row=3	3,1	3,2	3,3*	3,4	3,5
row=4	4,1*	4,2	4,3*	4,4	4,5
row=5	5,1*	5,2	5,3*	5,4	5,5
row=6	6,1*	6,2	6,3*	6,4	6,5
	7,1*	7,2*	7,3*	7,4	7,5

Please Pay Attention

Using logical AND we have restricted or limited the placing of * character at col=1.The condition is (col==1 && row>=4).

If condition were col==1 only, then the complete col=1 would have then get filled with * characters.

The additional condition &&row>=4 in the expression (col==1 && row>=4) actually limited the * characters at the column 1 started from row 4 till row 7.

Removing the (row, col) value pair which doesn't have * characters.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2			2,3*		
row=3			3,3*		
row=4	4,1*		4,3*		
row=5	5,1*		5,3*		
row=6	6,1*		6,3*		
row=7	7,1*	7,2*	7,3*		

OR

*	*	*	*	*
		*		
		*		
*		*		
*		*		
*		*		
	*			

Logical condition: (row==1) || (col==3) || (row==7 && col<=3) || (col==1 && row>=4).

ProblemYBJ: **Write the logical condition for the following style of alphabet J.**

	*	*	*	*	*
			*		
			*		
	*		*		
	*		*		
	*		*		
		*			

Note : If you are able to find your own correct logical expression, it means this book is successful in creating interest in this subject. If we are truly interested in any subject, we may get exhausted working with it but we won't feel burden.

eikhan theke main pora suru

***ExerciseYBK*: Alphabet K**

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2	1,3	1,4	1,5
row=2	2,1*	2,2	2,3	2,4	2,5
row=3	3,1*	3,2	3,3	3,4	3,5
row=4	4,1*	4,2	4,3	4,4	4,5
row=5	5,1*	5,2	5,3	5,4	5,5
row=6	6,1*	6,2	6,3	6,4	6,5
row=7	7,1*	7,2	7,3	7,4	7,5

We need to put * characters in the first column. So matching logical conditions will be **(col==1)** . We need to find a relationship among highlighted boxes as it suggests one of the arm of alphabet K.

The highlighted boxes which are represented by **(row, col)** value pairs as **(4,1),(3,2),(2,3),(1,4).**

If we pay little bit of attention we can observe that if we add row value/number to column value/number, we will get value 5 consistently.

(4,1),(3,2),(2,3),(1,4) ➔ (4+1),(3+2),(2+3),(1+4) ➔(5),(5),(5),(5),(5).

এই check টা করাটাই হল।

We need to verify if this holds true for other **row+col** value combinations or not. Let check by adding each **row+col** value for every box represented by **(row, col)** value pair. All **row+col** value are as follows and highlighted where **row+col=5.**

(1+1=2)	(1+2=3)	(1+3=4)	**(1+4=5)**	(1+5=6)
(2+1=3)	(2+2=4)	**(2+3=5)**	(2+4=6)	(2+5=7)
(3+1=4)	**(3+2=5)**	(3+3=6)	(3+4=7)	(3+5=8)
(4+1=5)	(4+2=6)	(4+3=7)	(4+4=8)	(4+5=9)
(5+1=6)	(5+2=7)	(5+3=8)	(5+4=9)	(5+5=10)
(6+1=7)	(6+2=8)	(6+3=9)	(6+4=10)	(6+5=11)
(7+1=8)	(7+2=9)	(7+3=10)	(7+4=11)	(7+5=12)

Nowhere **row + col = 5** holds true in other than our considered boxes. So our guess now becomes valid which uniquely identifies or selects the considered value pairs. This how we do selection of in general data by applying condition. In this case boxes with (**row, col)** combination are selected. Hence our next logical condition becomes **(row+col == 5)**.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2	1,3	1,4*	1,5
row=2	2,1*	2,2	2,3*	2,4	2,5
row=3	3,1*	3,2*	3,3	3,4	3,5
row=4	4,1*	4,2	4,3	4,4	4,5
row=5	5,1*	5,2	5,3	5,4	5,5
row=6	6,1*	6,2	6,3	6,4	6,5
row=7	7,1*	7,2	7,3	7,4	7,5

So logical condition becomes

(col == 1) || (row+col==5).

Similarly we need to find the relationship between

(4, 1), (5, 2), (6, 3), (7, 4) boxes, the other arm of the alphabet K shown by highlighted cell boarder for row value more than 4.

Previously, addition of (row +col) worked and this time seems to be subtraction may work. Let's study this case

(4,1),(5,2),(6,3),(7,4) ➔(4-1),(5-2),(6-3),(7-4) ➔(3),(3),(3),(3).

It's giving consistent result and we need to verify if any other **(row, col)** combination too follows this rule of selection or not. If we find any other **(row, col),** then we need to find some other relationship formula. The selection rule or logical condition should uniquely select **(4,1),(5,2),(6,3),(7,4).** Let check by subtracting each **row - col** value. All **(row – col)** value are shown as follows and highlighted ones were follows the rule **(row - col=3).**

(1-1=0)	(1-2=-1)	(1-3=-2)	(1-4=-3)	(1-5=-4)
(2-1=1)	(2-2=0)	(2-3=-1)	(2-4=-2)	(2-5=-3)
(3-1=2)	(3-2=1)	(3-3=0)	(3-4=-1)	(3-5=-2)
(4-1=3)	(4-2=2)	(4-3=1)	(4-4=0)	(4-5=-1)
(5-1=4)	**(5-2=3)**	(5-3=2)	(5-4=1)	(5-5=0)
(6-1=5)	(6-2=4)	**(6-3=3)**	(6-4=2)	(6-5=1)
(7-1=6)	(7-2=5)	(7-3=4)	**(7-4=3)**	(7-5=2)

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2	1,3	1,4*	1,5
row=2	2,1*	2,2	2,3*	2,4	2,5
row=3	3,1*	3,2*	3,3	3,4	3,5
row=4	4,1*	4,2	4,3	4,4	4,5
row=5	5,1*	5,2*	5,3	5,4	5,5
row=6	6,1*	6,2	6,3*	6,4	6,5
row=7	7,1*	7,2	7,3	7,4*	7,5

Nowhere **row – col = 3** holds true in other than our considered value pairs**.** So our guess now becomes valid to select these boxes under consideration**.**

Hence our next logical condition becomes

(row - col == 3). So final logical condition becomes

(col ==1) || (row+col==5) ||(row - col == 3).

Removing the (row, col) value pair where * character is not present, we will get as follows;

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*			1,4*	
row=2	2,1*		2,3*		
row=3	3,1*	3,2*			
row=4	4,1*				
row=5	5,1*	5,2*			
row=6	6,1*		6,3*		
row=7	7,1*			7,4*	

	*			*
	*		*	
	*	*		
OR	*			
	*	*		
	*		*	
	*			*

We have seen in this case addition and subtraction relationship among row and column values worked. Sometimes other arithmetic relationship like division or remainder among row and column value holds true to select the boxes means **(row, col)** value pairs. Logical condition can be implemented in many different ways. There could be possible different alternative ways of giving conditions for the same set of selection of **(row, col)** value pairs.

***ExerciseYBM*: Alphabet M needs to be fit in 7X5 box.**

*				*	
*	*		*	*	
*		*		*	
*				*	
*				*	
*				*	
*				*	

	col=1	col=2	col=3	col=4	col=5
row=1	1,1	1,2	1,3	1,4	1,5
row=2	2,1	2,2	2,3	2,4	2,5
row=3	3,1	3,2	3,3	3,4	3,5
row=4	4,1	4,2	4,3	4,4	4,5
row=5	5,1	5,2	5,3	5,4	5,5
row=6	6,1	6,2	6,3	6,4	6,5
row=7	7,1	7,2	7,3	7,4	7,5

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2	1,3	1,4	1,5*
row=2	2,1*	2,2	2,3	2,4	2,5*
row=3	3,1*	3,2	3,3	3,4	3,5*
row=4	4,1*	4,2	4,3	4,4	4,5*
row=5	5,1*	5,2	5,3	5,4	5,5*
row=6	6,1*	6,2	6,3	6,4	6,5*
row=7	7,1*	7,2	7,3	7,4	7,5*

Let start with putting * characters in the first column & in the last column.

So matching logical conditions will be: **(col==1)||(col==5).**

We need to find a relationship among highlighted boxes which are represented by

(row, col) value pairs as **(1, 1), (2, 2), (3, 3).**

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2	1,3	1,4	1,5*
row=2	2,1*	2,2	2,3	2,4	2,5*
row=3	3,1*	3,2	3,3	3,4	3,5*
row=4	4,1*	4,2	4,3	4,4	4,5*
row=5	5,1*	5,2	5,3	5,4	5,5*
row=6	6,1*	6,2	6,3	6,4	6,5*
row=7	7,1*	7,2	7,3	7,4	7,5*

Let's try to understand relationship that uniquely selects these specific set of row and column value pair.

It's easy to observe that row value/number and column value/number are the same. So we can express our selection of boxes by logical condition **(row==col).**

The logical condition (row==col) will select (4, 4) and (5, 5) also.

The box (5, 5) is already been selected by our previous condition.

We don't want box **(4, 4)** to be get selected in our case. We have to do selection till value pair **(3, 3)** starting from the value pair **(1, 1).** We have to use logical AND operator to apply restriction in selection of boxed and logical condition then can be given as follows;

(row==col) && (row <=3 && col<=3)

These selection of boxes till now can be expressed by logical condition

(col==1)|| (col==5) || ((row==col) && (row <=3 && col<=3))

Again, we need to find a relationship among given highlighted boxes in order to uniquely select them out all the given boxes and which are represented by (row, col) value pairs as **(3, 3), (2, 4), (1, 5).**

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2	1,3	1,4	1,5*
row=2	2,1*	2,2*	2,3	2,4	2,5*
row=3	3,1*	3,2	3,3*	3,4	3,5*
row=4	4,1*	4,2	4,3	4,4	4,5*
row=5	5,1*	5,2	5,3	5,4	5,5*
row=6	6,1*	6,2	6,3	6,4	6,5*
row=7	7,1*	7,2	7,3	7,4	7,5*

Let's try to understand relationship that uniquely selects these specific set of row and column value pairs which are as follows;

- ➔ **(3,3),(2,4),(1,5) or**
- ➔ **(3+3),(2+4),(1+5) or**
- ➔ **(6),(6),(6)**

If we pay little bit of attention we can observe that if we add row value/number to column value/number, we will get value 6 consistently. We need to verify if this holds true for other **row+col** value combinations or not. Let check by adding each **row+col** value for every box represented by **(row, col)** value pair**.** All **row+col value** are as follows and highlighted ones were that follows **row+col=6.**

	col=1	col=2	col=3	col=4	col=5
row=1	1+1=2	1+2=3	1+3=4	1+4=5	1+5=6
row=2	2+1=3	2+2=4	2+3=5	2+4=6	2+5=7
row=3	3+1=4	3+2=5	3+3=6	3+4=7	3+5=8
row=4	4+1=5	4+2=6	4+3=7	4+4=8	4+5=9
row=5	5+1=6	5+2=7	5+3=8	5+4=9	5+5=10
row=6	6+1=7	6+2=8	6+3=9	6+4=10	6+5=11
row=7	7+1=8	7+2=9	7+3=10	7+4=11	7+5=12

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2	1,3	1,4	1,5*
row=2	2,1*	2,2*	2,3	2,4	2,5*
row=3	3,1*	3,2	3,3*	3,4	3,5*
row=4	4,1*	4,2	4,3	4,4	4,5*
row=5	5,1*	5,2	5,3	5,4	5,5*
row=6	6,1*	6,2	6,3	6,4	6,5*
row=7	7,1*	7,2	7,3	7,4	7,5*

Now **row+col=6** also holds true for the value pair **(4, 2)** and we don't want to put character * over there. We have to restrict it to value pair **(3, 3).**The overall logical condition specifically for this can be given as; **(row+col ==6) && (row<=3 && col>=3)**

Let's put character * that satisfy this logical condition (row+col ==6) && (row<=3 && col>=3) and removing the other (row, col) value pairs.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2	1,3	1,4	1,5*
row=2	2,1*	2,2*	2,3	2,4*	2,5*
row=3	3,1*	3,2	3,3*	3,4	3,5*
row=4	4,1*	4,2	4,3	4,4	4,5*
row=5	5,1*	5,2	5,3	5,4	5,5*
row=6	6,1*	6,2	6,3	6,4	6,5*
row=7	7,1*	7,2	7,3	7,4	7,5*

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*				1,5*
row=2	2,1*	2,2*		2,4*	2,5*
row=3	3,1*		3,3*		3,5*
row=4	4,1*				4,5*
row=5	5,1*				5,5*
row=6	6,1*				6,5*
row=7	7,1*				7,5*

OR

*				*
*	*		*	*
*		*		*
*				*
*				*
*				*
*				*

The complete selection of boxes now can be expressed by logical condition

(col==1)||(col==5)||

((row==col) &&(row <=3 && col<=3))||

((row+col ==6) &&(row<=3 && col>=3))

In the logical condition **((row==col) &&(row <=3 && col<=3)),** due to logical condition **(row==col)** the part of the condition given by **(row <=3 && col<=3) is** either **row<=3 or col<=3** is enough.

This could be further given by

((row==col) && row <=3) or ((row == col) && col<=3).

In the logical condition **((row+col ==6) &&(row<=3 && col>=3)),** every **(row, col)** value pair in **(row+col ==6)** are given by **(5, 1), (4, 2) ,(3, 3), (2, 4), (1, 5) .**

(row, col) value pair	row value	col value
(1, 5)	**1**	**5**
(2, 4)	**2**	**4**
(3, 3)	**3**	**3**
(4, 2)	**4**	**2**
(5, 1)	**5**	**1**

Every row value is unique within their group of values and is similar with the case for col values too. Hence, considering either row value or col value is sufficient. It is not necessary to consider both the values of row and col to apply restriction condition as given by **&& (row <=3 && col>=3).**

So part of total logical condition which is given by **(row <=3 && col>=3**), considering either **row<=3 or col>=3** is enough**.** This could be further given by

((row+col ==6) && row <=3) or ((row+col ==6) && col>=3).

We can rewrite the actual logical condition as;

(col==1)||(col==5)||

((row==col) && row <=3)||

((row+col ==6) && row<=3)

Please notice the restriction condition **&&row <=3** is same for both logical condition expressions**.**

We can again rewrite logical condition in more compact form and can be given as follows:

(col==1)||(col==5)||

(((row==col)|| (row+col ==6)) && row<=3)

ProblemYBM1: **Change the following logical condition so that we should able to get following output.**

(col==1)||(col==5)||

((row==col) &&(row <=3 && col<=3))||

((row+col ==6) &&(row<=3 && col>=3))

```
*
*       *
*             *
*       *
*
*
*
```

ProblemYBM2: **Change the following logical condition so that we should able to get following output.**

(col==1)||(col==5)||

((row==col) &&(row <=3 && col<=3))||

((row+col ==6) &&(row<=3 && col>=3))

```
*                       *
*       *           *   *
*             *         *
*       *           *   *
*                       *
*                       *
*                       *
```

ProblemYBM3: **Change the following logical condition so that we should able to get following output.**

(col==1)||(col==5)||

((row==col) &&(row <=3 && col<=3))||

((row+col ==6) &&(row<=3 && col>=3))

```
*                   *
      *         *
            *
      *     *     *
*           *           *
            *
            *
```

ProblemYBM4: **Change the following logical condition so that we should able to get following output.**

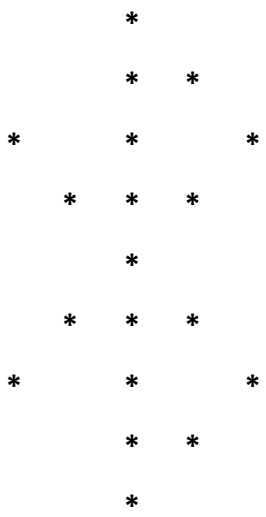
(col==1)||(col==5)||

((row==col) &&(row <=3 && col<=3))||

((row+col ==6) &&(row<=3 && col>=3))

```
*           *           *
      *     *     *
*     *     *     *     *
      *     *     *
*           *           *
            *
            *
```

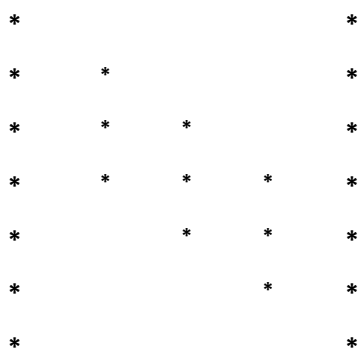
ProblemYBM5: **Write logical condition for the following <u>Bluetooth</u> like symbol.**



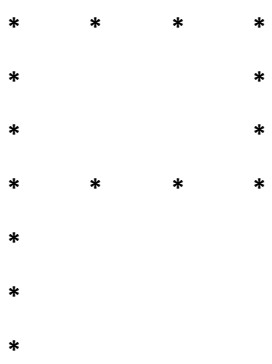
[Hint]

	col=1	col=2	col=3	col=4	col=5
row=1			1,3*		
row=2			2,3*	2,4*	
row=3	3,1*		3,3*		3,5*
row=4		4,2*	4,3*	4,4*	
row=5			5,3*		
row=6		6,2*	6,3*	6,4*	
row=7	7,1*		7,3*		7,5*
row=8			8,3*	8,4*	
row=9			9,3*		

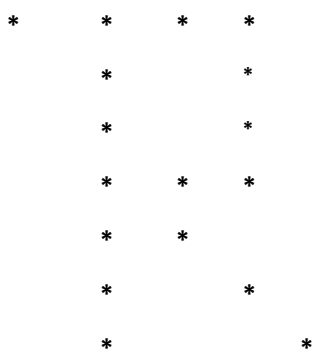
ProblemYBN: **Write logical condition for Alphabet N to be fit in 7X5 box.**



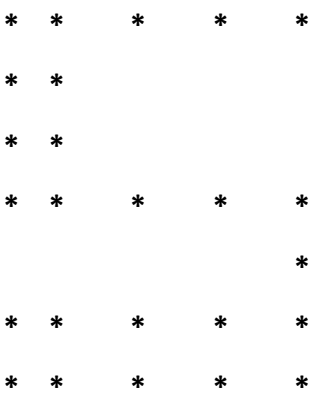
ProblemYBP: **Write logical condition for Alphabet P to be fit in 7X5 box.**



ProblemYBR: **Write logical condition for Alphabet R to be fit in 7X5 box.**



ProblemYBS: **Write logical condition for Alphabet S to be fit in 7X5 box.**



Decision making

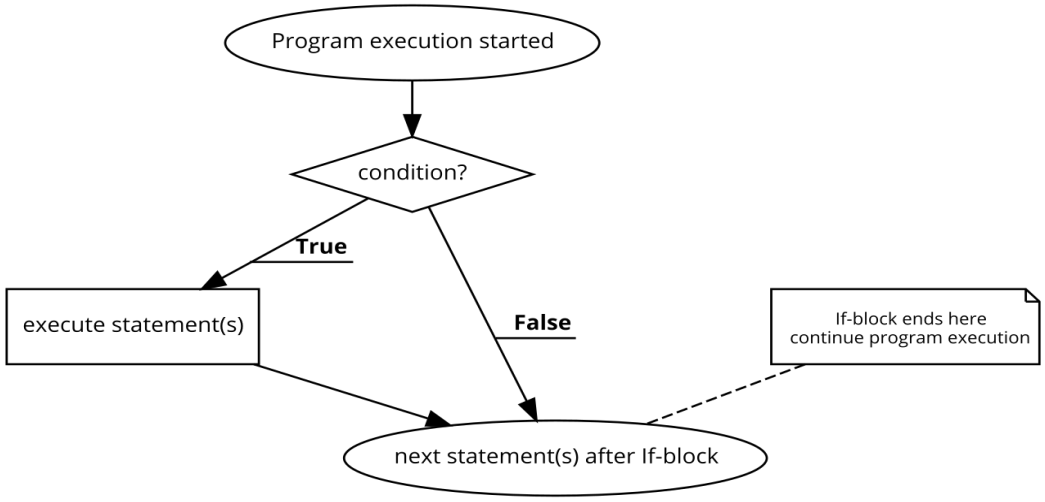
There are only two perfect answers in this world either **YES** or **NO**. In programming these perfect answers are represented by boolean data type. The boolean data type can take only single value at a time which is either **true** or **false**. So boolean value **true** is represented by **YES** and boolean value **false** is represented by **NO**. While running the program, the control of program may find logical expression. This logical expression is then evaluated to find the result. The result is expressed in terms of either **true** or **false**. Based on the boolean result of logical expression the control of program decides whether to execute bunch of programming statements or bypass it.

Such kind of decision making has to be programmed by using following different versions of decision making **if-statements**.

a) **If-statement**
b) **If-else statement**
c) **If-else if …statement (**Also known as **ladder if-else statement)**

Flow-chart helps to visualize the flow of control when condition becomes either **true** or **false**.

a) if-statement



The general syntax for **if-statement** can be given as:

```
If (condition) {
  execute statement(s);        If-block
}
next statement(s); // continue program execution
```

- **`condition:`** It will be an expression consisting of variables and operators in such a way so that whose outcome should be a boolean value either **true** or **false**. Since logical and comparison operators gives result in either **true** or **false** and commonly used in conditional expression.
- When the result of **`condition`** is **true**, it will execute statement(s) enclosed in the **If-block** {….}.

Note: - A block is placed in a program in order to group set of statement(s) by enclosing it in **open curly brace** and **closed curly brace**. An example as follows:

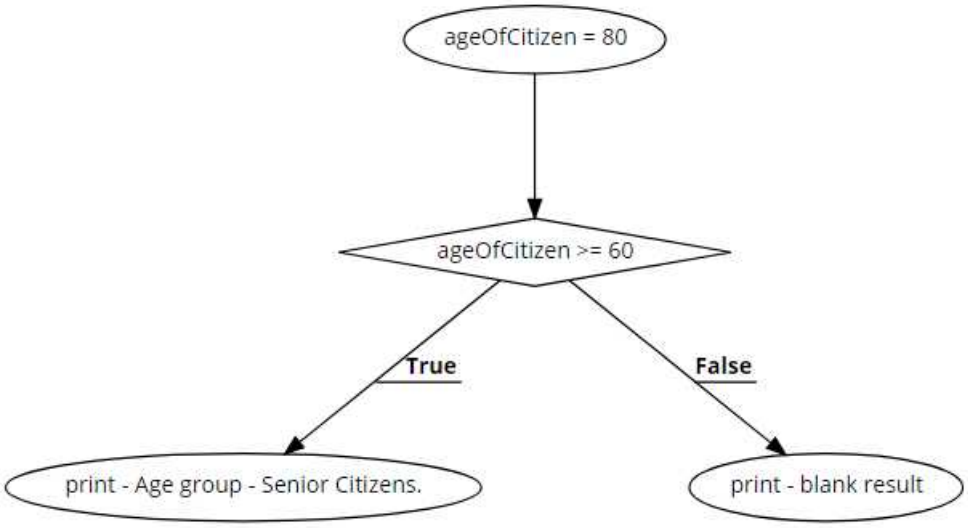
```
{
      statement(s);
}
```

Case 1.

`int ageOfCitizen = 80;`

condition: `(ageOfCitizen >= 60) will evaluate to “true”`

`So, if (80 >= 60) or if (condition) will be “true” and executes the if-block.`



StatementIf.c

```c
#include <stdio.h>

int main() {

      int ageOfCitizen = 80;

      if( ageOfCitizen >= 60 ) {

            printf("Age group - Senior Citizens.");

      }


      return 0;

}
```

Output

```
Age group - Senior Citizens.
```

Case 2. Change the value of ageOfCitizen in the program.

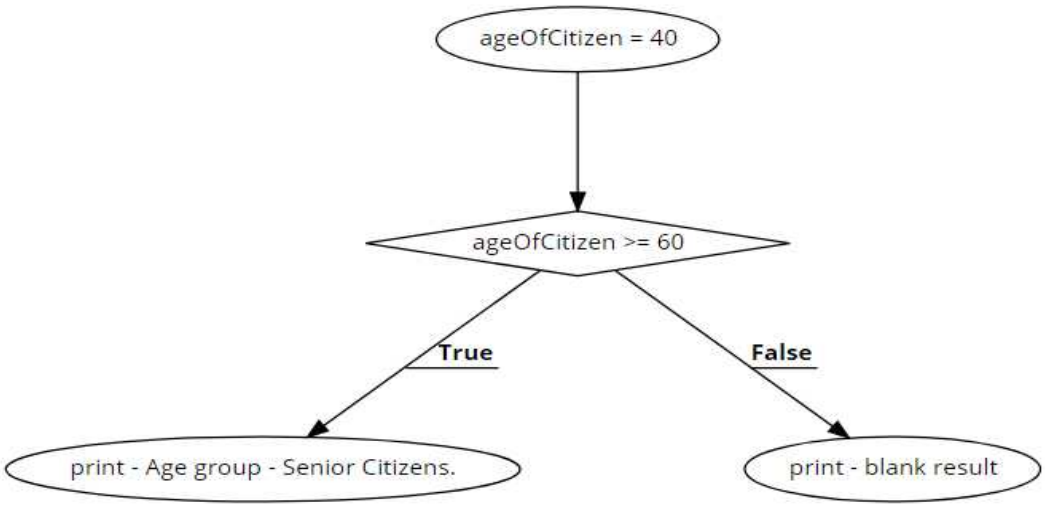
```
int ageOfCitizen = 40;
```

condition: (ageOfCitizen >= 60)

if (40 >= 60), will evaluate to “false”

if (condition) will be “false” and it will not execute the if-block.

The output will be blank.



StatementIf1.c

```c
#include <stdio.h>

int main() {

	int ageOfCitizen = 40;

	if( ageOfCitizen >= 60 ) {

		printf("Age group - Senior Citizens.");

	}

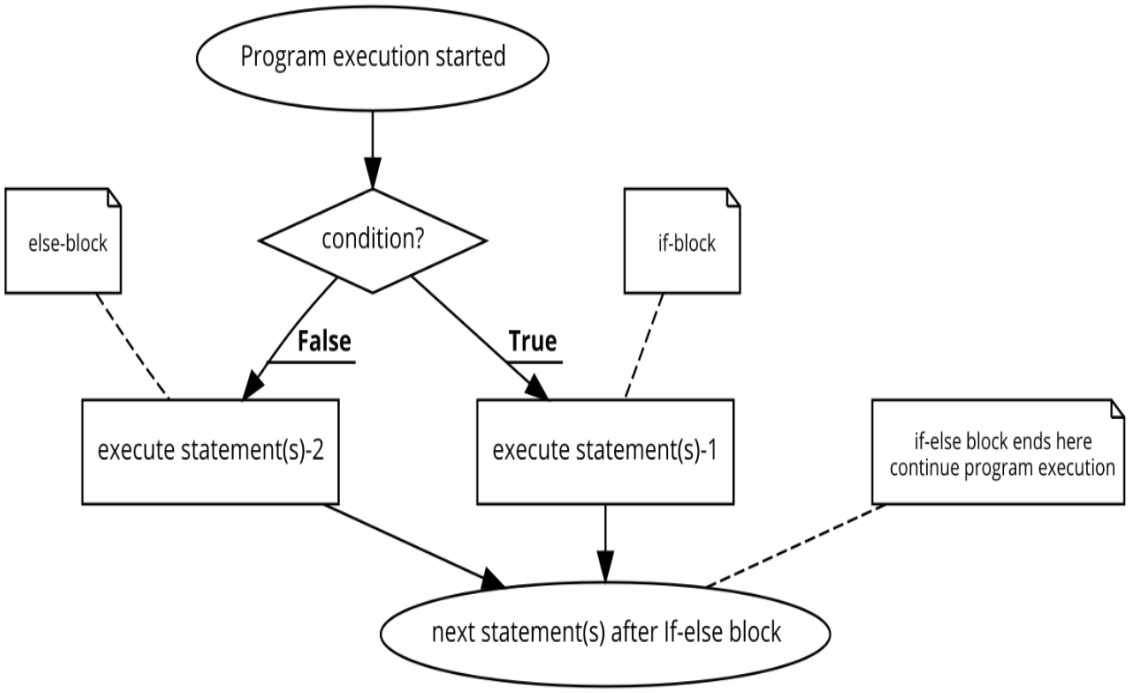
	return 0;

}
```

Output

It didn’t print anything, since condition becomes failed and code block within **if-statement** doesn’t get executed.

b) if-else statement



The general syntax for **if-else statement** can be given as:

```
If (condition) {
  execute statement(s)-1;     If-block
 } else {
  execute statement(s)-2;     else-block
}
next statement(s); // continue program execution
```

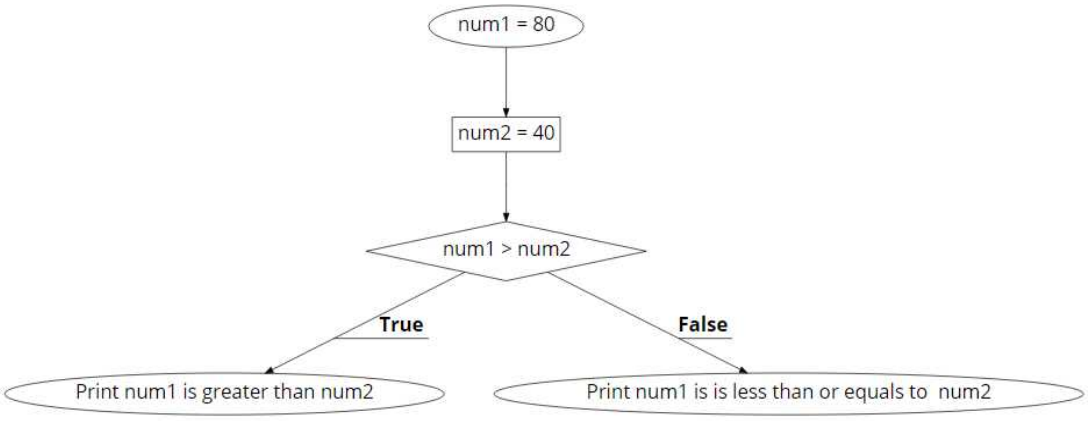
Case 1.

`int num1 = 80;`

`int num2 = 40;`

condition: `(num1 > num2) , if (80 > 40), will evaluate to “true”`

`if(condition) will be “true” and executes the if-block.`



StatementIfElse.c

```
#include <stdio.h>

int main() {

	int num1 = 80;

	int num2 = 40;

	if( num1 > num2) {

		printf("%d is greater than %d",num1,num2);

	}else {

		printf("%d is less than or equals to  %d", num1, num2);

	}

	return 0;

}
```

Output

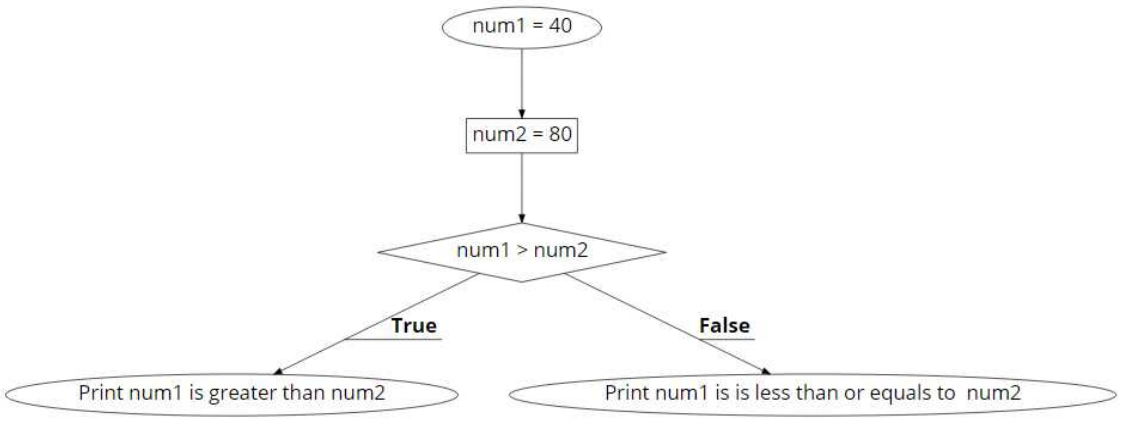
```
80 is greater than 40
```

Case 2. Change the value of num1 and num2 in the program.

```
int num1 = 40;
int num2 = 80;
```

condition: (num1 > num2),if (40 >80), will evaluate to “false”

If(condition) will be “false” and executes the else-block.



StatementIfElse1.c

```c
#include <stdio.h>

int main() {

	int num1 = 40;

	int num2 = 80;

	if( num1 > num2) {

	  printf( "%d is greater than %d", num1, num2);

	}else {

	  printf( "%d is less than %d", num1, num2);

	}

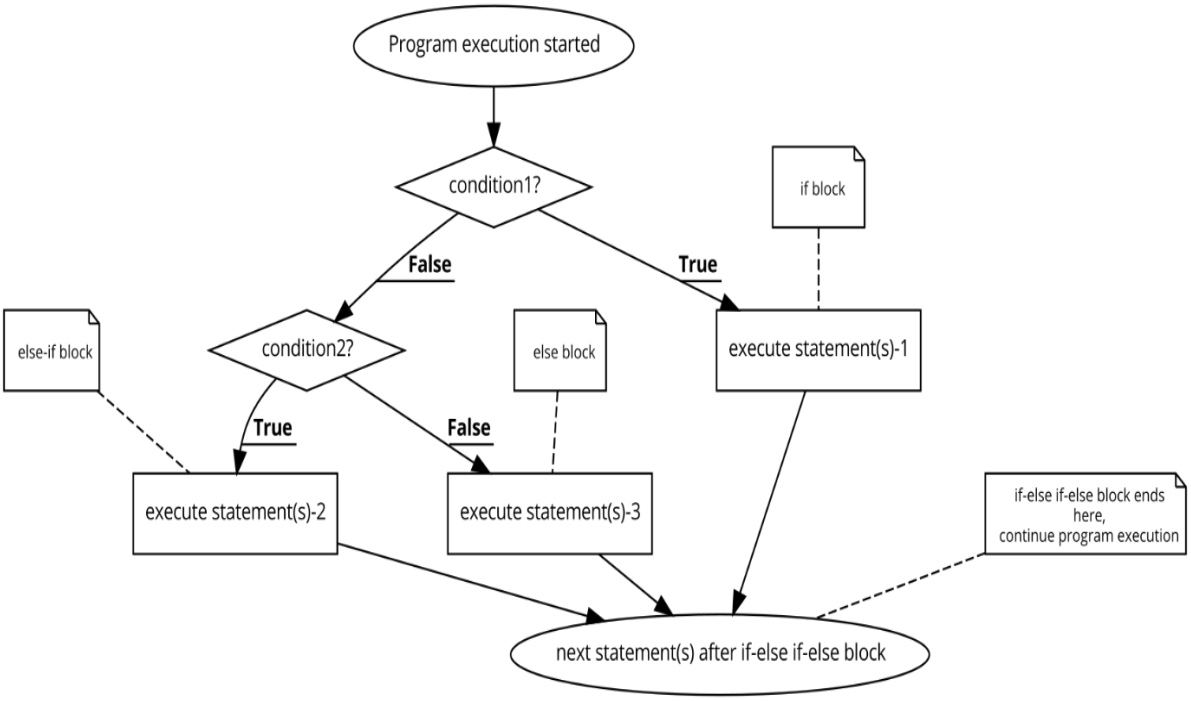
	return 0;

}
```

Output

```
40 is less than 80
```

c) if-else if-else statement



The general syntax for **if-else if-else statement** can be given as:

```
if (condition1) {
  execute statement(s)-1;          if-block
}else if(condition2) {
    execute statement(s)-2;       else-if-block
}else{
    execute statement(s)-3;        else-block
}
```

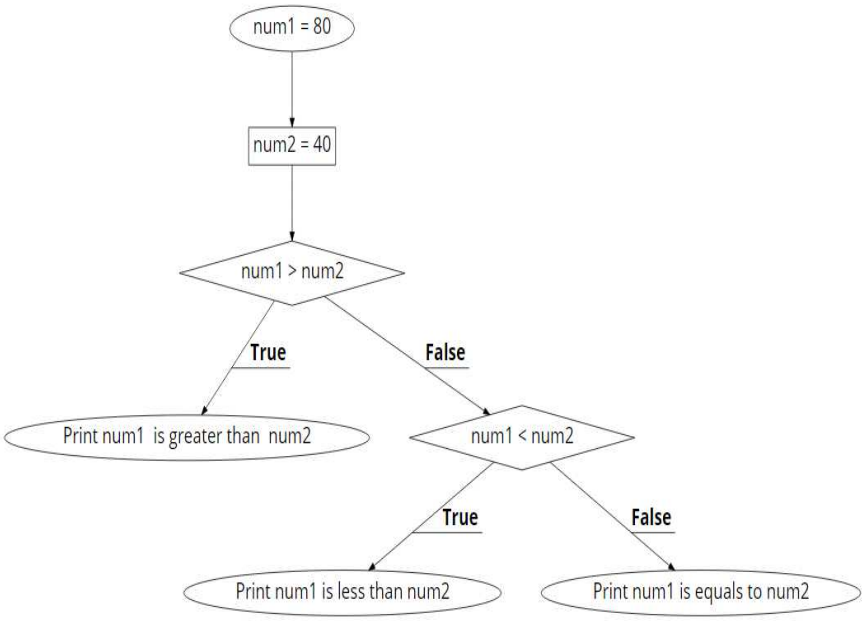
Case 1.

`int num1 = 80;`

`int num2 = 40;`

`condition : (num1 > num2) ,if (80 > 40), will evaluate to “true”`

`If(condition) will be “true” and executes the if-block.`



StatementIfElseIf.c

```c
#include <stdio.h>
int main() {

	int num1 = 80;
	int num2 = 40;

	if( num1 > num2 ) {
		printf("%d is greater than %d",num1, num2);
	}else if( num1 < num2 ) {
		printf("%d is less than %d",num1, num2);
	}else {
		printf("%d equals to %d" + num2);
	}

	return 0;
}
```

Output

```
80 is greater than 40
```

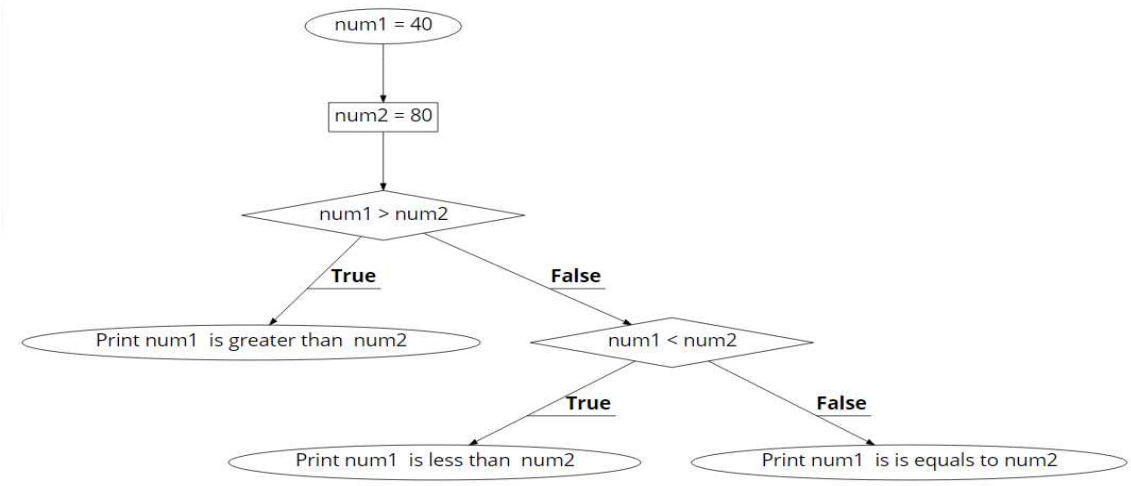
Case 2. Change the value of num1 and num2 in the program.

```
int num1 = 40;

int num2 = 80;
```

(num1 > num2) boolean expression will evaluate to “false” and will go to next boolean expression (num1 < num2).

(num1 < num2) will evaluate to “true” as (40 < 80).



So, else if (num1 < num2), else if(condition) will be “true” and executes the else if-block.

StatementIfElseIf1.c

```
#include <stdio.h>

int main() {

	int num1 = 40;

	int num2 = 80;

	if( num1 > num2 ) {

		printf("%d is greater than %d",num1, num2);

	}else if( num1 < num2 ) {

		printf("%d is less than %d",num1, num2);

	}else {

		printf("%d equals to %d" + num2);

	}

	return 0;

}
```

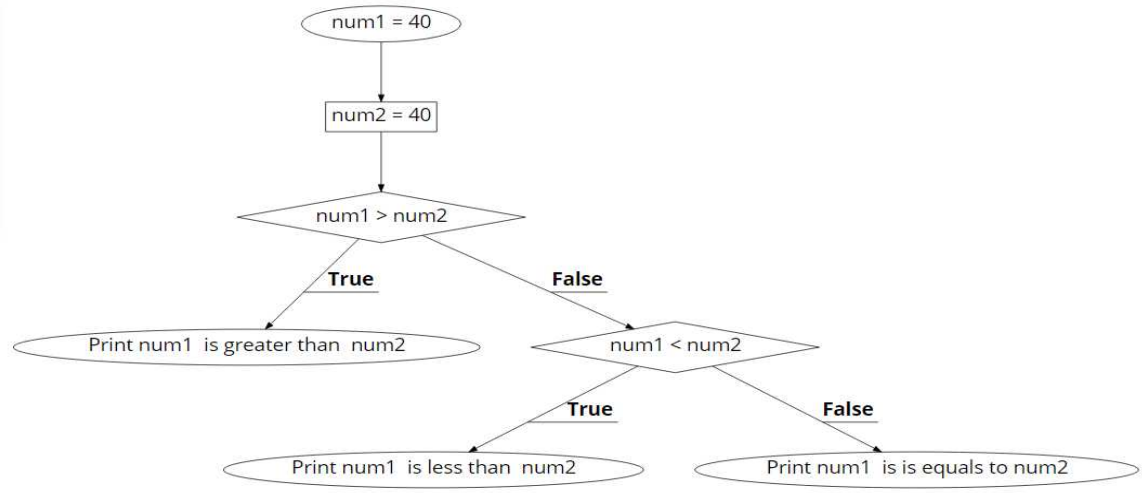
Output

```
40 is less than 80
```

Case 3. Change the value of num2 in the program.

int num1 = 40;

int num2 = 40;



if(num1 > num2) boolean expression will evaluate to "false" and then it will go to next boolean expression else if(num1 < num2). The (num1 < num2) will evaluate to "false" as (40 < 40) doesn't holds true. So, else if(num1 < num2) or else if (condition) will be "false" and finally executes the last block which is an else-block. The else-block has no condition attached to it.

The **last else block** will get executed when all the conditions will become failed. It's a default block and at-least this block will get executed if none other block.

StatementIfElseIf2.c

```
#include <stdio.h>

int main() {

	int num1 = 40;

	int num2 = 40;

	if( num1 > num2 ) {

		printf("%d is greater than %d",num1, num2);

	}else if( num1 < num2 ) {

		printf("%d is less than %d",num1, num2);

	}else {

		printf("%d equals to %d", num1, num2);

	}

	return 0;

}
```

Output

```
40 equals to 40
```

ProblemLadderIfElse: A college came up with an idea to encourage CS students by refunding some percentage of their bi-semester CS Course fees. College management laid the following conditions to be eligible for the same. Moreover, if a student scored in Grade-0, will be declared failed.

If a student scored in Grade-5, will be eligible for 35% refund of the college fees.

If a student scored in Grade-4, will be eligible for 25% refund of the college fees.

If a student scored in Grade-3, will be eligible for 20% refund of the college fees.

If a student score scored in Grade-2, will be eligible for 10% refund of the college fees.

If a student scored in Grade-1, will be passed with margin.

LadderIfElse.c

```c
#include <stdio.h>
int main() {

	int grade = 2;

	if(grade == 1) {
		printf("35%% fee refund.");
	}else if(grade == 2) {
		printf("25%% fee refund.");
	}else if(grade == 3) {
		printf("20%% fee refund.");
	}else if(grade == 4) {
		printf("10%% fee refund.");
	}else if(grade == 5) {
		printf("Passed with margin.");
	}else if(grade == 0) {
		printf("Failed,please retry.");
	}else  {
		printf("Invalid grade ? " + grade);
	}
	return 0;
}

//Note:-  %% double percentage symbols are used to print %
```

Output

```
25% fee refund.
```

It's going to test if-conditions sequence-wise, unless and until some if-condition becomes **true**.

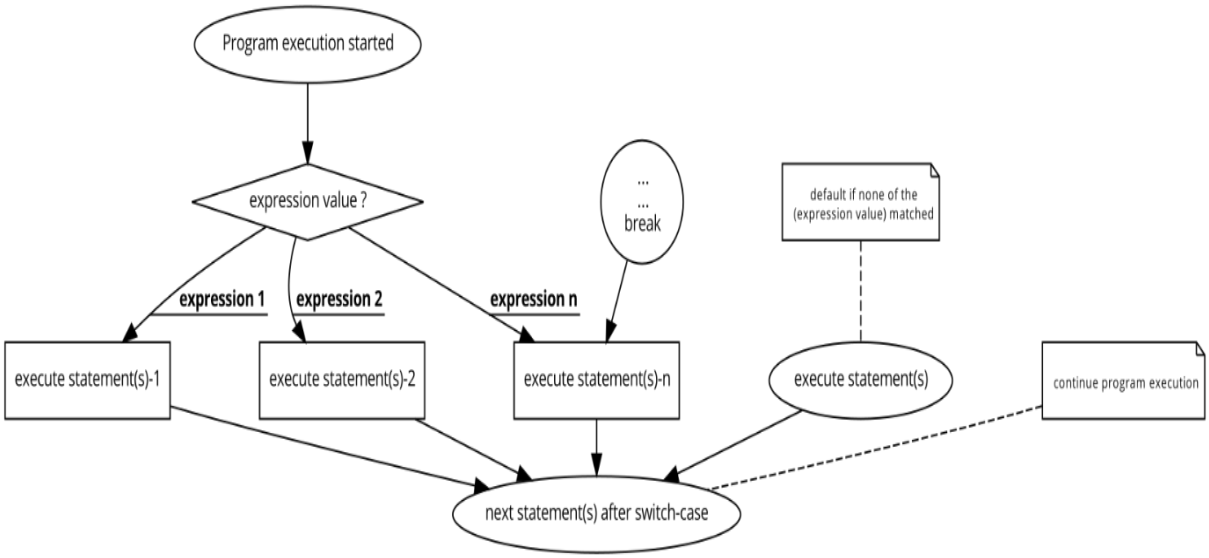
This type of many else-if is also called as ladder else-if, which is basically not a good practice.

When two or three **else-if condition** are used, is considered to be okay and further more **else-if conditions** are strongly discouraged as it reduces the code readability .

Then what is the way-out?

The answer is **switch-case** construct.

d) switch-case



The general syntax for **switch-case statement** can be given as:

```
switch(expression value ?){
  case expression 1:
            execute statement(s)-1;
    break;
  case expression 2:
            execute statement(s)-2;
      break;
   …
   …
   break;
  case expression n:
            execute statement(s)-n;
    break;
  default:
            // default if none of the case expression value matched
            execute statement(s);
  }// switch-case ends
next statement(s); // continue program execution
```

Let's convert ladder if-else statement to switch-case. The previous code `LadderIfElse.c` can be re-written as:

ConvertLadderIfElseToSwitchCase.c

```
#include <stdio.h>

int main() {

	int grade = 21;

	switch (grade) {

	case 1:

		printf("35%% fee refund.");

		break;

	case 2:

		printf("25%% fee refund.");

		break;

	case 3:

		printf("20%% fee refund.");

		break;

	case 4:

		printf("10%% fee refund.");

		break;

	case 5:

		printf("Passed with margin.");

		break;

	case 0:

		printf("Failed,please retry.");

		break;

	default:

		printf("Invalid grade ? %d", grade);

		break;

	}

	return 0;

}
```

Output

```
Invalid grade ? 21
```

switch-case is the elegant way of dealing with **ladder if-conditions.**

My Notes : programming Logic Experiences.

You can't manage what you can't measure.

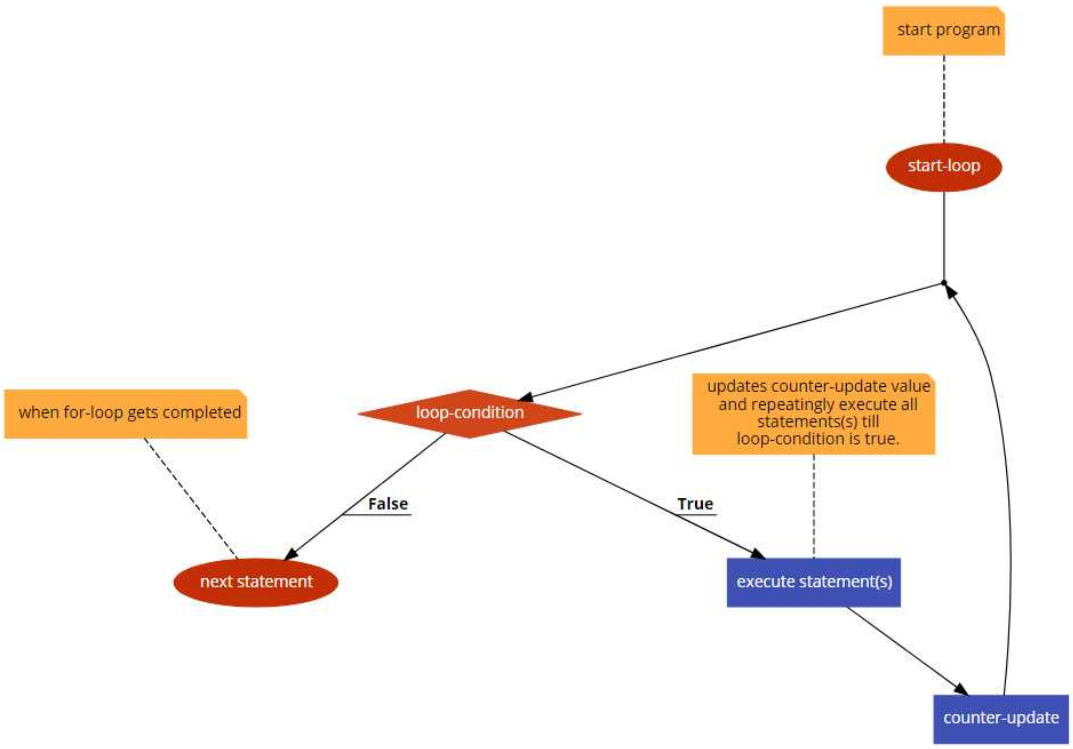
- Peter Drucker

Write the key points of your programming logic experiences from alphabetic exercises.

S.No	Short Description	Description

About *for-loop*

In case of high calculation intensive data or large amount of data processing activities, human resources are not reliable in terms of accuracy and productivity as compared to computers. Nothing is automatic unless someone does it. The feature of repetitiveness helps to automate tedious data processing work amazingly efficient and accurate as compared to doing it manually. One of the repetitive control feature in a programming is known as **for *statement*** or ***for-loop.*** In a very generic way ***for-loop*** can be expressed as follows:



for (***start-loop***; ***loop-condition*** ; ***counter-update***) {

    execute statement(s);

}

next statement;

- ***start-loop*** is a simple way of initialisation of a variable and anyway from some point the process of repetitive cycle has to be get started.
- ***loop-condition*** is a logical expression which evaluates a boolean value either ***true*** or ***false***. Based on this (***true/false***), **for-loop** control structure decides whether to repeat the set of statement(s) or not.
- ***counter-update*** is a change in variable's value in a fixed smaller steps (increment/decrement) only after all statement(s) gets executed in **for-loop** block. Usually it holds strong bonding with ***start-loop*** and ***loop-condition***.

The ***for-loop*** helps a programmer to repeat programming statement(s) over a "range of values". When we talk about a range, then it must have a minimum value and a maximum value.

We will use the following notations:

min_val minimum value from where the for-loop starts the repetition.

max_val maximum value to the extent where for-loop will do the repetition.

It means ***for-loop*** will operate between *min_val* and *max_val* values. We can understand this concept better by doing examples based on incrementing or decrementing ***counter-update*** expression of the ***for-loop.***

Basics

Here are some basic examples to get clarity about how the ***for-loop*** behaves.

Let's express range [*min_val*, *max_val*] where *min_val* < *max_val* with counter-update incremental value 1 .The for-loop can be expressed as;

for (**int** *variable_name* = *min_val* ; *variable_name* <= *max_val* ; *variable_name* ++){

statement(s);

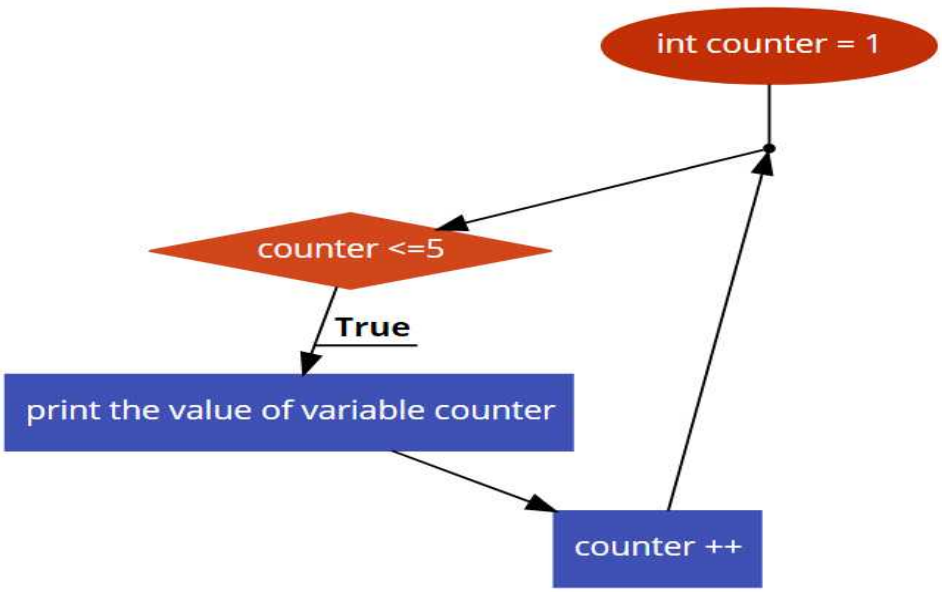
}

Case a:

Let the range be [1,5] with counter-update incremental value 1 ,then the for-loop can be expressed to print counter variable's value 1 to 5 in ascending order which is as follows;

```
for ( int counter = 1 ; counter <=5 ; counter ++ ){
      printf("%d\n",counter);//print output to console
}
```

Hence the above ***for-loop*** can be expressed as a flow chart which is as follows:



ForLoopIncrement.c

```c
#include <stdio.h>
int main() {
	for ( int counter = 1 ; counter <=5 ; counter ++ ){
		printf("%d\n",counter);//print to console
	}
	return 0;
}
```

Output

```
1
2
3
4
5
```

Case b: Changing the ***loop-condition*** counter <= 5 to counter < 5, and let's see the result.

ForLoopIncrement1.c

```c
#include <stdio.h>
int main() {
	for ( int counter = 1 ; counter < 5 ; counter ++ ){
		printf("%d\n",counter);//print to console
	}
	return 0;
}
```

Output

```
1
2
3
4
```

It printed from 1 to 4.

Case c: Let's modify `ForLoopIncrement1.c` keeping **loop-condition(** `counter <5 )` same and yet still should print 1 to 5.

`ForLoopIncrement2.c`

```
#include <stdio.h>
int main() {
	for ( int counter = 0 ; counter < 5 ; counter ++ ){
		int  result = counter + 1; //add 1 to the counter
		printf("%d\n",result);//print to console
	}
	return 0;
}
```

Output

```
1
2
3
4
5
```

The variable counter started with the value 0 instead of 1.

Case d: Let's express range[min_val, max_val] where min_val < max_val with counter-update decremented value 1 .

The for-loop can be expressed as;

for (**int** *variable_name* = *max_val*; *variable_name* >= *min_val* ; *variable_name* --){

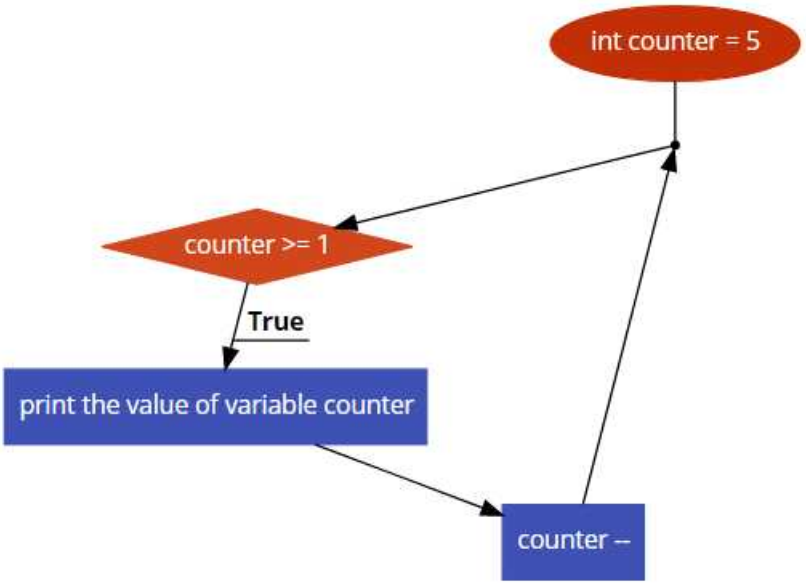
	statement(s);

}

To print numbers 1 to 5 in descending order with ***counter-update*** decremented value 1, then the ***for-loop*** can be expressed as;

```
	for ( int counter = 5 ; counter >= 1; counter -- ){
		printf( "%d\n", counter ); //print to console
	}
```

Hence decrement ***for-loop*** can be expressed as the given following flow chart.



ForLoopDecrement.c

```
#include <stdio.h>
int main() {
	for ( int counter = 5 ; counter >= 1; counter -- ){
		printf("%d\n", counter ); //print to console
	}
	return 0;
}
```

Output

```
5
4
3
2
1
```

For-loop with {break} and {continue} keywords

- "break " keyword within the for-loop stops further repeation of loop when condition for "break;" statement is reached.
  It's similar like a **moving car stops** when driver applies the **break**.
- "continue" keyword bounce back control of the executing program to the for-loop to continue without giving chance to execute further statement(s) after the **continue;** statement.

ForLoopContinueNext.c

```
#include <stdio.h>

int main() {

	for( int cnt = 1 ; cnt <= 10 ; cnt ++ ) {

		if( cnt <= 3) {

			printf("keyword{continue} is throwing ");

			printf("control of program back.\n");

			continue;

		}

		printf("Please Join Us...!!!\n");

		if( cnt == 5) {

			printf("keyword{break} at cnt = %d\n", cnt);

			break;

		}

		printf("Please look back...\n" );

	}

	return 0;

}
```

Output

```
keyword{continue} is throwing control of program back.
keyword{continue} is throwing control of program back.
keyword{continue} is throwing control of program back.
Please Join Us...!!!
Please look back...
Please Join Us...!!!
keyword{break} at cnt = 5
```

Nested for-loop/Inner for-loop

We have previously seen examples of for-loop that have if-statements or any other simple statements. Instead of a simple if- statement within for-loop we place another for-loop then how the things will be. For example;

```
for ( int counter = 5 ; counter >= 1; counter -- ){
      // print the variable counter's value
      printf( "%d\n", counter );  //statement1
      printf( "%d\n", 2*counter );//statement2
      printf( "%d\n", 3*counter );//statement3
}
```

We know that these considered statements will get execute in order. Firstly statement1 will get executed, then statement2 and lastly statement3 will get executed. All these statements will repeat 5 times when we execute the considered piece of program. If we place another for-loop instead of any simple statement, the major style of work doesn't change. We need to think in way that **inner for-loop** itself is like some other single programming statement. Whenever control of a program starts executing programming statements in a sequence and finds any **inner for-loop**, it will then fully executes **inner for-loop** and then executes next statement(s) if there are any available.

For example;

```
for ( int counter = 5 ; counter >= 1; counter -- ){
      // print the variable counter's value
      printf( "%d\n", counter );  //statement1
      //The for-loop is a statement2
      for(int inner = 1 ; inner <= 5; inner ++){
            printf("%d\n", inner );

      }
      printf("%d\n", 3* counter );//statement3
}
```

Please treat **nested for-loop** as a **single** programming statement

Now as usual **statement1** gets completed first. Then inner for-loop with 5 times repetitions will get executed completely and finally **statement3** will get executed. All these three statements will repeat 5 times as suggested by the structure of outer for-loop. Whenever a for-loop is placed within the code block of another for-loop is known as **inner for-loop** or **nested for-loop**.

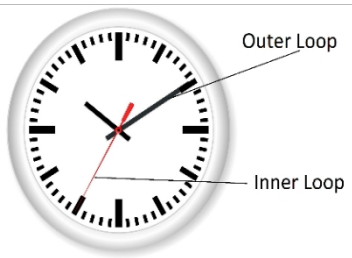
The inner for-loop will **restart from the beginning** for every new value of a outer for-loop.

It is easy to follow this concept if we try to relate it with time piece clock with following assumptions;

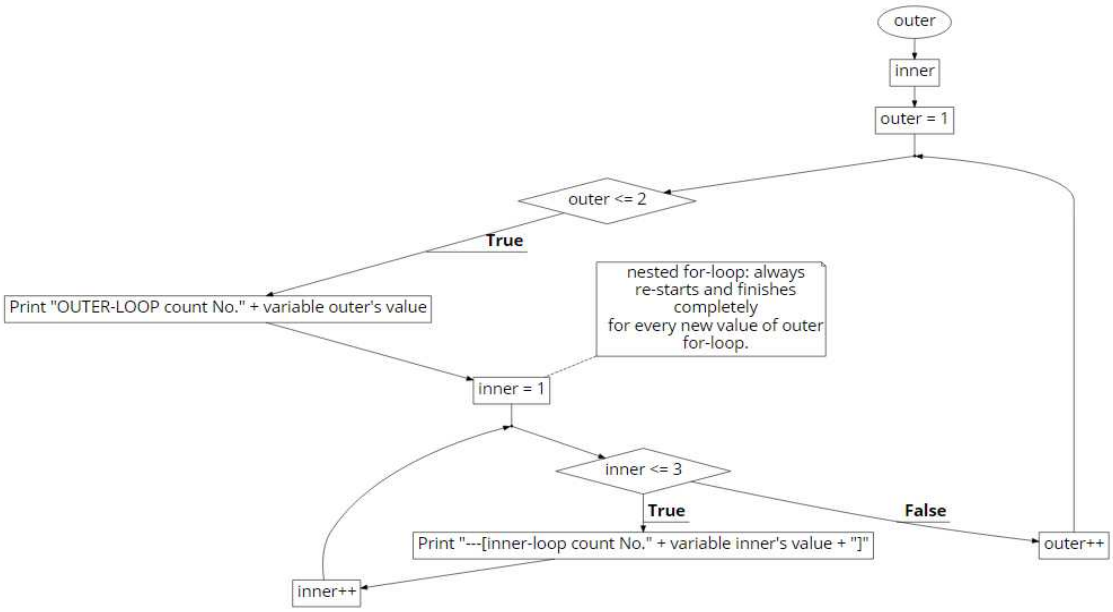
Inner Loop → It's like a "**seconds" hand** in a clock

Outer Loop → It's like a "**minute" hand** in a clock

In a clock when "**seconds**" hand completes one cycle then only afterwards "**minute**" hand will get updated by counter value 1 and not before.



The *inner* for-loop will execute completely before every new counter value of *outer* for-loop to continue repetition depends upon it's loop-condition.



NestedForLoop.c

```c
#include <stdio.h>
int main() {

	for( int out = 1 ; out <= 2 ; out++ ) {

		printf("OUTER count No.%d\n", out);

		// nested for-loop: always re-starts and finishes

		// completely for every new value of outer for-loop.

		for( int in = 1; in <= 3; in++ ) {

			printf(" <INNER> count No.%d\n", in );

		}

	}

	return 0;

}
```

Output

```
OUTER count No.1
 <INNER> count No.1
 <INNER> count No.2
 <INNER> count No.3
OUTER count No.2
 <INNER> count No.1
 <INNER> count No.2
 <INNER> count No.3
```

Nested Loop: Beginner Programmer's Best Friend

Best friend is a person not only who appreciates our good quality but also gives reality check by showing our weaknesses or mistakes whenever it is required. Nested loop plays such best friend's role while learning programming logic and it also gives a reality check how much true is our programming logic. This is my personal opinion that **practising a programming logic just by juggling with numbers is not a good idea** and specially for a person who had a less or no inclination towards higher school mathematics. Many of them feel bored to practice on such mathematical problems in order to learn skill of programming logic.

Everybody is different and everyone has a different buffer of patience and pace of learning. There should be a simple to understand approach. Any beginner who want to learn/improve programming should be able to do it with his own learning pace on the backdrop of minimal technical stuff.

It is broadly three-step learning process on pictorial based exercises so that beginners get comfortable in the programming.

Step1. Clarity about how logical condition is to be explored.

Step2. Clarity about how nested loop works. This is our next major hands-on training.

Step3. Finding a scope to improve implemented program so that programming logic can be further simplified. Looking for alternate possible solutions.

After going through these three steps, anyone will be in a confident position to learn and explore advanced features of programming language.

Why nested loops are such an important deal?

The outer for-loop and inner for-loop, each one are used for outputting data in different direction horizontally and vertically to give a 2-dimensional output. Usually outer for-loop moves the control vertically and inner for-loop outputs the data horizontally. This gives a grid kind of output. This particular feature of looping can be utilized to perfect the imagination of the learner. Yes, logic building is a skill and can be developed using simple maths and common sense. It's not a must for a person to be very good in maths in order to be a successful programmer or software developer. If a learner is able to imagine the behaviour of outer and nested loop correctly, the person will gain enough confidence that he/she can code.

The first and foremost requirement for any subject related teaching is to first create interest about the subject among the student or learner group. If a leaner gets deeply connected with the subject than rest of the learning with be without burden. Learner will enjoy learning and this course is

strictly meant for that. If any learner take this course and follow honestly, will surely see the fruits of effort without a doubt.

Nested loop gives good drilling exercise to a new/novice/beginner/unconfident programmer.

Once a learner becomes interested in programming then all the learning will become thrilling and exciting. That interest will come naturally when learner gets some confidence in developing programming logic on its own. For that purpose only very few things needs to be practice well and that is covered in appropriate depth. My sincere advice is please study and practice this book end to end before looking to any other resources.

This book helps you progress faster and it's kind of essential pre-workouts for programming.

I personally don't agree that those who give-up learning programming, that they are less capable but actually it doesn't excites them to put efforts. Whatever may be the learning pace of a person it doesn't matter but what matters is involvement in learning. True kind of involvement is not at all possible without interest. This book is all about proper installation of interest and confidence that any person can do programming by putting some efforts in the right direction.

The selection of exercises and problems are done for the main reason that learning should be naturally adaptable to any learner. The learner just need to follow step by step instructions mentioned in this book.

The confidence and interest in programming will be assured as you naturally learn and complete the training. This is not just a book, basically a training guide. Some initial efforts and patience is definitely required. Once a learner understood the required fundamentals, the further improvement will be sky-rocketed due to immense love for a subject which will come naturally during the practice. These set of exercises are result of experiments conducted in order to teach programming for the beginner's level and who had no or some programming experience.

After completion of these exercises everyone will able to develop a good level of confidence within a span of 2-3 months of time. High point was that students becomes interested in programming. Programming was never remained boring or burden for them. Learner will be able to naturally developed a tendency of patience in order to push himself/herself to solve the problems and being totally focused. In short, it makes the learning interesting.

ExerciseGB1.For every single value of outer for-loop in an iteration, inner for-loop executes completely.

The complete execution of inner for-loop is treated like an execution of a normal statement.

Unless and until inner for-loop get executed completely, the outer for-loop next iteration won't happen. The following program gives the demonstration of printing the value of columns vertically for every single row value.

ForLoopEveryRowAllColumns.c

```
#include <stdio.h>
int main() {
	int TOTAL_ROWS = 3;
	int TOTAL_COLUMNS = 3;
	for( int row = 1 ; row <= TOTAL_ROWS ; row ++ ) {
		for( int col = 1 ; col <= TOTAL_COLUMNS ; col ++ ) {
		  printf( "Row = %d |  Col = %d\n" , row, col);
		}
	   printf("--------------------------------\n");
	}

	return 0;
}
```

Output

```
Row = 1| Col = 1
Row = 1| Col = 2
Row = 1| Col = 3
--------------------------------
Row = 2| Col = 1
Row = 2| Col = 2
Row = 2| Col = 3
--------------------------------
Row = 3| Col = 1
Row = 3| Col = 2
Row = 3| Col = 3
--------------------------------
```

ExerciseGB2.Printing column iteration values horizontally for every iteration value of row. The row value should come in every different row. So row level iteration values print vertically.

ForLoopEveryRowAllColumnsHorizontally.c

```
public class ForLoopEveryRowAllColumnsHorizontally {

	public static void main(String[] args) {

		int TOTAL_ROWS = 3;

		int TOTAL_COLUMNS = 3;


		for( int row = 1 ; row <= TOTAL_ROWS ; row ++ ) {

			printf( "Row = " + row + " |" );

			for( int col = 1 ; col <= TOTAL_COLUMNS ; col ++ ) {

				printf( " Col = " + col);

			}

			// control moves to the next line and this how

			// printing of row and col values happens in the next row

			printf("\n");

		}

	}

}
```

Output

```
Row = 1| Col = 1 Col = 2 Col = 3
Row = 2| Col = 1 Col = 2 Col = 3
Row = 3| Col = 1 Col = 2 Col = 3
```

ProblemGB2: Write a program for the following output considering rows and columns are 3.

```
Row = 1| Col = 1
Row = 2| Col = 2
Row = 3| Col = 3
--------------------------------
Row = 4| Col = 1
Row = 5| Col = 2
Row = 6| Col = 3
--------------------------------
Row = 7| Col = 1
Row = 8| Col = 2
Row = 9| Col = 3
--------------------------------
```

ExerciseGB3.Printing row iteration and column iteration values as (row, col) as shown in the following output

(1,1) (1,2) (1,3) (1,4) (1,5)

(2,1) (2,2) (2,3) (2,4) (2,5)

(3,1) (3,2) (3,3) (3,4) (3,5)

(4,1) (4,2) (4,3) (4,4) (4,5)

(5,1) (5,2) (5,3) (5,4) (5,5)

ForLoopCoordinateRepresentation.c

```c
#include <stdio.h>

int main() {

	int TOTAL_ROWS = 5;

	int TOTAL_COLUMNS = 5;


	for( int row = 1 ; row <= TOTAL_ROWS ; row ++ ) {

		for( int col = 1 ; col <= TOTAL_COLUMNS ; col ++ ) {

			printf("(%d,%d) ", row, col);

		}

		// moves the control to the next line and

		// prints next row iteration value

		printf("\n");

		printf("\n");

	}

	return 0;

}
```

Output

```
(1,1) (1,2) (1,3) (1,4) (1,5)

(2,1) (2,2) (2,3) (2,4) (2,5)

(3,1) (3,2) (3,3) (3,4) (3,5)

(4,1) (4,2) (4,3) (4,4) (4,5)

(5,1) (5,2) (5,3) (5,4) (5,5)
```

ExerciseGB4.Generating pattern using character "*" by printing it for every (row, col) iteration values.

RowColumnStars.c

```c
#include <stdio.h>
int main() {
	int TOTAL_ROWS = 10;
	int TOTAL_COLUMNS = 10;
	for( int row = 1 ; row <= TOTAL_ROWS ; row ++ ) {
		for( int col = 1 ; col <= TOTAL_COLUMNS ; col ++ ) {
			printf("* " );
		}
		// moves the control to the next line
		// printing of "*" characters starts from next line
		printf("\n");
	}
	return 0;
}
```

Output

```
* * * * * * * * * *
* * * * * * * * * *
* * * * * * * * * *
* * * * * * * * * *
* * * * * * * * * *
* * * * * * * * * *
* * * * * * * * * *
* * * * * * * * * *
* * * * * * * * * *
* * * * * * * * * *
```

ProblemGB3: Write a program for the following output considering rows and columns are 5.

Output

```
*-*-*-*-*
*-*-*-*-*
*-*-*-*-*
*-*-*-*-*
*-*-*-*-*
```

ProblemsOnShapes: Write a program for each of the following different types of output.

Ouput-A

```
1==1 1==2 1==3 1==4 1==5

2==1 2==2 2==3 2==4 2==5

3==1 3==2 3==3 3==4 3==5

4==1 4==2 4==3 4==4 4==5

5==1 5==2 5==3 5==4 5==5
```

Ouput-B

Ouput-C

```
1<=1 1<=2 1<=3 1<=4 1<=5

2<=1 2<=2 2<=3 2<=4 2<=5

3<=1 3<=2 3<=3 3<=4 3<=5

4<=1 4<=2 4<=3 4<=4 4<=5

5<=1 5<=2 5<=3 5<=4 5<=5
```

Ouput-D

```
3<1 3<2 3<3 3<4 3<5 3<6 3<7 3<8

4<1 4<2 4<3 4<4 4<5 4<6 4<7 4<8

5<1 5<2 5<3 5<4 5<5 5<6 5<7 5<8

6<1 6<2 6<3 6<4 6<5 6<6 6<7 6<8

7<1 7<2 7<3 7<4 7<5 7<6 7<7 7<8

8<1 8<2 8<3 8<4 8<5 8<6 8<7 8<8
```

Ouput-E

```
1>1 1>2 1>3 1>4 1>5 1>6

2>1 2>2 2>3 2>4 2>5 2>6

3>1 3>2 3>3 3>4 3>5 3>6

4>1 4>2 4>3 4>4 4>5 4>6

5>1 5>2 5>3 5>4 5>5 5>6

6>1 6>2 6>3 6>4 6>5 6>6
```

Ouput-F

```
1>=1 1>=2 1>=3 1>=4 1>=5 1>=6 1>=7 1>=8

2>=1 2>=2 2>=3 2>=4 2>=5 2>=6 2>=7 2>=8

3>=1 3>=2 3>=3 3>=4 3>=5 3>=6 3>=7 3>=8

4>=1 4>=2 4>=3 4>=4 4>=5 4>=6 4>=7 4>=8

5>=1 5>=2 5>=3 5>=4 5>=5 5>=6 5>=7 5>=8
```

Ouput-G

Ouput-F

How to verify logical conditions?

Let's verify the logical condition of the alphabet M captured in 7 rows and 5 columns given as :

(col==1)||(col==5)|| (((row==col)|| (row+col ==6)) && row<=3)

VerifyAlphabetM.c

```
#include <stdio.h>
int main() {
	int TOTAL_ROWS = 7;
	int TOTAL_COLUMNS = 5;
	int condition; //logical condition variable
	for ( int row = 1; row <= TOTAL_ROWS; row++) {
		for ( int col = 1; col <= TOTAL_COLUMNS; col++) {
			condition = (col == 1) || (col == 5) ||
					( ( (row == col) || (row + col == 6) )
					 && row <= 3 );
			// space(s) are used just to show formatted output.
			if (condition) {
				printf("* "); // single space
			} else {
				printf("  ");// double space
			}
		}
		printf("\n");// Move cursor to the next row

	}
	return 0;
}
```

Output

```
*   *
** **
* * *
*   *
*   *
*   *
*   *
```

This way we can verify logical conditions by directly assigning logical condition to the variable named as **condition**. Code changes were shown in the bold text for the variable **condition.**

We can use same approach to verify logical condition for simple pictures of alphabets that were already done.

You can do it.

Task A. Please write logical condition for the alphabets M, T, U, V, W, X, Y, Z and single digits 0,1,2,3,4,5,6,7,8,9 by considering **number of rows and columns are 10 each**.

Write a program(s) to print only the mentioned alphabets and digits using "*" character.

Task B .If there is a variable national_id in a program and you assign a 8-10 digit length value it should output in digits made up of "*" characters already done in a part of it in **Task A**.

(**Optional Task)**

Try to make logical expression in such a way if **number of rows and columns were changes to** different values like **5,15,...**, the output should be correct. Basically, it should be able to resize into small or big shape but should be correct.

Note: We will deal more on this how to find generalised logical condition in the section **Programming Patterns**.

I strongly suggesting to try this optional task.

Key summary points

1) We can think of logically dividing our display screen into small-small boxes.

2) Each single box is uniquely identified by a combination of **row** and **col** variables value.

3) We have to wisely choose only those values of **row** & **col** so that we should be able to generate required picture. Many boxes that are shown empty means we should not print anything in that.

4) For every new value of **row**, the **col** values always re-starts from the beginning.

Learning logic building skills by guessing – A tedious strategy

The usual and common way to learn programming is just by randomly experimenting with it. This takes much more time to learn programming as it required hell lot of practice. No doubt it has advantages especially if we want to explore the api/in-built libraries or features of a programming language. This particular style of learning, by guessing logic of a program may act as a killer of interest in the programming subject itself for beginners. This way it takes more time to learn programming and for some percentage of people it is an interesting way. **Everyone can't make a good guess in every subject area and there is no such specific way of guessing that anybody can utilize it as a standard practice**. With this guessing strategy to learn programming, will actually add more difficulties to a person who couldn't able to make a good guess or even awkward guessing. Whenever the imagination regarding programming doesn't flow, the person's mind just goes blank. So It should not be a surprise if that person is good or smart in other activities and may lack in patience or find programming very hard to learn. This is not in question about programming as a subject, it could happen with any type of learning. The ground reality is that if imagination and thought process doesn't flow for a person who trying to learn something, it just doesn't go into the head.

Learning to program by guessing is a switch off to many beginners. Since it doesn't reflect any standard way of learning. Instead, I liked to introduce a systematic approach to learn programming based on simple maths and common sense. That approach has dealt in depth at different levels of difficulties to properly learn programming logic and covered in the next section **programming patterns**.

In order to do programming in patterns the person has to know for-loop and applying logical condition well. Looping are the hardest. Nested loop challenges us to think logically in two different directions at the same time to output a pattern and any pattern is a 2-dimensional simple picture. These type of activities stimulates logical thinking. While learning programming if learner gets confusions or doubts, it is a good sign. It suggests that learner's thought process and imagination become active and platform is getting ready to acquire the skill of programming. It does happen many times that clearing one doubt opens the door for further knowledge in many different areas. So doubt basically works like a surprise knowledge capsule to be explored. A learner just needs to keep patience in order to try to resolve that specific doubt or confusion in a systematic way not by guessing. So whenever we get any doubt, we just need to think that we got one more chance to improve intelligence. Clearing doubts are just a part of an iterative way of refining our logical skills.

Obviously, learning will not last longer and productive if it is not supported by a systematic approach. A person who is somewhat good in developing programming logic usually struck lesser when required to deal with harder subjects of programming. This book is all about developing program's logical skills. It doesn't matter if someone is initially a big zero in it. The content in this book surely can help a person who is ready to make an honest attempt and will eventually become interested in programming. The exercises are designed in a way to naturally suit any new learner in increasing order of difficulty. Some exercises are specially included only for the purpose of giving surprise to the learner to think out of the box.

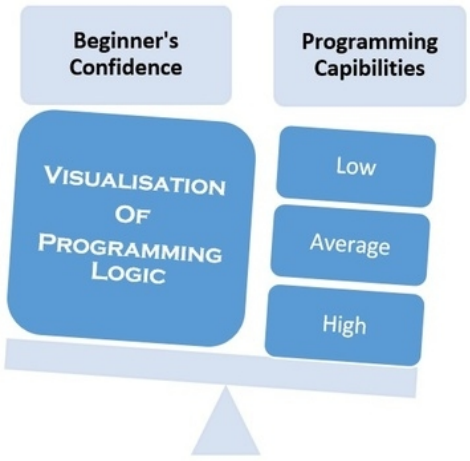
Good programming logic is not just the basics, for programmers, it's the key to confidence.

Programming Patterns

All Programming patterns can be considered 2-dimensional pictures. Mostly whether the output is seen on a Desktop/Web/Mobile Application, it is 2 dimensional. For this reason, we select simple, picture based problems to address common programming logic issues. Moreover, programmers who are proficient in solving two dimensional problems in programming are also capable of solving three dimensional problems, e.g., related to game programming. The main reason to select exercises based on pictorial patterns is that it clearly validates programmer's logic. The mismatching output forces learner to think for appropriate reason.

Repetitively working on programming logic to get the matching output greatly helps in perfecting the visualization of programming logic. It is a kind of reverse engineering, and we will see how it really works to improve programming skills. These exercises make learners alert, involved, self-motivated, and, most importantly, teaches them good amount of patience. These exercises are designed to create curiosity in the learner's mind. The programmer's imagination should work in a direction to trap the programming logic; this will make programming an interesting job. In a typical software development workplace, one programmer has to understand another's programming. So a programmer can learn from the work of other programmers. In all cases, some mastery over visualization of programming logic will definitely help to understand other programmer's programs. So practising visualization of programming logic should be the prime focus for any beginner programmer. Looping is the only hardest thing in a program. Mastering the flow of control in nested loops are enough to boost the confidence of beginner programmers.

In a nutshell, beginner's confidence in programming capabilities has a direct relation with low/average/high level of visualization of programming logic.



What we will be learning is a simple maths based approach to visualize the program logic. This technique requires a level of practice. The more we practice the more refined will be our imagination to trap the programming logic.

Just by practising nested loops along with correct sense of applying logical conditions, a novice programmer can immensely build self-confidence.

Only one quality, that is, having patience to stick to work ethics, can lead a learner to become talented in his/her field. The chosen set of exercises naturally help learners in that direction.

"Failure taught me things about myself that I could have learned no other way."

- J.K. Rowling

If you can imagine, you can do it.

- Enzo Ferrari

In general, programmer is always expected to be good in programming logic. If a programmer has got that confidence, then further advance technical learning will be with much confidence. Every programmer might wish to have that level of imaginative sense so that programming logic should get click naturally, matching the demand of the problem to be solved. It will be just a wishful thinking to expect clarity about visualization of programming logic without proper practice. Solid ground work is required to acquire talent to have fair level of clarity about visualization of programming logic.

The most important protocol which makes visualization of programming logic to work correctly is to understand the intelligence involved in the flow of data.

The trick to get this intelligence is to express flow of data in one or more logical expressions using our version of observation table known as logic trace table. The logical expression is made up of one or more variables and operators. Finally, programming logic is made up of one or more logical expressions. We can easily learn this art of making logical expressions through appropriate practice mentioned in this section.

Our approach will be to extract **logical expression(s)** by observing the output of the program. This thought process is captured in the following approach which is nothing but a programming logic building exercise of one of its kind. The main steps of the approach are depicted in the following picture. The programmer should be able to visualise the logic clearly even before writing the program. This quality is possible only when programmer is practised to such a level to be able to visualise clearly how any repeating control structure works or looping in general. Simple picture based exercises helps greatly in that regard which are basically based on nested loop.

Logic trace table: Brain behind programming logic

Do not deny the classical approach, simply as a reaction, or you will have created another pattern and trapped yourself there.

- Bruce Lee

Conceptually logic trace table just one type of observation table. All major observations are put into this table so that we should be able to relate the different pieces of information. You can create your own one or many depends upon your needs. In our case, the rows and columns of logic trace table are selected with the motive to trace programming logic. We can use observation table based approach to solve other types of problems also not just related to patterns. The main concern is to get involve in perfecting novice programmer's visualisation of logic through the usage of logic trace table and ultimately to install confidence in programming. Furthermore, clarity and work ethics helps to achieve our small goals. Small achievements raises our confidence level to achieve next level of difficult challenges. So clarity and work ethics are the foundation for every event of achievement to happen.

What purpose does logic trace table solve?

> **Simplicity is the key to brilliance.**
>
> **- Bruce Lee**

It's a simple trick to train novice programmer's mind on logical thinking. It's main purpose is to apply thinking in more structured manner to get programming logic. It is a simple table structure meant to derive the **logical expression**. The table structure basically gives clarity about relationship among the variable parameters.

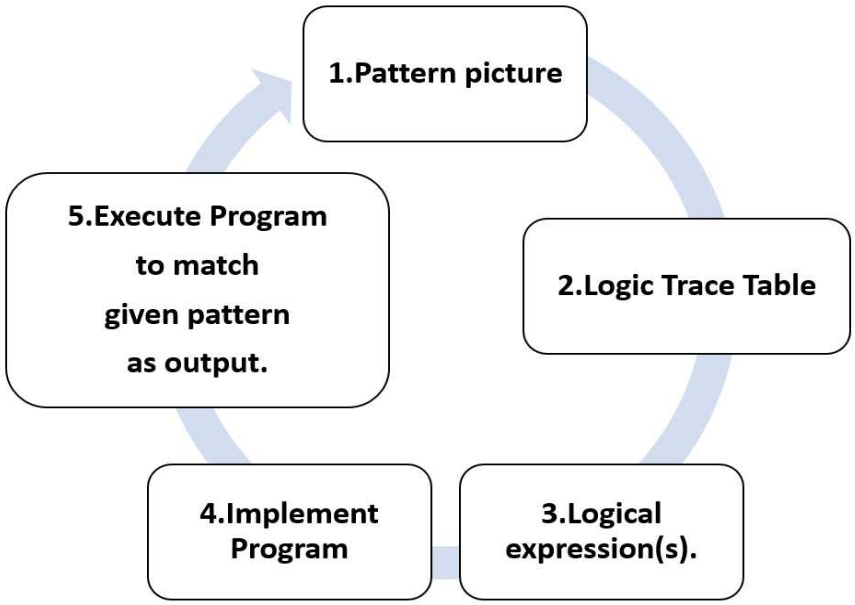
> **Clarity is power.**
>
> **-The more clear you are about EXACTLY what it is that you want, the more your brain knows how to get there.**
>
> **- Unknown**

In my opinion, without taking help of logic trace table structure, most of the novice programmer will find it difficult to think clearly in terms of building logic of a program. The clear thinking to find logical expression is a process which can be improved with a little bit of practice. Unless and until a person is able to put his/her understanding in a simplified way, it means that the person had not really understood the concept.

What is the importance of logical expression?

Programming logic is a brain behind program. It is made up of logical expression(s).So, at the very core level it is a logical expressions when executed helps in decision making. Most of the time it is applied in condition of looping, in decision structures like **if-condition**, **if-elseIf condition**.

THE FRAMEWORK FOR IMPROVING LOGIC BUILDING SKILLS.



The steps need to follow are in a specific order which are given as follows;

Step	**Activity**
1.	Select the **pattern** picture.
2.	Create the **Logic Trace Table** by seeing the pattern picture.
3.	Extract the **logical expression(s)** from Logic Trace Table.
4.	Implementing the **program** using logical expression(s).
5.	Execute the program and check **program's output to match selected pattern picture**.

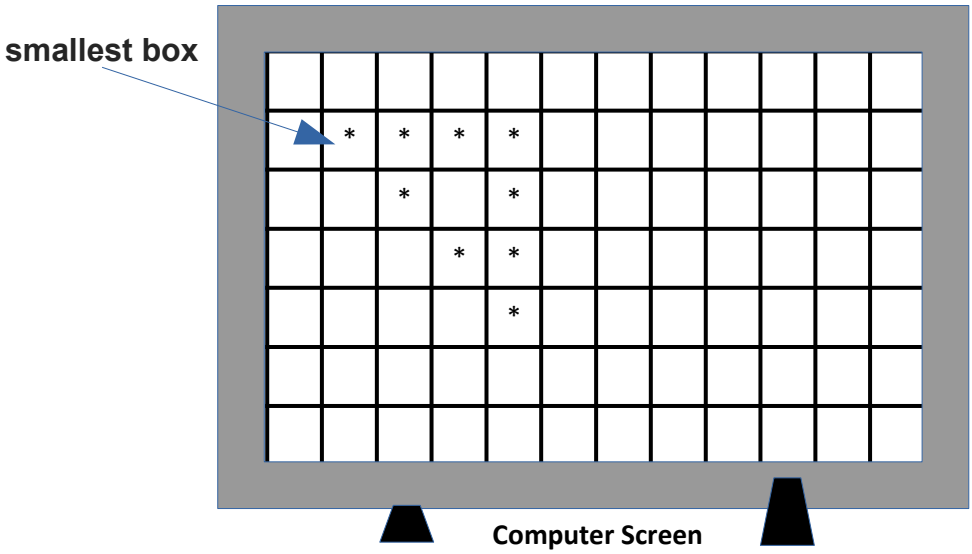
Note: **If step 5 goes wrong, need to repeat step2 to step5.**

PART 2

PROGRAMMING IN ACTION

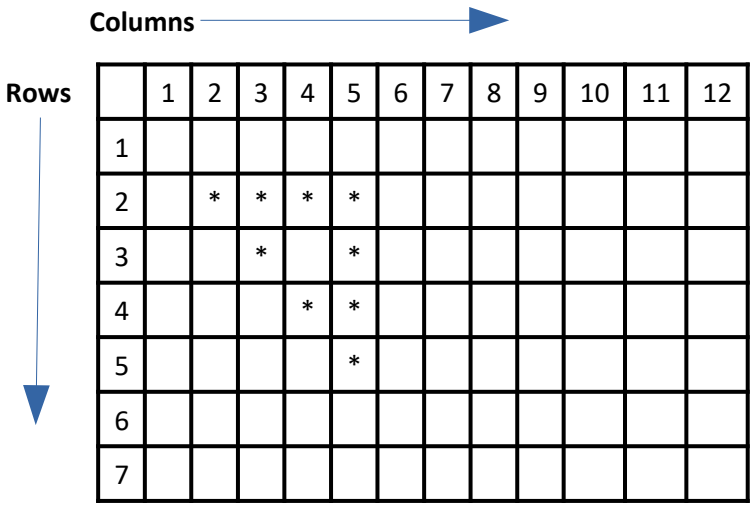
Outputting Simple Picture on Computer Screen

The key point here is to understand that the computer screen is nothing more than a logical collection of small boxes where individual small box can output individual keyboard character. These boxes are arranged in such a way to give a clear idea about rows and columns. Following is the conceptual diagram of computer screen showing some rows and columns of a simple picture outputted into the computer screen. Each keyboard character is in one of the single smallest boxes.



Before we begin programming with simple picture-based exercises, we must first learn to conceptually allocate row number and column number. This will give justice to uniquely identify each smallest box with a singular keyboard character. The (**row number, column number**) numeric combination will imply a single keyboard character. Mentioning (row number, column number) numeric combination is simply a reason to locate correctly, any keyboard character on the computer screen. It is quite analogous to locating a city on the geographical map using **(longitude, latitude)** combination. Hence, our version of single smallest box maps to a single keyboard character on the computer screen and equivalently maps to a unique (row number, column number) numeric pair. In this way, the collection of (row number, column number) numeric pairs can be used to represent a simple picture completely.

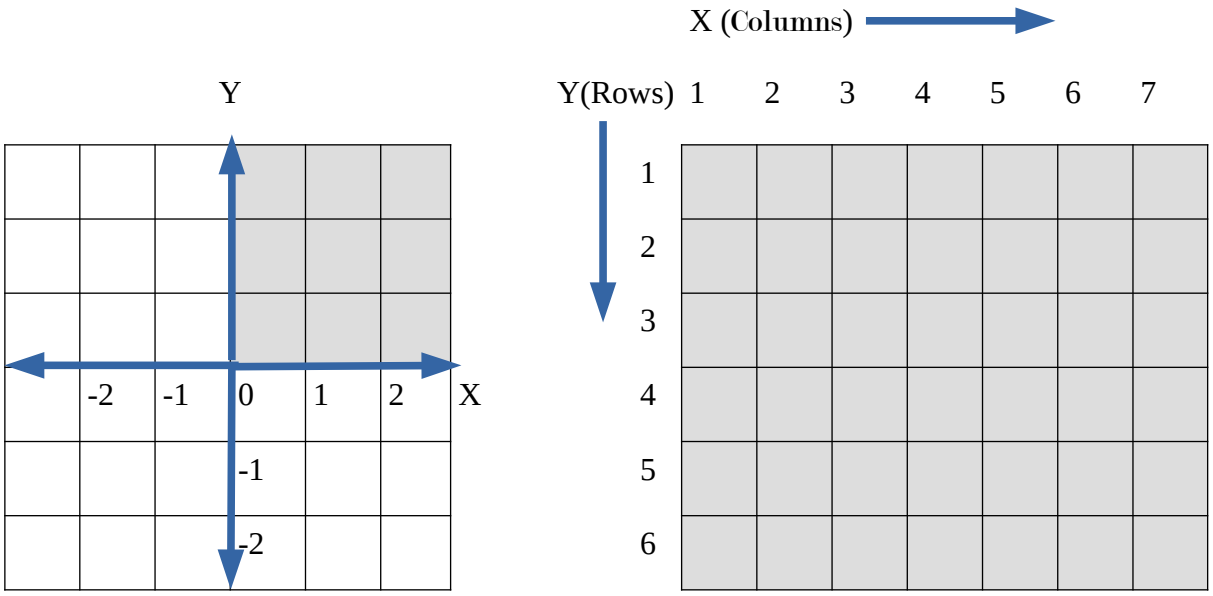
Single Smallest Box → Single keyboard character → (row number, column number) numeric pair.



	1	2	3	4	5	6	7	8	9	10	11	12
1												
2		*	*	*	*							
3			*		*							
4				*	*							
5					*							
6												
7												

Alternately, we can think of the computer screen as a sophisticated version of graph paper or grid paper. Each smallest box of the computer screen is a coordinate, a unique combination of two numbers, an "X (Columns)" (**horizontal**) and a "Y(Rows)" (**vertical**) - that determines the location of a keyboard character in computer screen space.

It is our programming job, how can a simple picture-based pattern should appear at these smallest box coordinates represented by (**row number, column number**) numeric pairs.



School Grade Graph Paper Computer Screen

As discussed previously, we can represent any 2-dimensional simple picture in terms of rows and columns on the computer screen. So, such simple picture which are made up of keyboard characters can be completely caged into collection of small boxes. Each keyboard character of a considered simple picture can have a unique (**row number, column number**) numeric pair. Hence, the collection of such numeric pairs, then can totally represent a complete simple picture.

The collection of related unique (**row number, column number**) numeric pairs of this simple picture can be given as;

(2,2), (2,3), (2,4), (2,5)

(3,3), (3,5), (4,4), (4,5),

(5,5)

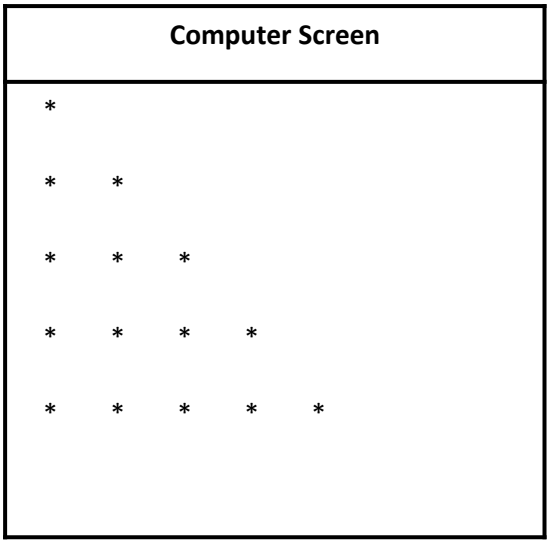
	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1					
Row=2		*(2,2)	*(2,3)	*(2,4)	*(2,5)
Row=3			*(3,3)		*(3,5)
Row=4				*(4,4)	*(4,5)
Row=5					*(5,5)

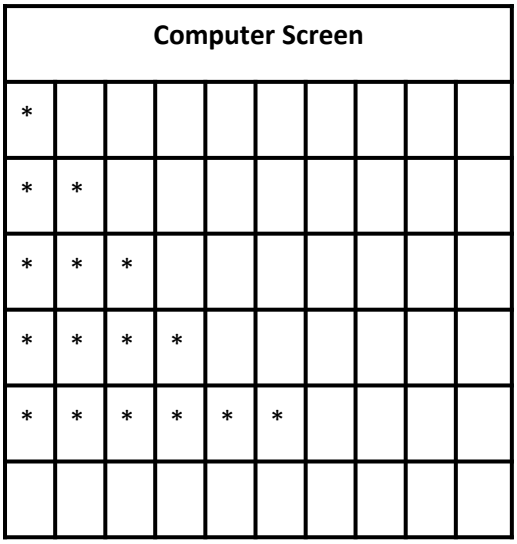
Cursor Movement

A moving position-indicator shown on a computer monitor where the next move or operation will occur. We will use this concept to explain better how each character of a pattern got generated on the computer screen or console screen of the programming IDE. In our case, keyboard characters are to be outputted using C inbuilt libraries onto the computer screen at specific locations. So the cursor movement to the next position on the screen and then printing of character will be taken care by C inbuilt methods.

Pattern1A

Our programming job is to output this simple picture-based pattern on a part of the computer screen as shown in the following. Our approach will be considering computer screen as a combination of rows and columns containing small boxes. We can consider the computer screen as a combination of (**row, column**) numeric value paired boxes spread all across the screen. Each box is treated as a resident of just a single character only. We also know that the complete pattern can be easily expressed as a collection of selected (**row, column**) numeric value pairs.





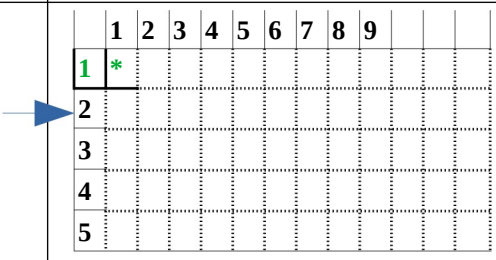
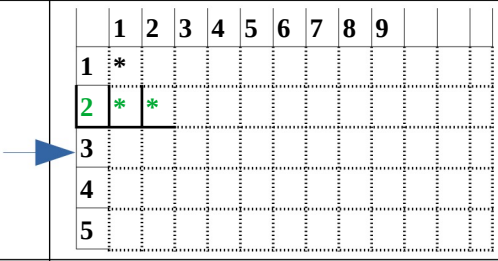
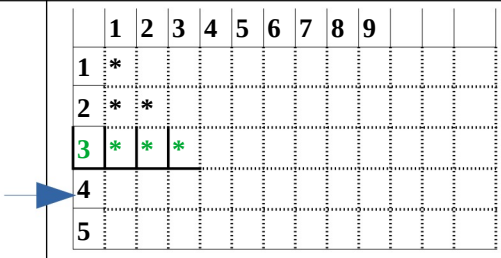
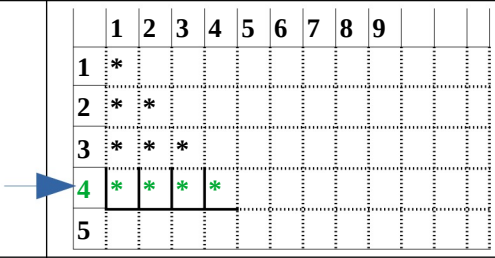
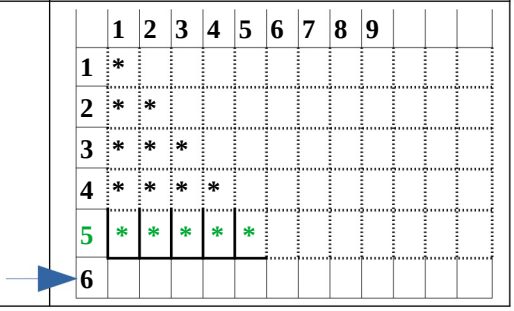
Analysis of output

We can know exact location of each "*" character by knowing it's unique (**row, column**) numeric combination on the computer screen.

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	* (1,1)				
Row=2	*** (2 ,1)**	*(2,2)			
Row=3	* (3,1)	*(3,2)	*(3,3)		
Row=4	*(4,1)	*(4,2)	*(4,3)	*(4,4)	
Row=5	*(5,1)	*(5,2)	*(5,3)	*(5,4)	*(5,5)

(2, 1) means character "*" at **second row** and **first column** in the table.

For example, for a numeric pair(2,1), the value 2 represents row number and the value 1 represents a column number. This is the actual measure by which we are able to identify a "*" character clearly at that particular location on the computer screen.

Steps for pattern generation (Total Rows = 5)		Computer Screen
For row = 1	Print “*” <u>once</u> in the same 1st row then move to row number 2	
For row = 2	Print “*” <u>twice</u> in the same 2nd row then move to row number 3	
For row = 3	Print “*” <u>thrice</u> in the same 3rd row then move to row number 4	
For row = 4	Print “*” <u>four</u> times in the same 4th row then move to row number 5	
For row = 5	Print “*” <u>five</u> times in the 5th row then move to row number 6	

Use of C’s inbuilt methods.	
In order to move to the <u>next row</u>	`printf("\n");`
In order to move to the next position in the <u>same row</u> and to print “*” character	`printf("*");`

Let's convert steps into C program.

Pattern1A_1.c

```c
#include <stdio.h>

int main() {

	printf("*");  // print * character 1 time
	printf("\n"); // Move cursor to the next row

	printf("*");
	printf("*");  //print * characters 2 times, same row
	printf("\n"); // Move cursor to the next row

	printf("*");
	printf("*");
	printf("*");  // print * characters 3 times, same row
	printf("\n"); // Move cursor to the next row

	printf("*");
	printf("*");
	printf("*");
	printf("*");  //print * characters 4 times, same row
	printf("\n"); // Move cursor to the next row

	printf("*");
	printf("*");
	printf("*");
	printf("*");
	printf("*");  //print 5 times, same row
	printf("\n"); // Move cursor to the next row

	return 0;

}
```

Output

```
*
**
***
****
*****
```

This solution will fail in a situation if we want to dynamically increase the total number of rows to 7,10,15…and so on. Whenever we execute this program will always get fixed output of 5 rows

There are two kinds of statements that are get repeated many times in the program.

```
printf("*");   and printf("\n");
```

Repetition means directly we need to think about ***for-loop.*** Hence we can use it for the same.

Let's try to deep dive how the ***for-loop*** should be constructed in order to match our targeted output. We will do the reverse engineering. By observing the pattern's output we try to build the logic.

Column Level Repetitive Action

Let's consider two attributes rows and columns of a 2-dimensional pattern as variables;

row → variable type of integer to store integer value one at a time and represents row number between 1 and 5.

col → variable type of integer to store integer value one at a time and represents column number between 1 and 5.

totalRows → variable type of integer to store integer value for total number of rows.

Logic Trace Tables

(row, col) value pair depicts the position of "*" character on the display screen				
row=1, col=1				
row=2, col=1	**row=2**, col=2			
row=3, col=1	**row=3**, col=2	**row=3**, col=3		
row=4, col=1	**row=4**, col=2	**row=4**, col=3	**row=4**, col=4	
row=5, col=1	**row=5**, col=2	**row=5**, col=3	**row=5**, col=4	**row=5**, col=5

Values of col variable for every row = 1 to 5.

Row level Repetitive Action		**Expected Output**
Variable: row	**Column level Repetitive Action**	
	Variable: col	
1	1	*
2	1,2	* *
3	1,2,3	* * *
4	1,2,3,4	* * * *
5	1,2,3,4,5	* * * * *

For defining for-loop control structure with counter-update incremental value 1, we need one set of **min_val** & **max_val** since somewhere loop should start and end.

The ***min_val*** is same for every row. For every different row value we have a different ***max_val***.

Row level Repetitive Action			**Expected Output**
Variable : row	**Column level Repetitive Action**		
	Variable : col		
	min_val	**max_val**	
1	1	1	*
2	1	2	* *
3	1	3	* * *
4	1	4	* * * *
5	1	5	* * * * *

5 different **max_val** implies 5 different **Column level** for-loop for 5 different rows.

Target to achieve
The **max_val** must also be the same for every row. We need to find a unique expression for the column **max_val** so that if we increase the number of rows, that much row sized pattern it should generate.

Approach to solution

Learning the approach to find a generalised solution for different input values of

total number of rows is the actual practice about programming logic.

We need to generalise the non-repeating numeric pattern of the column **max_val** using the available variables. The variables are row & totalRows=5.

Row level Repetitive Action			**Expected Output**
Variable : row	**Column level Repetitive Action**		
	Variable : col		
	min_val	**max_val**	
1	1	1	*
2	1	2	* *
3	1	3	* * *
4	1	4	* * * *
5	1	5	* * * * *

The numeric pattern of the column **max_val** matches exactly the every value of **row**.

So all the numeric pattern can be expressed in row variable.

Column level Repetitive Action			
Variable : row	loop-condition(Variable : col)		
	min_val	**max_val**	
		default	1
1	1	1	row
2	1	2	row
3	1	3	row
4	1	4	row
5	1	5	row

Now, **max_val** has the same expression and it is consistent for every value of **row.**

We can now define column level ***for-loop*** which is same for every row value.

Elements of* Column level *for-loop	
min_val	1
max_val	row
variable_name	**col**
statement(s)	`printf("*");`

Row Level Repetitive Action

The same following statements had been repeated 5 times(total number of rows).

Column level *for-loop* // *inner for-loop*

```
printf("\n");//Move cursor to the next row
```

We can observe from the pattern that the total number of rows are 5.

We have assigned unique value to each row starting from 1 till 5.So, it's an easy take now.

The minimum and maximum value can be given for the ***for-loop*** as;

min_val = 1 & *max_val* = 5 or totalRows

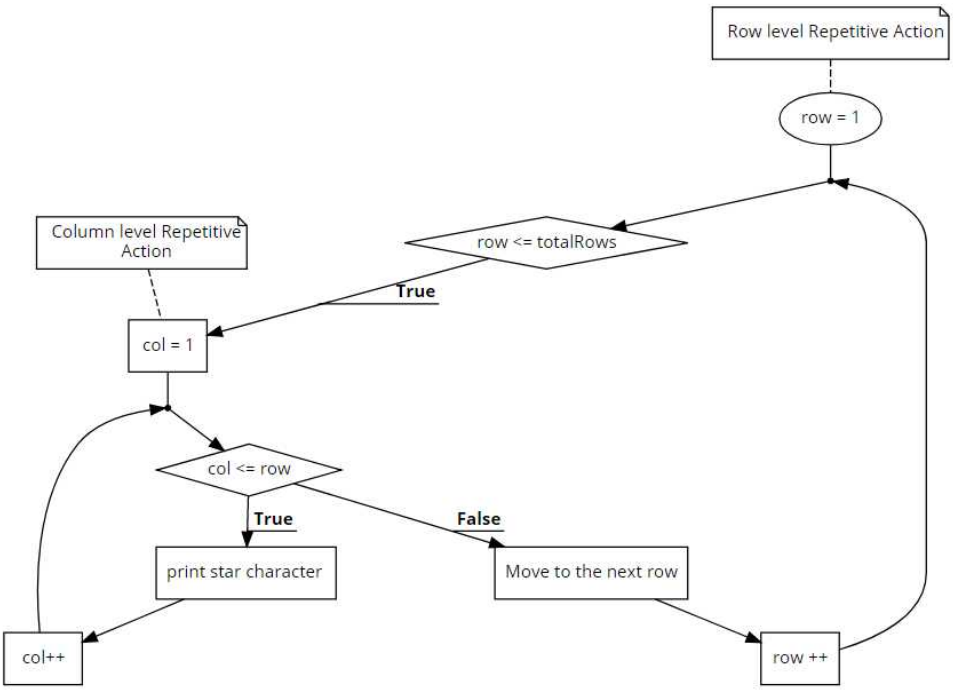
Let's consider;

row → variable type of integer to store integer value one at a time and represents row number between 1 and 5(totalRows). The **totalRows** has initially assigned the value 5.

Outer ***for-loop*** starts at row=1 & will repeat till row <= **totalRows**.

Elements Of* Row Level *for-loop	
min_val	1
max_val	totalRows
variable_name	**row**
statement(s)	**Column level *for-loop*** `printf("\n"); //Move cursor to the next row`

We got everything related to ***for-loop*** for row level repetition.



Let’s execute the following program.

Pattern1A_2.c

```c
#include <stdio.h>
int main() {

	int totalRows = 5;  //number of rows to display

	// Row level Repetitive Action:
	// Action1.Executes Column level Repetitive Action
	// Action2.Move cursor to next row.
	for( int row = 1 ; row <= totalRows ; row ++ ) {
		// Column level Repetitive Action :
		for( int col = 1 ; col <= row  ; col++ ) {
			// Action1.Move cursor in the same row
			// Action2.print “*” character
			printf("* ");
		}
		printf("\n");// Move cursor to the next row
	}

	return 0;
}
```

Output

```
*
* *
* * *
* * * *
* * * * *
```

Now, test for different number of lines, let it be 10. It should work correctly, so that we can be confirmed that our solution is generalised and fit for any number of lines.

Pattern1A_1.c

```
#include <stdio.h>

int main() {

	int totalRows = 10;  //number of rows to display


	// Row level Repetitive Action:

	// Action1.Executes Column level Repetitive Action

	// Action2.Move cursor to next row.

	for( int row = 1 ; row <= totalRows ; row ++ ) {

		// Column level Repetitive Action :

		for( int col = 1 ; col <= row  ; col++ ) {

			// Action1.Move cursor in the same row

			// Action2.print “*” character

			printf("* ");

		}

		printf("\n");// Move cursor to the next row

	}


	return 0;

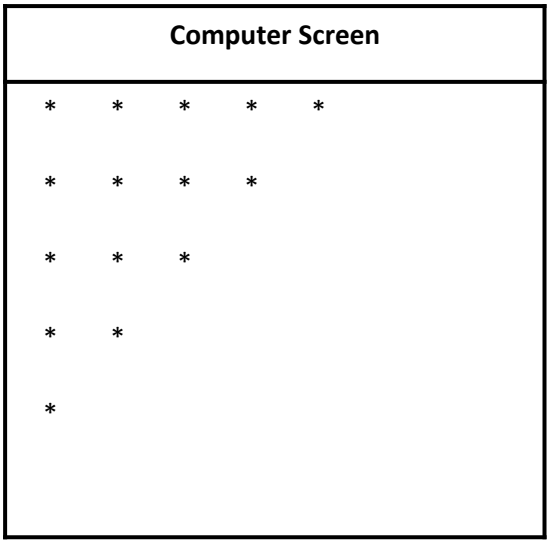
}
```

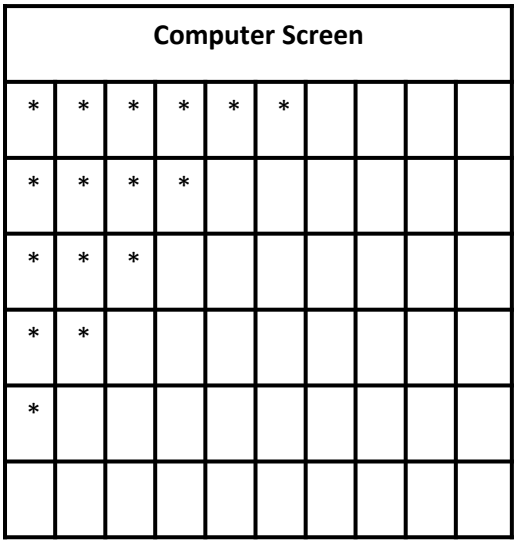
Output

```
*
* *
* * *
* * * *
* * * * *
* * * * * *
* * * * * * *
* * * * * * * *
* * * * * * * * *
* * * * * * * * * *
```

Pattern1B

Our programming job is to output this simple picture-based pattern on a part of the computer screen as shown in the following. Our approach will be considering computer screen as a combination of rows and columns containing small boxes. We can consider the computer screen as a combination of (**row, column**) numeric value paired boxes spread all across the screen. Each box is treated as a resident of just a single character only. We also know that the complete pattern can be easily expressed as a collection of selected (**row, column**) numeric value pairs.





*	*	*	*	*	*				
*	*	*	*						
*	*	*							
*	*								
*									

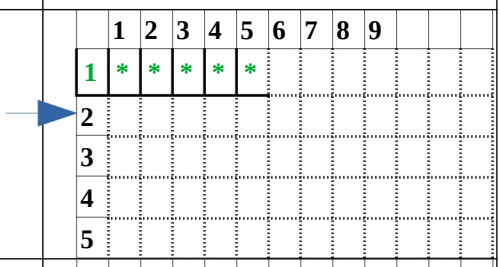
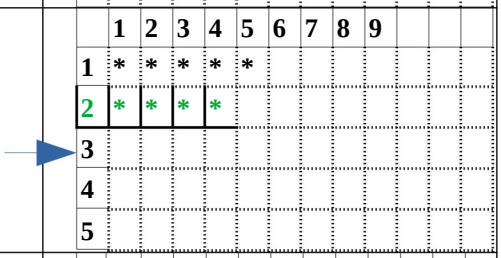
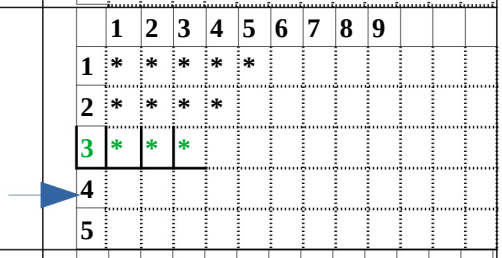
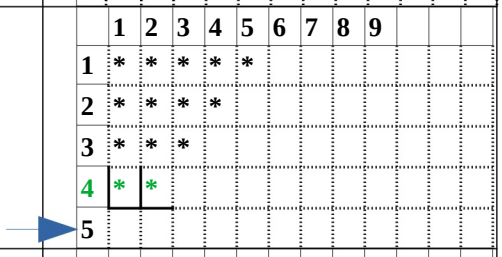
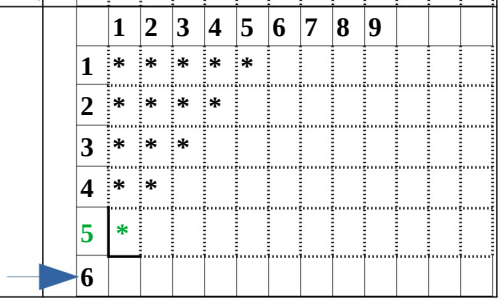
Analysis of output

We can know exact location of each "*" character by knowing it's unique (**row, column**) numeric combination on the computer screen.

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	* (1,1)	*(1,2)	*(1,3)	*(1,4)	*(1,5)
Row=2	*** (2 ,1)**	*(2,2)	*(2,3)	*(2,4)	
Row=3	* (3,1)	*(3,2)	*(3,3)		
Row=4	*(4,1)	*(4,2)			
Row=5	*(5,1)				

(2, 1) means character "*" at **second row** and **first column** in the table.

For example, for a numeric pair(2,1), the value 2 represents row number and the value 1 represents a column number. This is the actual measure by which we are able to identify a "*" character clearly at that particular location on the computer screen.

Steps for pattern generation (Total Rows = 5)		Computer Screen
For row = 1	Print "*" <u>five</u> times in the same 1st row then move to row number 2	
For row = 2	Print "*" <u>four</u> times in the same 2nd row then move to row number 3	
For row = 3	Print "*" <u>thrice</u> in the same 3rd row then move to row number 4	
For row = 4	Print "*" <u>twice</u> in the same 4th row then move to row number 5	
For row = 5	Print "*" <u>once</u> in the 5th row then move to row number 6	

Let's convert steps into C program.

Use of C in-built method	
In order to move cursor to the <u>next row</u>	`printf("\n" );`
In order to move cursor and to print "*" character in the <u>same row</u>	`printf("*");`

These are two kinds of statements that will get repeated many times in the program.

`printf("\n" ); & printf("*");`

Hence we can use ***for-loop*** for the same.

Column Level Repetitive Action

Let's consider two attributes rows and columns of a 2-dimensional pattern as variables;

row → variable type of integer to store integer value one at a time and represents row number between 1 and 5.

col → variable type of integer to store integer value one at a time and represents column number between 1 and 5.

totalRows → variable type of integer to store integer value for total number of rows.

Trace Table

(row, col) value pair depicts the position of "*" character on the display screen				
row=1, col=1	**row=1,** col=2	**row=1,** col=3	**row=1,** col=4	**row=1,** col=5
row=2, col=1	**row=2,** col=2	**row=2,** col=3	**row=2,** col=4	
row=3, col=1	**row=3,** col=2	**row=3,** col=3		
row=4, col=1	**row=4,** col=2			
row=5, col=1				

Values of col variable for every row = 1 to 5.

Row level Repetitive Action		**Expected Output**
Variable: row	**Column level Repetitive Action**	
	Variable: col	
1	1,2,3,4,5	* * * * *
2	1,2,3,4	* * * *
3	1,2,3	* * *
4	1,2	* *
5	1	*

For defining for-loop control structure with counter-update incremental value 1, we need one set of **min_val** & **max_val** since somewhere loop should start and end.

Row level Repetitive Action			**Expected Output**
Variable : row	**Column level Repetitive Action**		
	Variable : col		
	min_val	max_val	
1	1	5	* * * * *
2	1	4	* * * *
3	1	3	* * *
4	1	2	* *
5	1	1	*

For every different row value we have a different max_val.

Target to achieve
We need to find a unique expression for the column **max_val** so that if we increase the number of rows, that much row sized pattern it should generate.

Approach to solution

We need to generalise the non-repeating numeric pattern of the column **max_val** using the available variables. We can use variable **totalRows**. Variable **totalRows** has been assigned the value 5 as the total number of rows.

Column level Repetitive Action			
Variable : row	loop-condition(Variable : col)		
	min_val	**max_val**	
		default	1
1	1	**5**	**totalRows**
2	1	4	totalRows - 1
3	1	3	totalRows - 2
4	1	2	totalRows - 3
5	1	1	totalRows - 4

Again non-repeating numeric values are there. We can express **max_val 2nd time** as follows**.**

Column level Repetitive Action				
Variable : row	loop-condition(Variable : col)			
	min_val	**max_val**		
		default	1	2
1	1	**5**	**totalRows**	(totalRows +1) - 1
2	1	4	totalRows -1	(totalRows +1) - 2
3	1	3	totalRows - 2	(totalRows +1) - 3
4	1	2	totalRows - 3	(totalRows +1) - 4
5	1	1	totalRows - 4	(totalRows +1) - 5

Part of **column max_val's** expression followed the same numeric value pattern as **row.**

Column level Repetitive Action					
Variable : row	loop-condition(Variable : col)				
	min_val	**max_val**			
		default	1	2	3
1	1	5	totalRows	(totalRows +1) - 1	(totalRows +1) - row
2	1	4	totalRows -1	(totalRows +1) - 2	(totalRows +1) - row
3	1	3	totalRows - 2	(totalRows +1) - 3	(totalRows +1) - row
4	1	2	totalRows - 3	(totalRows +1) - 4	(totalRows +1) - row
5	1	1	totalRows - 4	(totalRows +1) - 5	(totalRows +1) - row

Further we can express it in terms of totalRows & row variables.

Now, **max_val** has the same expression and it is consistent for every value of **row.**

We can now define column level ***for-loop*** which is for every row value.

Elements of* Column level *for-loop	
min_val	1
max_val	(totalRows +1) - row
variable_name	col
statement(s)	**printf("*");**

Row Level Repetitive Action

The same following statements had been repeated 5 times(total number of rows).

Column level *for-loop* // *inner for-loop*

```
printf("\n" );//Move cursor to the next row
```

We can observe from the pattern that the total number of rows are 5.

We have assigned unique value to each row starting from 1 till 5.So, it's an easy take now.

The minimum and maximum value can be given for the ***for-loop*** as;

min_val = 1 & max_val = 5 or totalRows

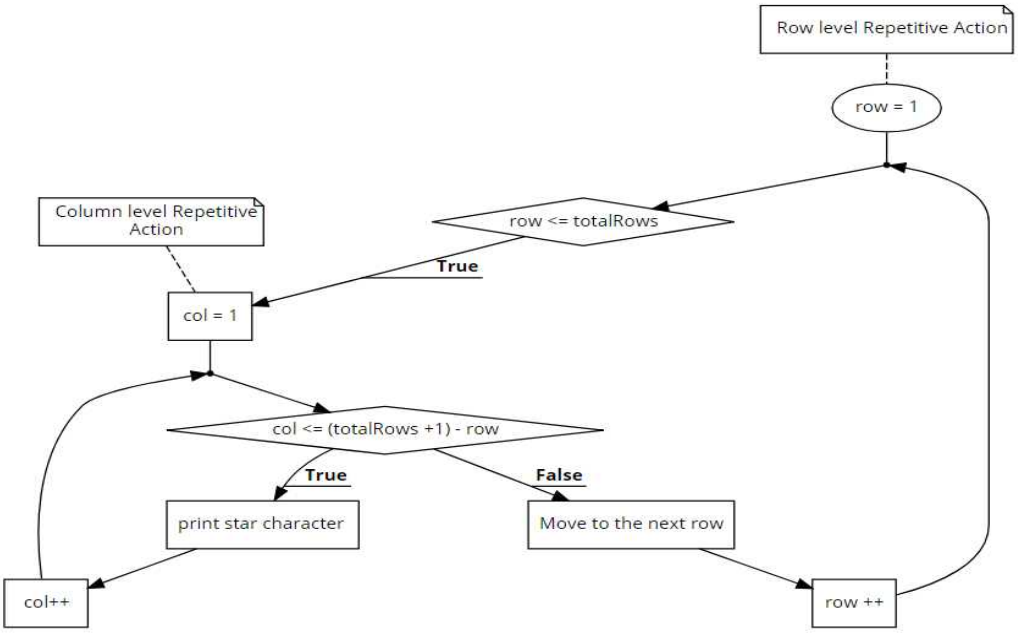
Let's consider;

row → variable type of integer to store integer value one at a time and represents row number between 1 and 5(totalRows). The **totalRows** has initially assigned the value 5.

Outer ***for-loop*** starts at row=1 & will repeat till row <= **totalRows**.

Elements Of* Row Level *for-loop	
min_val	1
max_val	totalRows
variable_name	**row**
statement(s)	**Column level *for-loop*** //Move cursor to the next row **printf("\n")**;

We got everything related to ***for-loop*** for row level repetition.



Let’s execute the following program.

PatternB1_1.c

```c
#include <stdio.h>
int main() {

	int  totalRows = 5;  //number of rows to display

	// Row level Repetitive Action :
	// Action1.Executes Column level Repetitive Action
	// Action2.Move cursor to next row.
	for( int row = 1 ; row <= totalRows ; row ++ ) {

		// Column level Repetitive Action :
		for(int col = 1 ; col <= (totalRows +1) - row ; col++) {
			// Action1.Move cursor in the same row
			// Action2.print “*” character
			printf("* ");
		}
		printf("\n");// Move cursor to the next row
	}
	return 0;
}
```

Output

```
* * * * *
* * * *
* * *
* *
*
```

Let's execute the following program for different row value equals to 10(`totalRows = 10`).

PatternB1_1.c

```c
#include <stdio.h>
int main() {

	int  totalRows = 10;  //number of rows to display

	// Row level Repetitive Action :
	// Action1.Executes Column level Repetitive Action
	// Action2.Move cursor to next row.
	for( int row = 1 ; row <= totalRows ; row ++ ) {

		// Column level Repetitive Action :
		for(int col = 1 ; col <= (totalRows +1) - row ; col++) {
			// Action1.Move cursor in the same row
			// Action2.print "*" character
			printf("* ");
		}
		printf("\n");// Move cursor to the next row
	}
	return 0;
}
```

Output

```
* * * * * * * * * *
* * * * * * * * *
* * * * * * * *
* * * * * * *
* * * * * *
* * * * *
* * * *
* * *
* *
*
```

Alternative solution2

From previous solution we got the following **Column level Repetitive Action** details as follows:

Let's try to simplify the expression (totalRows +1) – row.

Column level Repetitive Action		
Variable : row	loop-condition(Variable : col)	
	min_val	**max_val**
		3
1	1	(totalRows +1) - row
2	1	(totalRows +1) - row
3	1	(totalRows +1) - row
4	1	(totalRows +1) - row
5	1	(totalRows +1) - row

Subtracting 1 from row and **max_val**.

Column level Repetitive Action		
Variable : row	loop-condition(Variable : col)	
	min_val	**max_val**
		1
1- 1	1	(totalRows +1) - row - 1
2- 1	1	(totalRows +1) - row - 1
3- 1	1	(totalRows +1) - row - 1
4- 1	1	(totalRows +1) - row - 1
5- 1	1	(totalRows +1) - row - 1

Column level Repetitive Action		
Variable : row	loop-condition(Variable : col)	
	min_val	**max_val**
		1
0	1	(totalRows - row)
1	1	(totalRows - row)
2	1	(totalRows - row)
3	1	(totalRows - row)
4	1	(totalRows - row)

Now, **max_val** has the same expression and it is consistent for every value of **row.**

We can now define column level ***for-loop*** which is same for every row value.

Elements of* Column level *for-loop	
min_val	1
max_val	(totalRows - row)
variable_name	**col**
statement(s)	`printf("*" + " " );`

Row Level Repetitive Action

Outer ***for-loop*** starts at row=0 & will repeat till row <= **totalRows** – 1 or row < **totalRows**.

Elements Of* Row Level *for-loop	
min_val	0
max_val	totalRows
variable_name	**row**
Loop condition	**row < totalRows**
statement(s)	**Column level *for-loop*** `printf("\n");// Moves cursor to the next row`

The new solution changes are in the following program.

PatternB2.c

```
#include <stdio.h>
int main() {

	int  totalRows = 5;  //number of rows to display

	// Row level Repetitive Action :
	// Action1.Executes Column level Repetitive Action
	// Action2.Move cursor to next row.
	for( int row = 0 ; row < totalRows ; row ++ ) {

		// Column level Repetitive Action :
		for(int col = 1 ; col <= ( totalRows  - row ) ; col++) {
			// Action1.Move cursor in the same row
			// Action2.print “*” character
			printf("* ");
		}
		printf("\n");// Move cursor to the next row

	}

	return 0;
}
```

Output

```
* * * * *
* * * *
* * *
* *
*
```

Now, test for different number of rows, let it be 10.

PatternB2.c

```c
#include <stdio.h>
int main() {

	int  totalRows = 10;  //number of rows to display

	// Row level Repetitive Action :
	// Action1.Executes Column level Repetitive Action
	// Action2.Move cursor to next row.
	for( int row = 0 ; row < totalRows ; row ++ ) {

		// Column level Repetitive Action :
		for(int col = 1 ; col <= ( totalRows  - row ) ; col++) {
			// Action1.Move cursor in the same row
			// Action2.print “*” character
			printf("* ");
		}
		printf("\n");// Move cursor to the next row

	}

	return 0;
}
```

Output

```
* * * * * * * * * *
* * * * * * * * *
* * * * * * * *
* * * * * * *
* * * * * *
* * * * *
* * * *
* * *
* *
*
```

Alternative solution3

We have to have to use remaining variables row & **totalRows=5** to find the generalised expression for the column **max_val.**

The following table is the one that we already got from the **trace table** in the very beginning.

Column level Repetitive Action		
Variable : row	loop-condition(Variable : col)	
	min_val	**max_val**
		default
1	1	5
2	1	4
3	1	3
4	1	2
5	1	1

If we observe the numeric pattern for row (1, 2, 3, 4, and 5) is **reverse** of the numeric pattern of the column **max_val** (5, 4, 3, 2, and 1).

If we just reverse the counter of row variable it will match exactly the values of the column **max_val** (5, 4, 3, 2, and 1).It simply means instead of using the incrementing ***for-loop*** for the row variable we need to replace it with the decrementing ***for-loop*** for the row variable.

Reversing the flow of values for the variable row. It can be given as follows:

Column level Repetitive Action		
Variable : row	loop-condition(Variable : col)	
	min_val	**max_val**
		default
5	1	5
4	1	4
3	1	3
2	1	2
1	1	1

Column level Repetitive Action		
Variable : row	loop-condition(Variable : col)	
	min_val	**max_val**
		default
5	1	**row**
4	1	**row**
3	1	**row**
2	1	**row**
1	1	**row**

Now numeric pattern of variable row and column **max_val** matches exactly.

So, **max_val** can be expressed in variable row.

So, **max_val** is consistent for every value of **row.** Now, we are in good position to go for the ***for-loop*** since ***for-loop*** requires one minimum and one maximum value or an expression.

We can now define column level **decrementing *for-loop***.

It starts at value of variable **row** and will repeatedly decrease till **col** value becomes 1.

Elements of* Column level decrementing *for-loop	
max_val	row
min_val	1
variable_name	col
Loop condition	col >=1
statement(s)	printf("*");

Row Level Repetitive Action

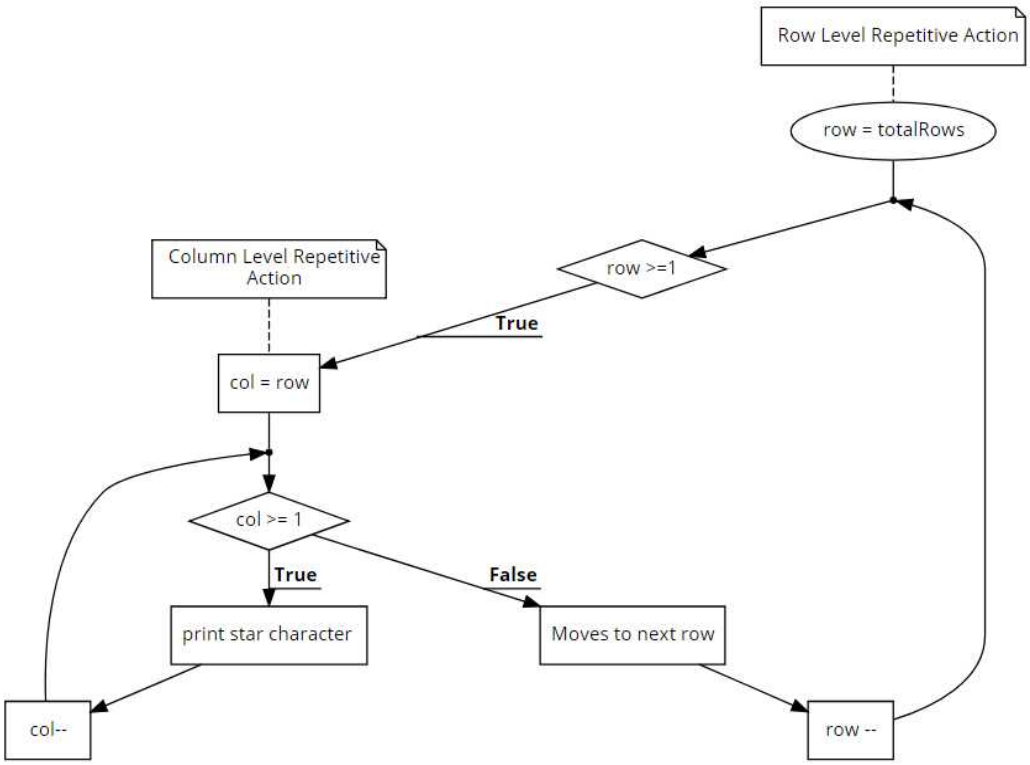
Outer **decrementing *for-loop*** starts at row= **totalRows** & will repeatedly decrease

till row value becomes 1.

Elements Of* Row Level decrementing *for-loop	
max_val	totalRows
min_val	1
variable_name	row
Loop condition	**row >=1**

statement(s)	**Column level for-loop**
	// Move cursor to the next row
	`printf("\n");`

Hence we got everything related to the nested ***for-loop.***



The **updated solution changes are made bold** in the following program.

PatternB3.c

```
#include <stdio.h>
int main() {

	int  totalRows = 10;  //10 rows to display

	// Row level Repetitive Action :
	// Action1.Executes Column level Repetitive Action
	// Action2.Move cursor to next row.
	for( int row = totalRows ; row >=1  ; row -- ) {

		// Column level Repetitive Action :
		for(int col = row; col >= 1 ; col--) {
			// Action1.Move cursor in the same row
			// Action2.print “*” character
			printf("* ");
		}
		printf("\n");// Move cursor to the next row
	}

	return 0;
}
```

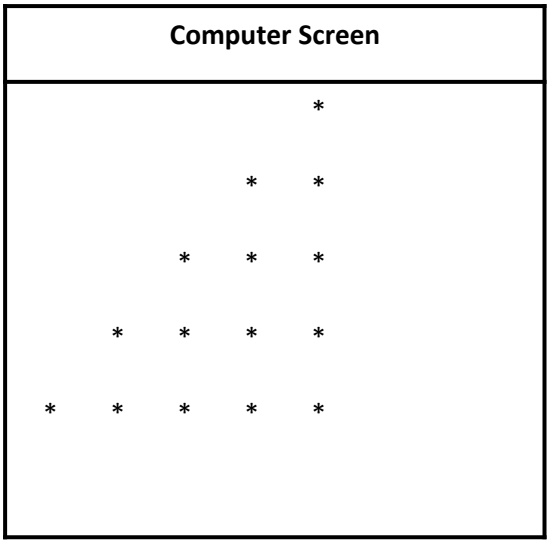
Output

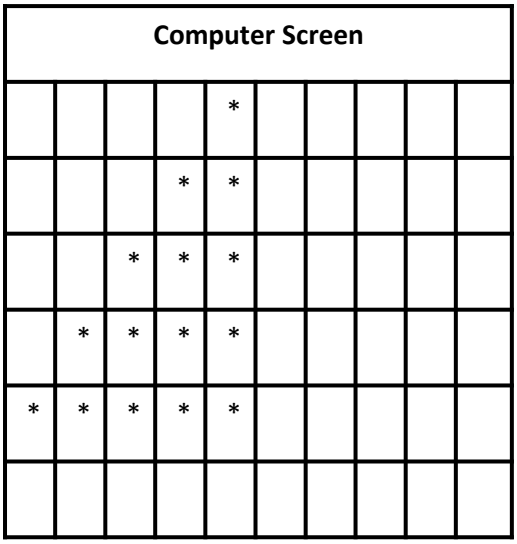
```
* * * * * * * * * *
* * * * * * * * *
* * * * * * * *
* * * * * * *
* * * * * *
* * * * *
* * * *
* * *
* *
*
```

This page intentionally left blank.

Pattern1C

Our programming job is to output this simple picture-based pattern on a part of the computer screen as shown in the following. Our approach will be considering computer screen as a combination of rows and columns containing small boxes. We can consider the computer screen as a combination of (**row, column**) numeric value paired boxes spread all across the screen. Each box is treated as a resident of just a single character only. We also know that the complete pattern can be easily expressed as a collection of selected (**row, column**) numeric value pairs.





Analysis of output

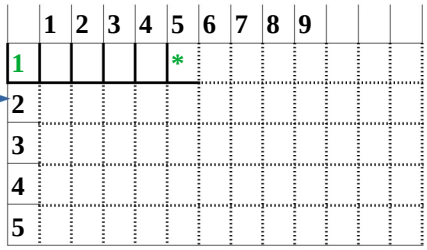
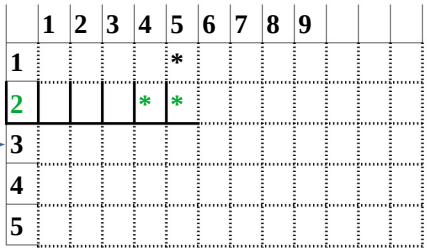
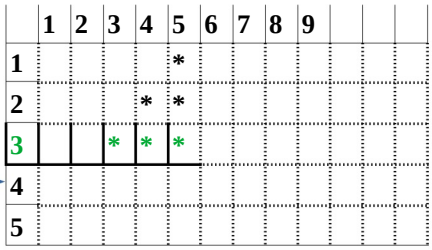
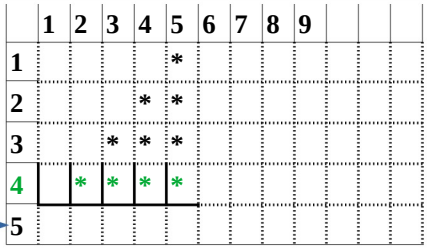
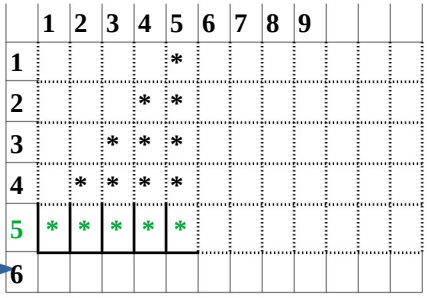
We can know exact location of each "*" character by knowing it's unique (**row, column**) numeric combination on the computer screen.

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	space	space	space	space	* (1,5)
Row=2	space	space	space	*** (2,4)**	*(2,5)
Row=3	space	space	*(3,3)	* (3,4)	*(3,5)
Row=4	space	*(4,2)	*(4,3)	*(4,4)	*(4,5)
Row=5	*(5,1)	*(5,2)	*(5,3)	*(5,4)	*(5,5)

(2, 4) means character "*" at **second row** and **fourth column** in the table.

For example, for a numeric pair(2,4), the value 2 represents row number and the value 4 represents a column number. This is the actual measure by which we are able to identify a "*" character clearly at that particular location on the computer screen.

Space → space is also a character like any other character. If a character or a text is displayed on the screen after a blank area or gap, it means that many space character has to be print before printing any other character(s) afterwards.

	Steps for pattern generation	Computer Screen
For row = 1	Print space " " 4 times in the same 1st row. Print star "*" once in the same 1st row then move to the next row number 2.	
For row = 2	Print space " " 3 times in the same 2nd row. Print star "*" 2 times in the same 2nd row then move to the next row number 3.	
For row = 3	Print space " " 2 times in the same 3rd row. Print star "*" 3 times in the same 3rd row then move to the next row number 4.	
For row = 4	Print space" " 1 time in the same 4th row. Print star "*" 4 times in the same 4th row then move to the next row number 5.	
For row = 5	Print space" " 0 times in the same 5th row. Print star "*" 5 times in the same 5rd row then move to the next row number 5.	

Let's convert these steps into C program.

Use of C in-built methods	
Move to the next row	`printf("\n")`
Move to the next (row, col) position and print space " " character in the same row	`printf(" ")`
Move to the next (row, col) position and print star "*" character in the same row	`printf("*")`

Column Level Repetitive Action

Let's consider two attributes rows and columns of a 2-dimensional pattern as variables;

row → variable type of integer to store integer value one at a time and represents row number between 1 and 5.

col → variable type of integer to store integer value one at a time and represents column number between 1 and 5.

spaces → variable type of integer to store integer value one at a time and represents space character at column number between 1 and 5.

totalRows → variable type of integer to store integer value for total number of rows.

Logic Trace Tables

(row, col) value pair depicts the position of "*" character on the display screen				
				row=1, col=5
			row=2, col=4	**row=2**, col=5
		row=3, col=3	**row=3**, col=4	**row=3**, col=5
	row=4, col=2	**row=4**, col=3	**row=4**, col=4	**row=4**, col=5
row=5, col=1	**row=5**, col=2	**row=5**, col=3	**row=5**, col=4	**row=5**, col=5

Values of col variable for every row = 1 to 5.

Row level Repetitive Action				**Expected Output**
Variable: row	**Column level Repetitive Action**			
	Variable : spaces	Variable: col	Character count	
1	1,2,3,4	5	4 Spaces + 1 star	*
2	1,2,3	4, 5	3 Spaces + 2 stars	* *
3	1,2	3, 4, 5	2 Spaces + 3 stars	* * *
4	1	2, 3, 4, 5	1 Spaces + 4 stars	* * * *
5	0(no space)	1, 2, 3, 4, 5	0 Spaces + 5 stars	* * * * *

The space (" ") and star("*") characters are repeated many times. So, based on the count

of the space (" ") and star("*") characters we can use ***for-loop*** for each.

We can put our initial observation in somewhat more clear form as given in the following table.

Row level Repetitive Action					**Expected Output**
Variable : row	**Column level Repetitive Action**				
	Variable : spaces		Variable : col		
	min_val	max_val	min_val	max_val	
1	1	4	1	1	*
2	1	3	1	2	* *
3	1	2	1	3	* * *
4	1	1	1	4	* * * *
5	0	0	1	5	* * * * *

Approach to solution

The variables are **row** & **totalRows**=5.

We need to generalise the non-repeating numeric pattern of the column **max_val**. The numeric pattern of the variable col's column **max_val** matches exactly the every value of **row**. So all the numeric pattern of variable col's column **max_val** can be expressed in row variable.

Let's try to use variable **totalRows** to generalise the variable **spaces**'s column **max_val.**

Row level Repetitive Action					
Variable : row	**Column level Repetitive Action**				
	Variable : spaces			Variable : col	
	min_val	**max_val**			
		default	1	**min_val**	**max_val**
1	1	4	totalRows -1	1	row
2	1	3	totalRows - 2	1	row
3	1	2	totalRows - 3	1	row
4	1	1	totalRows - 4	1	row
5	0	0	totalRows - 4	1	row

Part of variable **spaces** column **max_val's** expression follows the same numeric value pattern as **row.** Further we can express it in terms of totalRows & row variables.

Row level Repetitive Action					
Variable : row	**Column level Repetitive Action**				
	Variable : spaces			Variable : col	
	min_val	**max_val**			
		default	1	**min_val**	**max_val**
1	1	4	totalRows -row	1	row
2	1	3	totalRows -row	1	row
3	1	2	totalRows -row	1	row
4	1	1	totalRows -row	1	row
5	0	0	totalRows -row	1	row

We generalised the non-repeating numeric pattern of the column **max_val** into an expressions.

We can now define column level ***for-loop*** which is printing spaces for every row value.

Elements of* Column level *for-loop	
min_val	1
max_val	totalRows -row
variable_name	**spaces**
statement(s)	`printf(" ");`

We can now define column level ***for-loop*** which is printing (*) characters for every row value.

Elements of* Column level *for-loop	
min_val	1
max_val	row
variable_name	**col**
statement(s)	`printf("*");`

Row Level Repetitive Action

The same following statements had been repeated 5 times(total number of rows).

Column level *for-loop* for printing space characters / / ***inner for-loop***

Column level *for-loop* for printing star characters / / ***inner for-loop***

```
printf("\n");// Move cursor to the next row
```

We have assigned unique value to each row starting from 1 till 5.So, it's an easy take now.

The minimum and maximum value can be given for the ***for-loop*** as;

min_val = 1 & max_val = 5 or totalRows

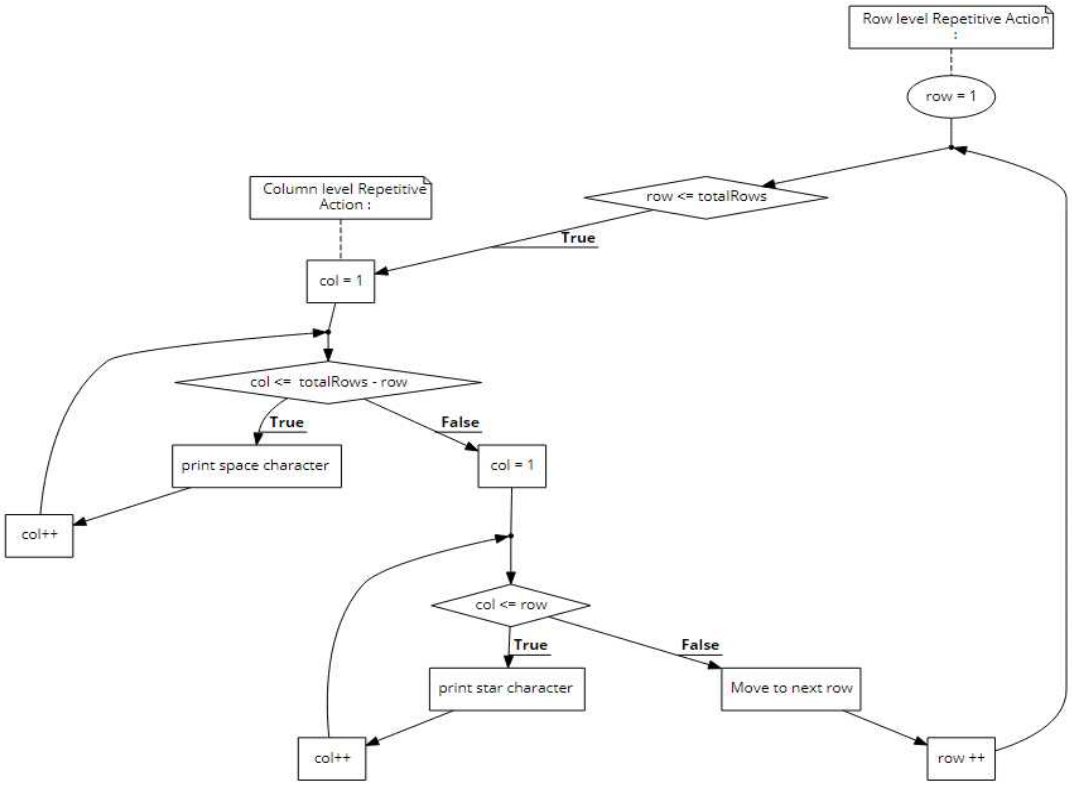
Let's consider;

row → variable type of integer to store integer value one at a time and represents row number between 1 and 5(totalRows). The **totalRows** has initially assigned the value 5.

Outer ***for-loop*** starts at row=1 & will repeat till row <= **totalRows**.

We got everything related to ***for-loop*** for row level repetition.

Elements Of* Row Level *for-loop	
min_val	1
max_val	totalRows
variable_name	row
statement(s)	**Column level for-loop** for printing space characters. **Column level for-loop** for printing star characters `printf("\n");`// Move cursor to the next row



Let’s write complete programming logic and execute the program.

PatternC1.c

```c
#include <stdio.h>
int main() {
	int  totalRows = 5;  //number of rows to display

	// Row level Repetitive Action :
	// Action1.Executes Column level Repetitive Action
	// Action2.Move cursor to next row.
	for( int row = 1 ; row <= totalRows ; row ++ ) {

		// Column level Repetitive Action :
		for(int sp = 1 ; sp <= ( totalRows - row ) ; sp++) {
			// Action1.Move cursor in the same row
			// Action2.print space “  ” character
			printf("  ");
		}
		for(int col = 1 ; col <= row  ; col++) {
			// Action1.Move cursor in the same row
			// Action2.print star “* ” character
			printf("* ");
		}
		printf("\n");// Move cursor to the next row
	}
	return 0;
}
```

Output

```
        *
      * *
    * * *
  * * * *
* * * * *
```

Alternative solution2

The available variables are row & totalRows=5.

The variable col takes the value 1 to 5, i.e., col=1, 2, 3, 4, 5.

The overall observation is that at each row, either it's printing space characters or star (*) characters in a continues fashion.

The columns in the pattern divided into two parts, either it's printing space " " character(s) or star("*") characters.

Row level Repetitive Action					**Expected Output**
Variable: row	**Column level Repetitive Action**				
	Variable: spaces	Variable: col	Character count	Total Characters	
1	1,2,3,4	5	4 Spaces + 1 star	5	*
2	1,2,3	4, 5	3 Spaces + 2 stars	5	* *
3	1,2	3, 4, 5	2 Spaces + 3 stars	5	* * *
4	1	2, 3, 4, 5	1 Spaces + 4 stars	5	* * * *
5	0(no space)	1, 2, 3, 4, 5	0 Spaces + 5 stars	5	* * * * *

The printing of space(s) **comes first** before print star("*") character(s).When care of the condition till it print space character(s),then the remaining are the star("*") character(s). If we closely observe It's cutting the values of variable col into two halves. One part goes to printing of space character(s) and the other goes to printing of star character(s).

Row level Repetitive Action					**Expected Output**
Variable: row	**Column level Repetitive Action**				
	Variable : col				
	Printing space character(s)		Printing star character(s)		
	min_val	**max_val**	**min_val**	**max_val**	
1	1	4	5	5	*
2	1	3	4	5	* *
3	1	2	3	5	* * *
4	1	1	2	5	* * * *
5	0	0	1	5	* * * * *

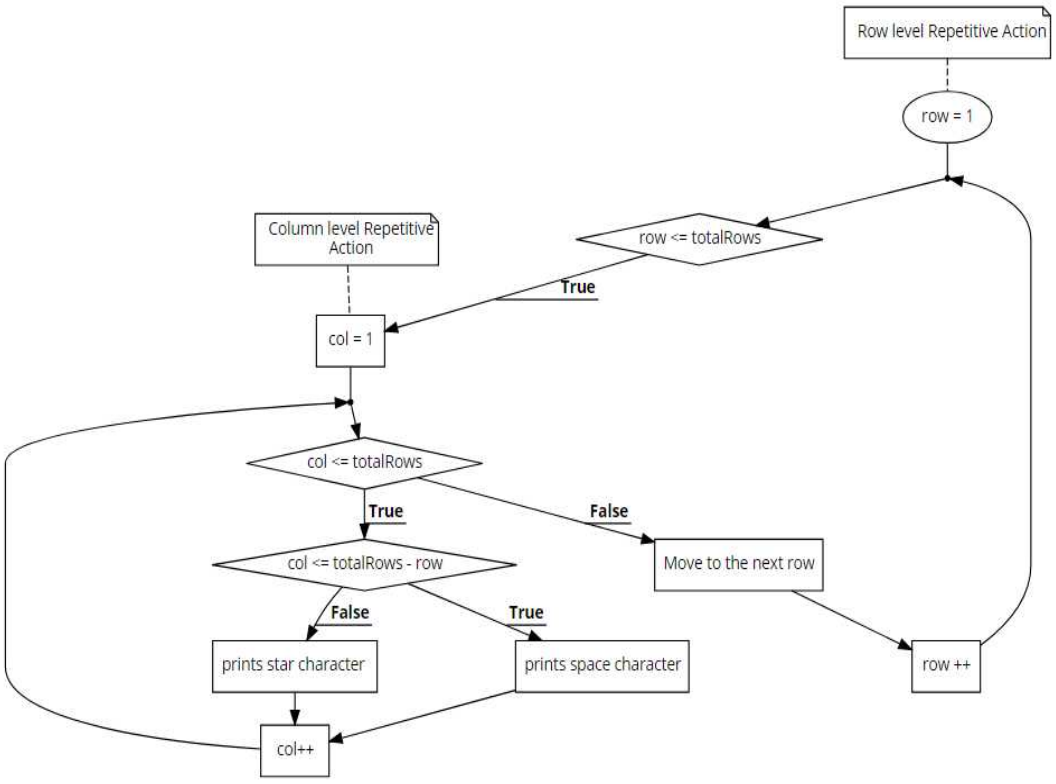
From the previous solution, we already had that generalised condition for printing spaces and using that generalised condition we can put all in the following table.

Row level Repetitive Action					
Variable : row	**Column level Repetitive Action**				
	Variable : col				
	Printing space character(s)			Printing star character(s)	
	min_val	**max_val**			
		default	**1**	**min_val**	**max_val**
1	1	4	**totalRows -row**	5	5
2	1	3	**totalRows -row**	4	5
3	1	2	**totalRows -row**	3	5
4	1	1	**totalRows -row**	2	5
5	0	0	**totalRows -row**	1	5

The **Column level for-loop** runs between the col variable's value 1 to 5.

The condition (**totalRows - row**) gives the maximum limit starting from 1 to which it will print space character(s).

So, we can easily use **if-else** statement for our situation for filtering and printing the space and star characters.



Prints a space when condition is **True** for every value of variable **col**				
col=1	col=2	col=3	col=4	col=5
	If (condition is **True)** Print space character **Else** Print star character			

The updated program with if-else statement is as follows:

PatternC2.c

```
#include <stdio.h>
int main() {

	int totalRows = 5; //number of rows to display
	// Row level Repetitive Action :
	// Action1.Executes Column level Repetitive Action
	// Action2.Move cursor to next row.
	for (int row = 1; row <= totalRows; row++) {

		// Column level Repetitive Action :
		// Action1.Move cursor in the same row
		for (int col = 1; col <= totalRows; col++) {
			if (col <= (totalRows - row)) {
				// Action2.prints space “ ” character
				printf("  ");
			} else {
				// Action3.prints star “* ” character
				printf("* ");
			}
		}
		printf("\n");// Move cursor to the next row

	}

	return 0;
}
```

Output

```
        *
       * *
     * * *
   * * * *
 * * * * *
```

Now, test for different number of lines, let it be 10.

PatternC2.c

```
#include <stdio.h>
int main() {

	int totalRows = 10; //number of rows to display
	// Row level Repetitive Action :
	// Action1.Executes Column level Repetitive Action
	// Action2.Move cursor to next row.
	for (int row = 1; row <= totalRows; row++) {

		// Column level Repetitive Action :
		// Action1.Move cursor in the same row
		for (int col = 1; col <= totalRows; col++) {
			if (col <= (totalRows - row)) {
				// Action2.prints space “ ” character
				printf("  ");
			} else {
				// Action3.prints star “* ” character
				printf("* ");
			}
		}
		printf("\n");// Move cursor to the next row

	}

	return 0;
}
```

Output

```
                  *
               * *
            * * *
         * * * *
      * * * * *
   * * * * * *
 * * * * * * *
* * * * * * * *
```

Alternative solution3

From the previous solution, we have checked the **if-condition** to be true to print spaces and in the else part of the **if-statement** we have printed the star("*") characters. This was captured in the following way;

Prints a space when condition is **True** for every value of variable **col**				
col=1	col=2	col=3	col=4	col=5
If (condition is **True)** Print space character **Else** Print star character				

In this solution we will reverse situation. When **if-condition** becomes true we will print a star("*") character and in the else part of the **if-statement** we will print spaces.

Let's start with the observation table that we got from the previous solution.

Row level Repetitive Action				
Variable : row	**Column level Repetitive Action**			
	Variable : col			
	Printing space character(s)		Printing star character(s)	
	min_val	**max_val**		
		default	**min_val**	**max_val**
1	1	4	5	5
2	1	3	4	5
3	1	2	3	5
4	1	1	2	5
5	0	0	1	5

We know that totalRows = 5 .

The column **min_val** for printing star character(s) can be given as;

Row level Repetitive Action				
Variable : row	**Column level Repetitive Action**			
	Variable : col			
	Printing space character(s)		Printing star character(s)	
	min_val	**max_val**		
		default	**min_val**	**max_val**
1	1	4	5 - **0**	totalRows
2	1	3	5 - **1**	totalRows
3	1	2	5 - **2**	totalRows
4	1	1	5 - **3**	totalRows
5	0	0	5 - **4**	totalRows

If we can try variable totalRows to generalise the numeric pattern of the column **min_val .**

Then it will be totalRows – **0** , totalRows -**1**,….so on. The part of this expression nearly following the numeric pattern of the variable row = 1,2,…,5.The former is having the numeric pattern starting from zero(0) and row variable's numeric pattern starting from one(1).Let it be ,start the numeric pattern of row variable too start from zero(0).

Let's make use of the variables totalRows & row to derive the generalised expression

for printing star character(s).

Row level Repetitive Action				
Variable : row	**Column level Repetitive Action**			
	Variable : col			
	Printing space character(s)		Printing star character(s)	
	min_val	**max_val**		
		default	**min_val**	**max_val**
0	1	4	totalRows - **0**	totalRows
1	1	3	totalRows - **1**	totalRows
2	1	2	totalRows - **2**	totalRows
3	1	1	totalRows - **3**	totalRows
4	0	0	totalRows - **4**	totalRows

We can use the variable **row** to generalise the column **min_val** for printing star character(s).

Row level Repetitive Action				
Variable : row	**Column level Repetitive Action**			
	Variable : col			
	Printing space character(s)		Printing star character(s)	
	min_val	**max_val**		
		default	**min_val**	**max_val**
0	1	4	totalRows - row	totalRows
1	1	3	totalRows - row	totalRows
2	1	2	totalRows - row	totalRows
3	1	1	totalRows - row	totalRows
4	0	0	totalRows - row	totalRows

The condition (col >= (**totalRows – row)**) gives the minimum limit, a starting point from which it will start printing star("*") character(s) till the maximum value of variable col which is **totalRows**.

The **if-condition** can be given for in between range of values (**totalRows - row**) and **totalRows** using the comparison and logical operators as:

If (col >= (**totalRows - row**) && col <= **totalRows)** and moreover we already know that;

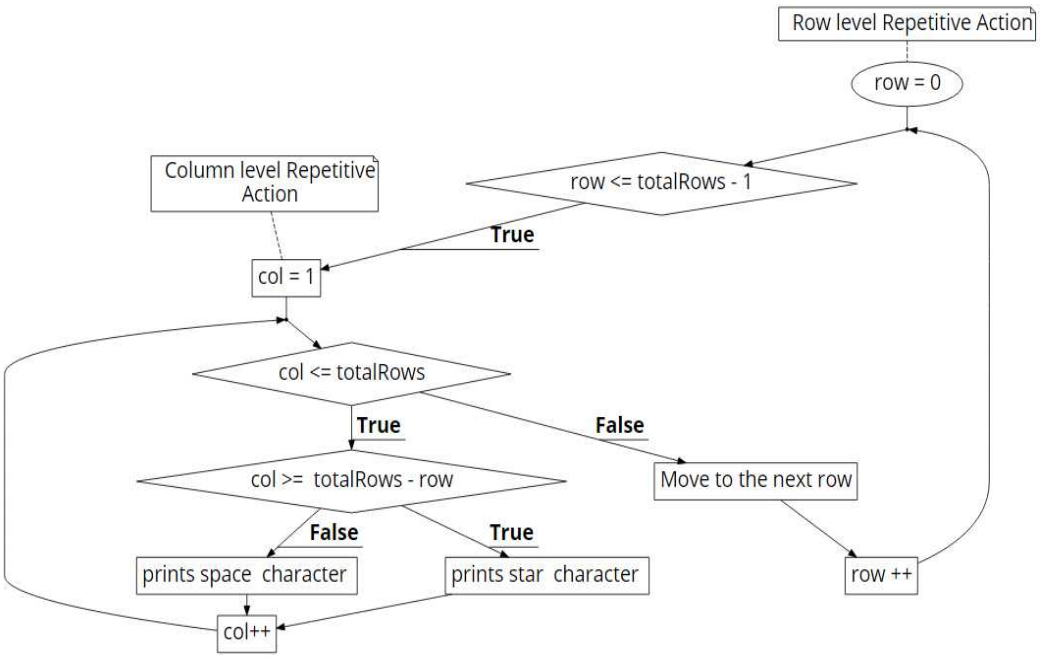
variable col = 1,2,3,4,5 or 1,2,3,4, **totalRows**

variable row = 0,1,2,3,4 or 0,1,2,3,4, **(totalRows - 1) .**

We can eliminate the logical and condition(col <= **totalRows) ,**since this condition will already be taken in the column level inner for-loop.

So the if-condition will reduced to: If (col >= (**totalRows - row**))

We can summarise it all in the following flow-chart and update it in the program.



Now, test for different number of lines, let it be 7.

PatternC3.c

```
#include <stdio.h>
int main() {

	int totalRows = 7; //7 rows to display

	// Row level Repetitive Action :
	// Action1.Executes Column level Repetitive Action
	// Action2.Move cursor to next row.
	for (int row = 0; row <= totalRows - 1; row++) {

		// Column level Repetitive Action :

		// Action1.Move cursor in the same row
		for (int col = 1; col <= totalRows; col++) {

			if (col >= (totalRows - row)) {
				// Action2.prints star “* ” character
				printf("* ");
			} else {
				// Action3.prints space “ ” character
				printf("  ");
			}
		}
		printf("\n");// Move cursor to the next row

	}

	return 0;

}
```

Output

```
            *
          * *
        * * *
      * * * *
    * * * * *
  * * * * * *
* * * * * * *
```

Different programming logic can be developed to tackle the same problem. We need to choose the right one based on the time and space complexity of a program and that is out of scope of this book.

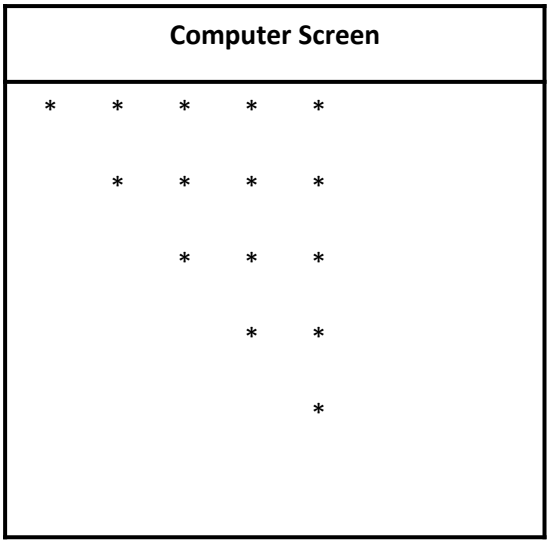
The major concern is learner should be able to develop programming logic on it’s own simply by using the available variables to **remove the hard-coded value dependencies and transform it into appropriate logical expressions**.

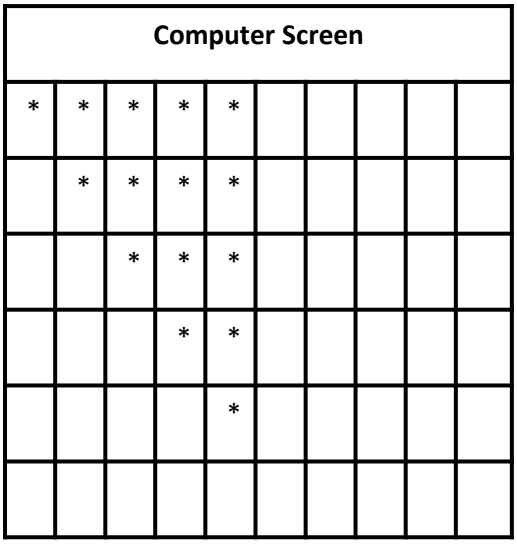
The programmer should able to think logically in different ways and in that way learner will become confident in programming. This is the major intention of this book to transform learner into a confident programmer or at least will become interested in the programming.

Let’s consider the following is a part of the display screen of a computer or laptop where we like to print the following pattern.

Pattern1D

Our programming job is to output this simple picture-based pattern on a part of the computer screen as shown in the following. Our approach will be considering computer screen as a combination of rows and columns containing small boxes. We can consider the computer screen as a combination of (**row, column**) numeric value paired boxes which are spread all over the screen. Each box is treated as a resident of just a single character only. We also know that the complete pattern can be easily expressed as a collection of selected (**row, column**) numeric value pairs.





Analysis of output

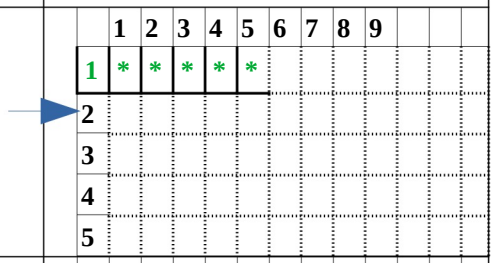
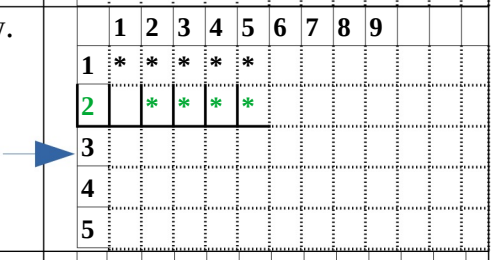
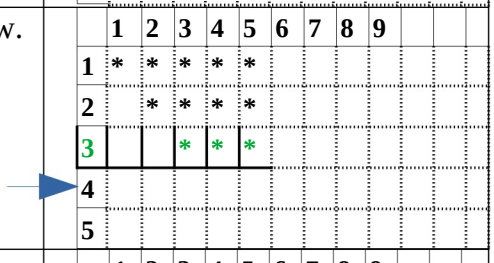
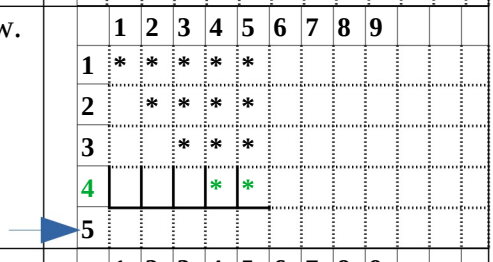
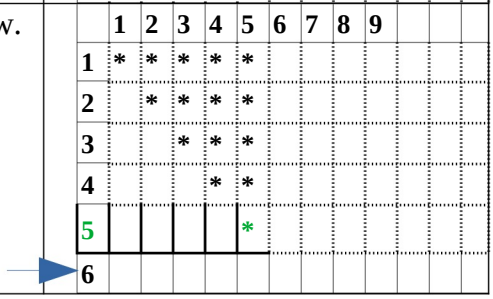
We can represent computer screen in terms of rows and columns. In order to know the placement of each "*" character and space " " character on the display screen, we can give each box a numeric representation in terms of row number and a column number.

The (**row number, column number**) numeric pair can be used to represent a pattern. The first value represents the row number and second value represents a column number. For example;

(2, 4) means character "*" at **second row** and **fourth column** in the table.

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	*(1,1)	*(1,2)	*(1,3)	*(1,4)	* (1,5)
Row=2	space	*(2,2)	*(2,3)	*** (2,4)**	*(2,5)
Row=3	space	space	*(3,3)	* (3,4)	*(3,5)
Row=4	space	space	space	*(4,4)	*(4,5)
Row=5	space	space	space	space	*(5,5)

Space → space is also a character like any other character. If a non-space character or a text is displayed on the screen after a blank area or gap, it means that many space character has to be print before printing any other non-space character(s) afterwards.

Steps for pattern generation (Total Rows = 5)		Computer Screen
For row = 1	Print "*" five times in the same 1st row then move to row number 2	
For row = 2	Print space " " 1 time in the same 2nd row. Print "*" four times in the same 2nd row then move to row number 3.	
For row = 3	Print space " " 2 times in the same 3rd row. Print "*" thrice in the same 3rd row then move to row number 4.	
For row = 4	Print space " " 3 times in the same 4th row. Print "*" twice in the same 4th row then move to row number 5.	
For row = 5	Print space " " 4 times in the same 5th row. Print "*" once in the same 5th row then move to row number 6.	

Let's convert steps into C program.

Use of C in-built method	
Move to the next row	`printf("\n")`
Move to the next (row, col) position and print space " " character in the same row	`printf(" ")`
Move to the next (row, col) position and print star "*" character in the same row	`printf("*")`

Since printing of these keyboard characters are performed using mentioned C inbuilt methods. These methods are going to repeat many a times in a program in order to print these repeated characters. Hence we can use ***for-loop*** for such repetitive task in a program.

Column Level Repetitive Action

Let's consider two attributes rows and columns of a 2-dimensional pattern as variables;

row → variable type of integer to store integer value one at a time and represents row number between 1 and 5.

col → variable type of integer to store integer value one at a time and represents column number between 1 and 5.

spaces → variable type of integer to store integer value one at a time and represents space character at column number between 1 and 5.

totalRows → variable type of integer to store integer value for total number of rows.

Logic Trace Tables

(row, col) value pair depicts the position of "*" character on the display screen				
row=1, col=1	**row=1**, col=2	**row=1**, col=3	**row=1**, col=4	**row=1**, col=4
	row=2, col=2	**row=2**, col=3	**row=2**, col=4	**row=2**, col=4
		row=3, col=3	**row=3**, col=4	**row=3**, col=4
			row=4, col=4	**row=4**, col=4
				row=5, col=5

Row level Repetitive Action				**Expected Output**
Variable: row	**Column level Repetitive Action**			
	Variable : spaces	Variable: col	Character count	
1	0(no space)	1, 2, 3, 4, 5	4 Spaces + 1 star	* * * * *
2	1	2, 3, 4, 5	3 Spaces + 2 stars	* * * *
3	1,2	3, 4, 5	2 Spaces + 3 stars	* * *
4	1,2,3	4, 5	1 Spaces + 4 stars	* *
5	1,2,3,4	5	0 Spaces + 5 stars	*

Row level Repetitive Action					**Expected Output**
Variable : row	**Column level Repetitive Action**				
	Variable : col				
	Printing space character(s)		Printing star character(s)		
	min_val	**max_val**			
		default	**min_val**	**max_val**	
1	0	0	1	5	* * * * *
2	1	1	2	5	* * * *
3	1	2	3	5	* * *
4	1	3	4	5	* *
5	1	4	5	5	*

We know that totalRows = 5.

If we observe the numeric pattern for variable row (1, 2, 3, 4, and 5) is **matching** the

numeric pattern of the column **min_val** of printing star character(s).

Our target is to express our program logic in terms of our chosen variables (**row, col, totalRows**).

Let's use these variables to find generalised expression for printing star character(s).

Row level Repetitive Action					**Expected Output**
Variable : row	**Column level Repetitive Action**				
	Variable : col				
	Printing space character(s)		Printing star character(s)		
	min_val	**max_val**			
		default	**min_val**	**max_val**	
1	0	0	row	totalRows	* * * * *
2	1	1	row	totalRows	* * * *
3	1	2	row	totalRows	* * *
4	1	3	row	totalRows	* *
5	1	4	row	totalRows	*

The condition (col >= **row**) gives the minimum limit ,a starting point from which it will start printing star("*") character(s) till the maximum value of variable col which is **totalRows**.

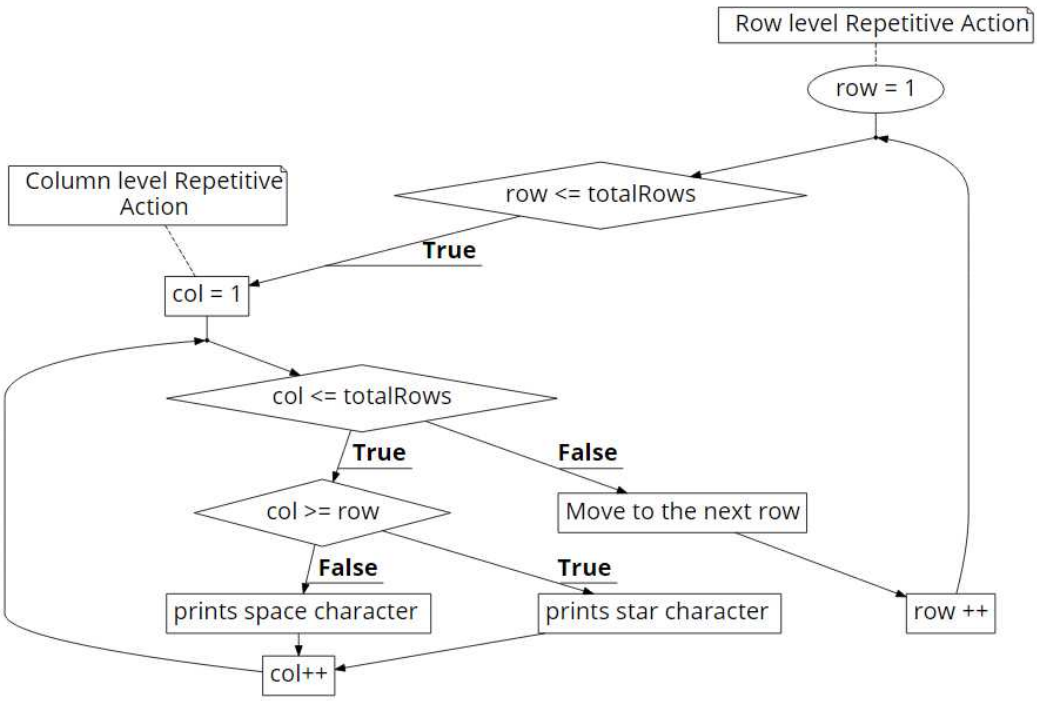
The if-condition can be given for in between range of values (**row**) and **totalRows** using the comparison and logical operators as:

If (col >= **row** && col <= **totalRows).**

We can eliminate the logical and condition(col <= **totalRows) ,** since this condition already be taken in the column level inner for-loop.

So the **if-condition** can be given as;

If (col >= **row**)We can summarise it which is as follows:



The program can be written as follows:

PatternD1.c

```c
#include <stdio.h>
int main() {

	int totalRows = 5; // total number of rows to display

	// Row level Repetitive Action :
	// Action1.Executes Column level Repetitive Action
	// Action2.Move cursor to next row.
	for (int row = 1; row <= totalRows; row++) {

		// Column level Repetitive Action :

		// Action1.Move cursor in the same row
		for (int col = 1; col <= totalRows; col++) {

			if (col >= row) {
				// Action2.prints star “* ” character
				printf("* ");
			} else {
				// Action3.prints space “ ” character
				printf("  ");
			}
		}
		printf("\n");// Move cursor to the next row

	}

	return 0;
}
```

Output

```
* * * * *
  * * * *
    * * *
      * *
        *
```

Now, test for different number of lines, let it be 10.

PatternD1.c

```c
#include <stdio.h>
int main() {

	int totalRows = 10; // total number of rows to display

	// Row level Repetitive Action :
	// Action1.Executes Column level Repetitive Action
	// Action2.Move cursor to next row.
	for (int row = 1; row <= totalRows; row++) {

		// Column level Repetitive Action :

		// Action1.Move cursor in the same row
		for (int col = 1; col <= totalRows; col++) {

			if (col >= row) {
				// Action2.prints star “* ” character
				printf("* ");
			} else {
				// Action3.prints space “ ” character
				printf("  ");
			}
		}
		printf("\n");// Move cursor to the next row

	}

	return 0;
}
```

Output

```
* * * * * * * * * *
  * * * * * * * * *
    * * * * * * * *
      * * * * * * *
        * * * * * *
          * * * * *
            * * * *
              * * *
                * *
                  *
```

Problem D1*:* Write a program to generate the pattern **Pattern1D** by following the condition expression for printing of space characters. When condition expression is **true**, the program should print space character and star character when it's **false**.

Prints a space when condition is **True** for every value of variable **col**				
col=1	col=2	col=3	col=4	col=5
If (condition=(?) is **True)** Print space character **Else** Print star character				

Problem D2. Write a program to generate the pattern **Pattern1D** by using the information given in the

columns →Variable: row, Character count and Total Characters.

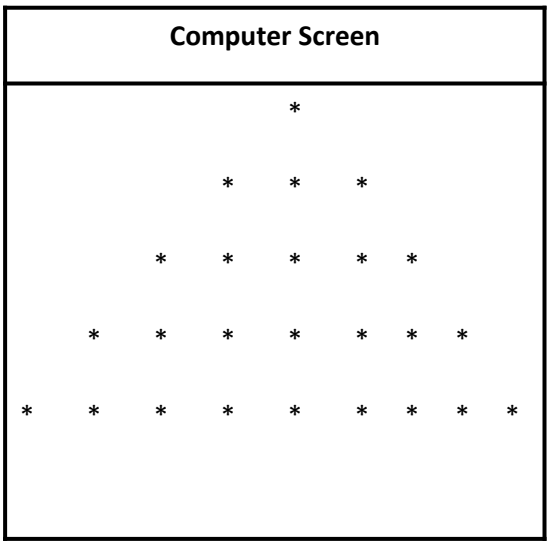
The variable totalRows = 5,denotes the number of rows. The program should work for 10 rows also and which should give output by changing the variable **totalRows = 10**.

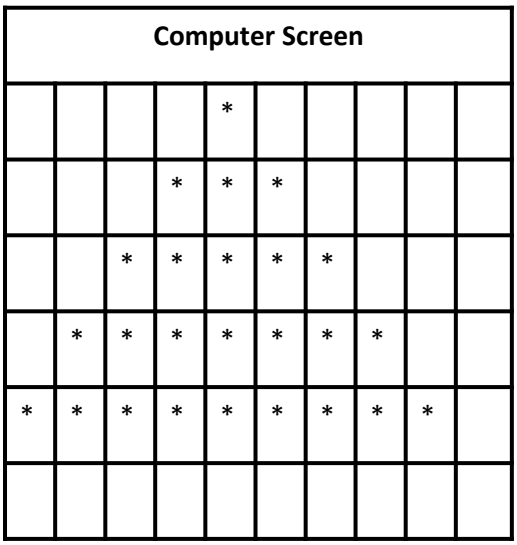
Row level Repetitive Action					**Expected Output**
Variable: row	**Column level Repetitive Action**				
	Variable : spaces	Variable: col	**Character count**	Total Characters	
1	0(no space)	1, 2, 3, 4, 5	0 Spaces + 5 stars	5	* * * * *
2	1	2, 3, 4, 5	1 Spaces + 4 stars	5	* * * *
3	1,2	3, 4, 5	2 Spaces + 3 stars	5	* * *
4	1,2,3	4, 5	3 Spaces + 2 stars	5	* *
5	1,2,3,4	5	4 Spaces + 1 star	5	*

This page intentionally left blank.

Pattern1E

Our programming job is to output this pattern on a part of the computer screen as shown in the following. We can consider the computer screen as a combination of (**row, column**) numeric value paired boxes which are spread all over the screen. Each box is treated as a resident of just a single character only.





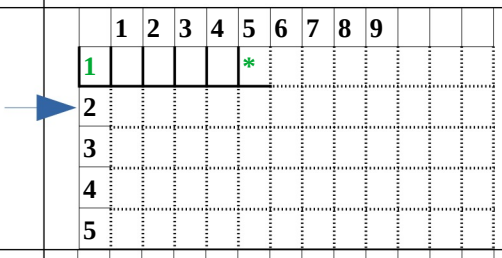
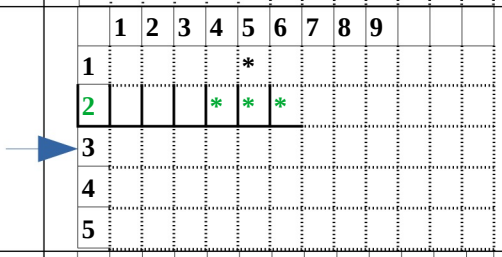
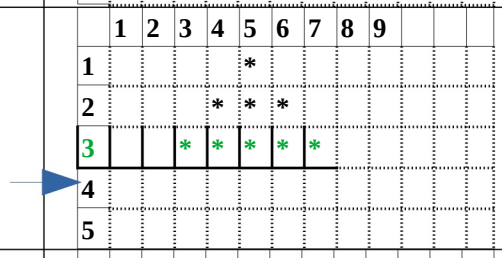
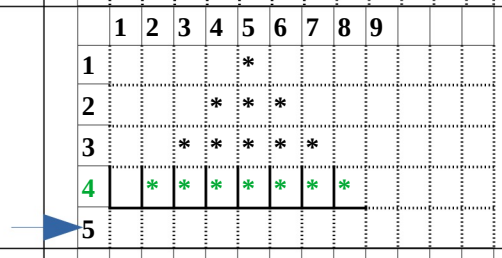
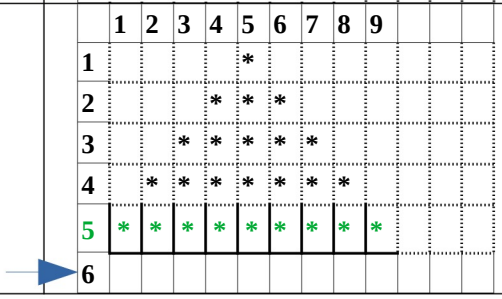
Analysis of output

We that the complete pattern can be easily expressed as a collection of selected (**row, column**) numeric value pairs. The selection of (**row number, column number**) numeric pairs will be based on the keyboard characters used to design the given pattern. With this numeric pair number scheme we can correctly able to locate a particular keyboard character on the computer screen.The first value in any selected numeric pair represents the row number and second value represents a column number. For example;

(2, 4) means character "*" at **second row** and **fourth column** in the table.

	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1	space	space	space	space	* (1,5)	space	space	space	space
Row=2	space	space	space	* (2,4)	*(2,5)	*(2,6)	space	space	space
Row=3	space	space	*(3,3)	* (3,4)	*(3,5)	*(3,6)	*(3,7)	space	space
Row=4	space	*(4,2)	*(4,3)	*(4,4)	*(4,5)	*(4,6)	*(4,7)	*(4,8)	space
Row=5	*(5,1)	*(5,2)	*(5,3)	*(5,4)	*(5,5)	*(5,6)	*(5,7)	*(5,8)	*(5,9)

Space → space is also a character like any other character. If a non-space character or a text is displayed on the screen after a blank area or gap, it means that many space character has to be print before printing any other non-space character(s) afterwards.

Steps for pattern generation		Computer Screen
For row = 1	Print space " " 4 times in the same 1st row. Print star "*" once in the same 1st row then move to the next row number 2.	
For row = 2	Print space " " 3 times in the same 2nd row. Print star "*" 3 times in the same 2nd row then move to the next row number 3.	
For row = 3	Print space " " 2 times in the same 3rd row. Print star "*" 5 times in the same 3rd row then move to the next row number 4.	
For row = 4	Print space" " 1 time in the same 4th row. Print star "*" 7 times in the same 4th row then move to the next row number 5.	
For row = 5	Print space" " 0 times in the same 5th row. Print star "*" 9 times in the same 5rd row then move to the next row number 6.	

Let's convert steps into C program.

Use of C in-built method	
Move to the next row	`printf("\n");`
Move to the next (row, col) position and print space " " character in the same row	`printf(" ");`
Move to the next (row, col) position and print star "*" character in the same row	`printf("*");`

Above three types of programming statements are expected to get repeat many times in the program. Hence we can use ***for-loop*** for the same.

Column Level Repetitive Action

Let's consider two attributes rows and columns of a 2-dimensional pattern as variables;

row → variable type of integer to store integer value one at a time and represents row number between 0 and 4.

col → variable type of integer to store integer value one at a time and represents column number between 1 and 5.

spaces → variable type of integer to store integer value one at a time and represents space character at column number between 1 and 5.

totalRows → variable type of integer to store integer value for total number of rows.

Logic Trace Tables

(row, col) value pair depicts the position of "*" character on the display screen								
				row=1, col=5				
			row=2, col=4	**row=2**, col=5	**row=2**, col=6			
		row=3, col=3	**row=3**, col=4	**row=3**, col=5	**row=3**, col=6	**row=3**, col=7		
	row=4, col=2	**row=4**, col=3	**row=4**, col=4	**row=4**, col=5	**row=4**, col=6	**row=4**, col=7	**row=4**, col=8	
row=5, col=1	**row=5**, col=2	**row=5**, col=3	**row=5**, col=4	**row=5**, col=5	**row=5**, col=6	**row=5**, col=7	**row=5**, col=8	**row=5**, col=9

Row level Repetitive Action			**Expected Output**
Variable: row	**Column level Repetitive Action**		
	Variable : spaces	Variable: col	
1	1,2,3,4	5	*
2	1,2,3	4, 5,6	* * *
3	1,2	3, 4, 5,6,7	* * * * *
4	1	2, 3, 4, 5,6,7,8	* * * * * * *
5	0(no space)	1, 2, 3, 4, 5,6,7,8,9	* * * * * * * * *

The space (" ") and star ("*") characters are repeated many times.

So, we can use ***for-loop*** for each. We can fill table as follows:

Row level Repetitive Action					**Expected Output**
Variable : row	**Column level Repetitive Action**				
	Variable : spaces		Variable : col		
	min_val	**max_val**	**min_val**	**max_val**	
1	1	4	5	5	*
2	1	3	4	6	* * *
3	1	2	3	7	* * * * *
4	1	1	2	8	* * * * * * *
5	0	0	1	9	* * * * * * * * *

We know that totalRows=5(total number of rows).

Let's try to use variable **totalRows** to generalise the variable **col**'s columns **min_val** & **max_val.**

Row level Repetitive Action					**Expected Output**
Variable : row	**Column level Repetitive Action**				
	Variable : spaces		Variable : col		
	min_val	**max_val**	**min_val**	**max_val**	
1	1	4	totalRows -0	totalRows + 0	*
2	1	3	totalRows -1	totalRows + 1	* * *
3	1	2	totalRows -2	totalRows + 2	* * * * *
4	1	1	totalRows -3	totalRows + 3	* * * * * * *
5	0	0	totalRows -4	totalRows + 4	* * * * * * * * *

Let start the **row** value from 0 to 4 instead of 1 to 5.This is in order to match the part

of the expression of variable **col**'s columns **min_val** & **max_val.**

Row level Repetitive Action					**Expected Output**
Variable : row	**Column level Repetitive Action**				
	Variable : spaces		**Variable : col**		
	min_val	**max_val**	**min_val**	**max_val**	
0	1	4	totalRows -0	totalRows + 0	*
1	1	3	totalRows -1	totalRows + 1	* * *
2	1	2	totalRows -2	totalRows + 2	* * * * *
3	1	1	totalRows -3	totalRows + 3	* * * * * * *
4	0	0	totalRows -4	totalRows + 4	* * * * * * * * *

Using variable **row** to generalise the variable **col**'s columns **min_val** & **max_val.**

Row level Repetitive Action					**Expected Output**
Variable : row	**Column level Repetitive Action**				
	Variable : spaces		Variable : col		
	min_val	**max_val**	**min_val**	**max_val**	
0	1	4	totalRows - row	totalRows + row	*
1	1	3	totalRows - row	totalRows + row	* * *
2	1	2	totalRows - row	totalRows + row	* * * * *
3	1	1	totalRows - row	totalRows + row	* * * * * * *
4	0	0	totalRows - row	totalRows + row	* * * * * * * * *

The condition (col >= (**totalRows – row**)) gives the minimum limit , a starting point from which it will start printing star("*") character(s) till the maximum value of variable col which is (**totalRows + row)**.

The if-condition can be given for between the range of values (**totalRows - row**) and

(**totalRows + row)** using the comparison and logical operators as:

If (col >= (**totalRows - row**) && col <= **(totalRows + row))**

We already know that;

variable row = 0,1,2,3,4 or

row = 0,1,2,3,4, **totalRows - 1**

variable col = 1,2,3,4,…,9 or

col=1,2,3,4,**…., (totalRows + row)**

Why the maximum value of variable col is kept dynamic?

We know that the printing star("*") character(s) will continue till the maximum value of variable **col** which is (**totalRows + row).**

We are not worried about the spaces when printing of all the star("*") character(s) gets completed in a row.

It makes sense to restrict the column level looping to the point where the maximum number of printing of star("*") characters reached.

Why the minimum value of variable col is kept 1?

It makes sense to start the col=1,2…from the value 1 ,since we have to print space characters before actual printing of star("*") characters required to start at the column level looping.

We can eliminate the logical AND(&&) condition which is (col <= **(totalRows + row))** in the

if-condition to print the star("*") characters.

It's already planned **to consider** this logical AND(&&) condition as the **loop-condition** already

been taken into the column level inner for-loop.

So the if-condition will now reduce to the following:

If (col >= (**totalRows - row**))

We can summarise it all as;

<table>
<tr><td colspan="6">Prints a star character when condition is True for every value of variable col</td></tr>
<tr><td>col=1</td><td>col=2</td><td>col=3</td><td>col=4</td><td>……</td><td>col= (totalRows + row)</td></tr>
<tr><td colspan="6">If (condition=(col >= (totalRows - row)) is True)
Print star character
Else
Print space character</td></tr>
</table>

We generalised the non-repeating numeric pattern of the column **max_val** into an expressions.

We can now define column level ***for-loop*** which is same for every row value.

Elements of* Column level *for-loop	
min_val	1
max_val	**totalRows + row**
variable_name	**col**
statement(s)	`printf("*");`

Row Level Repetitive Action

The following statements had been repeated 5 times(total number of rows).

Column level *for-loop* for printing star characters // ***inner for-loop***

```
printf("\n" );// Move cursor to the next row
```

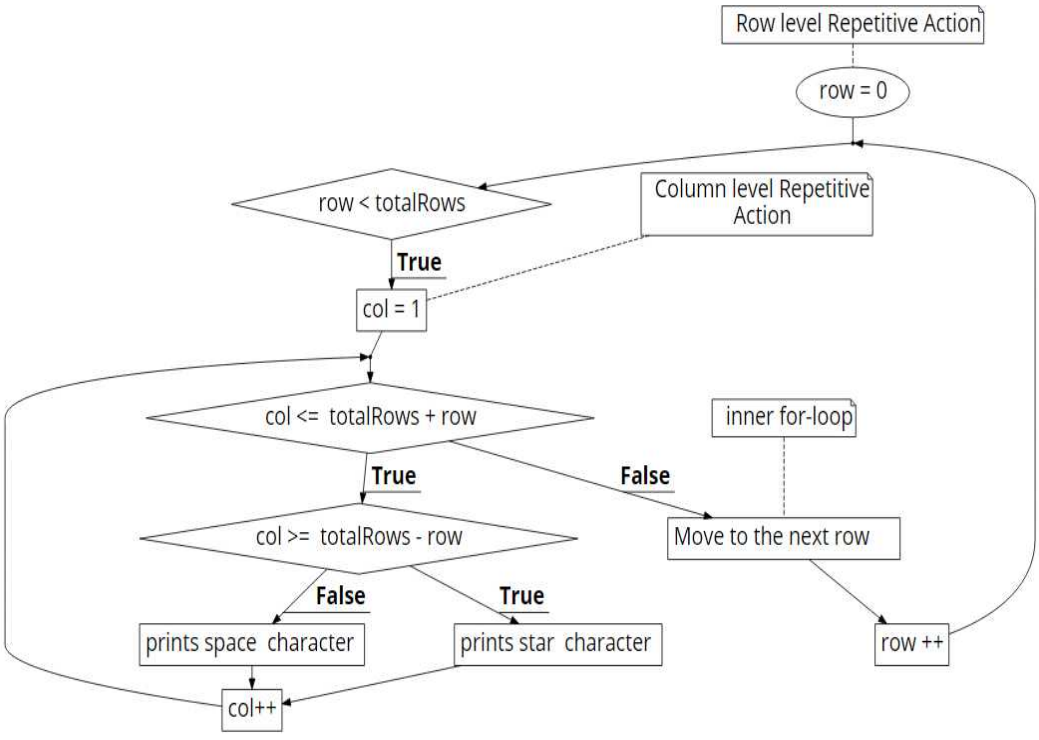
We have assigned unique value to each row starting from 0 till 4.

Let's consider;

row → variable type of integer to store integer value one at a time and represents row number between 0 and 4(totalRows - 1). The **totalRows** has initially assigned the value 5.

Elements Of* Row Level *for-loop	
min_val	0
max_val	totalRows - 1
variable_name	**row**
statement(s)	**Column level *for-loop*** for printing star characters `printf("\n" );// Move cursor to the next row`

We got everything related to ***for-loop*** for row level repetition and related flow-chart can be shown.



Let’s execute the following program.

PatternE1.c

```c
#include <stdio.h>
int main() {

	int totalRows = 5; // total number of rows to display

	// Row level Repetitive Action :
	// Action1.Executes Column level Repetitive Action
	// Action2.Move cursor to next row.
	for (int row = 0; row < totalRows; row++) {

		// Column level Repetitive Action :

		// Action1.Move cursor in the same row
		for (int col = 1; col <= (totalRows + row); col++) {

			if (col >= (totalRows - row)) {
				// Action2.prints star “* ” character
				printf("* ");
			} else {
				// Action3.prints space “ ” character
				printf("  ");
			}

		}// inner for-loop

		printf("\n");// Move cursor to the next row

	}// outer for-loop


	return 0;
}
```

Output

```
        *
      * * *
    * * * * *
  * * * * * * *
* * * * * * * * *
```

Alternative solution2

From the previous solution, when the **if-condition** is true we will print a star("*") character and in the else part of the **if-statement** we will print spaces. Let's start with the observation table that we got from the previous solution.

Row level Repetitive Action					**Expected Output**
Variable : row	**Column level Repetitive Action**				
	Variable : spaces		Variable : col		
	min_val	**max_val**	**min_val**	**max_val**	
1	1	4	5	5	*
2	1	3	4	6	* * *
3	1	2	3	7	* * * * *
4	1	1	2	8	* * * * * * *
5	0	0	1	9	* * * * * * * * *

Reversing the counter values of variable row, i.e. row = 5, 4, 3, 2, 1 so that it will match the column **min_val** of the variable **col.**

Row level Repetitive Action					**Expected Output**
Variable: row	**Column level Repetitive Action**				
	Variable : spaces		Variable : col		
	min_val	**max_val**	**min_val**	**max_val**	
5	1	4	5	5	*
4	1	3	4	6	* * *
3	1	2	3	7	* * * * *
2	1	1	2	8	* * * * * * *
1	0	0	1	9	* * * * * * * * *

We can use the variable **totalRows** to generalise the column **max_val** for printing star character(s).

Row level Repetitive Action						**Expected Output**
Variable : row	**Column level Repetitive Action**					
	Variable : spaces		Variable : col			
	min_val	**max_val**	**min_val**	**max_val**		
				default	**1**	
5	1	4	row	5 + 0	totalRows + 0	*
4	1	3	row	5 + 1	totalRows + 1	* * *
3	1	2	row	5 + 2	totalRows + 2	* * * * *
2	1	1	row	5 + 3	totalRows + 3	* * * * * * *
1	0	0	row	5 + 4	totalRows + 4	* * * * * * * * *

Part of variable **col's column max_val's** expression is still having distinct numeric value at each row.

Row level Repetitive Action						**Expected Output**
Variable : row	Column level Repetitive Action					
	Variable : col					
	min_val	max_val				
			1	2	3	
5	row	5	5 + 0	totalRows + 0	totalRows + 0	*
4	row	6	5 + 1	totalRows + 1	totalRows + 5- 4	* * *
3	row	7	5 + 2	totalRows + 2	totalRows + 5- 3	* * * * *
2	row	8	5 + 3	totalRows + 3	totalRows + 5- 2	* * * * * * *
1	row	9	5 + 4	totalRows + 4	totalRows + 5- 4	* * * * * * * * *

Row level Repetitive Action					Expected Output
Variable : row	Column level Repetitive Action				
	Variable : col				
	min_val	max_val			
			4	5	
5	row	5	totalRows + 5- 5	totalRows + totalRows - 5	*
4	row	6	totalRows + 5- 4	totalRows + totalRows - 4	* * *
3	row	7	totalRows + 5- 3	totalRows + totalRows - 3	* * * * *
2	row	8	totalRows + 5- 2	totalRows + totalRows - 2	* * * * * * *
1	row	9	totalRows + 5- 4	totalRows + totalRows - 4	* * * * * * * * *

Part of variable **col's column max_val's** expression matches with the row values.

Row level Repetitive Action				**Expected Output**
Variable : row	**Column level Repetitive Action**			
	Variable : col			
	min_val	**max_val**		
		5	6	
5	row	totalRows + totalRows - 5	2 * totalRows - row	*
4	row	totalRows + totalRows - 4	2 * totalRows - row	* * *
3	row	totalRows + totalRows - 3	2 * totalRows - row	* * * * *
2	row	totalRows + totalRows - 2	2 * totalRows - row	* * * * * * *
1	row	totalRows + totalRows - 4	2 * totalRows - row	* * * * * * * * *

The condition (col >= **row**) gives starting point from which ***for-loop*** will start printing star("*") character(s) till the maximum value of variable col which is **(**2 * **totalRows - row**) for every value of row. The if-condition can be given for between the range of values (**row**) and **(**2 * **totalRows - row**) using the comparison and logical operators as:

If (col >= (**row**) && col <= **(**2 * **totalRows - row))**

We already know that;

variable col = 1,2,3,4,5 or 1,2,3,4, **…,(**2 * **totalRows - row)**

Same expression

variable row = 5,4,3,2,1 or **totalRows**,4,3,2,1

We can eliminate the logical AND(&&) condition(col <= **(**2 * **totalRows - row))** in the **if-condition.** This logical AND(&&) condition already been taken care as the **loop-condition** in the column level inner for-loop.

So the if-condition will now reduce to If (col >= **row**).We can summarise it all as;

Prints a star when condition is **True** for every value of variable **col**					
col=1	col=2	col=3	col=4	.....	col= **(**2 * **totalRows - row)**
If (condition=(col >= **row**) is **True)** Print star character **Else** Print space character					

We generalised the non-repeating numeric pattern of the column **max_val** into an expressions.

We can now define column level ***for-loop***.

Elements of **Column level** ***for-loop***	
min_val	1
max_val	(2 * **totalRows - row)**
variable_name	**col**
statement(s)	`printf("*");`

Row Level Repetitive Action

The following statements had been repeated 5 times(total number of rows).

Column level *for-loop* for printing star characters**//** ***inner for-loop***

```
printf("\n" );// Move cursor to the next row
```

We have assigned unique value to each row starting from 5 till 1.

Let's consider;

row → variable type of integer to store integer value one at a time and represents row number between 5 and 1. The **totalRows** has initially assigned the value 5.

We got everything related to ***for-loop*** for row level repetition.

Elements Of **Row Level decrementing** ***for-loop***	
min_val	1
max_val	totalRows
variable_name	**row**
statement(s)	**Column level *for-loop*** for printing star characters `printf("\n");//Moves cursor to the next row`

Let’s execute the following program.

PatternE2.c

```c
#include <stdio.h>
int main() {

	int totalRows = 5; // total number of rows to display

	// Row level Repetitive Action:
	// Action1.Executes Column level Repetitive Action
	// Action2.Move cursor to next row.
	for (int row = totalRows; row >= 1; row--) {

		// Column level Repetitive Action :

		// Action1.Move cursor in the same row
		for (int col = 1; col <= (2 * totalRows - row); col++) {

			if (col >= row) {
				// Action2.prints star “* ” character
				printf("* ");
			} else {
				// Action3.prints space “ ” character
				printf("  ");
			}

		}// inner for-loop

		printf("\n");// Move cursor to the next row

	}// outer for-loop

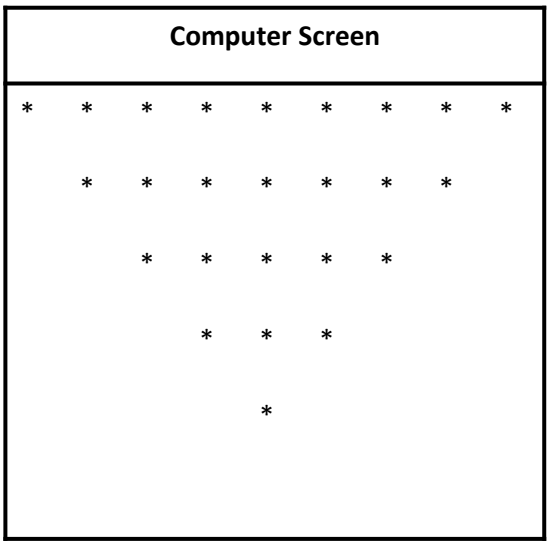
	return 0;
}
```

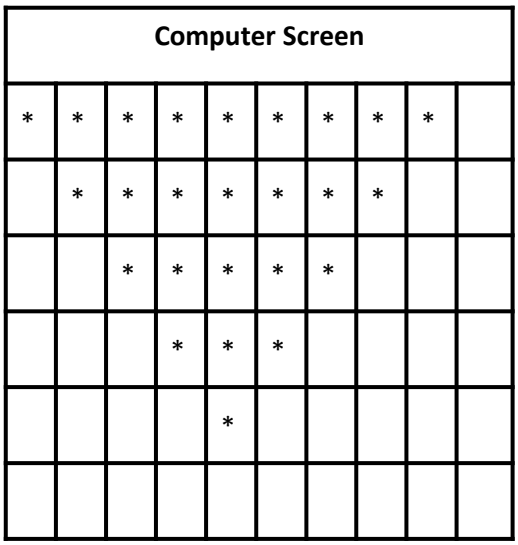
Output

```
        *
      * * *
    * * * * *
  * * * * * * *
* * * * * * * * *
```

Pattern1F

Our programming job is to output this pattern on a part of the computer screen as shown in the following. We can consider the computer screen as a combination of (**row, column**) numeric value paired boxes which are spread all over the screen. Each box is treated as a resident of just a single character only.





Analysis of output

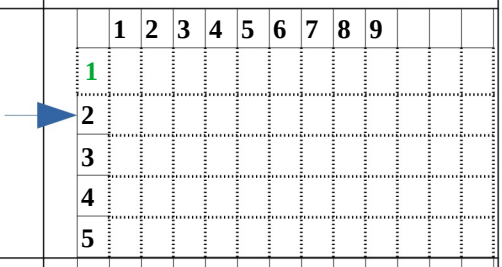
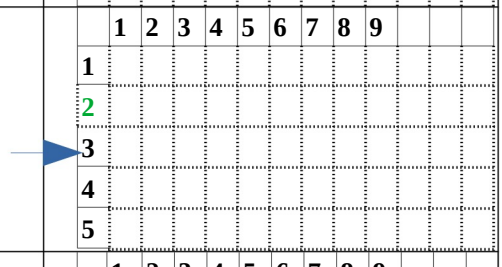
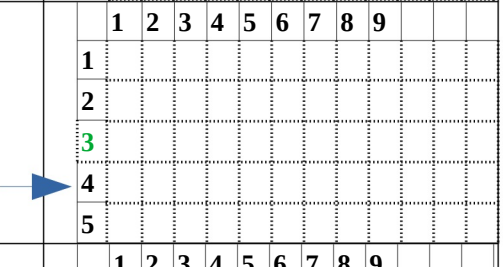
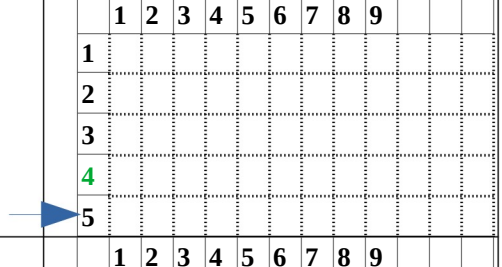
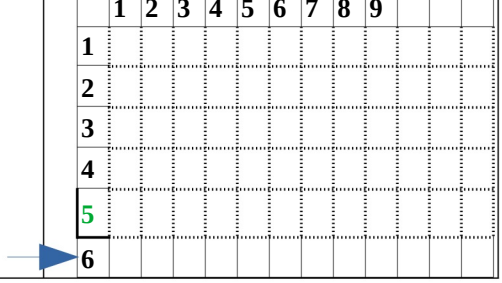
We that the complete pattern can be easily expressed as a collection of selected (**row, column**) numeric value pairs. The selection of (**row number, column number**) numeric pairs will be based on the keyboard characters used to design the given pattern. With this numeric pair number scheme we can correctly able to locate a particular keyboard character on the computer screen.The first value in any selected numeric pair represents the row number and second value represents a column number. For example;

(2, 4) means character "*" at **second row** and **fourth column** in the table.

	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1	*(1,1)	*(1,2)	*(1,3)	*(1,4)	* (1,5)	*(1,6)	*(1,7)	*(1,8)	*(1,9)
Row=2	space	*(2,2)	*(2,3)	* (2,4)	*(2,5)	*(2,6)	*(2,7)	*(2,8)	space
Row=3	space	space	*(3,3)	* (3,4)	*(3,5)	*(3,6)	*(3,7)	space	space
Row=4	space	space	space	*(4,4)	*(4,5)	*(4,6)	space	space	space
Row=5	space	space	space	space	*(5,5)	space	space	space	space

The actual challenge is to find correct (row, col) positions on the computer screen by writing a program. This will be carried out by applying for-loop along with appropriate logical conditions. This stimulates growth for algorithmic thinking, a fundamental requirement of a confident programmer.

Exercise. Complete the steps for the generation of this logic pattern for every row 1 to 5 by showing output on the computer screen diagram.

Steps for pattern generation(row = 1 to 5)	Computer Screen
For row = 1	
For row = 2	
For row = 3	
For row = 4	
For row = 5	

Let's convert steps into C program.

Use of C in-built method	
Move to the next row	**printf("\n");**
Move to the next (row, col) position and print space " " character in the same row	**Print (" ");**
Move to the next (row, col) position and print star "*" character in the same row	**Print ("*");**

Mentioned methods are bound to get repeat many a times in a program in order to output these characters repeatedly. Hence we will use these within ***for-loop*** for such repetitive task in a program.

Column Level Repetitive Action

Let's consider two attributes rows and columns of a 2-dimensional pattern as variables;

row → variable type of integer to store integer value one at a time and represents row number between 0 and 4.

col → variable type of integer to store integer value one at a time and represents column number between 1 and 5.

spaces → variable type of integer to store integer value one at a time and represents space character at column number between 1 and 5.

totalRows → variable type of integer to store integer value for total number of rows.

Logic Trace Tables

(row, col) value pair depicts the position of "*" character on the display screen								
row=1, col1	**row=1**, col2	**row=1**, col3	**row=1**, col4	**row=1**, col5	**row=1**, col6	**row=1**, col7	**row=1**, col8	**row=1**, col9
	row=2, col2	**row=2**, col3	**row=2**, col4	**row=2**, col5	**row=2**, col6	**row=2**, col7	**row=2**, col8	
		row=3, col3	**row=3**, col4	**row=3**, col5	**row=3**, col6	**row=3**, col7		
			row=4, col4	**row=4**, col5	**row=4**, col6			
				row=5, col5				

The space (" ") and star("*") characters are repeated many times.
So, we can use ***for-loop*** for space (" ") and star("*") characters.

Row level Repetitive Action			**Expected Output**
Variable: row	**Column level Repetitive Action**		
	Variable : spaces	Variable: col	
1	0(no space)	1, 2, 3, 4, 5,6,7,8,9	* * * * * * * * *
2	1	2, 3, 4, 5,6,7,8	* * * * * * *
3	1,2	3, 4, 5,6,7	* * * * *
4	1,2,3	4, 5,6	* * *
5	1,2,3,4	5	*

We can fill previous table as follows also in order to explore various components of ***for-loop.***

Row level Repetitive Action					**Expected Output**
Variable : row	**Column level Repetitive Action**				
	Variable : spaces		Variable : col		
	min_val	**max_val**	**min_val**	**max_val**	
1	0	0	1	9	* * * * * * * * *
2	1	1	2	8	* * * * * * *
3	1	2	3	7	* * * * *
4	1	3	4	6	* * *
5	1	4	5	5	*

We know that totalRows=5(total number of rows).

Let's try to use variable **totalRows** to generalise the variable **col**'s columns are **min_val** & **max_val.**

Row level Repetitive Action					**Expected Output**
Variable : row	**Column level Repetitive Action**				
	Variable : spaces		Variable : col		
	min_val	**max_val**	**min_val**	**max_val**	
1	0	0	totalRows -4	totalRows + 4	* * * * * * * * *
2	1	1	totalRows -3	totalRows + 3	* * * * * * *
3	1	2	totalRows -2	totalRows + 2	* * * * *
4	1	3	totalRows -1	totalRows + 1	* * *
5	1	4	totalRows -0	totalRows + 0	*

Let use decrementing ***for-loop*** with range 4 to at the row level repetition.

This is in order to match the part of the expression of variable **col**'s columns **min_val** & **max_val.**

Using variable **row** to generalise the variable **col**'s columns **min_val** & **max_val.**

Row level Repetitive Action					**Expected Output**
Variable : row	**Column level Repetitive Action**				
	Variable : spaces		Variable : col		
	min_val	**max_val**	**min_val**	**max_val**	
4	0	0	totalRows - row	totalRows + row	* * * * * * * * *
3	1	1	totalRows - row	totalRows + row	* * * * * * *
2	1	2	totalRows - row	totalRows + row	* * * * *
1	1	3	totalRows - row	totalRows + row	* * *
0	1	4	totalRows - row	totalRows + row	*

The condition (col >= (**totalRows – row**)) gives the minimum limit, a starting point from which it will start printing star("*") character(s) till the maximum value of variable col which is (**totalRows + row)**.

The **if-condition** can be given for between the range of values

(**totalRows - row**) and (**totalRows + row)** using the comparison and logical operators as:

If (col >= (**totalRows - row**) && col <= **(totalRows + row))**

We already know that;

Variable col = 1,2,3,4,…,9 or 1,2,3,4,**…., (totalRows + row)**

Same expression

Variable row = 4,3,2,1,0 or (totalRows – 1),3,2,1,0

We can eliminate the logical AND(&&) condition(col <= **(totalRows + row))** in the

if-condition to print the star("*") characters.This logical AND(&&) condition already been **considered** as the upper_limit in the **loop-condition** for column level inner for-loop. So the **if-condition** will now reduce to the following.

If (col >= (**totalRows - row**))It will allow to print star("*") character when result of **if-condition's expression** is **true** or else will prints the space character. We generalised the non-repeating numeric pattern of the column **max_val** into an expressions. We can now define column level ***for-loop***.

Elements of **Column level** *for-loop*	
min_val	1
max_val	**totalRows + row**
variable_name	**col**
statement(s)	`printf("*");`

Row Level Repetitive Action

The following statements had been repeated 5 times(total number of rows).

Column level *for-loop* for printing star characters **//** ***inner for-loop***

```
printf("\n" );// Move cursor to the next row
```

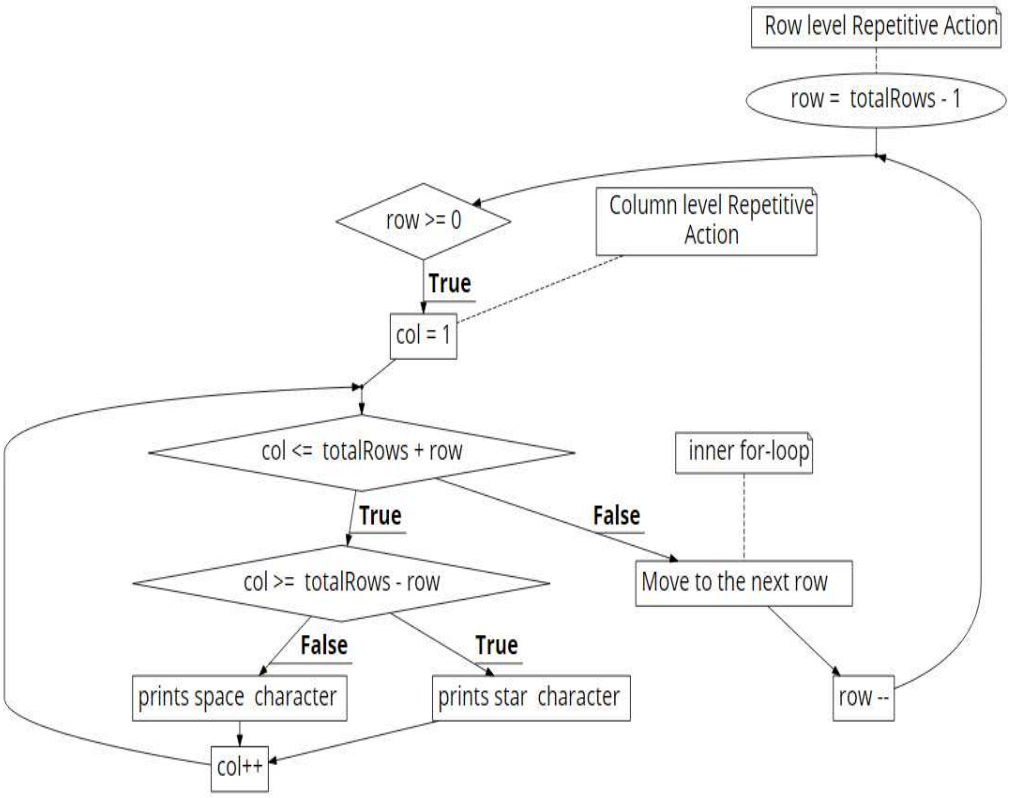
row → variable type of integer to store integer value one at a time and represents

row number between 4 or (**totalRows - 1**) and 0. The **totalRows** has initially assigned the value 5.

We got everything related to ***for-loop*** for row level repetition.

Elements Of **Row Level decrementing** *for-loop*	
min_val	0
max_val	totalRows - 1
variable_name	**row**
statement(s)	**Column level *for-loop*** for printing star characters `printf("\n" ); // Move to the next row`

The flow-chart can be given as follows:



Let’s execute the following program.

PatternF1.c

```
#include <stdio.h>
int main() {

	int totalRows = 5; // total number of rows to display

	// Row level Repetitive Action :

	// Action1.Executes Column level Repetitive Action
	// Action2.Move cursor to next row.
	for (int row = (totalRows - 1); row >= 0; row--) {

		// Column level Repetitive Action :

		// Action1.Move cursor in the same row
		for (int col = 1; col <= (totalRows + row); col++) {

			if (col >= (totalRows - row)) {
				// Action2.prints star “* ” character
				printf("* ");
			} else {
				// Action3.prints space “ ” character
				printf("  ");
			}

		}// inner for-loop

		printf("\n");// Move cursor to the next row

	}// outer for-loop


	return 0;
}
```

Output

```
* * * * * * * * *
  * * * * * * * *
    * * * * *
      * * *
        *
```

Alternative solution2

By using incrementing ***for-loop*** at the row level repetition we can give alternative solutions.

***Problem* F1**.Find the different logic and write the program to print the following pattern for 5 rows. Test the logic for 12 rows. Some of the details are mentioned in the form of table to start with.

Row level Repetitive Action					**Expected Output**
Variable : row	**Column level Repetitive Action**				
	Variable : spaces		Variable : col		
	min_val	**max_val**	**min_val**	**max_val**	
1	0	0	1	9	* * * * * * * * *
2	1	1	2	8	* * * * * * *
3	1	2	3	7	* * * * *
4	1	3	4	6	* * *
5	1	4	5	5	*

When the **if-condition** is true program should print a star("*") character and in the **else** part of the **if-else-statement** program should print space character.

Problem* F2**.Continuation to previous problem (Problem* F1**) write the program for the following situation.

When the **if-condition** is true, then program should print a space(" ") character and in the **else** part of the **if-else-statement** program should print star("* ")character.

Pattern1G

Our programming job is to output this pattern on a part of the computer screen as shown in the following. We can treat computer screen as a combination of (**row, column**) numeric value paired boxes which are spread all over the screen. Each box can hold a single character only.

Computer Screen
*
* *
* * *
* * * *
* * * * *
* * * *
* * *
* *
*

Computer Screen								
*								
*	*							
*	*	*						
*	*	*	*					
*	*	*	*	*				
*	*	*	*					
*	*	*						
*	*							
*								

Analysis of output

The complete pattern can be easily expressed as a collection of selected (**row, column**) numeric value pairs. The first value represents the row number and second value represents a column number. For example;

(2, 1) means character "*" at **second row** and **first column** in the table.

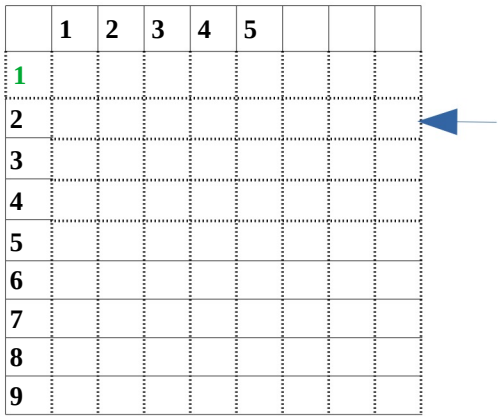
The real challenge is to find the correct (row, col) positions and to print required character on the computer screen by writing a program. This will be carried out by applying ***for-loop*** in combination with appropriate logical conditions. Finding logical conditions in a step-by-step approach stimulates the growth for algorithmic thinking, a fundamental requirement to become a confident programmer.

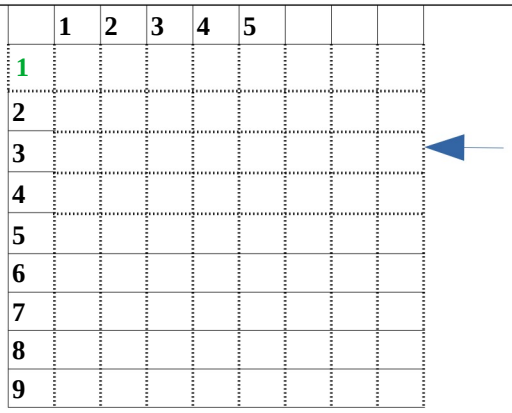
	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	* (1,1)				
Row=2	*** (2 ,1)**	*(2,2)			
Row=3	* (3,1)	*(3,2)	*(3,3)		
Row=4	*(4,1)	*(4,2)	*(4,3)	*(4,4)	
Row=5	*(5,1)	*(5,2)	*(5,3)	*(5,4)	*(5,5)
Row=6	*(6,1)	*(6,2)	*(6,3)	*(6,4)	
Row=7	*(7,1)	*(7,2)	*(7,3)		
Row=8	*(8,1)	*(8,2)			
Row=9	*(9,1)				

Exercise. Explain the steps for the generation of this logic pattern for every row 1 to 9 by showing output on the computer screen diagram.

Steps for pattern generation(row = 1 to 9) with computer screen diagram.

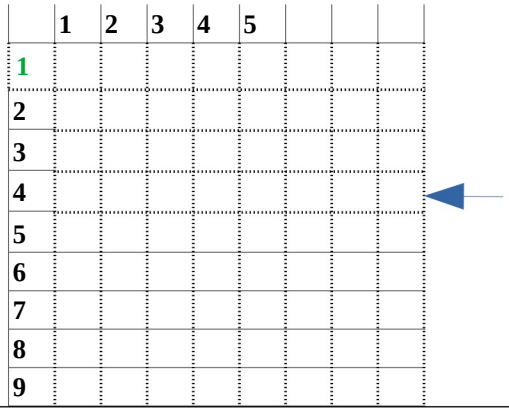
For row = 1

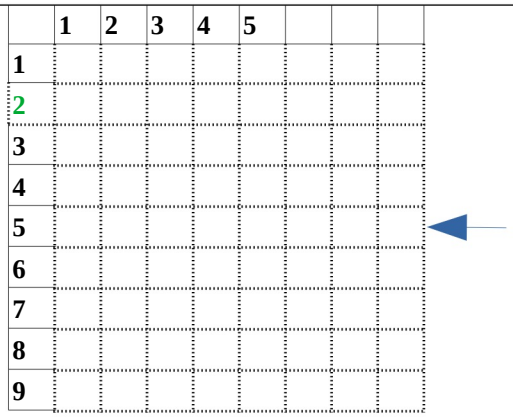




For row = 2

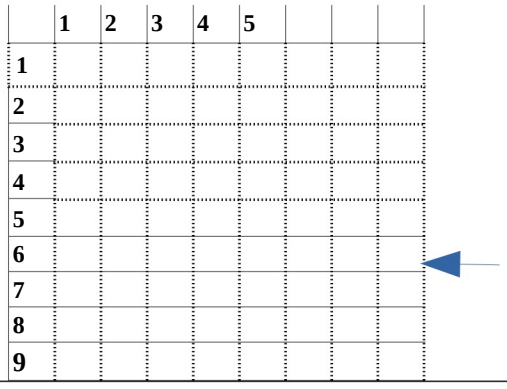
For row = 3

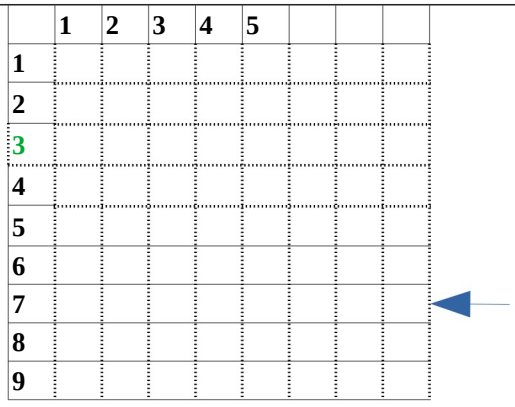




For row = 4

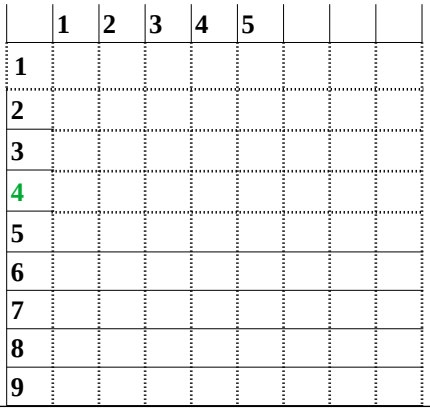
For row = 5



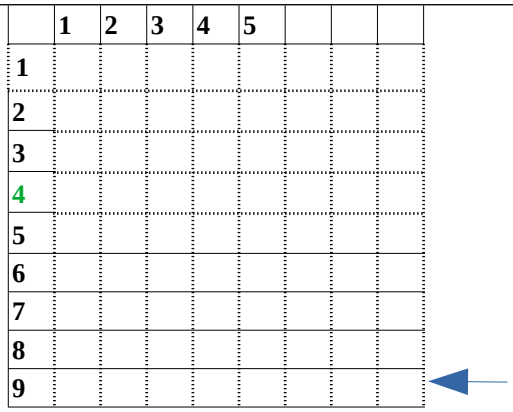


For row = 6

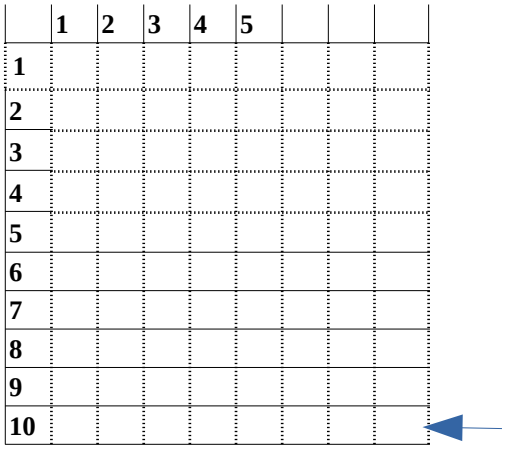
For row = 7



For row = 8



For row = 9



Let's convert steps into C program.

Use of C in-built method	
To move cursor to the next row	printf("\n");
To move to next(row, col) position and to print star "*" character in the same row	printf("*");

These statements were getting repeat many times and need to use within ***for-loop*** for the same.

Column Level Repetitive Action

We can consider two attributes rows and columns of a 2-dimensional pattern as variables.

Let's consider following variables which are type of integer to store integer value one at a time.

row → represents row number between 1 and 9.

col → represents column number between 1 and 5.

totalRows → variable type of integer to store integer value for total number of rows.

We need to explore various components of required **outer and inner *for-loops.***

(row, col) value pair depicts the position of "*" character on the display screen				
row=1, col=1				
row=2, col=1	**row=2**, col=2			
row=3, col=1	**row=3**, col=2	**row=3**, col=3		
row=4, col=1	**row=4**, col=2	**row=4**, col=3	**row=4**, col=4	
row=5, col=1	**row=5**, col=2	**row=5**, col=3	**row=5**, col=4	**row=5**, col=5
row=6, col=1	**row=6**, col=2	**row=6**, col=3	**row=6**, col=4	
row=7, col=1	**row=7**, col=2	**row=7**, col=3		
row=8, col=1	**row=8**, col=2			
row=9, col=1				

We required a set of **min_val** & **max_val** values within where loop will repeat.

Row level Repetitive Action		**Expected Output**
Variable: row	**Column level Repetitive Actio**n	
	Variable: col	
1	1	*
2	1,2	* *
3	1,2,3	* * *
4	1,2,3,4	* * * *
5	1,2,3,4,5	* * * * *
6	1,2,3,4	* * * *
7	1,2,3	* * *
8	1,2	* *
9	1	*

Row level Repetitive Action			**Expected Output**
Variable: row	**Column level Repetitive Actio**n		
	Variable: col		
	min_val	**max_val**	
1	1	1	*
2	1	2	* *
3	1	3	* * *
4	1	4	* * * *
5	1	5	* * * * *
6	1	4	* * * *
7	1	3	* * *
8	1	2	* *
9	1	1	*

Approach to solution

The variables are row & totalRows=5.We need to generalise the non-repeating numeric pattern of the column **max_val** using these remaining variables.

The numeric pattern (1 to 5) of the column **max_val** matches exactly the every value of row. Hence, it can be expressed in row variable.

Row level Repetitive Action					**Expected Output**
Variable : row		**Column level Repetitive Action**			
		Variable : col			
		min_val	**max_val**		
default	**1**		**default**	**1**	
1	1	1	1	row	*
2	2	1	2	row	* *
3	3	1	3	row	* * *
4	4	1	4	row	* * * *
5	5	1	5	totalRows	* * * * *
6	6	1	4	totalRows - 1	* * * *
7	7	1	3	totalRows - 2	* * *
8	8	1	2	totalRows - 3	* *
9	totalRows + 4	1	1	totalRows - 4	*

We need find the generalised expression for the maximum value of **row** variable also.

There are still non-repeating numeric values in the column **max_val** of the col variable.

Row level Repetitive Action					**Expected Output**
Variable : row		**Column level Repetitive Action**			
	1	Variable : col			
		min_val	**max_val**		
				2	**3**
1	1	1	1	row	row
2	2	1	2	row	row
3	3	1	3	row	row
4	4	1	4	row	row
5	5	1	5	totalRows	totalRows
6	6	1	4	totalRows – (6- 5)	totalRows – (row -5)
7	7	1	3	totalRows – (7- 5)	totalRows – (row -5)
8	8	1	2	totalRows – (8 -5)	totalRows – (row -5)
9	totalRows + (5-1)	1	1	totalRows - (9 -5)	totalRows - (row -5)

Row level Repetitive Action					
Variable : row			**Column level Repetitive Action**		
default	**1**	**2**	Variable : col		
			min_val	**max_val**	
				default	**4**
1	1	1	1	1	row
2	2	2	1	2	row
3	3	3	1	3	row
4	4	4	1	4	row
5	5	5	1	5	totalRows
6	6	6	1	4	totalRows – row +5
7	7	7	1	3	totalRows – row +5
8	8	8	1	2	totalRows – row +5
9	totalRows + (totalRows -1)	2*totalRows - 1	1	1	totalRows – row +5

Row level Repetitive Action					
Variable : row		**Column level Repetitive Action**			
	2	Variable : col			
		min_val	**max_val**		
				5	**6**
1	1	1	1	row	row
2	2	1	2	row	row
3	3	1	3	row	row
4	4	1	4	row	row
5	5	1	5	totalRows	totalRows
6	6	1	4	totalRows - row +totalRows	2*totalRows – row
7	7	1	3	totalRows - row +totalRows	2*totalRows – row
8	8	1	2	totalRows - row +totalRows	2*totalRows – row
9	2*totalRows - 1	1	1	totalRows - row +totalRows	2*totalRows - row

The numeric pattern of the column **max_val** of variable **col** matches exactly values(1 to 5) of **row**.

So that specific part of the numeric pattern can be expressed in row variable.

The condition (col >=1) gives a starting point from which it will start printing star("*") character(s) till the value of variable **row** reaches (**totalRows)**.

The variable **col** then start decreases which is captured in the expression (**2*totalRows - row**).

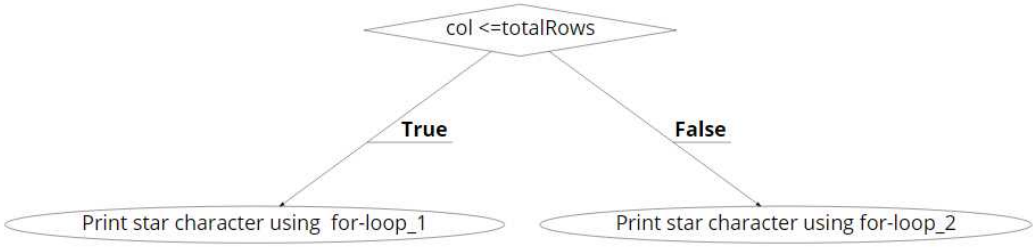
So in this case, there is not the one but two generalised expressions for values of the variable **col**. This indicates that we need to use two separate **for-loop** for the variable col.

Let the following are the two versions of column level for-loop for printing star ("*") character(s).

Column level *for-loop_1* for printing star characters / / ***inner for-loop***

Column level *for-loop_2* for printing star characters / / ***inner for-loop***

We can summarise it as:

<table>
<tr><td colspan="7">Prints a star for every value of variable row</td></tr>
<tr><td>row=1</td><td>row=2</td><td>row=3</td><td>row=4</td><td>row =totalRows</td><td>……</td><td>row= (2*totalRows - 1)</td></tr>
<tr><td colspan="7">If (condition=(col <=totalRows) is True)
Print star character using Column level <i>for-loop_1</i>
Else
Print star character using Column level <i>for-loop_2</i>
 
</td></tr>
</table>

We can now define these two column level ***for-loop***.

Elements of* Column level *for-loop_1	
min_val	1
max_val	**totalRows**
variable_name	**col**
statement(s)	`printf("*");`

Elements of* Column level *for-loop_2	
min_val	1
max_val	**2*totalRows - row**
variable_name	**col**
statement(s)	`printf("*");`

Row Level Repetitive Action

We have assigned unique value to each row starting from 1 till 9.

We can represent value 9 as (2*5 – 1).The variable **totalRows** has initially assigned the value 5.

So, we can generalised the maximum limit of the **row** variable using the variable. Let's consider;

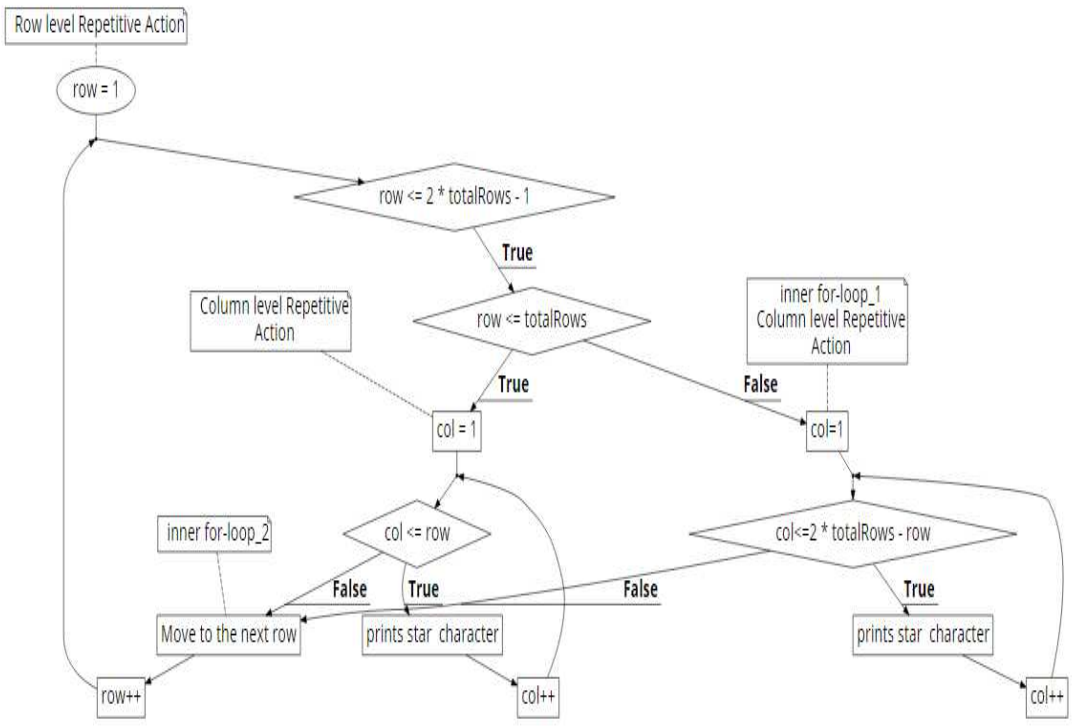
row → variable type of integer to store integer value one at a time and represents row number between 1 and (**2*totalRows** - 1).

We have logically divided row level repeation ***for-loop*** into two parts due to two different versions of the column level **for-loop**.

Elements Of* Row Level decrementing *for-loop	
min_val	1
totalRows	totalRows
variable_name	**row**
statement(s)	**Column level for-loop_1** for printing star characters `printf("\n");` // Move cursor to the next row

Elements Of* Row Level decrementing *for-loop	
totalRows + 1	totalRows + 1
max_val	2*totalRows - 1
variable_name	row
statement(s)	**Column level for-loop_2** for printing star characters `printf("\n");`// Move cursor to the next row

Since we got all the details related to considered for-loops, we can summarise these related information with the help of flow-chart.



Let's execute the following program.

PatternG1.c

```
#include <stdio.h>
int main() {

	int totalRows = 5;// number of rows to display

	for (int row = 1; row <= (2 * totalRows - 1); row++) {

		if (row <= totalRows) {
			for (int col = 1; col <= row; col++) {
				printf("* ");
			}
		} else {
			for (int col = 1; col <= (2 * totalRows - row); col++) {
				printf("* ");
			}
		}
		printf("\n");
	}
	return 0;
}
```

Output

```
*
* *
* * *
* * * *
* * * * *
* * * *
* * *
* *
*
```

The inner for-loop_1 and inner for-loop_2 differ only in the maximum limit value of the **col** variable.

It will be better if we capture maximum limit of **col** variable in a different variable.

Then we will be in a position to use only one inner for-loop for the complete column level repetition.

Let's update the previous program using single inner for-loop.

PatternG2.c

```c
#include <stdio.h>
int main() {
	int totalRows = 5;// number of rows to display
	int colMax = 0;// defines the maximum limit for col variable.

	// Row level Repetitive Action :
	// Action1.Executes Column level Repetitive Action
	// Action2.Move cursor to next row.
	for (int row = 1; row <= (2 * totalRows - 1); row++) {
		// sets the maximum limit of the inner for-loop
		if (row <= totalRows) {
			colMax = row;
		} else {
			colMax = (2 * totalRows - row);
		}
		// Column level Repetitive Action :
		// Action1.Move cursor in the same row
		// Action2.prints star "* " character
		for (int col = 1; col <= colMax; col++) {
			printf("* ");
		}// inner for-loop
		printf("\n");// Move cursor to next row.
	}// outer for-loop

	return 0;
}
```

Output

```
*
* *
* * *
* * * *
* * * * *
* * * *
* * *
* *
*
```

Alternative solution2

The remaining variables are row & totalRows=5.Let's start with the trace table that we got from the previous solution.

Row level Repetitive Action			**Expected Output**
Variable : row	**Column level Repetitive Action**		
	Variable : col		
	min_val	**max_val**	
1	1	1	*
2	1	2	* *
3	1	3	* * *
4	1	4	* * * *
5	1	5	* * * * *
6	1	4	* * * *
7	1	3	* * *
8	1	2	* *
9	1	1	*

We can try to express column **max_val** of **col** variable in terms of numeric pattern of row values.

Row level Repetitive Action							**Expected Output**
Variable : row	**Column level Repetitive Action**						
	Variable : col						
	min_val	**max_val**					
		default	**1**	**2**	**3**	**4**	
1	1	1	1	row	row	row – 2*0	*
2	1	2	2	row	row	row – 2*0	* *
3	1	3	3	row	row	row – 2*0	* * *
4	1	4	4	row	row	row – 2*0	* * * *
5	1	5	5	row	row	row – 2*0	* * * * *
6	1	4	6 - 2	row - 2	row – 2*1	row – 2*1	* * * *
7	1	3	7 - 4	row - 4	row – 2*2	row – 2*2	* * *
8	1	2	8 - 6	row - 6	row – 2*3	row – 2*3	* *
9	1	1	9 - 8	row - 8	row – 2*4	row – 2*4	*

We can express column **max_val** of **col** variable in terms of row variable and part of the expression follows a multiple of 2 but for row values greater than 5 or **totalRows**.

It has condition when to increment the multiple of 2 or not. The condition is handled by conditional statements mostly by **if-statement** or **if-else-statement** or **if-elseif-else** .This we need to handle using such conditional statement by declaring another integer variable ,for example;

int count ; // counter to increment the multiple of 2.

We can give **if-statement** something like the following by incrementing the value of count variable when row value crosses the **totalRows** value **which is value 5,**the initial total number of rows.

```
If( count > totalRows){
        count = count + 1; // Increment the count value by 1.
}
```

Now we can use variable count to generalise the column **max_val** of **col** variable.

Row level Repetitive Action						**Expected Output**
Variable : row	Variable : count	**Column level Repetitive Action**				
		Variable : col				
		min_val	**max_val**			
			default	**4**	**5**	
1	0	1	1	row – 2*0	row – 2*count	*
2	0	1	2	row – 2*0	row – 2*count	* *
3	0	1	3	row – 2*0	row – 2*count	* * *
4	0	1	4	row – 2*0	row – 2*count	* * * *
5	0	1	5	row – 2*0	row – 2*count	* * * * *
6	1	1	4	row – 2*1	row – 2*count	* * * *
7	2	1	3	row – 2*2	row – 2*count	* * *
8	3	1	2	row – 2*3	row – 2*count	* *
9	4	1	1	row – 2*4	row – 2*count	*

Also, we could try to express the maximum value of row variable in terms of **totalRows** which is as; row = 1,2,3,……,9 or 1,2,3,……,(10 – 1) or 1,2,3,……,(2*5 – 1) or 1,2,3,…..,(2* **totalRows** – 1)

Let's implement these observations as a different program.

PatternG3.c

```
#include <stdio.h>
int main() {

	int totalRows = 5;// number of rows to display
	int count = 0;// counter to increment the multiple of 2.

	// Row level Repetitive Action :
	// Action1.Executes Column level Repetitive Action
	// Action2.Move cursor to next row.
	for (int row = 1; row <= (2 * totalRows - 1); row++) {
		// increment count only when row value is greater
		  // than the total number of rows.
		if (row > totalRows) {
			count = count + 1;
		}

		// Column level Repetitive Action :

		// Action1.Move cursor in the same row
		// Action2.prints star "* " character
		for (int col = 1; col <= row - 2*count ; col++) {
			printf("* ");
		}// inner for-loop

		printf("\n");// Move cursor to next row.
	}// outer for-loop

	return 0;
}
```

Now, test for different number of lines, let it be 7.

PatternG3.c

```c
#include <stdio.h>
int main() {

	int totalRows = 7;// number of rows to display
	int count = 0;// counter to increment the multiple of 2.

	// Row level Repetitive Action :
	// Action1.Executes Column level Repetitive Action
	// Action2.Move cursor to next row.
	for (int row = 1; row <= (2 * totalRows - 1); row++) {
		// increment count only when row value is greater
		  // than the total number of rows.
		if (row > totalRows) {
			count = count + 1;
		}

		// Column level Repetitive Action :

		// Action1.Move cursor in the same row
		// Action2.prints star “* ” character
		for (int col = 1; col <= row - 2*count ; col++) {
			printf("* ");
		}// inner for-loop

		printf("\n");// Move cursor to next row.
	}// outer for-loop

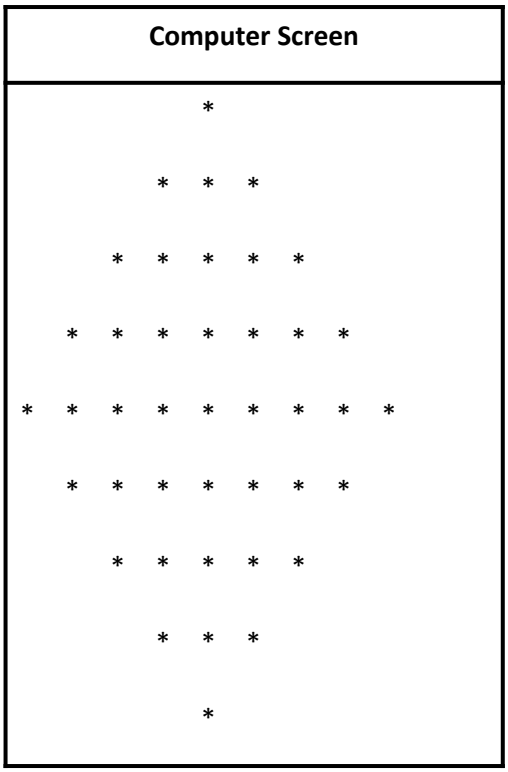
	return 0;
}
```

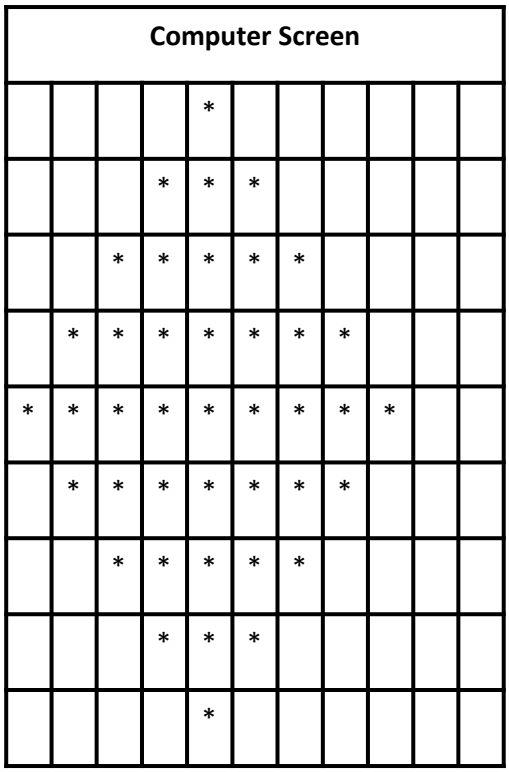
Output

```
*
* *
* * *
* * * *
* * * * *
* * * * * *
* * * * * * *
* * * * * *
* * * * *
* * * *
* * *
* *
*
```

Pattern1H

Our programming job is to output this pattern on a part of the computer screen as shown in the following. We can treat computer screen as a combination of (**row, column**) numeric value paired boxes which are spread all over the screen. Each box can hold a single character only.





Analysis of output

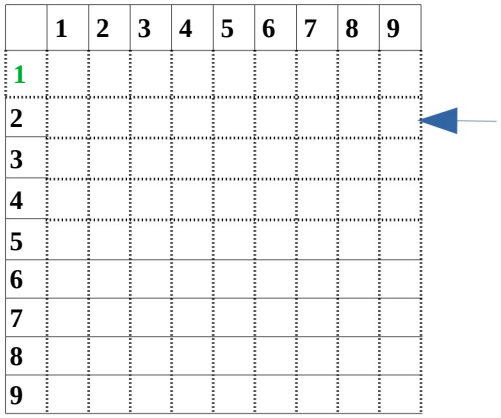
The complete pattern can be easily expressed as a collection of selected (**row, column**) numeric value pairs. Those selected (row, column) numeric value pairs are shown in the given table structure. For example; (2, 1) means character "*" at **second row** and **first column** in the table.

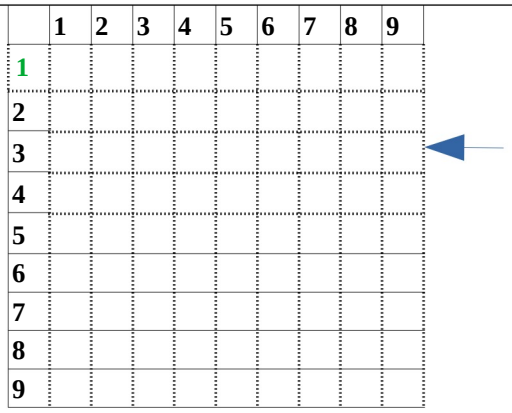
	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1					* (1,5)				
Row=2				*(2,4)	* (2 ,5)	*(2,6)			
Row=3			*(3,3)	*(3,4)	* (3,5)	*(3,6)	*(3,7)		
Row=4		*(4,2)	*(4,3)	*(4,4)	* (4,5)	*(4,6)	*(4,7)	*(4,8)	
Row=5	*(5,1)	*(5,2)	*(5,3)	*(5,4)	* (5,5)	*(5,6)	*(5,7)	*(5,8)	*(5,9)
Row=6		*(6,2)	*(6,3)	*(6,4)	* (6,5)	*(6,6)	*(6,7)	*(6,8)	
Row=7			*(7,3)	*(7,4)	* (7,5)	*(7,6)	*(7,7)		
Row=8				*(8,4)	* (8,5)	*(8,6)			
Row=9					*(9,5)				

Exercise. Explain the steps for the generation of this logic pattern for every row 1 to 9 by showing output on the computer screen diagram.

Steps for pattern generation(row = 1 to 9) with computer screen diagram.

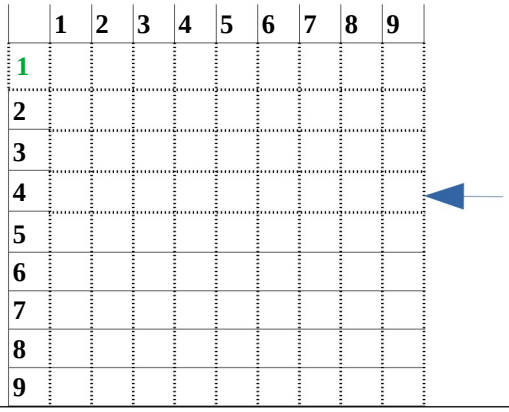
For row = 1

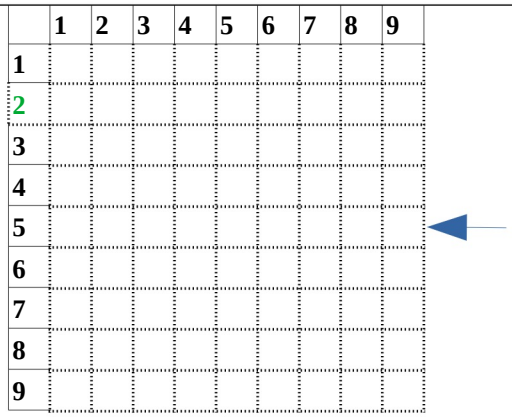




For row = 2

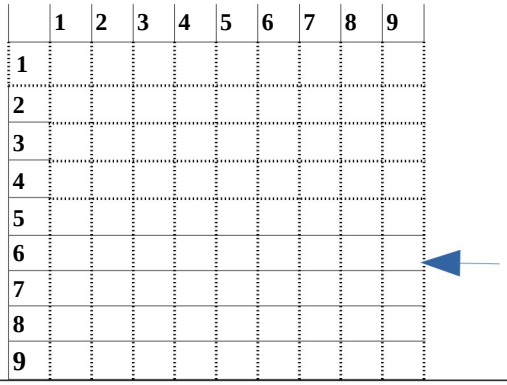
For row = 3

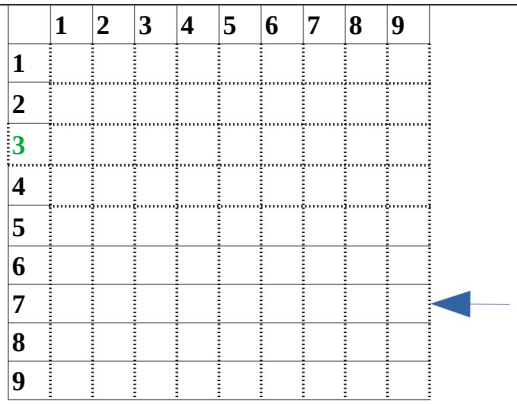




For row = 4

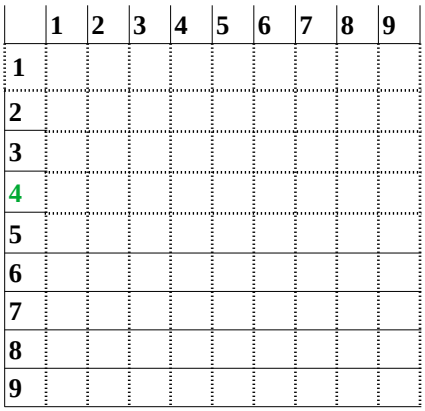
For row = 5



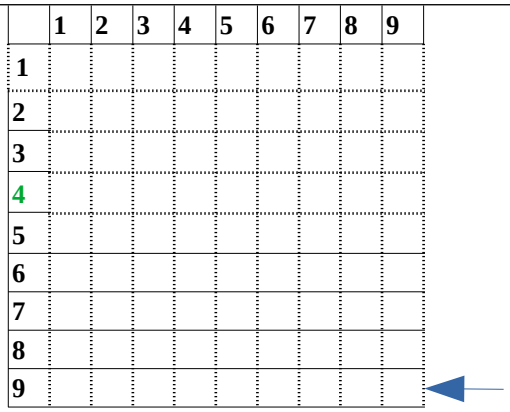


For row = 6

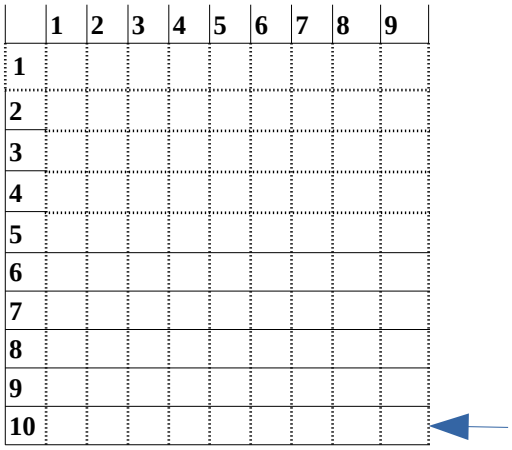
For row = 7



For row = 8



For row = 9



Let's convert steps into C program.

Use of C in-built method	
Move cursor to the next row.	`printf("\n");`
On the computer screen each movement will be a character sized step in a forward direction from left hand side to right hand side in the same row. When cursor moved to the next (row, column) required position, then print space " " character.	`printf(" ");`
On the computer screen each movement will be a character sized step in a forward direction from left hand side to right hand side in the same row. When cursor moved to the next (row, column) required position, then print star "*" character.	`printf("*");`

These programming statements are expected to get repeated many times in the program. Hence we need to use these statements within ***for-loop*** for printing characters after calculating the required (row, column) position. Finding the generalized logical expression using available variables to calculate the (row, column) position for printing logic pattern is the main brainy stuff in such exercises. We can easily confirm our logical expression by observing the output of a pattern. If outputted pattern doesn't match, then the learner can work upon to get the correct output. This way it helps in refining and building the skill of the program's logic.

Column Level Repetitive Action

Let's consider two attributes rows and columns of a 2-dimensional pattern as variables;

row → variable type of integer to store integer value one at a time and represents row number between 1 and 9.

col → variable type of integer to store integer value one at a time and represents column number between 1 and 9.

spaces → variable type of integer to store integer value one at a time and represents space character at column number between 1 and 5.

totalRows → variable type of integer to store integer value for total number of rows which is 5 by default.

(row, col) value pair depicts the position of "*" character on the display screen								
				row=1, col=5				
			row=2, col=4	**row=2**, col=5	**row=2**, col=6			
		row=3, col=3	**row=3**, col=4	**row=3**, col=5	**row=3**, col=6	**row=3**, col=7		
	row=4, col=2	**row=4**, col=3	**row=4**, col=4	**row=4**, col=5	**row=4**, col=6	**row=4**, col=7	**row=4**, col=8	
row=5, col=1	**row=5**, col=2	**row=5**, col=3	**row=5**, col=4	**row=5**, col=5	**row=5**, col=6	**row=5**, col=7	**row=5**, col=8	**row=5**, col=9
	row=6, col=2	**row=6**, col=3	**row=6**, col=4	**row=6**, col=5	**row=6**, col=6	**row=6**, col=7	**row=6**, col=8	
		row=7, col=3	**row=7**, col=4	**row=7**, col=5	**row=7**, col=6	**row=7**, col=7		
			row=8, col=4	**row=8**, col=5	**row=8**, col=6			
				row=9, col=5				

The real challenge is to find the correct (row, col) positions and to print required character on the computer screen by writing a program. This will be carried out by applying ***for-loop*** in combination with appropriate logical conditions. Finding logical conditions in a step-by-step approach stimulates the growth for algorithmic thinking, a fundamental requirement to become a confident programmer. For this logic pattern, we can give (row, column) numeric value pairs as shown in the above table structure. It is quite easy to locate these (row, col) positions manually in the table structure. It is that much equally challenging to find logical conditions. The learner should be able to output this logic pattern in different sizes based on different input values of total number of rows. With all these understandings, we will learn in a systematic way to explore the logic of program.

Approach to solution1

The overall observation is that at each row, either it's printing space characters or star (*) characters in a continues fashion.

The columns in the pattern divided into two parts, either it's printing space " " character(s) or star("*") characters. These comma separated values given in the table basically represents the column positions where space and star character(s) are placed.

Logic Trace Table					
Row level Repetitive Action					**Expected Output**
Variable: row	**Column level Repetitive Action**				
	Variable : spaces	Variable: col	Character count	Total Characters	
1	1,2,3,4	5	4 Spaces + 1 star	5	*
2	1,2,3	4, 5,6	3 Spaces + 3 stars	6	* * *
3	1,2	3, 4, 5,6,7	2 Spaces + 5 stars	7	* * * * *
4	1	2, 3, 4, 5,6,7,8,	1 Space + 7 stars	8	* * * * * * *
5	0 (no space)	1, 2, 3, 4, 5,6,7,8,9	0 Space + 9 stars	9	* * * * * * * * *
6	1	2, 3, 4, 5,6,7,8,	1 Space + 7 stars	8	* * * * * * *
7	1,2	3, 4, 5,6,7	2 Spaces + 5 stars	7	* * * * *
8	1,2,3	4, 5,6	3 Spaces + 3 stars	6	* * *
9	1,2,3,4	5	4 Spaces + 1 star	5	*

The printing of space(s) **comes first** before print star("*") character(s).If we take care of the condition till it print space character(s),then the remaining are the star("*") character(s). If we closely observe It's cutting the values of variable col into two halves. One part goes to printing of space character(s) and the other goes to printing of star character(s).

We know that the comma separated values basically represents the column positions that uniquely identifies the position of space and star character(s).

Let's take out the minimum value (**min_val)** and maximum value(**max_val)** from these comma separated values which were given for printing space and star character(s).

From the numeric pattern of minimum value (**min_val)** and maximum value(**max_val)** we need to derive the generalised expression.

Those generalised expressions will help us to define the ***for-loop*** at the row and the column level.

Row level Repetitive Action					**Expected Output**
Variable : row	**Column level Repetitive Action**				
	Variable : col				
	Printing space character(s)		Printing star character(s)		
	min_val	**max_val**	**min_val**	**max_val**	
1	1	4	5	5	*
2	1	3	4	6	* * *
3	1	2	3	7	* * * * *
4	1	1	2	8	* * * * * * *
5	0	0	1	9	* * * * * * * * *
6	1	1	2	8	* * * * * * *
7	1	2	3	7	* * * * *
8	1	3	4	6	* * *
5 + 4	1	4	5	5	*

We know that the total number of rows is given by the variable **totalRows** and has value 5 by default. So **totalRows** = 5.

The variables **row** and **col** both takes the value 1 to 9, i.e., row =1,2,3,4,5,..,9 and col=1,2,3,4,5,..,9.

Let's focus our attention for printing of star character(s) and try to find out the expression in terms of the variable **totalRows** = 5.

Row level Repetitive Action					**Expected Output**
Variable : row	**Column level Repetitive Action**				
	Variable : col				
	Printing star character(s)				
	min_val		**max_val**		
	default	**1**	**default**	**1**	
1	5	5-0	5	5+0	*
2	4	5-1	6	5+1	* * *
3	3	5-2	7	5+2	* * * * *
4	2	5-3	8	5+3	* * * * * * *
5	1	5-4	9	5+4	* * * * * * * * *
6	2	5-3	8	5+3	* * * * * * *
7	3	5-2	7	5+2	* * * * *
8	4	5-1	6	5+1	* * *
5 + 5 -1	5	5-0	5	5+0	*

Part of **min_val** and **max_val** expression both follows the same numeric pattern. We can consider this as a separate integer variable as count.

Then the logical expression can be expressed in terms of **totalRows** and count.

Row level Repetitive Action					
Variable : row	Variable : count	**Column level Repetitive Action**			
		Variable : col			
		Printing star character(s)			
		min_val		**max_val**	
		default	**2**	**default**	**2**
1	0	5	totalRows -count	5	totalRows + count
2	1	4	totalRows -count	6	totalRows + count
3	2	3	totalRows -count	7	totalRows + count
4	3	2	totalRows -count	8	totalRows + count
5	4	1	totalRows -count	9	totalRows + count
6	3	2	totalRows -count	8	totalRows + count
7	2	3	totalRows -count	7	totalRows + count
8	1	4	totalRows -count	6	totalRows + count
2*totalRows - 1	0	5	totalRows -count	5	totalRows + count

The variable count starts increases by value 1 from the start of the row=1 till the row=**totalRows** (**total number of rows**) and then starts decreases further by value 1 till (**totalRows** – 1) times.

We can deal this using if-else conditional statement.

The condition (col >=**(totalRows – count**)) gives starting point from which ***for-loop*** will start printing star("*") character(s) till the maximum value of variable col which is **(totalRows + count**) for every value of row. The **if-condition** can be given using the comparison and logical operators for the range of values between (**totalRows – count**) and **(totalRows + count**) as:

If (col >= (**totalRows – count**) && col <= **(totalRows + count))**

We already know that;

variable col = 1,2,3,4,5 or 1,2,3,4, **…,(totalRows + count)**

Same expression

variable row = 1,2,3,….,9 or 1,2,3,….,(2 * **totalRows - 1)**

We can eliminate the logical AND(&&) condition(col <= **(totalRows + count))** in the **if-condition.** This logical AND(&&) condition already been taken care in the **loop-condition** of the column level inner ***for-loop***. So the if-condition will now reduce to the following:

If (col >= **(totalRows – count)**)

We can generalised the non-repeating numeric pattern of the column **max_val** into an expressions.

We can now define column level ***for-loop***.

Elements of* Column level *for-loop	
min_val	1
max_val	**(totalRows + count)**
variable_name	**col**
statement(s)	`printf("*");`

Row Level Repetitive Action

The following statements had been repeated 9 times(total number of rows).

Column level *for-loop* for printing star characters**//** ***inner for-loop***

```
printf("\n" ); // Move cursor to the next row
```

We have assigned unique value to each row starting from 1 till 9(**2*totalRows – 1)**.

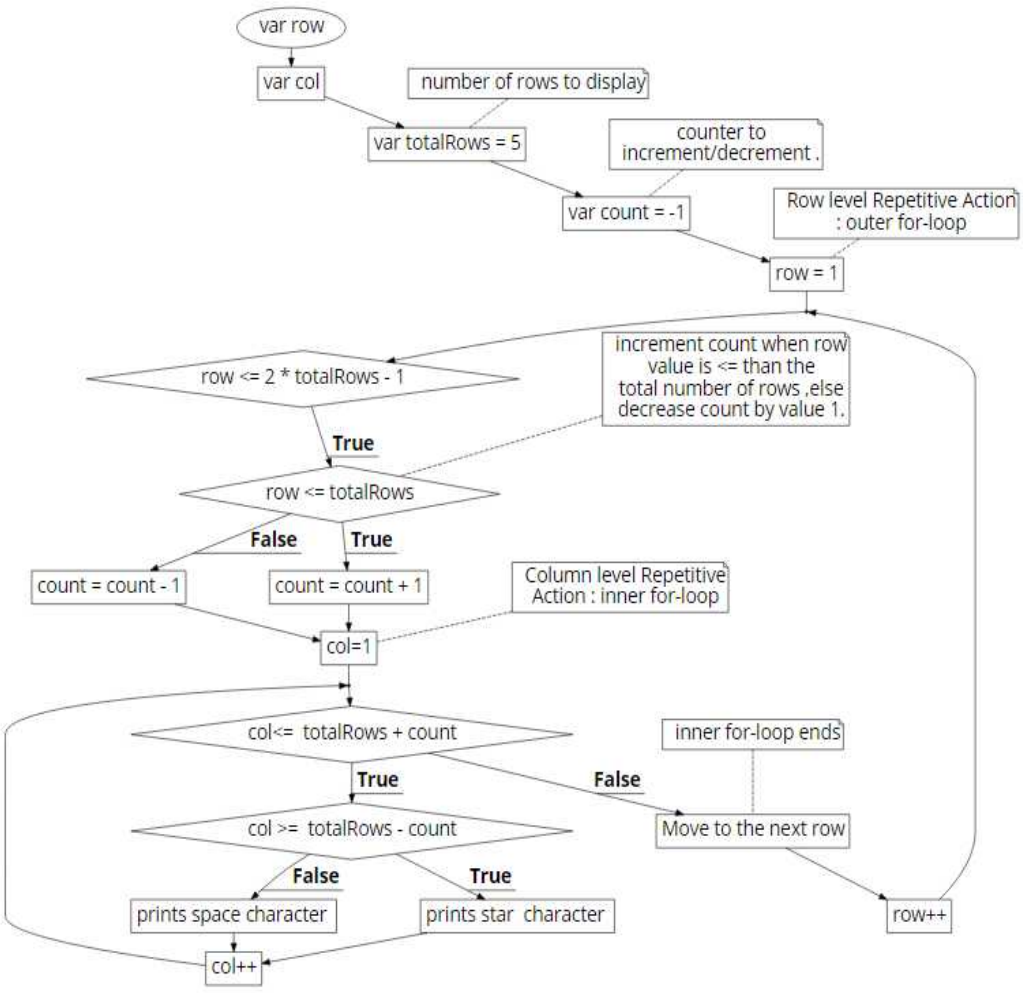
Let's consider;

row → variable type of integer to store integer value one at a time and represents row number between 1 and 9. The **totalRows** has initially assigned the value 5.

We got everything related to ***for-loop*** for row level repetition.

Elements Of **Row Level decrementing *for-loop***	
min_val	1
max_val	**2*totalRows - 1**
variable_name	**row**
statement(s)	**Column level *for-loop*** for printing star characters `printf("\n" );//Move cursor to the next row`

We can summarise it all in a flow-chart as;



Let’s execute the following program.

PatternH1.c

```c
#include <stdio.h>
int main() {

	int totalRows = 7;// number of rows to display
	int count = -1;// counter to increment/decrement .

	// Row level Repetitive Action :
	// Action1.Executes Column level Repetitive Action
	// Action2.Move cursor to next row.
	for (int row = 1; row <= (2 * totalRows - 1); row++) {
		// increment count when row value is <= than the
		// total number of rows
		// else decrease count by value 1.
		if (row <= totalRows) {
			count = count + 1;
		} else {
			count = count - 1;
		}

		// Column level Repetitive Action :

		// Action1.Move cursor in the same row
		// Action2.prints star “* ” character
		for (int col = 1; col <= (totalRows + count); col++) {
			if (col >= (totalRows - count)) {
				printf("* ");
			} else {
				printf("  ");
			}

		}// inner for-loop
		printf("\n");// Move cursor to next row.
	}// outer for-loop
	return 0;
}
```

Output

```
            *
          * * *
        * * * * *
      * * * * * * *
    * * * * * * * * *
  * * * * * * * * * * *
* * * * * * * * * * * * *
  * * * * * * * * * * *
    * * * * * * * * *
      * * * * * * *
        * * * * *
          * * *
            *
```

Problem* H1**.Continuation to previous program (PatternH1.c***), modify the program for the following situation.

In program (***PatternH1.c***), the count variable started with the value -1 given as :

```
int count = -1;// counter to increment/decrement
```

Required to match the output (**Pattern1H*)*** of the modified program (***PatternH1_2.c***) by making the following change:

```
int count = 0;// counter to increment/decrement
```

This time program has to initialise the variable count with the value `0` instead of -1 and yet still it should give the correct matching output as that of pattern program (**Pattern1H.c)**.

PatternH1_2.c

```
#include <stdio.h>
int main() {
            int totalRows = 7;// number of rows to display
            int count = 0;// counter to increment/decrement .

            // Row level Repetitive Action :
            // Action1.Executes Column level Repetitive Action
            // Action2.Move cursor to next row.

            // Write your logic

      return 0;
}
```

Output

```
            *
          * * *
        * * * * *
      * * * * * * *
    * * * * * * * * *
  * * * * * * * * * * *
* * * * * * * * * * * * *
  * * * * * * * * * * *
    * * * * * * * * *
      * * * * * * *
        * * * * *
          * * *
            *
```

Alternative solution2

Let's start with the one of the logic trace table that we got from the previous solution.

Row level Repetitive Action					**Expected Output**
Variable : row	**Column level Repetitive Action**				
	Variable : col				
	Printing space character(s)		Printing star character(s)		
	min_val	**max_val**	**min_val**	**max_val**	
1	1	4	5	5	*
2	1	3	4	6	* * *
3	1	2	3	7	* * * * *
4	1	1	2	8	* * * * * * *
5	0	0	1	9	* * * * * * * * *
6	1	1	2	8	* * * * * * *
7	1	2	3	7	* * * * *
8	1	3	4	6	* * *
9	1	4	5	5	*

Let's focus our attention for printing of star character(s) and try to find out the expression in terms of the variables i.e. row & **totalRows**=5.

Row level Repetitive Action						
Variable : row	**Column level Repetitive Action**					
	Variable : col					
	Printing star character(s)					
	min_val			**max_val**		
	default	**1**	**2**	**default**	**1**	**2**
1	5	5-0	totalRows +1 - 1	5	totalRows +0	totalRows +1 - 1
2	4	5-1	totalRows +1 - 2	6	totalRows +1	totalRows +2 - 1
3	3	5-2	totalRows +1 - 3	7	totalRows +2	totalRows +3 - 1
4	2	5-3	totalRows +1 - 4	8	totalRows +3	totalRows +4 - 1
5	1	5-4	totalRows +1 - 5	9	totalRows +4	totalRows +5 - 1
6	2	2	2	8	totalRows +3	totalRows +5 - 2
7	3	3	3	7	totalRows +2	totalRows +5 - 3
8	4	4	4	6	totalRows +1	totalRows +5 - 4
9	5	5	5	5	totalRows +0	totalRows +5 - 5

We can use row=1,2,3,4,5 values to partially express the columns **min_val** and **max_val.**

Row level Repetitive Action					
Variable : row	**Column level Repetitive Action**				
	Variable : col				
	Printing star character(s)				
	min_val		**max_val**		
		3		**3**	**4**
1	5	totalRows +1 - row	5	totalRows + row - 1	totalRows + row - 1
2	4	totalRows +1 - row	6	totalRows + row - 1	totalRows + row - 1
3	3	totalRows +1 - row	7	totalRows + row - 1	totalRows + row - 1
4	2	totalRows +1 - row	8	totalRows + row - 1	totalRows + row - 1
5	1	totalRows +1 - row	9	totalRows + row - 1	totalRows + row - 1
6	2	2	8	totalRows +5 - 2	2*totalRows - 2
7	3	3	7	totalRows +5 - 3	2*totalRows - 3
8	4	4	6	totalRows +5 - 4	2*totalRows - 4
9	5	5	5	totalRows +5 - 5	2*totalRows - 5

Variable : row	**Column level Repetitive Action**				
	Variable : col				
	Printing star character(s)				
	min_val		**max_val**		
	0	**3**	**0**	**4**	**5**
1	5	totalRows +1 - row	5	totalRows + row - 1	totalRows + row - 1
2	4	totalRows +1 - row	6	totalRows + row - 1	totalRows + row - 1
3	3	totalRows +1 - row	7	totalRows + row - 1	totalRows + row - 1
4	2	totalRows +1 - row	8	totalRows + row - 1	totalRows + row - 1
5	1	totalRows +1 - row	9	totalRows + row - 1	totalRows + row - 1
6	2	(6 – 4)	8	2*totalRows - (6 – 4)	2*totalRows - 6 + 4)
7	3	(7 – 4)	7	2*totalRows - (7 – 4)	2*totalRows - 7 + 4)
8	4	(8 – 4)	6	2*totalRows - (8 – 4)	2*totalRows - 8 + 4)
9	5	(9 – 4)	5	2*totalRows - (9 – 4)	2*totalRows - 9 + 4)

Variable : row	**Column level Repetitive Action**			
	Variable : col			
	Printing star character(s)			
	min_val		**max_val**	
	default	3	default	6
1	5	totalRows +1 - row	5	totalRows + row - 1
2	4	totalRows +1 - row	6	totalRows + row - 1
3	3	totalRows +1 - row	7	totalRows + row - 1
4	2	totalRows +1 - row	8	totalRows + row - 1
5	1	totalRows +1 - row	9	totalRows + row - 1
6	2	(row – 4)	8	2*totalRows - row + 4)
7	3	(row – 4)	7	2*totalRows - row + 4)
8	4	(row – 4)	6	2*totalRows - row + 4)
9	5	(row – 4)	5	2*totalRows - row + 4)

If we express value 4 as (totalRows - 1) ,then it will be more generalised.

row	**Column level Repetitive Action**			
	col			
	Printing star character(s)			
	min_val		**max_val**	
	default	4	default	7
1	5	totalRows +1 - row	5	totalRows + row - 1
2	4	totalRows +1 - row	6	totalRows + row - 1
3	3	totalRows +1 - row	7	totalRows + row - 1
4	2	totalRows +1 - row	8	totalRows + row - 1
5	1	totalRows +1 - row	9	totalRows + row - 1
6	2	row – (totalRows - 1)	8	2*totalRows - row + totalRows - 1)
7	3	row – (totalRows - 1)	7	2*totalRows - row + totalRows - 1)
8	4	row – (totalRows - 1)	6	2*totalRows - row + totalRows - 1)
9	5	row – (totalRows - 1)	5	2*totalRows - row + totalRows - 1)

row	**Column level Repetitive Action**			
	col			
	Printing star character(s)			
	min_val		**max_val**	
	default	3	default	6
1	5	totalRows +1 - row	5	totalRows + row - 1
2	4	totalRows +1 - row	6	totalRows + row - 1
3	3	totalRows +1 - row	7	totalRows + row - 1
4	2	totalRows +1 - row	8	totalRows + row - 1
5	1	totalRows +1 - row	9	totalRows + row - 1
6	2	row – (totalRows - 1)	8	2*totalRows - row + totalRows - 1)
7	3	row – (totalRows - 1)	7	2*totalRows - row + totalRows - 1)
8	4	row – (totalRows - 1)	6	2*totalRows - row + totalRows - 1)
9	5	row – (totalRows - 1)	5	2*totalRows - row + totalRows - 1)

row	**Column level Repetitive Action**			
	Variable : col			
	Printing star character(s)			
	min_val		**max_val**	
	default	**4**	**default**	**7**
1	5	totalRows +1 - row	5	totalRows + row - 1
2	4	totalRows +1 - row	6	totalRows + row - 1
3	3	totalRows +1 - row	7	totalRows + row - 1
4	2	totalRows +1 - row	8	totalRows + row - 1
5	1	totalRows +1 - row	9	totalRows + row - 1
6	2	row – totalRows + 1	8	3*totalRows - row – 1
7	3	row – totalRows + 1	7	3*totalRows - row – 1
8	4	row – totalRows + 1	6	3*totalRows - row – 1
9	5	row – totalRows + 1	5	3*totalRows - row – 1

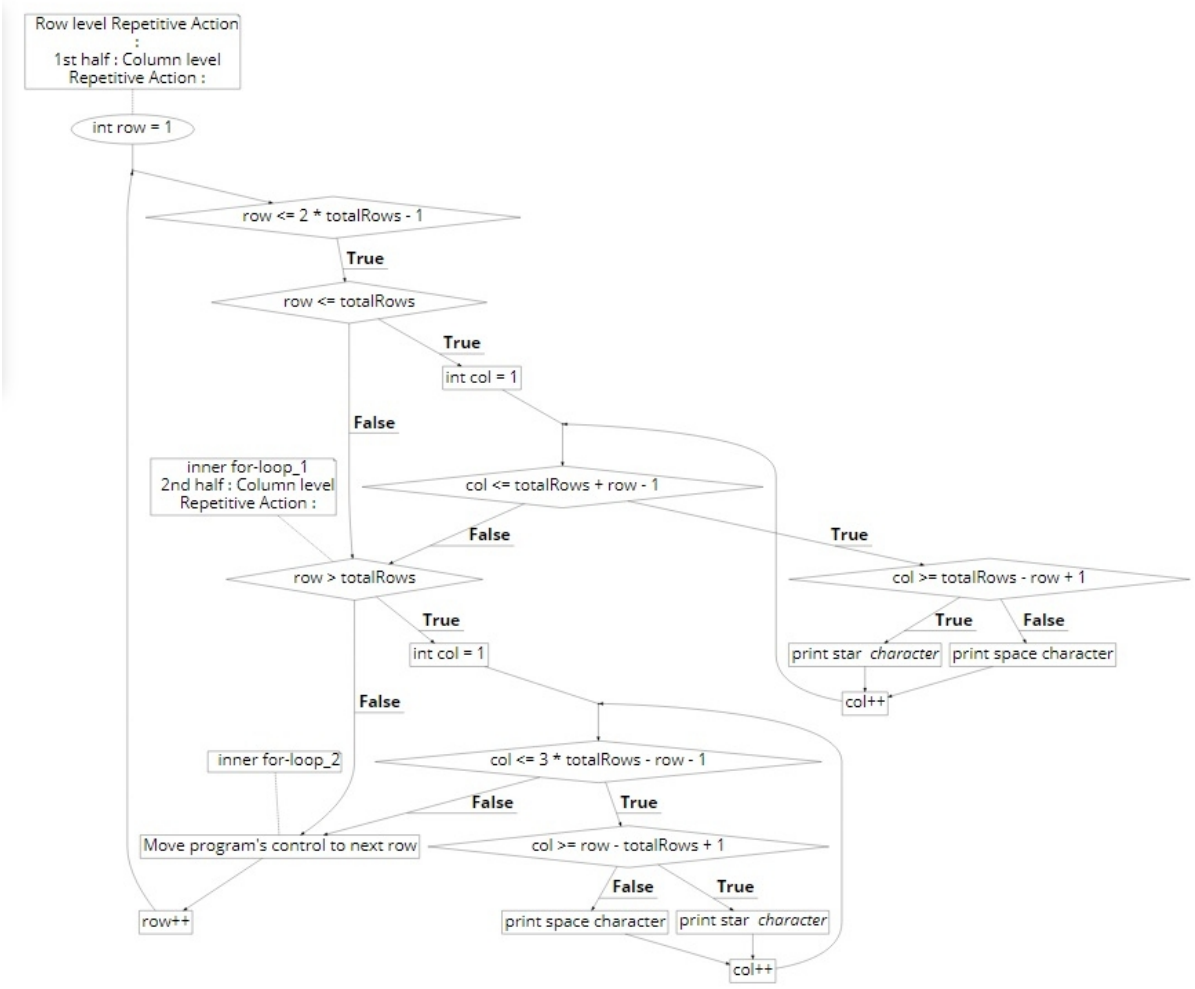
Usually it has been observed that usually we get one minimum consistent value and a logical expression for maximum value in an incrementing loop.

In this case, we got two set of logical expressions for minimum values and maximum values. Minimum value is the position from where for-loop at the column level starts printing * character and ends at the maximum value given by a logical expression.

Since two set of generalised expressions, so we need two set of column level ***for-loop.***

	min_val	**max_val**
Logical expression (set1)	totalRows +1 - row	totalRows + row - 1
Logical expression (set 2)	row – totalRows + 1	3*totalRows - row – 1

The following given flow-chart easily shows our logical expressions consideration.



Let's execute the following program.

PatternH2_1.c

```c
#include <stdio.h>
int main() {

	int totalRows = 4; //number of rows to display

	// Row level Repetitive Action :
	// Executes Repetitive Action at Column level and
	// then Move control to    next row.
	for (int row = 1; row <= (2 * totalRows - 1); row++) {
		//1st half : Column level Repetitive Action :
		if (row <= totalRows) {
			for (int col = 1; col <= (totalRows + row - 1); col++) {
				if (col >= (totalRows - row + 1)) {
					printf("* ");
				} else {
					printf("  ");
				}
			}//inner for-loop_1
		}
		// 2nd half : Column level Repetitive Action :
		if (row > totalRows) {
			for (int col = 1; col <= (3 * totalRows - row - 1); col++) {
				if (col >= (row - totalRows + 1)) {
					printf("* ");
				} else {
					printf("  ");
				}
			}//inner for-loop_2
		}
		printf("\n");// Move cursor to next row.
	}// outer for-loop

	return 0;
}
```

Output

```
      *
    * * *
  * * * * *
* * * * * * *
  * * * * *
    * * *
      *
```

Let's try to simplify program further by simplifying the **if-conditions** and **loop-condition** expressions.

For example, let's analyse the following expression when **col** is a variable which **starts from value 0.**

When variable col=5,6,…., will have the values, which means the logical condition to cover these values can be expressed as **col >= (4 + 1) or col > 4**, both means the same.

Similarly,

The variable **col=0,1,2,3 when contains the values**, means col <= (4 – 1) or col < 4 ,both means the same.

In the same way the **if-conditions** and **loop-condition** expressions can be simplified.

The **if-condition** gives the minimum limit, a starting point for which program will start printing star("*") character(s) till the maximum value of variable col given by the **loop-condition.**

Let's consider;

colMin → variable type of integer to store integer value of column , a starting point from which program will start printing star("*") character.

colMax → variable type of integer to store integer value of column , a maximum value till program will stop printing star("*") character(s).

Variable : row	**if-conditions**	
	Variable : colMin	
	default	**1**
`row <= totalRows`	`col >= ( totalRows - row + 1 )`	`col > ( totalRows - row )`
`row >totalRows`	`col >= ( row - totalRows + 1 )`	`col > ( row - totalRows )`

Variable : row	**loop-conditions**	
	Variable : colMax	
	default	**1**
`row <= totalRows`	`col<=( totalRows + row - 1 )`	`col<( totalRows + row )`
`row >totalRows`	`col<=(3* totalRows - row - 1)`	`col<( 3 * totalRows - row )`

Using variables **colMin & colMax** we can merge the two inner-loops by dynamically assigning these expression to these variables.

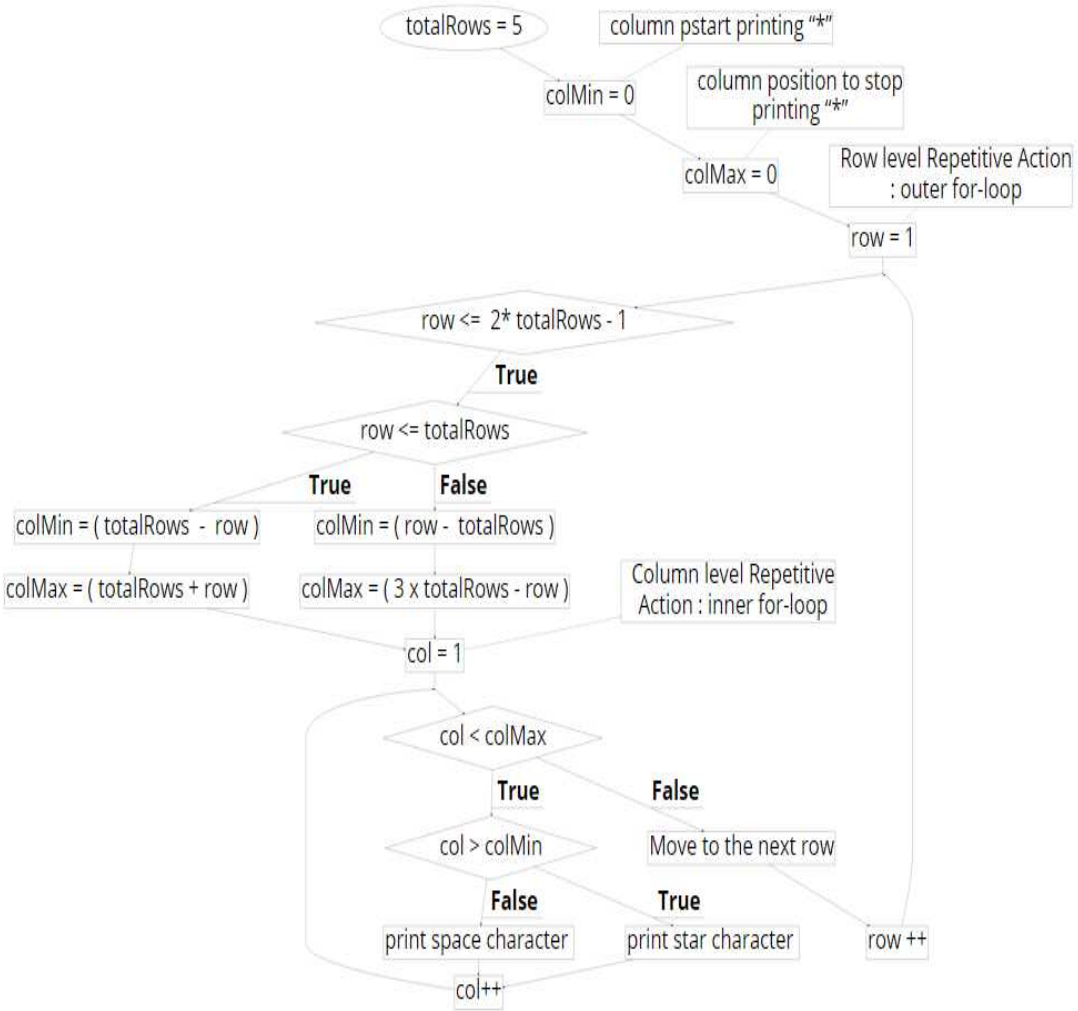
We can use **if-else** statement to update the variables **colMin & colMax** dynamically which are as follows.

```
if ( row <= totalRows ){
        colMin = ( totalRows  -  row );
        colMax = ( totalRows + row );
}else{
        colMin = ( row -  totalRows );
        colMax = ( 3 * totalRows - row );
}
```

Further, we can use variables **colMin & colMax** in inner ***for-loop*.**

In this way, we can try merging <u>two inner for-loops into **single inner *for-loop***</u>.

The following flow-chart gives the overall idea how program following this should get updated as per our previous discussion.



So, let's update our previous program with these changes and executes the program.

PatternH2_2.c

```c
#include <stdio.h>
int main() {

	int totalRows = 5; //number of rows to display
	int colMin = 0;// column position to start printing “*”
	int colMax = 0;// column position to stop printing “*”

	// Row level Repetitive Action :
	// Action1.Executes Column level Repetitive Action
	// Action2.Move cursor to next row.
	for (int row = 1; row <= (2 * totalRows - 1); row++) {

		if (row <= totalRows) {
			colMin = (totalRows - row);
			colMax = (totalRows + row);
		} else {
			colMin = (row - totalRows);
			colMax = (3 * totalRows - row);
		}

		//Column level Repetitive Action :
		for (int col = 1; col < colMax; col++) {
			if (col > colMin) {
				printf("* ");
			} else {
				printf("  ");
			}
		}//inner for-loop

		printf("\n");// Move cursor to next row.

	}// outer for-loop

	return 0;
}
```

Output

```
        *
      * * *
    * * * * *
  * * * * * * *
* * * * * * * * *
  * * * * * * *
    * * * * *
      * * *
        *
```

Now, test for different number of lines by changing value to 7 for the variable `totalRows` in the same program (**PatternH2_2.c**).

PatternH2_2.c

```
#include <stdio.h>
int main() {

	int totalRows = 7; //number of rows to display
	int colMin = 0;// column position to start printing “*”
	int colMax = 0;// column position to stop printing “*”

	// Row level Repetitive Action :
	// Action1.Executes Column level Repetitive Action
	// Action2.Move cursor to next row.
	for (int row = 1; row <= (2 * totalRows - 1); row++) {

		if (row <= totalRows) {
			colMin = (totalRows - row);
			colMax = (totalRows + row);
		} else {
			colMin = (row - totalRows);
			colMax = (3 * totalRows - row);
		}

		//Column level Repetitive Action :
		for (int col = 1; col < colMax; col++) {
			if (col > colMin) {
				printf("* ");
			} else {
				printf("  ");
			}
		}//inner for-loop

		printf("\n");// Move cursor to next row.

	}// outer for-loop

	return 0;
}
```

Output

```
            *
          * * *
        * * * * *
      * * * * * * *
    * * * * * * * * *
  * * * * * * * * * * *
* * * * * * * * * * * * *
  * * * * * * * * * * *
    * * * * * * * * *
      * * * * * * *
        * * * * *
          * * *
            *
```

Alternative solution3

The variables are row & totalRows=5.

Let's start with the logic trace table that we got from the previous solution1.

The variable spaces **representing the printing of space characters considered before printing of star(*) characters.**

The variable **col representing the printing of star(*) characters**

Row level Repetitive Action					**Expected Output**
Variable: row	**Column level Repetitive Action**				
	Variable : spaces	Variable: col	Character count	Total Characters	
1	1,2,3,4	5	4 Spaces + 1 star	5	*
2	1,2,3	4, 5,6	3 Spaces + 3 stars	6	* * *
3	1,2	3, 4, 5,6,7	2 Spaces + 5 stars	7	* * * * *
4	1	2, 3, 4, 5,6,7,8,	1 Spaces + 7 stars	8	* * * * * * *
5	0(nospace)	1, 2, 3, 4, 5,6,7,8,9	0 Spaces + 9 stars	9	* * * * * * * * *
6	1	2, 3, 4, 5,6,7,8,	1 Spaces + 7 stars	8	* * * * * * *
7	1,2	3, 4, 5,6,7	2 Spaces + 5 stars	7	* * * * *
8	1,2,3	4, 5,6	3 Spaces + 3 stars	6	* * *
9	1,2,3,4	5	4 Spaces + 1 star	5	*

The printing of space(s) **comes first** before print star("*") character(s). If we closely observe the total character count is divided into two halves. One part goes to printing of space character(s) and the other goes to printing of star character(s) as shown in the column **Character count.**

Let's take out the minimum value (**min_val)** and maximum value(**max_val)** from the column **character count** which gives the count for printing space and star character(s).From the numeric pattern of minimum value (**min_val)** and maximum value(**max_val)** we need to derive the generalised expression. Those generalised expressions will help us to define the ***for-loop*** at the row and the column level.

At the column level two inner for-loops are required one for printing space character(s) and other for printing star character(s).

Hence the following trace table can be drawn.

Row level Repetitive Action : totalRows=5					**Expected Output**
Variable : row	**Column level Repetitive Action**				
	Variable : col				
	space character(s) count		star character(s) count		
	min_val	**max_val**	**min_val**	**max_val**	
1	1	4	1	1	*
2	1	3	1	3	* * *
3	1	2	1	5	* * * * *
4	1	1	1	7	* * * * * * *
5	0	0	1	9	* * * * * * * * *
6	1	1	1	7	* * * * * * *
7	1	2	1	5	* * * * *
8	1	3	1	3	* * *
9	1	4	1	1	*

- The column **max_val** for space character(s) count started at value 4 (**totalRows** – 1) and then decreases by value 1 consistently till reached 0 at the row=5(total number of rows).

  Then it start increases by value 1 consistently till reached maximum row value 9(2*5 – 1) or (2*totalRows – 1) ,since totalRows=5.

- The column **max_val** for star character(s) count started at value 1 and increases by value 2 consistently till reached 9 at the row=5(total number of rows). Then it start decreases by value 2 consistently till **maximum row value 9 or (2*5 – 1) or (2*totalRows – 1)** ,since totalRows=5.

Row level Repetitive Action : totalRows=5							**Expected Output**
Variable : row	**Column level Repetitive Action**						
	Variable : col						
	space character(s) count			star character(s) count			
	min_val	**max_val**		**min_val**	**max_val**		
			1			**1**	
1	1	4	totalRows - 1	1	1	1	*
2	1	3	4 - 1	1	3	1 + 2	* * *
3	1	2	3 - 1	1	5	3 + 2	* * * * *
4	1	1	2 - 1	1	7	5 + 2	* * * * * * *
5	0	0	1 - 1	1	9	7 + 2	* * * * * * * * *
6	1	1	0 + 1	1	7	9 - 2	* * * * * * *
7	1	2	1 + 1	1	5	7 - 2	* * * * *
8	1	3	2 + 1	1	3	5 - 2	* * *
9	1	4	3 + 1	1	1	3 - 2	*

Let's consider;

spaceMax → variable type of integer to store integer value one at a time and starts at the value (**totalRows** – 1).

starMax → variable type of integer to store integer value one at a time and starts at the value 1.

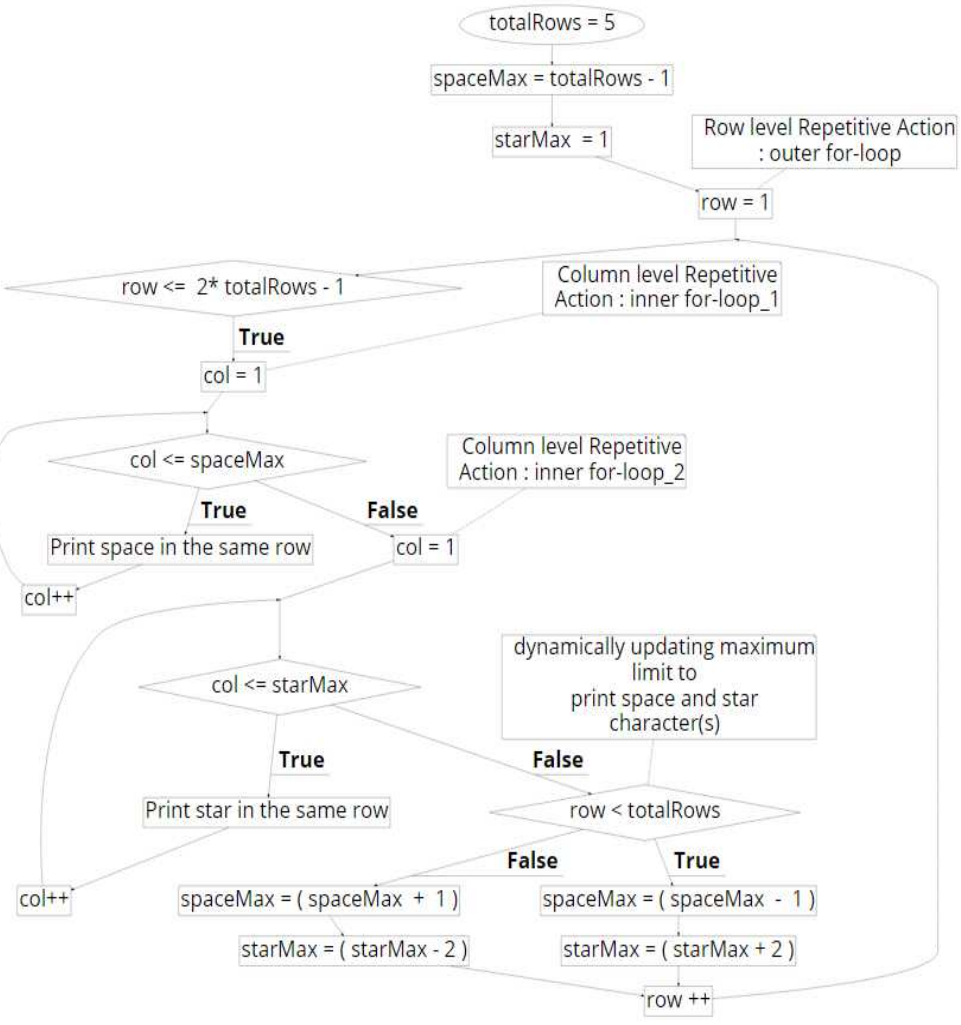
Using variables **spaceMax & starCount** we can use two inner-loops by dynamically assigning maximum limit to print space and star character(s) respectively by these variables.

Further, we can use variables **spaceMax & starMax** in inner ***for-loops*.**

In this way, we can use two inner for-loops for **printing single row of space and star character(s)** which are as follows.

`row < totalRows`	`row >= totalRows`
spaceMax = `( spaceMax - 1 )` **starMax =** `( starMax + 2 )`	**spaceMax =** `( spaceMax + 1 )` **starMax =** `( starMax - 2 )`

We can use **if-else** statement to dynamically update these variables, **spaceMax & starMax.**
We can overall idea with the help of following flow-chart.



Let’s execute the following program.

PatternH3.c

```
#include <stdio.h>
int main() {

	int totalRows = 5; //number of rows to display
	int spaceMax = totalRows - 1;// maximum count to print space “  ”
	int starMax = 1;// maximum count to print star “* ”

	// Row level Repetitive Action :
	// Action1.Executes Column level Repetitive Action
	// Action2.Move cursor to next row.
	for (int row = 1; row <= (2 * totalRows - 1); row++) {

		//Column level Repetitive Action : Print space in the same row
		for (int col = 1; col <= spaceMax; col++) {
			printf("  ");
		}//inner for-loop_1

		//Column level Repetitive Action : Print star in the same row
		for (int col = 1; col <= starMax; col++) {
			printf("* ");
		}//inner for-loop_2

		printf("\n");// Move cursor to next row.

		if (row < totalRows) {
			spaceMax = (spaceMax - 1);
			starMax = (starMax + 2);
		} else {
			spaceMax = (spaceMax + 1);
			starMax = (starMax - 2);
		}

	}// outer for-loop

	return 0;
}
```

Output

```
        *
      * * *
    * * * * *
  * * * * * * *
* * * * * * * * *
  * * * * * * *
    * * * * *
      * * *
        *
```

Problem H1:

The program (**PatternH3.c**) used the conditional operator < (less than) as shown:

```
if ( row < totalRows ){
        spaceMax = (spaceMax  -  1 );
        starMax = (starMax + 2 );
}else{
        spaceMax = (spaceMax  +  1 );
        starMax = (starMax - 2 );
}
```

Write a program **PatternH4.c** which will use conditional operator <= (less than equals to) instead of conditional operator < (less than) to generate the same output as done by **PatternH3.c** program**.** Make changes to the program (**PatternH3.c**) to generate the pattern **Pattern1H** by using conditional operator <= (less than equals to).

```
if ( row <= totalRows ){
        spaceMax = ( spaceMax  -  1 );
        starMax = ( starMax + 2 );
}else{
        spaceMax = ( spaceMax  +  1 );
        starMax = ( starMax - 2 );
}
```

Test the output for total number of rows = 10 or variable **totalRows=10**.

Problems. Write programs to output the following by making use of logic trace table.

Output1

```
* * * * * * * * * * *
* *               * *
*   *           *   *
*     *       *     *
*       * * *       *
*       * * *       *
*     *       *     *
*   *           *   *
* *               * *
* * * * * * * * * * *
```

Output2

```
* * * *       * * * *
* *               * *
*   *           *   *
*     *       *     *
          * *
          * *
*     *       *     *
*   *           *   *
* *               * *
* * * *       * * * *
```

Output3

```
  * * * * * * *
*   * * * * * *
* *   * * * * *
* * *   * * * *
* * * *   * * *
* * * * *   * *
* * * * * *   *
* * * * * * *
```

Output4

```
*
  *
    *
      *
        *
          *
            *
              *
                *
```

PatternAA

```
                *
              * *
            * * *
          * * * *
        * * * * *
      * * * * * *
    * * * * * * *
  * * * * * * * *
* * * * * * * * *
  * * * * * * * *
    * * * * * * *
      * * * * * *
        * * * * *
          * * * *
            * * *
              * *
                *
```

PatternBB

```
                *
              * *
            * * *
          * * * *
        * * * * *
      * * * * * *
    * * * * * * *
  * * * * * * * *
* * * * * * * * *
                * * * * * * * * * *
                * * * * * * * * *
                * * * * * * * *
                * * * * * * *
                * * * * * *
                * * * * *
                * * * *
                * * *
                * *
                *
```

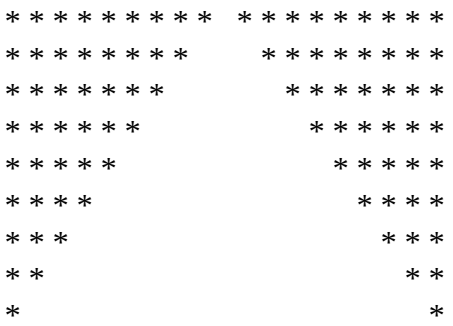
PatternCC

```
* * * * * * * * * * *
* * * * * * * * * *
* * * * * * * * *
* * * * * * *
* * * * * *
* * * * *
* * * *
* * *
* *
*
*
* *
* * *
* * * *
* * * * *
* * * * * *
* * * * * * *
* * * * * * * *
* * * * * * * * *
* * * * * * * * * *
```

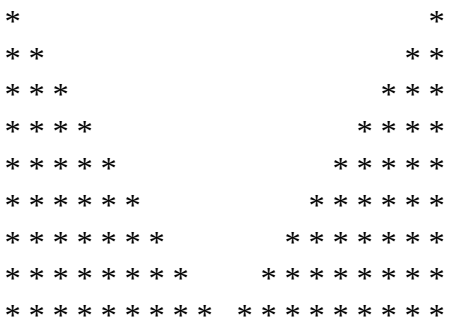
PatternDD

```
* * * * * * * * * * *
  * * * * * * * * * *
    * * * * * * * * *
      * * * * * * * *
        * * * * * * *
          * * * * * *
            * * * * *
              * * * *
                * * *
                  * *
                    *
                    *
                  * *
                * * *
              * * * *
            * * * * *
          * * * * * *
        * * * * * * *
      * * * * * * * *
    * * * * * * * * *
  * * * * * * * * * *
* * * * * * * * * * *
```

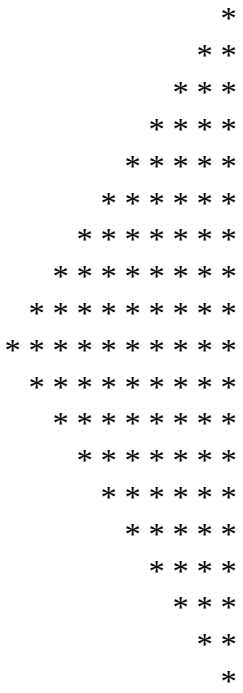
PatternEE



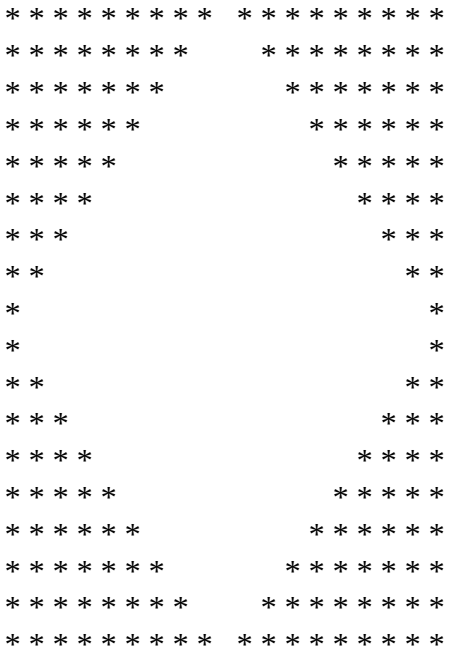
PatternFF



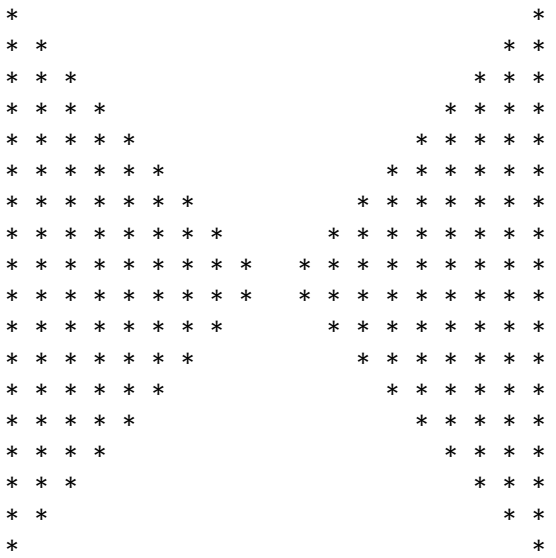
PatternGG



PatternHH



PatternII



ProblemGBSwitchCase: Please apply switch-case statement to generate the following output based on the value of border style values.

Border Style = 1

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
1												0											
2											0		0										
3										0				0									
4									0						0								
5										0				0									
6											0		0										
7												0											
8					0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
9				0						0				0						0			
10			0						0						0						0		
11		0						0								0						0	
12	0	0	0	0	0	0	0										0	0	0	0	0	0	0

Border Style = 2

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
1												0											
2											0	0	0										
3										0	0		0	0									
4									0	0				0	0								
5										0	0		0	0									
6											0	0	0										
7												0											
8					0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
9				0	0	0	0	0	0	0				0	0	0	0	0	0	0			
10			0	0				0	0						0	0				0	0		
11		0	0	0	0	0	0	0								0	0	0	0	0	0	0	
12	0	0	0	0	0	0	0										0	0	0	0	0	0	0

Border Style = 0

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
1												0											
2											0	0	0										
3										0	0	0	0	0									
4									0	0	0	0	0	0	0								
5										0	0	0	0	0									
6											0	0	0										
7												0											
8					0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
9				0	0	0	0	0	0	0				0	0	0	0	0	0	0			
10			0	0	0	0	0	0	0						0	0	0	0	0	0	0		
11		0	0	0	0	0	0	0								0	0	0	0	0	0	0	
12	0	0	0	0	0	0	0										0	0	0	0	0	0	0

Number Based Pattern

1

2 2

3 3 3

4 4 4 4

5 5 5 5 5

NumberPattern1A

Tabular representation.

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	1				
Row=2	2	2			
Row=3	3	3	3		
Row=4	4	4	4	4	
Row=5	5	5	5	5	5

(Row, Col) value pairs representation

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	1,1				
Row=2	2 ,1	2,2			
Row=3	3,1	3,2	3,3		
Row=4	4,1	4,2	4,3	4,3	
Row=5	5,1	5,2	5,3	5,4	5,5

Analysis of output

It can also be interpreted in a tabular form.

row	1				
Col	1				
Output	1				

For row =1, 1<=col<=1

row	2				
Col	1	2			
Output	2	2			

For row =2, 1<=col<=2

row	3				
Col	1	2	3		
Output	3	3	3		

For row =3, 1<=col<=3

row	4				
Col	1	2	3	4	
Output	4	4	4	4	

For row =4, 1<=col<=4

row	5				
Col	1	2	3	4	5
Output	5	5	5	5	5

For row =5, 1<=col<=5

Column Level Repetitive Action

Row level Repetitive Action			**Expected Output**
row	**Column level Repetitive Action**		
	If-condition(lower_limit <=col<= upper_limit)		
	lower_limit	upper_limit	
1	1	1	1
2	1	2	2 2
3	1	3	3 3 3
4	1	4	4 4 4 4
5	1	5	5 5 5 5 5

The *upper_limit* value is based on how many times the value repeated in a particular row.

The *lower_limit* of col variable has the same numeric value expression for every value of row. It's consistent for all the values rows.

Likewise we need to come out with such a consistent expression for upper_limit which should suit any number of rows.

The other remaining variables are row & totalRows=5. Let's try to generalise using these variables.

For all the rows, the row value prints exactly the number of times the upper_limit of col value.

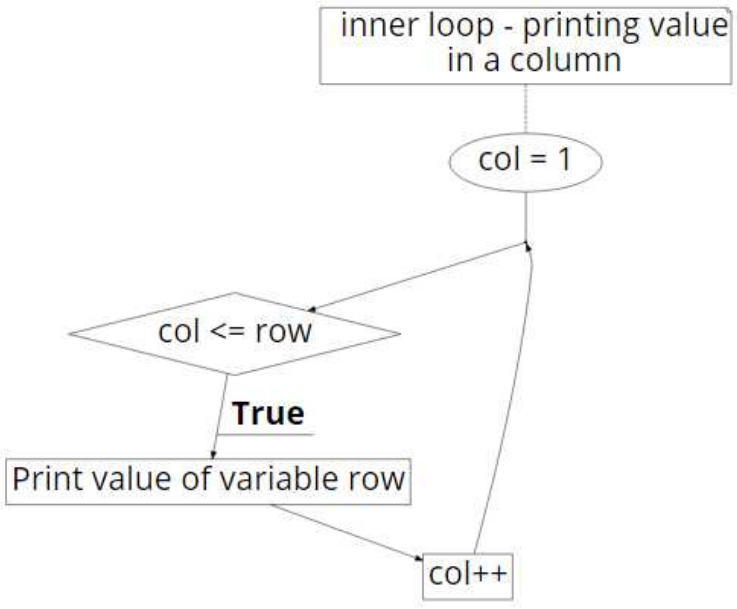
Column level Repetitive Action			
row	*lower_limit*	*upper_limit*	
		default	1
1	1	1	row
2	1	2	row
3	1	3	row
4	1	4	row
5	1	5	row

So the *upper_limit* of col variable is generalised to row variable.

The *lower_limit & upper_limit* are consistent for all the rows. It's now generalised for any number of rows.

We can now define column level ***for-loop*** which is for every row value.

Elements of* Column level *for-loop	
min_val	1
max_val	row
variable_name	**col**
statement(s)	`printf("%d ", row);`



Row Level Repetitive Action

The same following statements had been repeated 5 times(total number of rows).

Column level *for-loop* // *inner for-loop*

```
printf("\n");// Move cursor to the next row
```

The minimum and maximum value can be given for the ***for-loop*** as;

min_val = 1 & *max_val* = 5 or totalRows

Let's consider;

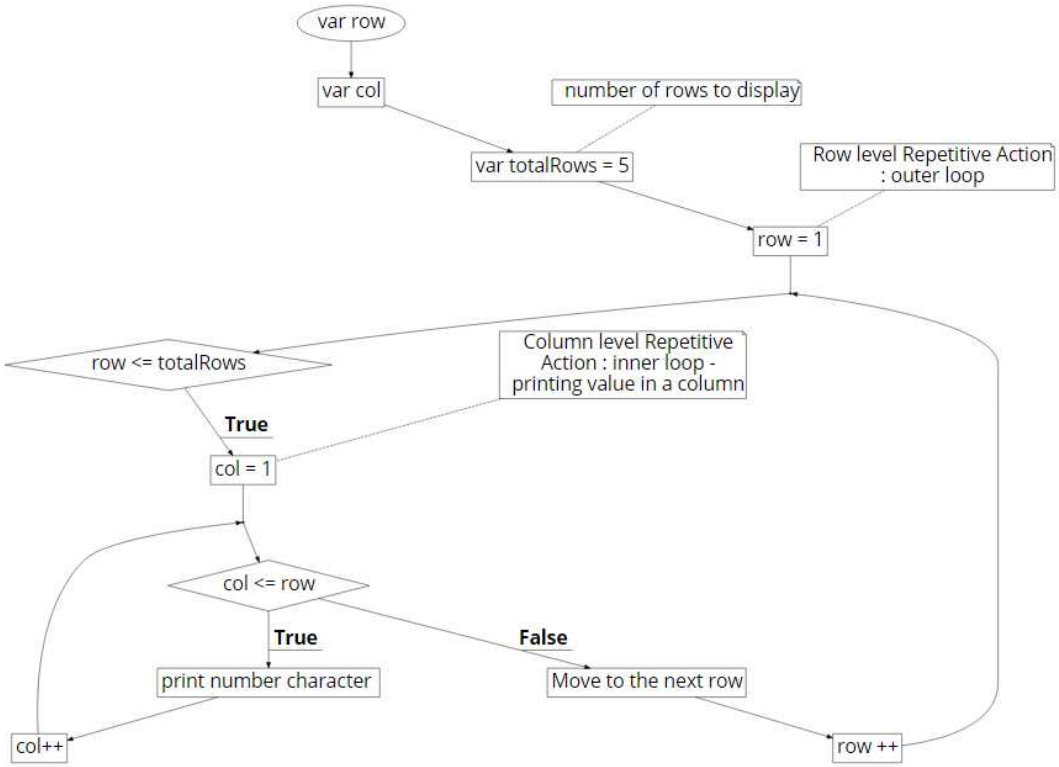
row → variable type of integer to store integer value one at a time and represents row number between 1 and 5(totalRows). The **totalRows** has initially assigned the value 5.

Outer ***for-loop*** starts at row=1 & will repeat till row <= **totalRows**.

Elements Of* Row Level *for-loop	
min_val	1
max_val	totalRows
variable_name	**row**
statement(s)	**Column level *for-loop*** `printf("\n");// Move cursor to the next row`

We got everything related to ***for-loop*** in order to implement row level repetition.

We can summarise all our finding in the following flow-chart.



Let’s execute the following program.

NumberPattern1A.c

```c
#include <stdio.h>
int main() {
	int totalRows = 5; //number of rows to display

	for (int row = 1; row <= totalRows; row++) {

		for (int col = 1; col <= row; col++) {
			// added extra space for clarity in output.
			printf("%d ", row);
		}

		printf("\n");// move to the next line

	}

	return 0;
}
```

Output

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

Now, test for different number of lines, let it be 7. It should work correctly, so that we can confirm that our solution is generalised and fit for any number of line.

NumberPattern1A.c

```c
#include <stdio.h>
int main() {
	int totalRows = 7; //number of rows to display

	for (int row = 1; row <= totalRows; row++) {

		for (int col = 1; col <= row; col++) {

			// added extra space for clarity in output.
			printf("%d ", row);
		}

		printf("\n");// move to the next line

	}

	return 0;
}
```

Output

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
6 6 6 6 6 6
7 7 7 7 7 7 7
```

```
        1
      2 2
    3 3 3
  4 4 4 4
5 5 5 5 5
```

NumberPattern1B

Tabular representation

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	space	space	space	space	1
Row=2	space	space	space	2	2
Row=3	space	space	3	3	3
Row=4	space	4	4	4	4
Row=5	5	5	5	5	5

(Row, Col) value pairs representation.

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1					1,5
Row=2				2,4	2,5
Row=3			3,3	3,4	3,5
Row=4		4,2	4,3	4,4	4,5
Row=5	5,1	5,2	5,3	5,4	5,5

Analysis of output

In this case we need to take care of printing two different types of character at a particular column in a particular row. The character types are the following.

1) Space character

2) Numeric character

Tabular form representation

row	1				
col	1	2	3	4	5
Output	space	space	space	space	1

For row =1, 1 <=spaces<=4, 1>=num>=1

row	2				
col	1	2	3	4	5
Output	space	space	space	2	2

For row =2, 1 <=spaces<=3, 2>=num>=1

row	3				
col	1	2	3	4	5
Output	space	space	3	3	3

For row =3, 1 <=spaces<=2, 3>=num>=1

row	4				
col	1	2	3	4	5
Output	space	4	4	4	4

For row =4, 1 <=spaces<=1, 4>=num>=1

row	5				
col	1	2	3	4	5
Output	5	5	5	5	5

For row =5, 5>=num>=1

Observations

1) There is repetition at row level vertically, i.e., Row = 1 to 5.
2) There is repetition at col level horizontally. , i.e., Col = 1 to 5.
   This gets divided into two parts .One for repetitively printing spaces and remaining column for printing numeric characters.

For Loop	**Lower limit**	**Upper limit**
Outer loop	row=1	row<=5
Inner Loop - spaces	spaces=1	spaces <= ?(Unkown)
Inner Loop - Number	num = ?(Unkown)	num = 1

Column Level Repetitive Action

Row level Repetitive Action					**Expected Output**
row	**Column level Repetitive Action**				
	lower_limit <=spaces<= upper_limit		upper_limit >=num >= lower_limit		
	lower_limit	upper_limit			
1	1	4	1	1	1
2	1	3	2	1	2 2
3	1	2	3	1	3 3 3
4	1	1	4	1	4 4 4 4
5	1	0	5	1	5 5 5 5 5

The lower_limit for spaces & num variable has the same numeric value expression for all the rows.

It's consistent for all the rows. Likewise we need to come out with such a consistent expression for upper_limit for which it should suit any number of rows.

The other remaining variables are row & totalRows=5. Let's try get generalised logical condition using these variables.

Column level Repetitive Action							
row	lower_limit <=spaces<= upper_limit				upper_limit >=num >= lower_limit		
	lower_limit	upper_limit			upper_limit		lower_limit
		default	1	2	default	1	
1	1	4	totalRows - 1	totalRows - row	1	row	1
2	1	3	totalRows - 2	totalRows - row	2	row	1
3	1	2	totalRows - 3	totalRows - row	3	row	1
4	1	1	totalRows - 4	totalRows - row	4	row	1
5	1	0	totalRows - 5	totalRows - row	5	row	1

So the upper_limit of spaces & num variables are generalised.

The lower_limit & upper_limit are consistent for all the rows. It's now generalised for n number of rows.

So our **If-condition**(lower_limit <=spaces<= upper_limit) becomes

1 <= spaces <= (totalRows – row)

The I**f-condition**(upper_limit >=num >= lower_limit) becomes

row >=num >= 1

So it has two repetition process one for printing spaces and second for printing numeric values.

It means it requires **two inner for-loops** (nested for – loops).

For Loop	Lower limit	Upper limit
Outer loop	row=1	row<= totalRows
Inner Loop - spaces	col=1	col <= (totalRows – row)
Inner Loop - Number	num = row	num = 1

We can now define column level ***for-loop*** which is for every row value.

Elements of **Column level** ***for-loop***	
min_val	1
max_val	totalRows -row
variable_name	**col**
statement(s)	`printf( " " );// print spaces`

Elements of **Column level** ***for-loop***	
min_val	1
max_val	row
variable_name	**num**
statement(s)	`printf("%d ", num );`

Row Level Repetitive Action

The same following statements had been repeated 5 times(total number of rows).

Column level *for-loop* for printing space characters// ***inner for-loop***

Column level *for-loop* for printing star characters// ***inner for-loop***

```
printf("\n");// Moves cursor to the next row
```

We can observe from the pattern that the total number of rows are 5.

We have assigned unique value to each row starting from 1 till 5.

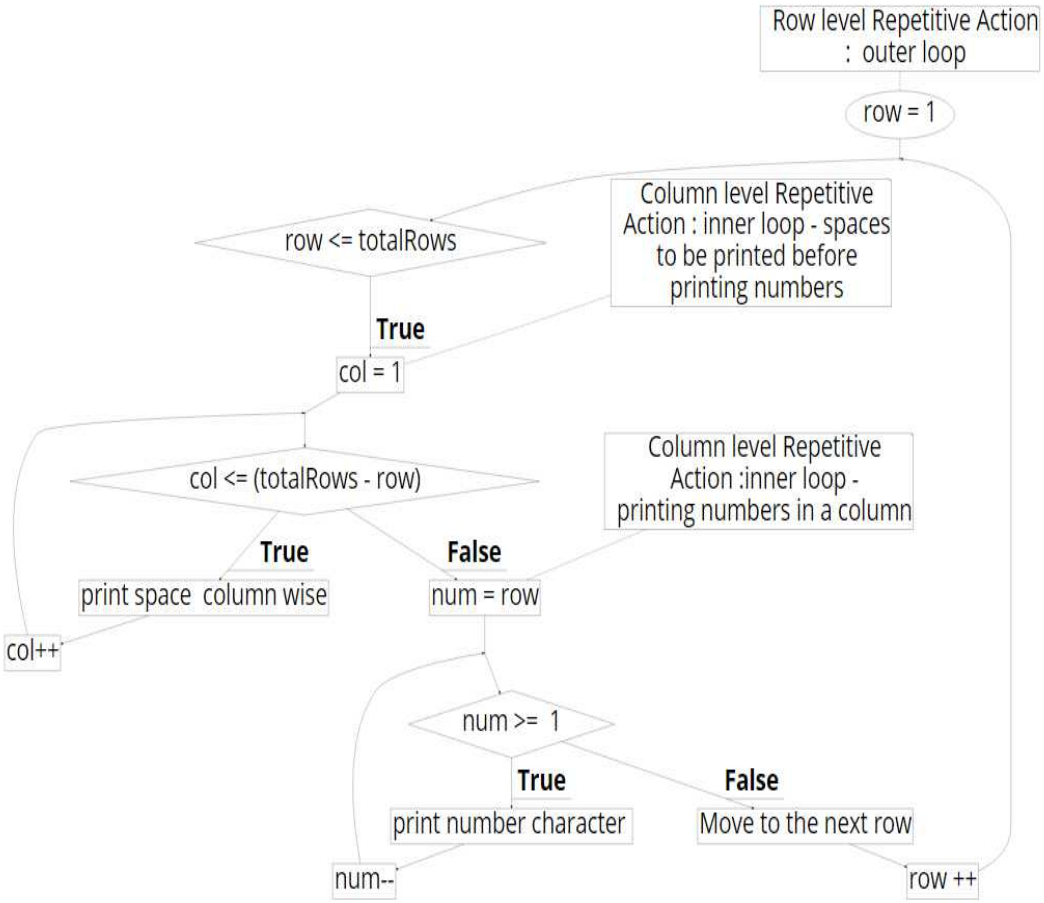
So, the minimum and maximum value can be given for the ***for-loop*** at row level as;

min_val = 1 & *max_val* = 5 or totalRows

Outer ***for-loop*** starts at row=1 & will repeat till row <= **totalRows**.

Elements Of* Row Level *for-loop	
min_val	1
max_val	totalRows
variable_name	**row**
statement(s)	**Column level *for-loop*** for printing space characters. **Column level *for-loop*** for printing star characters `printf("\n");// Move cursor to the next row`

We got everything related to ***for-loop*** required for row level repetition.



Let's execute the following program.

NumberPattern1B.c

```c
#include <stdio.h>
int main() {
	int  totalRows = 5;  //number of rows to display

	for( int row = 1 ; row <= totalRows ; row ++ ) {

		// spaces to be printed before printing numbers
		for(int col = 1 ; col <= (totalRows - row)   ; col++) {
			printf( "  " );
		}

		for(int num = row ; num >=  1 ; num--) {

		// added extra space for clarity in output.
			printf("%d ", num );
		}

		printf("\n");// move control to the next line
	}
	return 0;
}
```

Output

```
        1
      2 2
    3 3 3
  4 4 4 4
5 5 5 5 5
```

Now, test for different number of lines, let it be 7. It should work correctly, so that we can be confirmed that our solution is generalised and fit for any number of lines.

NumberPattern1B.c

```c
#include <stdio.h>
int main() {
	int  totalRows = 7;  //number of rows to display

	for( int row = 1 ; row <= totalRows ; row ++ ) {

		// spaces to be printed before printing numbers
		for(int col = 1 ; col <= (totalRows - row)   ; col++) {
			printf( "  " );
		}

		for(int num = row ; num >=  1 ; num--) {

		// added extra space for clarity in output.
			printf("%d ", num );
		}

		printf("\n");// move control to the next line
	}
	return 0;
}
```

Output

```
          1
        2 2
      3 3 3
    4 4 4 4
  5 5 5 5 5
6 6 6 6 6 6
7 7 7 7 7 7 7
```

Alternative solution

We observed previously that for row=5, the values of col and numeric output are just reversed.

Let's device a solution by decrementing the for-loop for col variable (5 to 1) so the numeric output will match with the value of row variable.

row	5				
col	1	2	3	4	5
output	5	4	3	2	1

For row =5, 5>=num>=1 , 1 <=col<=5

row	1				
col	5	4	3	2	1
output	space	space	space	space	1

row =1 , 1>=col>=1 (print row value) or row >=col>=1 (print row value)

row	2				
col	5	4	3	2	1
output	space	space	space	2	2

row = 2 , 2>=col>=1 (print row value) or row >=col>=1 (print row value)

row	3				
col	5	4	3	2	1
output	space	space	3	3	3

row = 3 , 3>=col>=1 (print row value) or row >=col>=1 (print row value)

row	4				
col	5	4	3	2	1
output	space	4	4	4	4

row = 4 , 4>=col>=1 (print row value) or row >=col>=1 (print row value)

row	5				
col	5	4	3	2	1
output	5	5	5	5	5

row = 5 , 5>=col>=1 (print row value) or row >=col>=1 (print row value)

So, **if –condition** = (col >=1 and col<=row)

col >=1 is redundant condition ,since it's a lower limit in the inner for-loop for col = 5 to 1.

So, **if –condition** reduced to (col<=row)

The overall observation is at each row, either it's printing space characters or row values in

a continues fashion. If we take care of only the logical condition of printing spaces, then it's quite obvious that the condition can be expressed in if-else.

The printing of space or numeric characters are continuous.

Let's execute the following program.

NumberPattern1B_2.c

```c
#include <stdio.h>
int main() {

	int  totalRows = 5;  //number of rows to display
	for( int row = 1 ; row <= totalRows ; row ++ ) {

		//prints variable col value for (row >= col >=1)
		for(int col = totalRows ; col >= 1 ; col--) {
			if( col<=row )   {
				printf( "%d ", row );
			}else {
				printf( "  " );
			}
		}

		printf("\n");// moves control to the next line
	}
	return 0;
}
```

Output

```
          1
        2 2
      3 3 3
    4 4 4 4
  5 5 5 5 5
```

Now, test for different number of lines, let it be 7. It should work correctly.

NumberPattern1B_2.c

```c
#include <stdio.h>
int main() {

	int  totalRows = 7;  //number of rows to display
	for( int row = 1 ; row <= totalRows ; row ++ ) {

		//prints variable col value for (row >= col >=1)
		for(int col = totalRows ; col >= 1 ; col--) {
			if( col<=row )   {
				printf( "%d ", row );
			}else {
				printf( "  " );
			}
		}

		printf("\n");// moves control to the next line
	}
	return 0;
}
```

Output

```
          1
        2 2
      3 3 3
    4 4 4 4
  5 5 5 5 5
6 6 6 6 6 6
7 7 7 7 7 7 7
```

5 5 5 5 5

4 4 4 4

3 3 3

2 2

1

NumberPattern1C

Tabular representation

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	5	5	5	5	5
Row=2	4	4	4	4	
Row=3	3	3	3		
Row=4	2	2			
Row=5	1				

(Row, Col) value pairs representation

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	1,1	1,2	1,3	1,4	1,5
Row=2	2,1	2,2	2,3	2,4	
Row=3	3,1	3,2	3,3		
Row=4	4,1	4,2			
Row=5	5,1				

Analysis of output

Let's device a solution by incrementing the for-loop for col variable (1 to 5) and determining the if-condition(lower_limit <=col<= upper_limit) of col variable to give the desired numeric output.

row	1				
col	1	2	3	4	5
output	5	5	5	5	5

row =1 , 1 <=Col <= 5 (print upper_limit=5)

row	2				
col	1	2	3	4	5
output	4	4	4	4	

row = 2 , (1 <=Col <= 4) (print upper_limit=4)

row	3				
col	1	2	3	4	5
output	3	3	3		

row = 3 , (1 <=Col <= 3) (print upper_limit=3)

row	4				
col	1	2	3	4	5
output	2	2			

row = 4 , (1 <=Col <= 2) (print upper_limit=2)

row	5				
col	1	2	3	4	5
output	1				

row = 5 , (1 <=Col <= 1) (print upper_limit=1)

Column Level Repetitive Action

The numeric output will be the value of row variable printed that many times equals the upper_limit of col variable. The upper_limit of col variable varies with the value of row variable.

Row level Repetitive Action			**Expected Output**
row	**Column level Repetitive Action**		
	lower_limit <=col<= upper_limit		
	lower_limit	*upper_limit*	
1	1	5	5 5 5 5 5
2	1	4	4 4 4 4
3	1	3	3 3 3
4	1	2	2 2
5	1	1	1

The lower_limit has same numeric value expression for the rows .It's consistent for all the rows.

Likewise we need to come out with such a consistent expression for upper_limit which should suit any number of rows. The other remaining variables are **row, totalRows**=5.

Let's try to generalised using these variables.

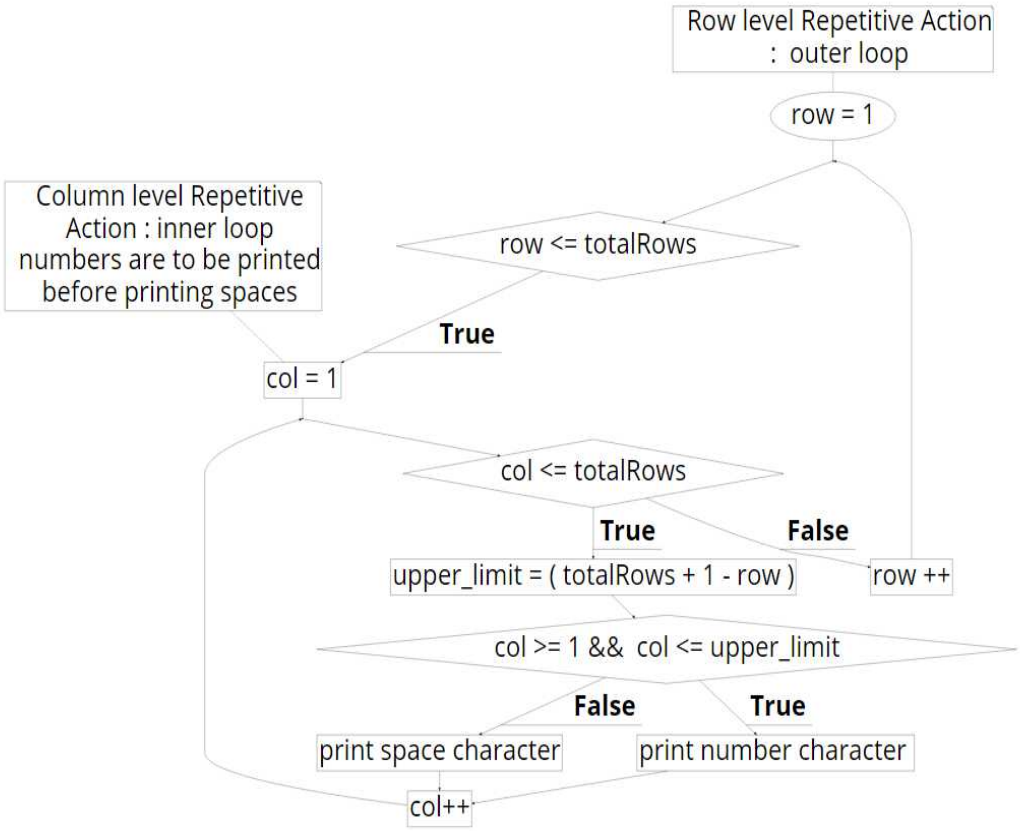
Column Level Repetitive Action					
row	*lower_limit <=col<= upper_limit*				
	lower_limit	*upper_limit*			
		default	1	2	3
1	1	5	totalRows	(totalRows +1) - 1	(totalRows +1) - row
2	1	4	totalRows -1	(totalRows +1) - 2	(totalRows +1) - row
3	1	3	totalRows - 2	(totalRows +1) - 3	(totalRows +1) - row
4	1	2	totalRows - 3	(totalRows +1) - 4	(totalRows +1) - row
5	1	1	totalRows - 4	(totalRows +1) - 5	(totalRows +1) - row

So the upper_limit of col variable is generalised to (totalRows + 1 – row).

The lower_limit & upper_limit are consistent for all the rows. It's now generalised for n number of rows. So our If-condition becomes

If-condition	lower_limit <=col<= upper_limit
Mathematical expression	1 <= col <= ((totalRows +1) – row)
`Syntax`	`col >= 1 &&  col <= ( totalRows + 1 - row )`

The flow-chart can be shown which is as follows.



Since inner for-loop's *lower_limit* matches with the if-condition's *lower_limit*.

```
for(int col = 1 ; col <= totalRows ; col++) {
        if( col >= 1 &&  col <= ( totalRows + 1 – row ) ) {
                ---- statements ------
        }else{
         // print space(s)
      }
}
```

So, we can safely neglect *lower_limit* in the if-condition as it is already been taken care by it's own for-loop's starting value. Hence, our final if-condition will be,

If-condition	lower_limit <=col<= upper_limit
Mathematical expression	col <= ((totalRows +1) – row)
Syntax	`col <= ( totalRows + 1 - row )`

If we need to print something after some space(s), then in that case putting program logic to

print space(s) make some sense. The other way round will not be a sensible exercise.

NumberPattern1C_1.c

```c
#include <stdio.h>
int main() {

	int  totalRows = 5;  //number of rows to display

	for( int row = 1 ; row <= totalRows ; row ++ ) {

		// numbers to be printed before printing spaces
		for(int col = 1 ; col <= totalRows    ; col++) {

			int upper_limit = ( totalRows + 1 - row );

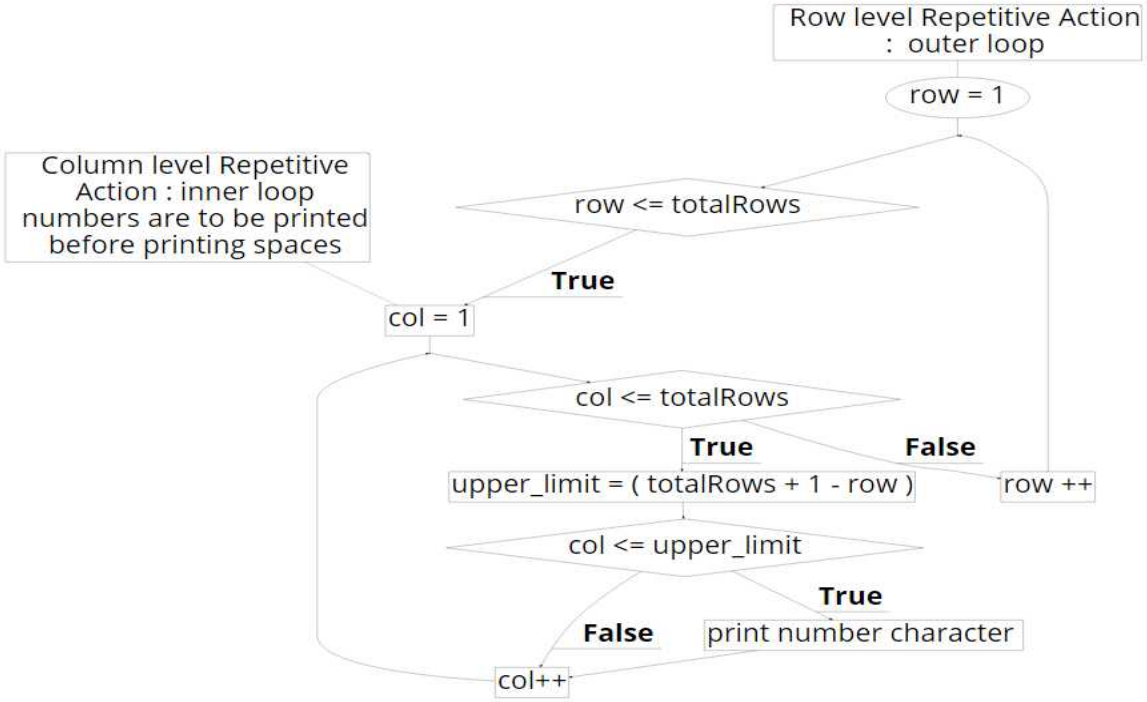
			if( col >= 1 &&  col <= upper_limit  ) {
				printf( "%d ", upper_limit );
			}else {
				printf( "  " );
			}
		}//inner loop

		printf("\n");// moves control to the next line

	}// outer loop

	return 0;
}
```

So in this particular case we can avoid participation of else-condition which further simplifies our condition logic. We can further refine our flow-chart given as follows.



NumberPattern1C_2.c

```c
#include <stdio.h>
int main() {

	int  totalRows = 5;  //number of rows to display

	for( int row = 1 ; row <= totalRows ; row ++ ) {

		// spaces to be printed before printing numbers
		for(int col = 1 ; col <= totalRows    ; col++) {

			int upper_limit = ( totalRows + 1 - row );
			if( col <= upper_limit  ) {
				printf( "%d ", upper_limit );
			}
		}// inner for-loop

		printf("\n");// moves control to the next line

	}// outer for-loop

	return 0;
}
```

We know that the total number of column level repetition are the total number of rows. Furthermore, inner loop's if-condition is responsible for decreasing the count of printing of numeric characters in columns. So the total number of repetition should also go on decreases as the number of rows goes on increases. Rows where not all column places are filled with numeric character(s) are unnecessarily looping more. We can stop this extra looping by dynamically matching the *upper_limit* of inner for-loop with the if-condition.

Hence, we can completely remove the if-condition and replace the logical expression of *upper_limit* of inner for-loop with the logical expression used in **If-condition**.

NumberPattern1C_3.c

```
#include <stdio.h>
int main() {

	int  totalRows = 5;  //number of rows to display

	for( int row = 1 ; row <= totalRows ; row ++ ) {

		int upper_limit = ( totalRows + 1 - row );

		for(int col = 1 ; col <= upper_limit ; col++ ) {
			printf( "%d ", upper_limit );
		}
		printf("\n");// moves control to the next line
	}

	return 0;
}
```

Output

```
5 5 5 5 5
4 4 4 4
3 3 3
2 2
1
```

Now, test for different number of lines, let it be 7. It should work correctly.

NumberPattern1C_3.c

```c
#include <stdio.h>
int main() {

	int  totalRows = 7;  //number of rows to display

	for( int row = 1 ; row <= totalRows ; row ++ ) {

		int upper_limit = ( totalRows + 1 - row );

		for(int col = 1 ; col <= upper_limit ; col++ ) {
			printf( "%d ", upper_limit );
		}
		printf("\n");// moves control to the next line
	}

	return 0;
}
```

Output

```
7 7 7 7 7 7 7
6 6 6 6 6 6
5 5 5 5 5
4 4 4 4
3 3 3
2 2
1
```

Alternative solution

This time let's use decrementing outer for-loop using row variable (5 to 1) and try to determine the logical condition in order to position the numeric character(s) in our row by column picture format.

row	5				
col	1	2	3	4	5
output	5	5	5	5	5

row =5 , 1 <=Col <= 5 (print upper_limit=5)

row	4				
col	1	2	3	4	5
output	4	4	4	4	

row = 4 , (1 <=Col <= 4) (print upper_limit=4)

row	3				
col	1	2	3	4	5
output	3	3	3		

row = 3 , (1 <=Col <= 3) (print upper_limit=3)

row	2				
col	1	2	3	4	5
output	2	2			

row = 2 , (1 <=Col <= 2) (print upper_limit=2)

row	1				
col	1	2	3	4	5
output	1				

row = 1 , (1 <=Col <= 1) (print upper_limit=1)

Column level Repetitive Action			**Expected Output**
row	*lower_limit <=col<= upper_limit*		
	lower_limit	*upper_limit*	
5	1	5	5 5 5 5 5
4	1	4	4 4 4 4
3	1	3	3 3 3
2	1	2	2 2
1	1	1	1

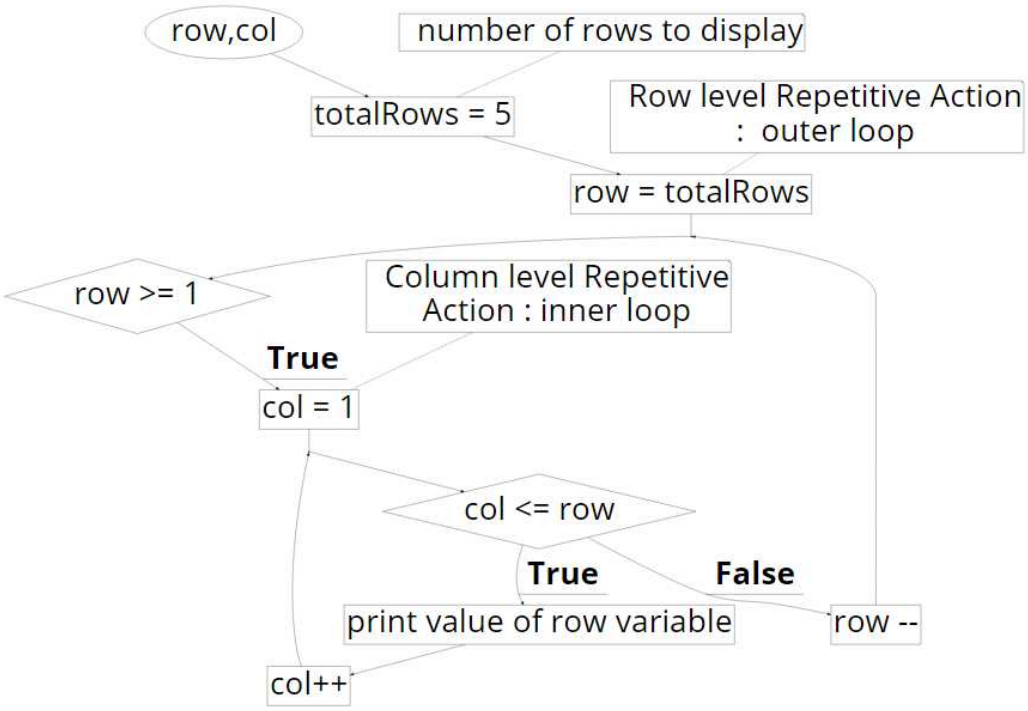
The numeric output will be the value of row variable printed that many times equals the upper_limit of row variable.

The upper_limit of row variable varies and matches with the value of row variable.

Column level Repetitive Action			**Expected Output**
row	*If-condition(lower_limit <=col<= upper_limit)*		
	lower_limit	*upper_limit*	
5	1	row	5 5 5 5 5
4	1	row	4 4 4 4
3	1	row	3 3 3
2	1	row	2 2
1	1	row	1

So our loop-condition(lower_limit <=col<= upper_limit) becomes

loop-condition	lower_limit <=col<= upper_limit
Mathematical expression	1 <= col <= row
`Syntax`	`for( int col = 1; col <=row ; col++)`



Hence, our final program will be,

NumberPattern2C.c

```c
#include <stdio.h>
int main() {

	int  totalRows = 5;  //number of rows to display

	for( int row = totalRows ; row >= 1 ; row -- ) {

		// row value to be printed
		for(int col = 1 ; col <= row ; col++) {
				printf( "%d ", row );
		}// inner for-loop

		printf("\n");// moves control to the next line

	}// outer for-loop

	return 0;
}
```

Output

```
5 5 5 5 5
4 4 4 4
3 3 3
2 2
1
```

Now, test for different number of lines, let it be 7. It should work correctly.

NumberPattern2C.c

```c
#include <stdio.h>
int main() {

	int  totalRows = 7;  //number of rows to display

	for( int row = totalRows ; row >= 1 ; row -- ) {

		// row value to be printed
		for(int col = 1 ; col <= row ; col++) {
			printf( "%d ", row );
		}// inner for-loop

		printf("\n");// moves control to the next line

	}// outer for-loop

	return 0;
}
```

Output

```
7 7 7 7 7 7 7
6 6 6 6 6 6
5 5 5 5 5
4 4 4 4
3 3 3
2 2
1
```

```
5 5 5 5 5
  4 4 4 4
    3 3 3
      2 2
        1
```

NumberPattern1D

Tabular representation

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	5	5	5	5	5
Row=2	space	4	4	4	4
Row=3	space	space	3	3	3
Row=4	space	space	space	2	2
Row=5	space	space	space	space	1

(Row, Col) value pairs representation

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	1,1	1,2	1,3	1,4	1,5
Row=2		2,2	2,3	2,4	2,5
Row=3			3,3	3,4	3,5
Row=4				4,4	4,5
Row=5					5,5

Analysis of output

In this case we need to take care of printing two different types of characters at a particular column in a particular row. The character types are the following.

1) Space character

2) Numeric character

row	5				
Col	1	2	3	4	5
Output	5	5	5	5	5

For row =1, 0 <=spaces<=0, 1<=num<=5

row	4				
Col	1	2	3	4	5
Output	space	4	4	4	4

For row =2, 1 <=spaces<=1, 1<=num<=4

row	3				
Col	1	2	3	4	5
Output	space	space	3	3	3

For row =3, 1 <=spaces<=2, 1<=num<=3

row	2				
Col	1	2	3	4	5
Output	space	space	space	2	2

For row =4, 1 <=spaces<=3, 1<=num<=2

row	1				
Col	1	2	3	4	5
Output	space	space	space	space	1

For row =5, 1 <=spaces<=4, 1<=num<=1

Observations

1) There is repetition at row level vertically, i.e., row = 1 to 5.
2) There is repetition at col level horizontally. , i.e., col = 1 to 5.
   This gets divided into two parts. One for repetitively printing spaces and remaining column for printing numeric characters.

For Loop	**Lower limit**	**Upper limit**
Outer loop(decrementing)	row=1	row>=5
Inner Loop - spaces	spaces=1	spaces <= ?(Unkown)
Inner Loop - Number	num = 1	num =?(Unkown)

Column Level Repetitive Action

So it has two repetition process one for printing spaces and second for printing numeric values.

It means require 2 inner for-loops and 1 outer loop for iterating rows. The relation between row value and upper limit for inner for-loop for spaces and numbers.

Row level Repetitive Action					**Expected Output**
row	**Column level Repetitive Action**				
	lower_limit<=col_sp<= upper_limit		***lower_limit <=col_num <= upper_limit***		
	lower_limit	**upper_limit**	**lower_limit**	**upper_limit**	
5	1	0	1	5	5 5 5 5 5
4	1	1	1	4	4 4 4 4
3	1	2	1	3	3 3 3
2	1	3	1	2	2 2
1	1	4	1	1	1

The lower_limit for spaces & num variable has the same numeric value expression for all the rows.

It's consistent for all the rows. Likewise we need to come out with such a consistent expression

for upper_limit for which it should suit any number of rows.

The other remaining variables are row & totalRows=5. Let's try to generalise using these variables.

Column level Repetitive Action						
row	***lower_limit <=col_sp<= upper_limit***			***lower_limit <=col_num <= upper_limit***		
	lower_limit	**upper_limit**		**upper_limit**	**lower_limit**	
		default	1		default	
5	1	0	totalRows - row	1	5	5 5 5 5 5
4	1	1	totalRows - row	1	4	4 4 4 4
3	1	2	totalRows - row	1	3	3 3 3
2	1	3	totalRows - row	1	2	2 2
1	1	4	totalRows - row	1	1	1

Now the uppler_limit of col_sp variable related inner for-loop is generalised, same expression on all the rows. The lower_limit of col_num variable related inner for-loop is not generalised as not having same expression on all the rows.

Since lower_limit of col_num variable values matches with row variable values.

So, we can use row variable to make it consistent given as follows.

Column level Repetitive Action						
row	***lower_limit <=col_sp<= upper_limit***			***lower_limit <=col_num <= upper_limit***		
	lower_limit	**upper_limit**		**upper_limit**	**lower_limit**	
		default	1		default	1
5	1	0	totalRows - row	1	5	row
4	1	1	totalRows - row	1	4	row
3	1	2	totalRows - row	1	3	row
2	1	3	totalRows - row	1	2	row
1	1	4	totalRows - row	1	1	row

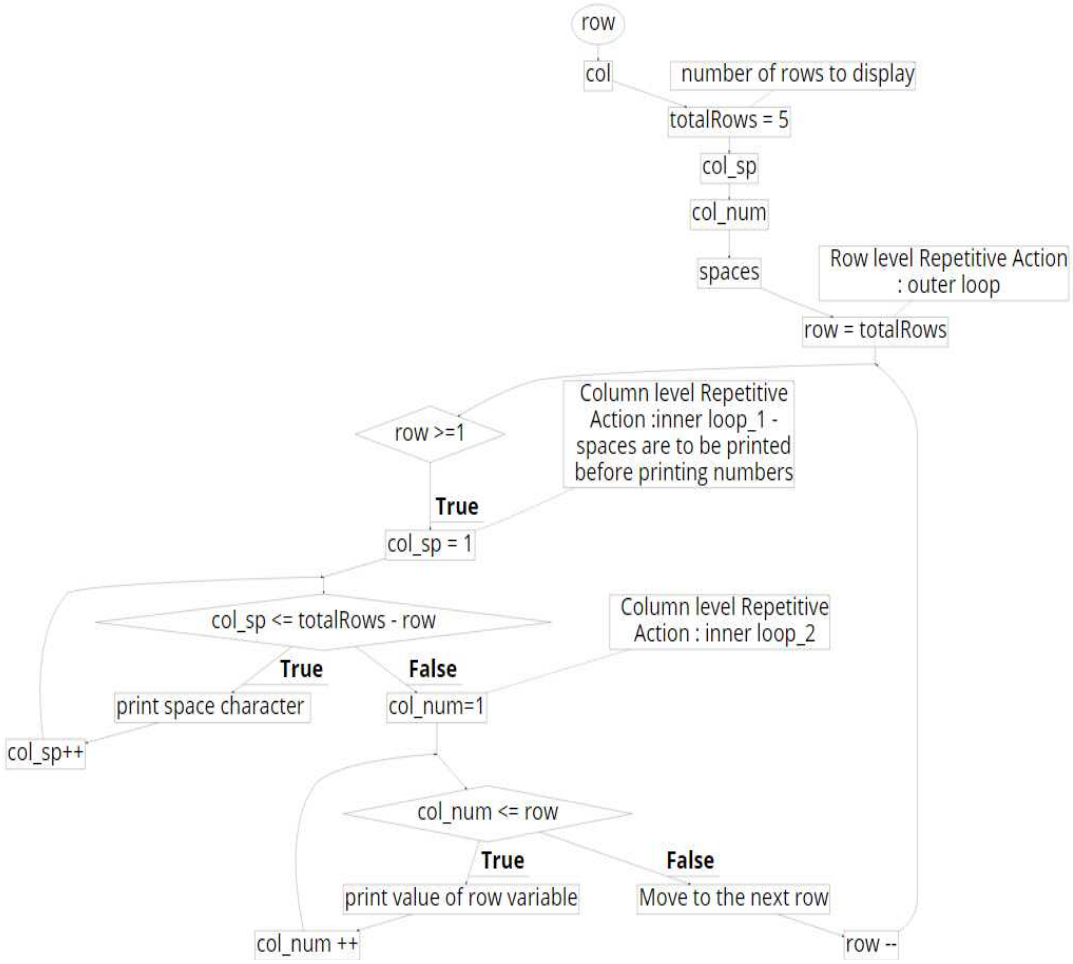
The lower_limit of col_sp variable is same for all the rows .The upper_limit of col_num variable is same for all the rows. It means the upper_limit of col_sp & col_num variables are generalised for any number of rows. So our inner for-loop condition for printing space characters becomes;

loop-condition	lower_limit <=col_sp<= upper_limit
Mathematical expression	1 <= col_sp <= (totalRows – row)
`Syntax`	`for(col_sp = 1;col_sp<=(totalRows - row); col_sp++ )`

So our inner for-loop condition for printing numeric characters becomes;

loop-condition	lower_limit <=col_num<= upper_limit
Mathematical expression	1 <= col_num <= row
`Syntax`	`for( col_num=1; col_num <= row ; col_num ++ )`

The flow-chart for our observations can be given as follows.



Let's write the program as per the flow-chart.

NumberPattern1D.c

```
#include <stdio.h>
int main() {

	int  totalRows = 5;  //number of rows to display

	for( int row = totalRows ; row >=1  ; row -- ) {

		// spaces to be printed
		for( int col = 1; col <= ( totalRows - row ) ; col++ ) {
			printf( "  " );
		}

		// row value to be printed
		for( int num=1; num<= row ; num++ ){
			printf( "% d", row );
		}// inner for-loop

		printf("\n");// moves control to the next line
	}// outer for-loop
	return 0;
}
```

Output

```
5 5 5 5 5
  4 4 4 4
    3 3 3
      2 2
        1
```

Now, test for different number of lines, let **change** it to be 7 and execute it. It should work correctly.

NumberPattern1D.c

```
#include <stdio.h>
int main() {


	int  totalRows = 7;  //number of rows to display


	for( int row = totalRows ; row >=1  ; row -- ) {


		// spaces to be printed
		for( int col = 1; col <= ( totalRows - row ) ; col++ ) {
			printf( "  " );
		}


		// row value to be printed
		for( int num=1; num<= row ; num++ ){
			printf( "% d", row );
		}// inner for-loop


		printf("\n");// moves control to the next line
	}// outer for-loop
	return 0;
}
```

Output

```
7 7 7 7 7 7 7
  6 6 6 6 6 6
    5 5 5 5 5
      4 4 4 4
        3 3 3
          2 2
            1
```

Alternative solution

Since the column values need to split clearly between printing spaces and numbers. We can try to do this using if-else condition construct.

NumberPattern2D.c

```
#include <stdio.h>
int main() {

	int totalRows = 5; //number of rows to display

	for ( int row = totalRows; row >= 1; row-- ) {

		for ( int col = 1; col <= ( totalRows ); col++ ) {

			if ( col <= ( totalRows - row ) ) {

				// spaces to be printed
				printf( "  " );

			} else {

				// row value to be printed
				printf( "%d ", row );

			}

		}// inner for-loop

		printf( "\n" );// moves control to the next line

	}// outer for-loop

	return 0;
}
```

Output

```
5 5 5 5 5
  4 4 4 4
    3 3 3
      2 2
        1
```

Alternative solution

Let's device a solution by decrementing the **for-loop** for col variable (5 to 1) so the numeric output will match with the value of row variable.

row	5				
col	5	4	3	2	1
output	5	5	5	5	5

row =5 , 5>=col>=1 (print row value) or row >=col>=1 (print row value)

row	4				
col	5	4	3	2	1
output	space	4	4	4	4

row = 4 , 4>=col>=1 (print row value) or row >=col>=1 (print row value)

row	3				
col	5	4	3	2	1
output	space	space	3	3	3

row = 3 , 3>=col>=1 (print row value) or row >=col>=1 (print row value)

row	2				
col	5	4	3	2	1
output	space	space	space	2	2

row = 2 , 2>=col>=1 (print row value) or row >=col>=1 (print row value)

row	1				
col	5	4	3	2	1
output	space	space	space	space	1

row = 1 , 1>=col>=1 (print row value) or row >=col>=1 (print row value)

Row level Repetitive Action				**Expected Output**
row	**Column level Repetitive Action**			
	upper_limit *>=col>= upper_limit*			
	upper_limit		**lower_limit**	
	default	1		
5	5	row	1	5 5 5 5 5
4	4	row	1	4 4 4 4
3	3	row	1	3 3 3
2	2	row	1	2 2
1	1	row	1	1

So, if –condition can be given as

if-condition	upper_limit >=col>= lower_limit
Mathematical expression	row >= col >= 1
Syntax	`If( col>=1 && col <= row )`

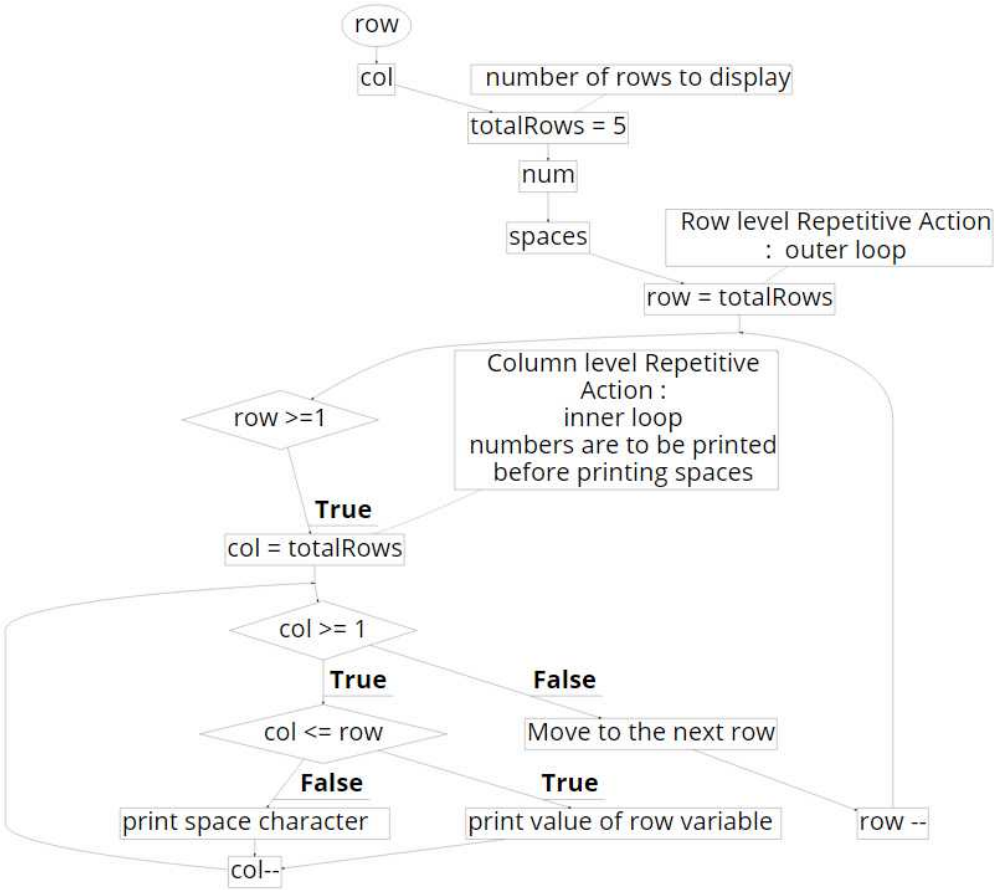
The part of if-condition col >=1 is already taken care in the lower_limit of inner for-loop.

So, if –condition is now becomes;

if-condition	upper_limit >=col>= lower_limit
Mathematical expression	col<=row
Syntax	`if( col <= row )`

The overall observation at each row is either it's printing space characters or row values in a continues fashion in every column but in a different way.

If we take care of condition of printing numbers only, then the else part will for printing spaces and can be easily expressed in **if-else statement** for printing desired output.



Let’s try to execute the following program.

NumberPattern3D.c

```
#include <stdio.h>
int main() {

	int  totalRows = 5;  //number of rows to display

	for( int row = totalRows ; row >=1  ; row -- ) {

		for( int col = totalRows; col >= 1 ; col-- ) {

			if( col <=  row ) {
				// row value to be printed
				printf( "%d ", row );
			}else {
				// spaces to be printed
				printf( "  " );
			}

		}
		printf("\n");// move control to the next line
	}

	return 0;
}
```

Output

```
5 5 5 5 5
  4 4 4 4
    3 3 3
      2 2
        1
```

Now, test for different number of lines, let it be 7. It should work correctly.

NumberPattern3D.c

```
#include <stdio.h>
int main() {

	int  totalRows = 7;  //number of rows to display

	for( int row = totalRows ; row >=1  ; row -- ) {

		for( int col = totalRows; col >= 1 ; col-- ) {

			if( col <=  row ) {
				// row value to be printed
				printf( "%d ", row );
			}else {
				// spaces to be printed
				printf( "  " );
			}

		}
		printf("\n");// moves control to the next line
	}

	return 0;
}
```

Output

```
7 7 7 7 7 7 7
  6 6 6 6 6 6
    5 5 5 5 5
      4 4 4 4
        3 3 3
          2 2
            1
```

```
                1
            2   2   2
        3   3   3   3   3
    4   4   4   4   4   4   4
5   5   5   5   5   5   5   5   5
```

NumberPattern1E

Tabular representation

	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1	space	space	space	space	1	space	space	space	space
Row=2	space	space	space	2	2	2	space	space	space
Row=3	space	space	3	3	3	3	3	space	space
Row=4	space	4	4	4	4	4	4	4	space
Row=5	5	5	5	5	5	5	5	5	5

(Row, Col) value pairs representation

	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1					1,5				
Row=2				2,4	2,5	2,6			
Row=3			3,3	3,4	3,5	3,6	3,7		
Row=4		4,2	4,3	4,4	4,5	4,6	4,7	4,8	
Row=5	5,1	5,2	5,3	5,4	5,5	5,6	5,7	5,8	5,9

Analysis of output

row	1								
col	1	2	3	4	5	6	7	8	9
output					1				

For row = 1, 5 <=Col <= 5, (printing value of row)

row	2								
col	1	2	3	4	5	6	7	8	9
output				2	2	2			

For row = 2, 4 <=Col <= 6, (printing value of row)

row	3								
col	1	2	3	4	5	6	7	8	9
output			3	3	3	3	3		

For row = 3, 3 <=Col <= 7, (printing value of row)

row	4								
col	1	2	3	4	5	6	7	8	9
output		4	4	4	4	4	4	4	

For row = 4, 2 <=Col <= 8, (printing value of row)

row	5								
col	1	2	3	4	5	6	7	8	9
output	5	5	5	5	5	5	5	5	5

For row = 5, 1 <=Col <= 9, (printing value of row)

Column Level Repetitive Action

Total maximum of 9 columns are needed to print the bottom of pattern.

Row level Repetitive Action			**Expected Output**
row	**Column level Repetitive Action**		
	(lower_limit <= col <= upper_limit)		
	lower_limit	upper_limit	
1	5	5	1
2	4	6	2 2 2
3	3	7	3 3 3 3 3
4	2	8	4 4 4 4 4 4 4
5	1	9	5 5 5 5 5 5 5 5 5

The lower_limit & upper_limit ,both are not consistent for all the rows. We need to come out with consistent expressions for both upper_limit and lower_limit which should suit for n number of rows. The other remaining variables are row, totalRows=5.

Let's try to generalise using these variables.

Column level Repetitive Action						
row	*lower_limit <= col <= upper_limit*					
	lower_limit			upper_limit		
		1	2		1	2
1	5	totalRows	totalRows +1 - 1	5	totalRows	totalRows +1 - 1
2	4	totalRows -1	totalRows +1 - 2	6	totalRows + 1	totalRows + 2 - 1
3	3	totalRows -2	totalRows +1 - 3	7	totalRows + 2	totalRows + 3 - 1
4	2	totalRows -3	totalRows +1 - 4	8	totalRows + 3	totalRows + 4 - 1
5	1	totalRows -4	totalRows +1 - 5	9	totalRows + 4	totalRows + 5 - 1

Column level Repetitive Action				
row	*If-condition(lower_limit <= col <= upper_limit)*			
	lower_limit		upper_limit	
	default	3	default	3
1	5	totalRows +1 - row	5	totalRows + row - 1
2	4	totalRows +1 - row	6	totalRows + row - 1
3	3	totalRows +1 - row	7	totalRows + row - 1
4	2	totalRows +1 - row	8	totalRows + row - 1
5	1	totalRows +1 - row	9	totalRows + row - 1

The lower_limit & upper_limit are consistent for all the rows. It's now generalised for any number of rows to print numeric value. So our If-condition becomes

if-condition	lower_limit <=col<= upper_limit
Mathematical expression	(totalRows +1 - row) <= col <= (totalRows + row - 1)
Syntax	`if( col >= ( ( totalRows - row ) + 1 ) &&` `col <= ( ( totalRows + row ) - 1 ) )`

When the if-condition satisfied, it will print the row value at that position at a particular row or else will print space character. Since we know that the total maximum of 9 columns are needed to print the bottom of pattern. The for-loop condition is given as;

loop-condition	lower_limit <=col<= upper_limit
Mathematical expression	1<= col <= 9
Syntax	`for( col = 1 ; col <= 9 ; col++ )`

The *upper_limit* of col variable of inner for-loop which is a **value 9** is required to be generalised.

The hard-coded values in logical condition makes program to be fail for different input values. Somehow, it has to be generalised using variables considered for program.

loop-condition	lower_limit <=col<= upper_limit
Mathematical expression	1<= col <= totalRows + 4
	1<= col <= totalRows + (totalRows – 1)
	1<= col <= (2*totalRows – 1)
`Syntax`	`for( col = 1 ; col <= ( 2* totalRows - 1 ) ; col++ )`

We are now able to generalised using variable **totalRows**.

Let's try to verify the upper limit for total columns which we will display for each row

for **totalRows = 5** , the col <= 9 (2*5 – 1)

for **totalRows = 4** , the col <= 7 (2*4 – 1)

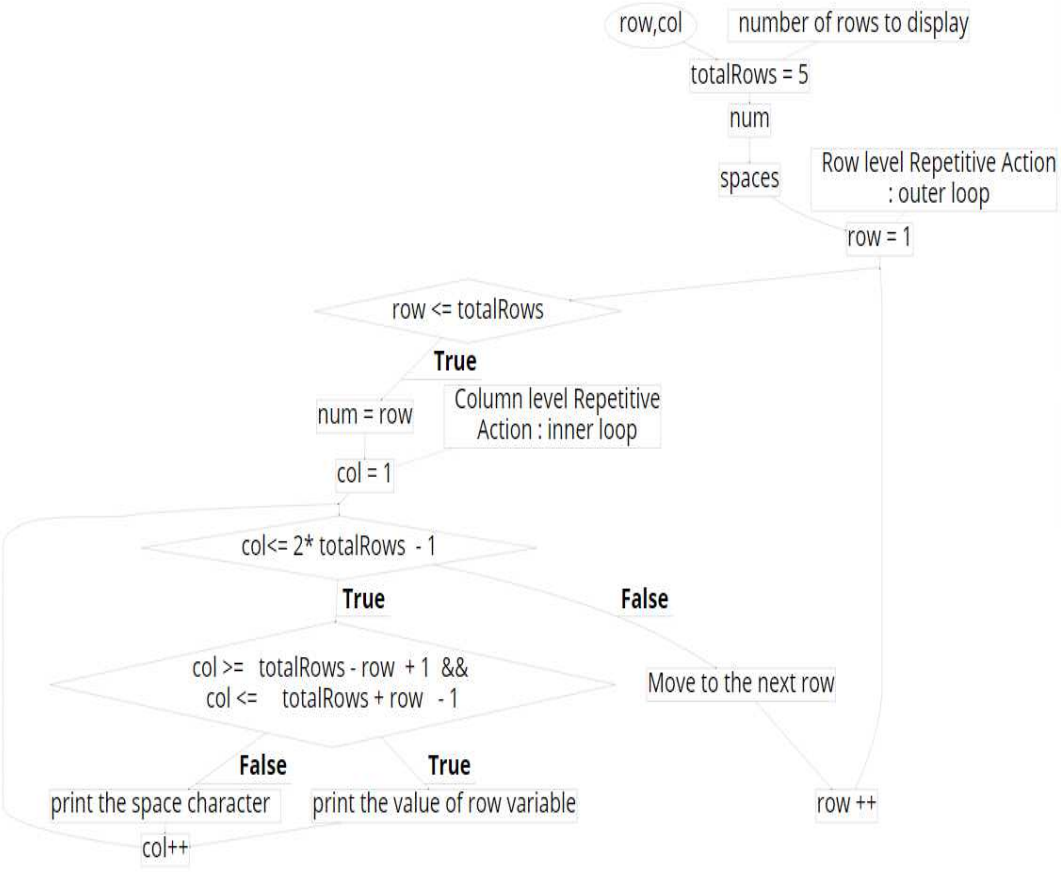
for **totalRows = 3** , the col <= 5 (2*3 – 1)

for **totalRows = 2** , the col <= 3 (2*2 – 1)

for **totalRows = 1** , the col <= 1 (2*1 – 1)

So this generalisation for total columns as per given rows holds valid.

All these observations can be easily captured in a flow-chart given as follows.



Let's write the program based on the observations captured in a flow-chart.

NumberPattern1E.c

```
#include <stdio.h>
int main() {

	int totalRows = 5; //number of rows to display

	for ( int row = 1; row <= totalRows; row++ ) {

		// repetition happens ( 2* totalRows  - 1) times for columns

		for ( int col = 1; col <= ( 2 * totalRows - 1 ); col++ ) {

			if ( col >= ( ( totalRows - row ) + 1 ) &&
			     col <= ( ( totalRows + row ) - 1 ) ) {

				// row value to be printed
				printf( "%d ", row );

			} else {

				// spaces to be printed
				printf( "  " );

			}

		}// end  inner for - loop

		printf( "\n" );// moves control to the next line

	} // end  outer for - loop

	return 0;
}
```

Output

```
        1
      2 2 2
    3 3 3 3 3
  4 4 4 4 4 4 4
5 5 5 5 5 5 5 5 5
```

Now, test for different number of lines, let it be 8. It should work correctly.

NumberPattern1E.c

```
#include <stdio.h>
int main() {

	int totalRows = 8; //number of rows to display

	for ( int row = 1; row <= totalRows; row++ ) {

		// repetition happens ( 2* totalRows  - 1) times for columns

		for ( int col = 1; col <= ( 2 * totalRows - 1 ); col++ ) {

			if ( col >= ( ( totalRows - row ) + 1 ) &&
			     col <= ( ( totalRows + row ) - 1 ) ) {

				// row value to be printed
				printf( "%d ", row );

			} else {

				// spaces to be printed
				printf( "  " );

			}

		}// end inner for-loop

		printf( "\n" );// moves control to the next line

	} // end outer for-loop

	return 0;
}
```

Output

```
              1
            2 2 2
          3 3 3 3 3
        4 4 4 4 4 4 4
      5 5 5 5 5 5 5 5 5
    6 6 6 6 6 6 6 6 6 6 6
  7 7 7 7 7 7 7 7 7 7 7 7 7
8 8 8 8 8 8 8 8 8 8 8 8 8 8
```

1

0 0

1 1 1

0 0 0 0

1 1 1 1 1

NumberPattern1F

Tabular representation.

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	1				
Row=2	0	0			
Row=3	1	1	1		
Row=4	0	0	0	0	
Row=5	1	1	1	1	1

(Row, Col) value pairs representation.

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	1,1				
Row=2	2 ,1	2,2			
Row=3	3,1	3,2	3,3		
Row=4	4,1	4,2	4,3	4,3	
Row=5	5,1	5,2	5,3	5,4	5,5

Analysis of output

It can also be interpreted in a tabular form.

row	1				
Col	1				
Output	1				

For row =1, 1<=col<=1

row	2				
Col	1	2			
Output	0	0			

For row =2, 1<=col<=2

row	3				
Col	1	2	3		
Output	1	1	1		

For row =3, 1<=col<=3

row	4				
Col	1	2	3	4	
Output	0	0	0	0	

For row =4, 1<=col<=4

row	5				
Col	1	2	3	4	5
Output	1	1	1	1	1

For row =5, 1<=col<=5

Column Level Repetitive Action

Row level Repetitive Action				**Expected Output**
row	**Column level Repetitive Action**			
	Values repeated	lower_limit <=col<= upper_limit		
		lower_limit	upper_limit	
1	1	1	1	1
2	0	1	2	0 0
3	1	1	3	1 1 1
4	0	1	4	0 0 0 0
5	1	1	5	1 1 1 1 1

The *upper_limit* value is based on how many times the value repeated in a particular row.

The *lower_limit* of col variable has the same numeric value expression for every value of row. It's consistent for all the values rows. Likewise we need to come out with such a consistent expression for upper_limit which should suit any number of rows using available variables. The variables are row & totalRows=5.

Moreover, the repeated values in column appears alternatively in row. It seems we can establish a relationship between appearing of 0 or 1 value with the row value.

If we observe it is following even-odd rule. For even value of row, means when row value is a multiple of 2, we need to print zeros or ones otherwise. If we try we can generalise 0 or 1 in terms of row variable. Let's try to generalise columns **upper_limit** and **values_repeated** by expressing in terms of available variables.

row	**Column level Repetitive Action**					
	values_repeated		lower_limit <=col<= upper_limit			
			lower_limit	upper_limit		
	default	**1**		**default**	**1**	
1	1	row%2==1	1	1	row	1
2	0	row%2==0	1	2	row	0 0
3	1	row%2==1	1	3	row	1 1 1
4	0	row%2==0	1	4	row	0 0 0 0
5	1	row%2==1	1	5	row	1 1 1 1 1

For all the rows, the value prints exactly the number of times the upper_limit of col value.

So the *upper_limit* of col variable is generalised to **row** variable.

The *lower_limit & upper_limit* are consistent for all the rows.

It's now generalised for any number of rows.

We can now define column level ***for-loop*** which is for every row value.

Elements of* Column level *for-loop	
min_val	1
max_val	row
variable_name	**col**
statement(s)	`printf("%d ", row);`

Also, printing 0 or 1 can be given by checking the if-condition (row%2==1) or (row%2==0).

General Rule
Such situation may come where we need to check whether logical condition is following pattern of (**row+col**) values or (**row-col**) values or (**row*col**) values or (**row%col**) values or it could be anything like that. The general rule is, somehow we need to express hard-coded values in terms of available variable(s) if following same numeric pattern/variations or simple maths formula made up of available variable(s).

Row Level Repetitive Action

The same following statements had been repeated 5 times(total number of rows).

Column level *for-loop* // *inner for-loop*

```
printf("\n" );// Move cursor to the next row
```

The minimum and maximum value can be given for the ***for-loop*** as;
min_val = 1 & *max_val* = 5 or totalRows
Let's consider;

row → variable type of integer to store integer value one at a time and represents row number between 1 and 5(totalRows). The **totalRows** has initially assigned the value 5.

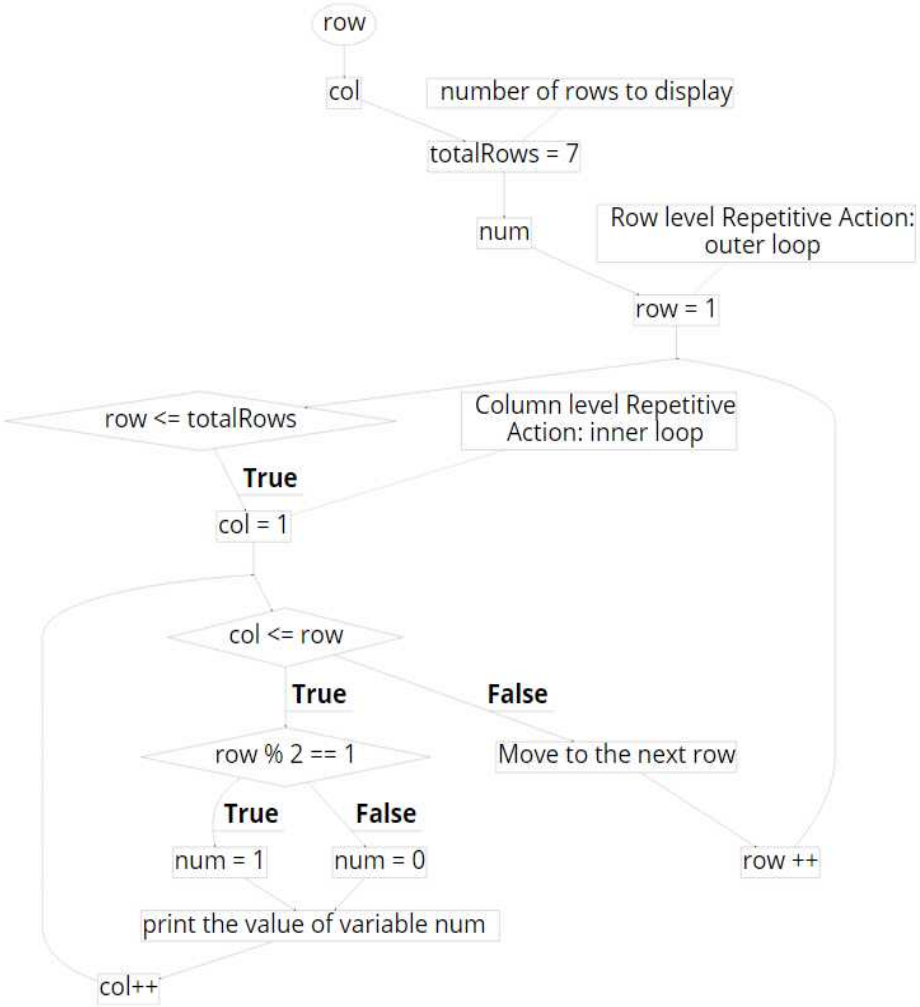
Outer ***for-loop*** starts at row=1 & will repeat till row <= **totalRows**.

We got everything related to ***for-loop*** for row level repetition.

Elements Of* Row Level *for-loop	
min_val	1
max_val	totalRows
variable_name	**row**

statement(s)	**Column level *for-loop***
	`// Moves cursor to the next row`
	`printf("\n" );`

We can summarise our observations in a flow-chart as shown in the following.



Let’s execute the following program.

NumberPattern1F.c

```c
#include <stdio.h>
int main() {

	int totalRows = 7; // number of rows to display
	int num;

	// Row level Repetitive Action:
	for ( int row = 1; row <= totalRows; row++ ) {

		// Column level Repetitive Action:
		// print the value of variable num column-wise
		for ( int col = 1; col <= row; col++ ) {

			if ( row % 2 == 1 ) {
				num = 1;
			} else {
				num = 0;
			}

			printf( "%d ", num );

		} // inner loop

		printf( "\n" );// Moves cursor to the next row

	} // outer loop

	return 0;
}
```

Output

```
1
0 0
1 1 1
0 0 0 0
1 1 1 1 1
0 0 0 0 0 0
1 1 1 1 1 1 1
```

Level – 2: Number Pattern.

1

1 2

1 2 3

1 2 3 4

1 2 3 4 5

NumberPatternAA

Tabular representation

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	1				
Row=2	1	2			
Row=3	1	2	3		
Row=4	1	2	3	4	
Row=5	1	2	3	4	5

(Row, Col) value pairs representation.

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	1,1				
Row=2	2 ,1	2,2			
Row=3	3,1	3,2	3,3		
Row=4	4,1	4,2	4,3	4,3	
Row=5	5,1	5,2	5,3	5,4	5,5

Analysis of output

It can also be interpreted in a tabular form.

row	1				
Col	1				
Output	1				

For row =1, 1<=col<=1

row	2				
Col	1	2			
Output	1	2			

For row =2, 1<=col<=2

row	3				
Col	1	2	3		
Output	1	2	3		

For row =3, 1<=col<=3

row	4				
Col	1	2	3	4	
Output	1	2	3	4	

For row =4, 1<=col<=4

row	5				
Col	1	2	3	4	5
Output	1	2	3	4	5

For row =5, 1<=col<=5

Observations

1) There is repetition required at row level vertically, i.e., row = 1 to 5.
2) There is repetition required at col level horizontally. , i.e., col = 1 to 5.
   So it's a 2-way repetition process means require 2 for-loops (nested for – loop).

For Loop	**Lower limit**	**Upper limit**
Outer loop	row=1	row<=5
Inner Loop	col=1	(Unknown) Need to find a relation how exactly it's varying.

Column Level Repetitive Action

Row level Repetitive Action			**Expected Output**
row	**Column Level Repetitive Action**		
	loop-condition(lower_limit <=col<= upper_limit)		
	lower_limit	upper_limit	
1	1	1	1
2	1	2	1 2
3	1	3	1 2 3
4	1	4	1 2 3 4
5	1	5	1 2 3 4 5

The lower_limit has same numeric value expression for the rows .It's consistent for all the rows. Likewise we need to come out with such a consistent expression for upper_limit which should suit any number of rows.

The other remaining variables are row & totalRows=5. Let's try to generalise using these variables.

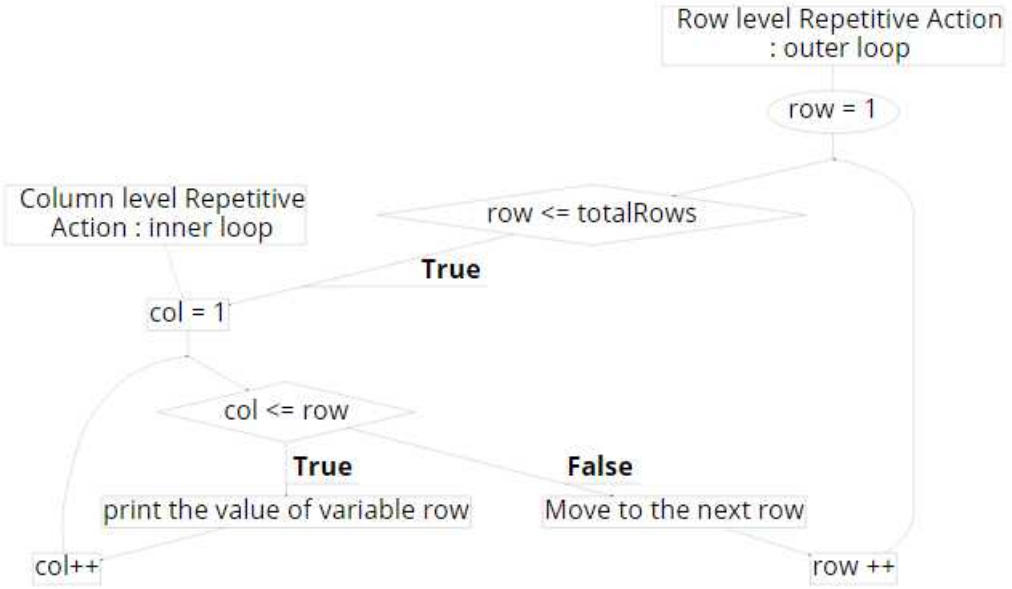
Column level Repetitive Action			
row	loop-condition(lower_limit <=col<= upper_limit)		
	lower_limit	upper_limit	
		default	1
1	1	1	row
2	1	2	row
3	1	3	row
4	1	4	row
5	1	5	row

So the upper_limit of col variable is generalised to (row). For all the rows, the col values matches exactly the output that we need.The lower_limit & upper_limit are consistent for all the rows.

It's now generalised for any number of rows. So our **loop-condition** becomes;

loop-condition	lower_limit <=col<= upper_limit
Mathematical expression	1 <= col <= row
`Syntax`	`for( col = 1 ; col <= row ; col++)`

The related flow-chart can be given as follows.



Let’s implement the program.

NumberPatternAA.c

```c
#include <stdio.h>
int main() {

	int totalRows = 5; //number of rows to display

	for ( int row = 1; row <= totalRows; row++ ) {

		for ( int col = 1; col <= row; col++ ) {

			// added extra space for clarity in output.
			printf( "%d ", col );
		} // end inner for-loop

		printf( "\n" );// moves control to the next line

	} // end outer for-loop

	return 0;
}
```

Output

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Now, test for different number of lines, let it be 10. It should work correctly.

NumberPatternAA.c

```
#include <stdio.h>
int main() {

	int totalRows = 10; //number of rows to display

	for ( int row = 1; row <= totalRows; row++ ) {

		for ( int col = 1; col <= row; col++ ) {

			// added extra space for clarity in output.
			printf( "%d ", col );
		} // end inner for-loop

		printf( "\n" );// moves control to the next line

	} // end outer for-loop

	return 0;
}
```

Output

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
1 2 3 4 5 6
1 2 3 4 5 6 7
1 2 3 4 5 6 7 8
1 2 3 4 5 6 7 8 9
1 2 3 4 5 6 7 8 9 10
```

```
    1
  2 1
3 2 1
4 3 2 1
5 4 3 2 1
```

NumberPatternBB

Tabular representation

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	space	space	space	space	1
Row=2	space	space	space	2	1
Row=3	space	space	3	2	1
Row=4	space	4	3	2	1
Row=5	5	4	3	2	1

Trace Table: (Row, Col) co-ordinate style representation.

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1					1,5
Row=2				2,4	2,5
Row=3			3,3	3,4	3,5
Row=4		4,2	4,3	4,4	4,5
Row=5	5,1	5,2	5,3	5,4	5,5

Analysis of output

In this case we need to take care of printing two different types of characters at a particular column in a particular row:

1) Space character

2) Numeric character

row	1				
col	1	2	3	4	5
output	space	space	space	space	1

For row =1, 1 <=spaces<=4, 1>=num>=1

row	2				
Col	1	2	3	4	5
Output	space	space	space	2	1

For row =2, 1 <=spaces<=3, 2>=num>=1

row	3				
Col	1	2	3	4	5
Output	space	space	3	2	1

For row =3, 1 <=spaces<=2, 3>=num>=1

row	4				
Col	1	2	3	4	5
Output	space	4	3	2	1

For row =4, 1 <=spaces<=1, 4>=num>=1

row	5				
Col	1	2	3	4	5
Output	5	4	3	2	1

For row =5, 5>=num>=1

Observations

1) There is repetition at row level vertically, i.e., Row = 1 to 5.
2) There is repetition at col level horizontally. , i.e., Col = 1 to 5.
   This gets divided into two parts .One for repetitively printing spaces and remaining column for printing numeric characters.

For Loop	**Lower limit**	**Upper limit**
Outer loop	row=1	row<=5
Inner Loop - spaces	spaces=1	spaces <= ?(Unkown)
Inner Loop - Number	num = ?(Unkown)	num = 1

Column Level Repetitive Action

Row level Repetitive Action					**Expected Output**
row	**Column level Repetitive Action**				
	lower_limit <=spaces<= upper_limit		*upper_limit>=num >= lower_limit*		
	lower_limit	upper_limit	upper_limit	lower_limit	
1	1	4	1	1	1
2	1	3	2	1	2 1
3	1	2	3	1	3 2 1
4	1	1	4	1	4 3 2 1
5	1	0	5	1	5 4 3 2 1

The lower_limit for spaces & num variable has the same numeric value expression for all the rows.

It's consistent for all the rows. Likewise we need to come out with such a consistent expression

for upper_limit for which it should suit any number of rows.

The other remaining variables are row & totalRows=5. Let's try to generalise using these variables.

Column Level Repetitive Action							
row	*lower_limit <=spaces<= upper_limit*				*upper_limit >=num >= lower_limit*		
	lower_limit	upper_limit			upper_limit		lower_limit
			1	2		1	
1	1	4	totalRows - 1	totalRows - row	1	row	1
2	1	3	totalRows - 2	totalRows - row	2	row	1
3	1	2	totalRows - 3	totalRows - row	3	row	1
4	1	1	totalRows - 4	totalRows - row	4	row	1
5	1	0	totalRows - 5	totalRows - row	5	row	1

So the upper_limit of spaces & num variables are generalised.

The lower_limit & upper_limit are consistent for all the rows.

It's now generalised for any number of rows.

Since it has two repetition process;

a) one repetition process for printing spaces.

b) second repetition process for printing numeric values.

So our inner for-loop condition for printing space characters becomes;

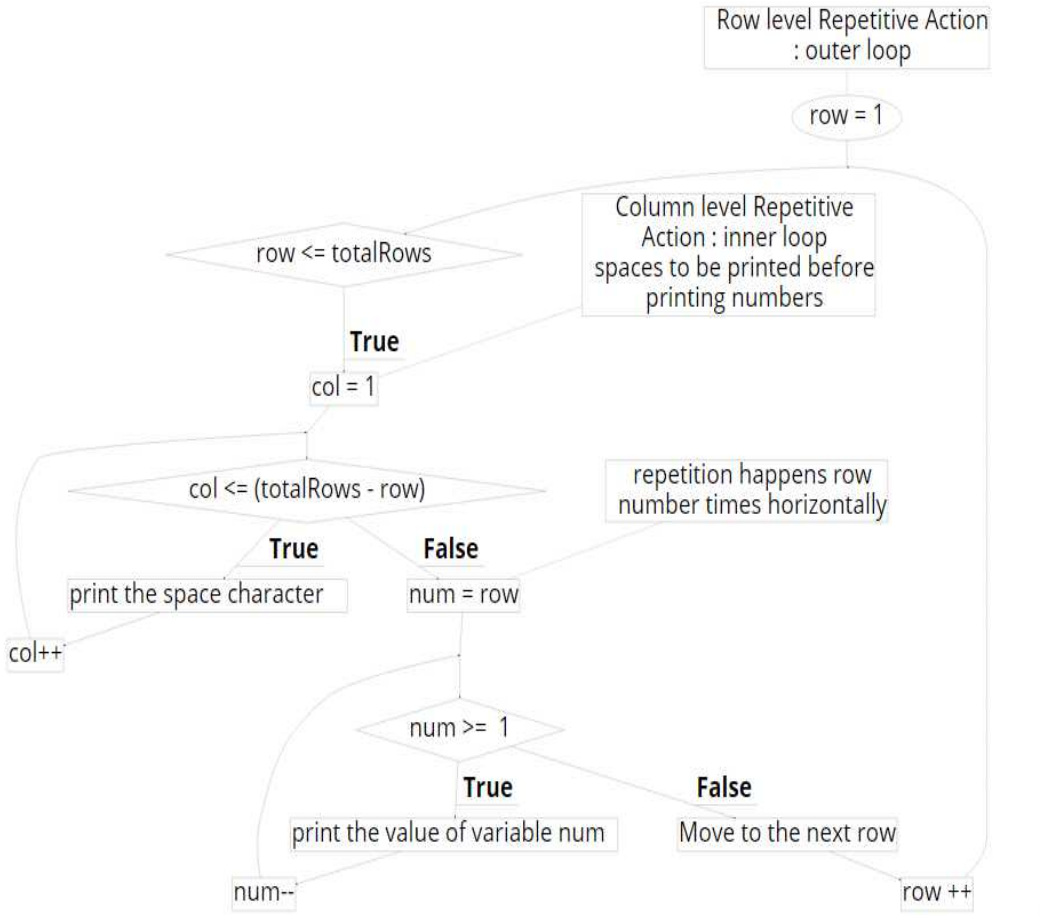
loop-condition	lower_limit <=col<= upper_limit
Mathematical expression	1 <= col <= (totalRows – row)
`Syntax`	`for( col = 1;col <= (totalRows - row ); col++ )`

So our inner for-loop condition for printing numeric characters becomes;

loop-condition	lower_limit <=num<= upper_limit
Mathematical expression	1 <= num <= row
`Syntax`	`for(  num=1; num <= row ; num ++ )`

The flow-chart can be used to summarise all the gathered information.

It is given as follows.



Let’s execute the following program.

NumberPatternBB.c

```
#include <stdio.h>
int main() {

	int totalRows = 5; // number of rows to display

	for ( int row = 1; row <= totalRows; row++ ) {

		// spaces to be printed before printing numbers
		for ( int col = 1; col <= ( totalRows - row ); col++ ) {
			printf( "  " );
		}

		for ( int num = row; num >= 1; num-- ) {
			// added extra space for clarity in output.
			printf( "%d ", num );
		}

		printf( "\n" );// moves control to the next line

	}

	return 0;
}
```

Output

```
        1
      2 1
    3 2 1
  4 3 2 1
5 4 3 2 1
```

Now, test for different number of lines, let it be 7. It should work correctly.

NumberPatternBB.c

```c
#include <stdio.h>
int main() {

	int totalRows = 7; // number of rows to display

	for ( int row = 1; row <= totalRows; row++ ) {

		// spaces to be printed before printing numbers
		for ( int col = 1; col <= ( totalRows - row ); col++ ) {
			printf( "  " );
		}

		for ( int num = row; num >= 1; num-- ) {
			// added extra space for clarity in output.
			printf( "%d ", num );
		}

		printf( "\n" );// moves control to the next line

	}

	return 0;
}
```

Output

```
            1
          2 1
        3 2 1
      4 3 2 1
    5 4 3 2 1
  6 5 4 3 2 1
7 6 5 4 3 2 1
```

Alternative solution

We observed previously that for row=5, the values of col and numeric output are just reversed. Let's device a solution by decrementing the for-loop for col variable (5 to 1) so the numeric output will match partially or wholly with the values of col variable.

row	5				
col	1	2	3	4	5
output	5	4	3	2	1

For row =5, 5>=num>=1 , 1 <=col<=5

row			1		
col	5	4	3	2	1
output	space	space	space	space	1

row =1 , 1>=col>=1 (print col value) or row >=col>=1 (print col value)

row			2		
col	5	4	3	2	1
output	space	space	space	2	1

row = 2 , 2>=col>=1 (print col value) or row >=col>=1 (print col value)

row			3		
col	5	4	3	2	1
output	space	space	3	2	1

row = 3 , 3>=col>=1 (print col value) or row >=col>=1 (print col value)

row			4		
col	5	4	3	2	1
output	space	4	3	2	1

row = 4 , 4>=col>=1 (print col value) or row >=col>=1 (print col value)

row			5		
col	5	4	3	2	1
output	5	4	3	2	1

row = 5 , 5>=col>=1 (print col value) or row >=col>=1 (print col value)

So, if –condition now becomes **(col >=1 and col<=row)**

col >=1 is redundant condition, since it's a lower limit in the inner for-loop for col = 5 to 1.

So, if –condition is reduced to **(col<=row)**

if-condition	lower_limit <=col<= upper_limit
Mathematical expression	1 <= col <= row
Syntax	`if( col<=row )`

The overall observation is at each row, either it's printing space characters or col values in a continues fashion. It's quite obvious that we can make use of **if-else statement** as printing of characters are continuous.

Let's update the program.

NumberPatternBB2.c

```
#include <stdio.h>
int main() {

	int totalRows = 5; //number of rows to display

	// Row level Repetitive Action :
	for ( int row = 1; row <= totalRows; row++ ) {

		// Column level Repetitive Action :
		// Action1.Move cursor in the same row
		// Action2.print numeric character
		for ( int col = totalRows; col >= 1; col-- ) {

			if ( col <= row ) {
				printf( "%d ", col );
			} else {
				printf( "  " );
			}
		}// inner for-loop

		printf( "\n" ); // moves control to the next line

	}// outer for-loop

	return 0;
}
```

Output

```
        1
      2 1
    3 2 1
  4 3 2 1
5 4 3 2 1
```

1 2 3 4 5

1 2 3 4

1 2 3

1 2

1

NumberPatternCC

Tabular representation

	Col=1	**Col=2**	**Col=3**	**Col=4**	**Col=5**
Row=1	1	2	3	4	5
Row=2	1	2	3	4	
Row=3	1	2	3		
Row=4	1	2			
Row=5	1				

(Row, Col) value pairs representation.

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	1,1	1,2	1,3	1,4	1,5
Row=2	2,1	2,2	2,3	2,4	
Row=3	3,1	3,2	3,3		
Row=4	4,1	4,2			
Row=5	5,1				

Analysis of output

Let's device a solution by incrementing the for-loop for col variable (1 to 5) so that the numeric output will match partially or wholly with the values of col variable. So **if-condition** can be expressed using col variable.

row	1				
col	1	2	3	4	5
output	1	2	3	4	5

row =1 , 1 <=Col <= 5 (print col value)

row	2				
col	1	2	3	4	5
output	1	2	3	4	

row = 2 , (1 <=Col <= 4) (print col value)

row	3				
col	1	2	3	4	5
output	1	2	3		

row = 3 , (1 <=Col <= 3) (print col value)

row	4				
col	1	2	3	4	5
output	1	2			

row = 4 , (1 <=Col <= 2) (print col value)

row	5				
col	1	2	3	4	5
output	1				

row = 5 , (1 <=Col <= 1) (print col value)

Column Level Repetitive Action

Row level Repetitive Action			**Expected Output**
row	**Column level Repetitive Action**		
	If-condition(lower_limit <=col<= upper_limit)		
	lower_limit	upper_limit	
1	1	5	1 2 3 4 5
2	1	4	1 2 3 4
3	1	3	1 2 3
4	1	2	1 2
5	1	1	1

The lower_limit has same numeric value expression for the rows .

It's consistent for all the rows. Likewise we need to come out with such a consistent expression for *upper_limit* which should suit any number of rows.

The considered variables are row, totalRows=5.Let try to generalise using these variables.

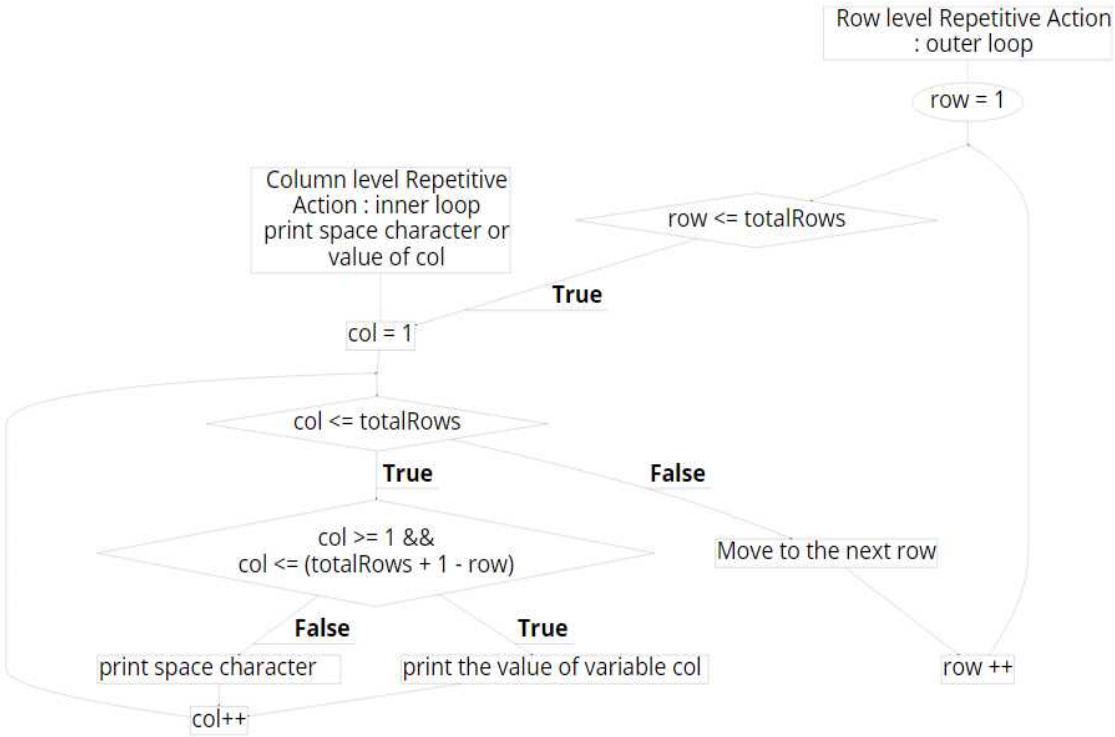
Column level Repetitive Action					
row	*If-condition(lower_limit <=col<= upper_limit)*				
	lower_limit	upper_limit			
			1	2	3
1	1	5	totalRows	(totalRows +1) - 1	(totalRows +1) - row
2	1	4	totalRows -1	(totalRows +1) - 2	(totalRows +1) - row
3	1	3	totalRows - 2	(totalRows +1) - 3	(totalRows +1) - row
4	1	2	totalRows - 3	(totalRows +1) - 4	(totalRows +1) - row
5	1	1	totalRows - 4	(totalRows +1) - 5	(totalRows +1) - row

So the upper_limit of col variable is generalised to (**totalRows + 1 – row**).

The lower_limit & upper_limit are consistent for all the rows. It's now generalised for any number of rows. So our If-condition becomes;

if-condition	lower_limit <=col<= upper_limit
Mathematical expression	1 <= col <= ((totalRows +1) – row)
`Syntax`	`if( col >= 1 &&  col <= ( totalRows + 1 - row) )`

We can summarise our findings in a flow-chart given as follows.



Let's implement the program.

NumberPatternCC1.c

```c
#include <stdio.h>
int main() {

	int totalRows = 5; //number of rows to display

	// Row level Repetitive Action :
	for ( int row = 1; row <= totalRows; row++ ) {

		// Column level Repetitive Action :
		// Action1.Move cursor in the same row
		// Action2.print character

		// spaces to be printed before printing numbers
		for ( int col = 1; col <= totalRows; col++ ) {

			if ( col >= 1 && col <= ( totalRows + 1 - row ) ) {
				printf( "%d ", col );
			} else {
				printf( "  " );
			}

		}// inner for-loop

		printf( "\n" );// move control to the next line

	}// outer for-loop

	return 0;
}
```

Output

```
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
```

The inner for-loop's *lower_limit* matches with the if-condition's lower_limit.

```
for(int col = 1 ; col <= totalRows ; col++) {
	if( col >= 1 &&  col <= (totalRows + 1 - row) ) {
		---- statements ------
	}
}
```

We can safely neglect the *lower_limit* in if-condition as it will become redundant condition, since already been taken care in inner for-loop.

Hence, our final if-condition can be given as;

if-condition	lower_limit <=col<= upper_limit
Mathematical expression	col <= ((totalRows +1) – row)
Syntax	`if( col <= (totalRows + 1 - row) )`

NumberPatternCC2.c

```c
#include <stdio.h>
int main() {

	int totalRows = 5; //number of rows to display

	// Row level Repetitive Action :
	for ( int row = 1; row <= totalRows; row++ ) {

		// Column level Repetitive Action :
		// Action1.Move cursor in the same row
		// Action2.print character

		// spaces to be printed before printing numbers
		for ( int col = 1; col <= totalRows; col++ ) {

			if ( col <= ( totalRows + 1 - row ) ) {
				printf( "%d ", col );
			} else {
				printf( "  " );
			}
		}// inner for-loop

		printf( "\n" );// moves control to the next line
	}// outer for-loop

	return 0;
}
```

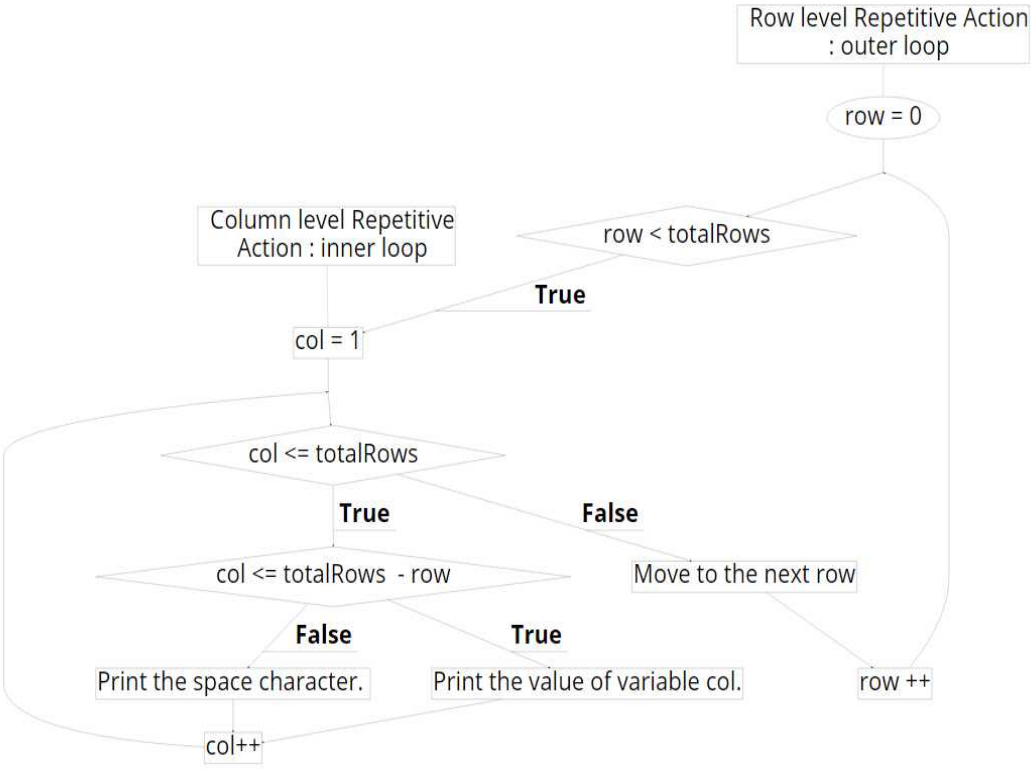
Output

```
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
```

Further, simplifying on subtracting 1 from row values and upper_limit of col values by utilizing the previous obtained trace table.

Column level Repetitive Action				
row		col		
	1	lower_limit	upper_limit	
			1	2
1 -1	**0**	1	(totalRows +1) – row - 1	**(totalRows – row)**
2 - 1	**1**	1	(totalRows +1) – row - 1	**(totalRows – row)**
3 -1	**2**	1	(totalRows +1) – row - 1	**(totalRows – row)**
4 - 1	**3**	1	(totalRows +1) – row - 1	**(totalRows – row)**
5 - 1	**4**	1	(totalRows +1) – row - 1	**(totalRows – row)**

if-condition	lower_limit <=col<= upper_limit
Mathematical expression	col <= (totalRows – row)
`Syntax`	`if( col <= (totalRows - row)`



Let's update the program with our latest logical condition modification.

NumberPatternCC3.c

```c
#include <stdio.h>
int main() {

	int totalRows = 5; //number of rows to display

	// Row level Repetitive Action :
	for ( int row = 0; row < totalRows; row++ ) {

		// Column level Repetitive Action :
		// Action1.Move cursor in the same row
		// Action2.print character

		// spaces to be printed before printing numbers
		for ( int col = 1; col <= totalRows; col++ ) {

			if ( col <= ( totalRows - row ) ) {
				printf( "%d ", col );
			} else {
				printf( "  " );
			}

		}// inner for-loop

		printf( "\n" );// move control to the next line

	}// outer for-loop

	return 0;
}
```

Output

```
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
```

```
5 4  3 2 1
  4 3 2 1
    3 2 1
      2 1
        1
```

NumberPatternDD

Tabular representation

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	5	4	3	2	1
Row=2	space	4	3	2	1
Row=3	space	space	3	2	1
Row=4	space	space	space	2	1
Row=5	space	space	space	space	1

(Row, Col) value pairs representation.

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	1,1	1,2	1,3	1,4	1,5
Row=2		2,2	2,3	2,4	2,5
Row=3			3,3	3,4	3,5
Row=4				4,4	4,5
Row=5					5,5

Analysis of output

Let's apply decrementing for-loop for col variable (5 to 1) or (5 >=col>=1) so that the numeric output will match partially or wholly with the values of col variable.

row			1		
col	5	4	3	2	1
output	5	4	3	2	1

row =1 , 5 >=col>=1 (print col value)

row			2		
col	5	4	3	2	1
output		4	3	2	1

row =2 , 4 >=col>=1 (print col value)

row			3		
col	5	4	3	2	1
output			3	2	1

row =3 , 3 >=col>=1 (print col value)

row			4		
col	5	4	3	2	1
output				2	1

row =4 , 2>=col>=1 (print col value)

row			1		
col	5	4	3	2	1
output					1

row =5 , 1>=Col >= 1 (print col value)

Column Level Repetitive Action

Row level Repetitive Action			**Expected Output**
row	**Column level Repetitive Action**		
	upper_limit >=col>= lower_limit		
	lower_limit	upper_limit	
1	1	5	5 4 3 2 1
2	1	4	4 3 2 1
3	1	3	3 2 1
4	1	2	2 1
5	1	1	1

The lower_limit has same consistent expression as numeric value for the all the rows.

Likewise we need to come out with such a consistent expression for upper_limit which should suit any number of rows.

The available variables are row& totalRows=5. Let's try to generalise using these variables.

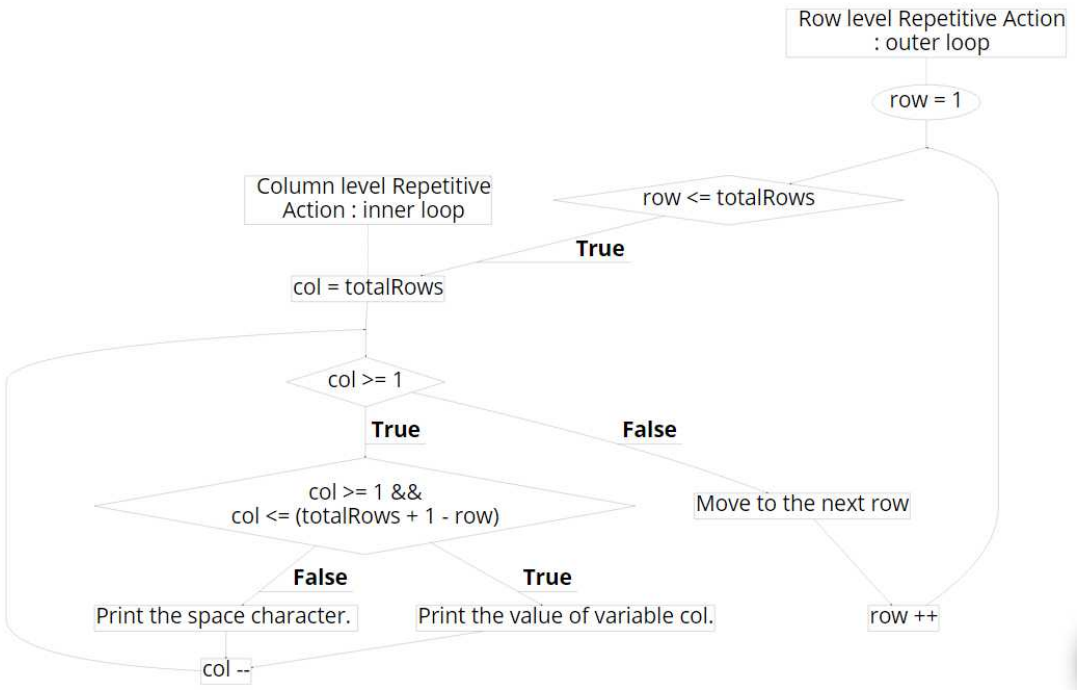
Column level Repetitive Action					
row	*If-condition(upper_limit >=col>= lower_limit)*				
	lower_limit	upper_limit			
			1	2	3
1	**1**	5	totalRows	totalRows +1 - 1	**totalRows +1 - row**
2	**1**	4	totalRows - 1	totalRows +1 - 2	**totalRows +1 - row**
3	**1**	3	totalRows - 2	totalRows +1 - 3	**totalRows +1 - row**
4	**1**	2	totalRows - 3	totalRows +1- 4	**totalRows +1 - row**
5	**1**	1	totalRows - 4	totalRows +1- 5	**totalRows +1 - row**

The lower_limit & upper_limit are become consistent for all the rows. The upper_limit is generalised to (**totalRows + 1 – row**).This logical expression is such a formula, commonly applied on all rows and columns which in turn helps to give matching output.

So our If-condition becomes;

if-condition	*upper_limit >=col>= lower_limit*
Mathematical expression	1 <= col <= ((totalRows +1) – row)
`Syntax`	`if( col >= 1 &&  col <= (totalRows + 1 - row) )`

The flow-chart can be given as follows.



NumberPatternDD1.c

```c
#include <stdio.h>
int main() {

	int totalRows = 5; //number of rows to display

	// Row level Repetitive Action:
	for ( int row = 1; row <= totalRows; row++ ) {

		// Column level Repetitive Action:
		// Action1.Move cursor in the same row
		// Action2.print numeric character

		// spaces to be printed before printing numbers
		for ( int col = totalRows; col >= 1; col-- ) {

			if ( col >= 1 && col <= ( totalRows + 1 - row ) ) {
				printf( "%d ", col );
			} else {
				printf( "  " );
			}

		}// inner for-loop

		printf( "\n" );// Moves control to the next line

	}// outer for-loop

	return 0;
}
```

Output

```
5 4 3 2 1
  4 3 2 1
    3 2 1
      2 1
        1
```

The decrementing inner for-loop's lower_limit matches with the if-condition's lower_limit as shown with bold.

The **lower_limit** in if-condition becomes an extra condition as inner for-loop's **loop-condition** (**col >= 1)** already taking care of it.

```
for(int col = totalRows    ; col >= 1 ; col--) {

        if( col >= 1 &&  col <= (totalRows + 1 - row) ) {
                ---- statements ------
        }
}
```

Hence, our final if-condition will be;

if-condition	*upper_limit >=col>= lower_limit*
Mathematical expression	col <= ((totalRows +1) – row)
Syntax	`if( col <= (totalRows + 1 - row) )`

NumberPatternDD2.c

```c
#include <stdio.h>
int main() {

	int totalRows = 5; //number of rows to display

	// Row level Repetitive Action :
	for ( int row = 1; row <= totalRows; row++ ) {

		// Column level Repetitive Action :
		// Action1.Move cursor in the same row
		// Action2.print character

		// spaces to be printed before printing numbers
		for ( int col = totalRows; col >= 1; col-- ) {

			if ( col <= ( totalRows + 1 - row ) ) {
				printf( "%d ", col );
			} else {
				printf( "  " );
			}

		}// inner for-loop

		printf( "\n" );// moves control to the next line

	}// outer for-loop

	return 0;
}
```

Output

```
5 4 3 2 1
  4 3 2 1
    3 2 1
      2 1
        1
```

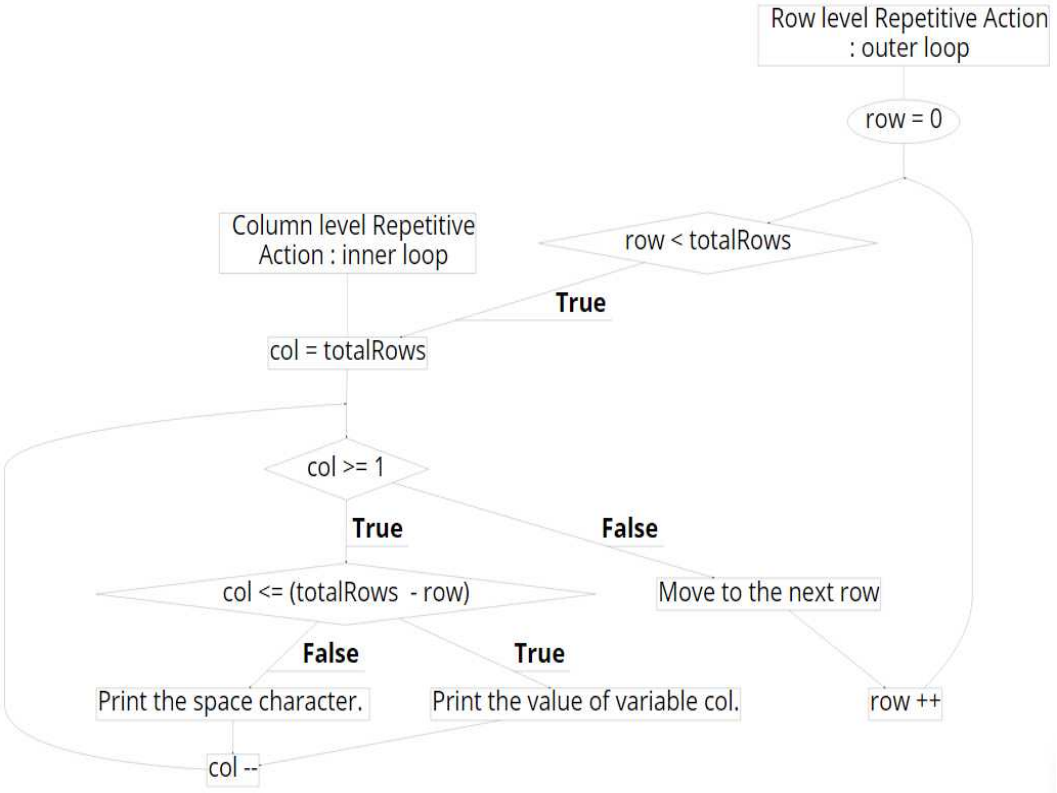
Further, simplifying upper_limit by subtracting 1 as shown in the following;

Column level Repetitive Action				
row		*upper_limit >=col>= lower_limit*		
		lower_limit	upper_limit	
1 - 1	0	1	(totalRows +1) – row - 1	(totalRows – row)
2 - 1	1	1	(totalRows +1) – row - 1	(totalRows – row)
3 - 1	2	1	(totalRows +1) – row - 1	(totalRows – row)
4 - 1	3	1	(totalRows +1) – row - 1	(totalRows – row)
5 - 1	4	1	(totalRows +1) – row - 1	(totalRows – row)

Hence, our improved if-condition will be,

if-condition	*upper_limit >=col>= lower_limit*
Mathematical expression	col <= ((totalRows – row)
`Syntax`	`if( col <= (totalRows - row) )`

The modification can be easily expressed in the flow-chart.



The updated final solution can be expressed in the following program.

NumberPatternDD3.c

```c
#include <stdio.h>
int main() {

	int totalRows = 5; //number of rows to display

	// Row level Repetitive Action :
	for ( int row = 0; row < totalRows; row++ ) {

		// Column level Repetitive Action :
		// Action1.Move cursor in the same row
		// Action2.print character

		// spaces to be printed before printing numbers
		for ( int col = totalRows; col >= 1; col-- ) {

			if ( col <= ( totalRows - row ) ) {
				printf( "%d ", col );
			} else {
				printf( "  " );
			}

		}

		// move control to the next line
		printf( "\n" );
	}

	return 0;
}
```

Output

```
5 4 3 2 1
  4 3 2 1
    3 2 1
      2 1
        1
```

```
        1
      2 1 2
    3 2 1 2 3
  4 3 2 1 2 3 4
5 4 3 2 1 2 3 4 5
```

NumberPatternEE

Tabular representation

	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1	space	space	space	space	1	space	space	space	space
Row=2	space	space	space	2	1	2	space	space	space
Row=3	space	space	3	2	1	2	3	space	space
Row=4	space	4	3	2	1	2	3	4	space
Row=5	5	4	3	2	1	2	3	4	5

(Row, Col) value pairs representation.

	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1					1,5				
Row=2				2,4	2,5	2,6			
Row=3			3,3	3,4	3,5	3,6	3,7		
Row=4		4,2	4,3	4,4	4,5	4,6	4,7	4,8	
Row=5	5,1	5,2	5,3	5,4	5,5	5,6	5,7	5,8	5,9

Analysis of output

row	1								
col	1	2	3	4	5	6	7	8	9
output					1				

For row = 1, 5 <=Col <= 5

row	2								
col	1	2	3	4	5	6	7	8	9
output				2	1	2			

For row = 2, 4 <=Col <= 6

row	3								
col	1	2	3	4	5	6	7	8	9
output			3	2	1	2	3		

For row = 3, 3 <=Col <= 7

row	1								
col	1	2	3	4	5	6	7	8	9
output		4	3	2	1	2	3	4	

For row = 4, 2 <=Col <= 8

row	1							
col	1	3	4	5	6	7	8	9
output	5	3	2	1	2	3	4	5

For row = 5, 1 <=Col <= 9

Column Level Repetitive Action

Row level Repetitive Action			**Expected Output**
row	**Column level Repetitive Action**		
	lower_limit <= col <= upper_limit		
	lower_limit	upper_limit	
1	5	5	1
2	4	6	2 1 2
3	3	7	3 2 1 2 3
4	2	8	4 3 2 1 2 3 4
5	1	9	5 4 3 2 1 2 3 4 5

The lower_limit & upper_limit ,both are not consistent for all the rows. We need to come out with consistent expressions for both upper_limit and lower_limit which should suit for any number of rows. The other remaining variables are **row, totalRows = 5**. Let's try to generalise using these variables.

Column level Repetitive Action						
row	*If-condition(lower_limit <= col <= upper_limit)*					
	lower_limit			upper_limit		
		1	2		1	2
1	5	totalRows	totalRows +1 - 1	5	totalRows	totalRows +1 - 1
2	4	totalRows -1	totalRows +1 - 2	6	totalRows + 1	totalRows + 2 - 1
3	3	totalRows -2	totalRows +1 - 3	7	totalRows + 2	totalRows + 3 - 1
4	2	totalRows -3	totalRows +1 - 4	8	totalRows + 3	totalRows + 4 - 1
5	1	totalRows -4	totalRows +1 - 5	9	totalRows + 4	totalRows + 5 - 1

Column level Repetitive Action				
row	*lower_limit <= col <= upper_limit*			
	lower_limit		upper_limit	
	default	3	default	3
1	5	totalRows +1 - row	5	totalRows + row - 1
2	4	totalRows +1 - row	6	totalRows + row - 1
3	3	totalRows +1 - row	7	totalRows + row - 1
4	2	totalRows +1 - row	8	totalRows + row - 1
5	1	totalRows +1 - row	9	totalRows + row - 1

The lower_limit & upper_limit are become consistent for all the rows.

The upper_limit is generalised to (**totalRows + row – 1**).

This logical expression is such a formula, commonly applied on all rows and columns which in turn helps to give matching output. So our If-condition becomes;

if-condition	lower_limit <=col_sp<= upper_limit
Mathematical expression	(totalRows +1 - row) <= col <= (totalRows + row - 1)
`Syntax`	`if( col >= ( (totalRows - row) + 1 ) && col <= ( ( totalRows + row ) - 1 ) )`

When the if-condition satisfied, it will print the numeric character at that position at a particular row or else will print the space character. We can observe two things from the given output which can be given as;

a) The values at each row starts with the row value and goes on decreases till the value 1 and further increases till it reaches the value of row value. The value basically oscillating like a pendulum of a wall clock. Let's choose a variable **num** which represents a row value, every time a new row starts. The value of variable **num** then oscillates to give output in a single row. It can be shown by following table.

Table for num variable starting at row value.

row	num	Col<totalRows	Col=totalRows	Col>totalRows
1	1		1	
2	2	2	1	2
3	3	3 2	1	2 3
4	4	4 3 2	1	2 3 4
5	5	5 4 3 2	1	2 3 4 5

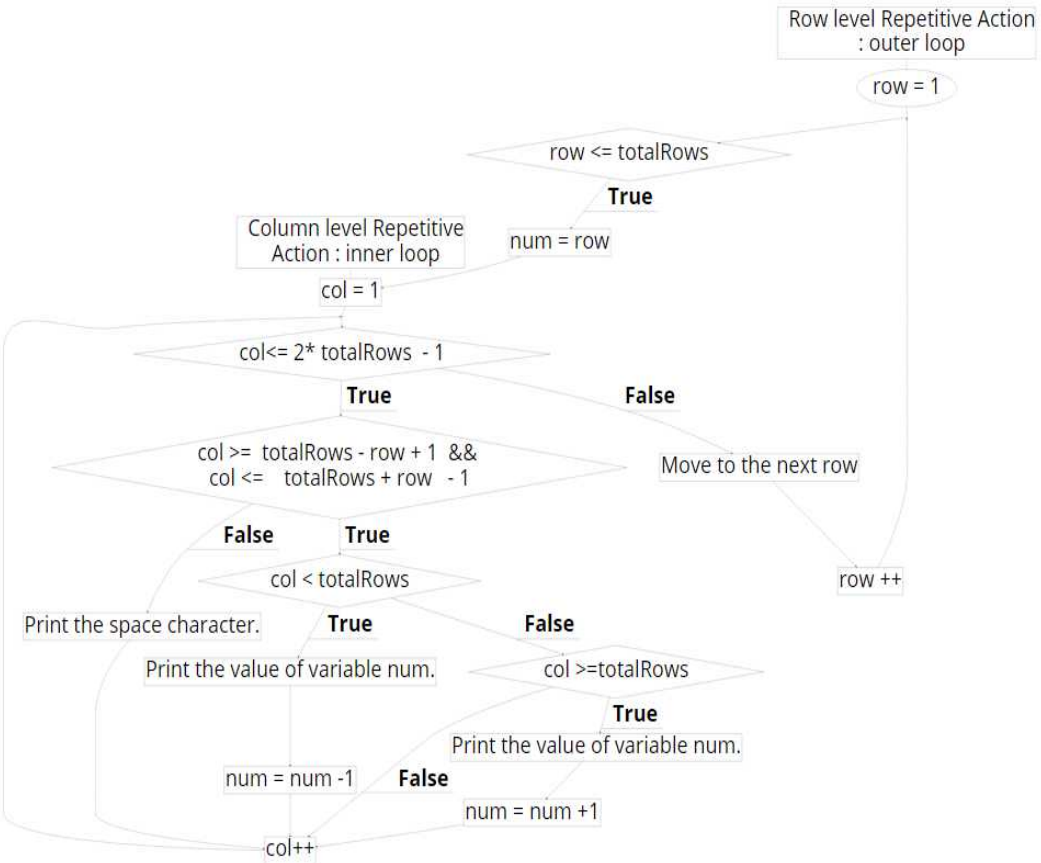
```
If(  col < totalRows ){
        num value gets printed.
        num value get decreased by 1.
 }
If( col >= totalRows ){
        num value gets printed.
        num value get increased by 1.
 }
```

b) The column counts are more than the **totalRows** count other than the first row and the input value of variable **totalRows** is 5.The column count is different on each row. Considering all these factors the upper_limit of inner for–loop need to be generalised.

loop-condition	lower_limit <=col<= upper_limit
Mathematical expression	1 <= col <= 9
	1 <=col <= **totalRows + 4**
	1 <=col <= **totalRows + (**totalRows – 1)
	1<=col<=2* totalRows - 1
`Syntax`	`for( col = 1 ; col <= ( 2* totalRows  - 1 ) ; col++)`

So this generalisation to calculate total columns for a given row value is satisfactory.

The flow-chart can be given as;



NumberPatternEE.c

```c
#include <stdio.h>
int main() {

	int totalRows = 8; //number of rows to display

	// Row level Repetitive Action :
	for ( int row = 1; row <= totalRows; row++ ) {

		//repetition count (2* totalRows  - 1) happens column-wise
		int num = row;

		// Column level Repetitive Action :
		// Action1.Move cursor in the same row
		// Action2.print character
		for ( int col = 1; col <= ( 2 * totalRows - 1 ); col++ ) {

			if ( col >= ( ( totalRows - row ) + 1 ) &&
				col <= ( ( totalRows + row ) - 1 ) ) {

				if ( col < totalRows ) {
					// 1st half triangle
					printf( "%d ", num );
					num = num - 1;
				} else if ( col >= totalRows ) {
					// 2nd half traingle
					printf( "%d ", num );
					num = num + 1;
				}

			} else {
				printf( "  " );
			}
		}//end  inner for - loop

		printf( "\n" );// move control to the next line

	}// end outer for-loop

	return 0;
}
```

Output

```
            1
          2 1 2
        3 2 1 2 3
      4 3 2 1 2 3 4
    5 4 3 2 1 2 3 4 5
  6 5 4 3 2 1 2 3 4 5 6
7 6 5 4 3 2 1 2 3 4 5 6 7
8 7 6 5 4 3 2 1 2 3 4 5 6 7 8
```

```
1 2 3 4 5 4 3 2 1
1 2 3 4   4 3 2 1
1 2 3       3 2 1
1 2           2 1
1               1
```

NumberPatternFF

Tabular representation

	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1	1	2	3	4	5	4	3	2	1
Row=2	1	2	3	4	space	4	3	2	1
Row=3	1	2	3	space	space	space	3	2	1
Row=4	1	2	space	space	space	space	space	2	1
Row=5	1	space	space	space	space	space	space	space	1

(Row, Col) value pairs representation.

	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1	(1,1) 1	(1,2) 2	(1,3) 3	(1,4) 4	(1,5) 5	(1,6) 4	(1,7) 3	(1,8) 2	(1,9) 1
Row=2	(2,1) 1	(2,2) 2	(2,3) 3	(2,4) 4	**2,5 space**	(2,6) 4	(2,7) 3	(2,8) 2	(2,9) 1
Row=3	(3,1) 1	(3,2) 2	(3,3) 3	**3,4 space**	**3,5 space**	**3,6 space**	(3,7) 3	(3,8) 2	(3,9) 1
Row=4	(4,1) 1	(4,2) 2	**4,3 space**	**4,4 space**	**4,5 space**	**4,6 space**	**4,7 space**	(4,8) 2	(4,9) 1
Row=5	(5,1) 1	**5,2 space**	**5,3 space**	**5,4 space**	**5,5 space**	**5,6 space**	**5,7 space**	**5,8 space**	(5,9) 1

Analysis of output

In general, three repetitive action takes place.

a) Printing of numbers starting from 1 in incrementing order.

b) Printing of spaces

c) Printing of numbers in decrementing order and it's upper limit will start from last updated incremented value.

Column Level Repetitive Action

Row level Repetitive Action			
row	**Column level Repetitive Action**		
	Variable :numInc	Variable :space (space counts)	Variable :numDec
1	1, 2 ,3 ,4, 5	0(no space)	4, 3, 2, 1
2	1, 2, 3, 4	1	4, 3, 2, 1
3	1, 2, 3	1,2,3	3, 2, 1
4	1, 2	1,2,3,4,5	2, 1
5	1	1,2,3,4,5,6,7	1

Row level Repetitive Action						
row	**Column level Repetitive Action**					
	Variable :numInc		Variable :space		Variable :numDec	
	lower_ limit	upper_limit	lower_limit	upper_limit	upper_limit	lower_limit
1	1	5	0	0	4	1
2	1	4	1	1	4	1
3	1	3	1	3	3	1
4	1	2	1	5	2	1
5	1	1	1	7	1	1

We know that total number of rows is given by the variable **totalRows**=5.We need to generalise to get unique expression. The **upper_limit** of variable **numInc** and **numDec** can be expressed in terms of variable **totalRows**.

Row level Repetitive Action						
row	**Column level Repetitive Action**					
	Variable :numInc		Variable :space		Variable :numDec	
	lower_ limit	upper_limit	lower_limit	upper_limit	upper_limit	lower_limit
		1			1	
1	1	totalRows - 0	0	0	totalRows - 1	1
2	1	totalRows - 1	1	1	totalRows - 1	1
3	1	totalRows - 2	1	3	totalRows - 2	1
4	1	totalRows - 3	1	5	totalRows - 3	1
5	1	totalRows - 4	1	7	totalRows - 4	1

If we take values of row variable starting from 0 instead of 1, then it will help us to generalise the **upper_limit** of variable **numInc** and **numDec.** Let try to generalise both of these using variable **row**.

Row level Repetitive Action						
row	**Column level Repetitive Action**					
	Variable :numInc		Variable :space		Variable :numDec	
	lower_ limit	upper_limit	lower_limit	upper_limit	upper_limit	lower_limit
		1				
0	1	totalRows - **0**	0	0	totalRows - **1**	1
1	1	totalRows - **1**	1	1	totalRows - **1**	1
2	1	totalRows - **2**	1	3	totalRows - **2**	1
3	1	totalRows - **3**	1	5	totalRows - **3**	1
4	1	totalRows - **4**	1	7	totalRows - **4**	1

The part of **upper_limit** of variable **numInc** and **numDec** matches with numeric pattern of row variable. Hence, we can use row variable to generalise the expressions.

row	**Column level Repetitive Action**					
	Variable :numInc		Variable :space		Variable :numDec	
	lower_limit	upper_limit 1	lower_limit	upper_limit	upper_limit	lower_limit
0	1	totalRows - **row**	0	0	totalRows - **1**	1
1	1	totalRows - **row**	1	1	totalRows - **row**	1
2	1	totalRows - **row**	1	3	totalRows - **row**	1
3	1	totalRows - **row**	1	5	totalRows - **row**	1
4	1	totalRows - **row**	1	7	totalRows - **row**	1

For the uppler_limit of variable **numDec** we got two expressions. Somehow by putting a condition we need to switch the expression depends upon the value of row variable. Let's take one temporary variable **ul _ numDec** to represent upper_limit of the variable **numDec,** then it can be given as **;**

```
Variable : ul _ numDec

if(row == 0 ){
   ul_ numDec is ( totalRows – 1)
 }else{
   ul_ numDec is (totalRows - row).
 }
```

Till now we are able to generalise variables **numInc** and **numDec,** since their lower_limit and upper_limit are showing same expression for all the row values.

Row level Repetitive Action						
row	**Column level Repetitive Action**					
	Variable :numInc		Variable :space		Variable :numDec	
	lower_limit	upper_limit 1	lower_limit	**upper_limit**	upper_limit	lower_limit
0	1	totalRows - **row**	0	**0**	ul _ numDec	1
1	1	totalRows - **row**	1	**1**	ul _ numDec	1
2	1	totalRows - **row**	1	**3**	ul _ numDec	1
3	1	totalRows - **row**	1	**5**	ul _ numDec	1
4	1	totalRows - **row**	1	**7**	ul _ numDec	1

What remained now is to generalise the **upper_limit** for the variable **spaces.**

row	**Column level Repetitive Action**						
	Variable :space						
	lower_limit	upper_limit					
			1	2	3	4	5
0	0	0	0	0	0	0	0
1	1	1	1	1	1 + 1 - 1	2*1 - 1	2*row -1
2	1	3	2 + 1	2 + 2 - 1	2 + 2 - 1	2*2 - 1	2*row -1
3	1	5	3 + 2	3 + 3 - 1	3 + 3 - 1	2*3 - 1	2*row -1
4	1	7	4 + 3	4 + 4 - 1	4 + 4 - 1	2*4 - 1	2*row -1

row	**Column level Repetitive Action**						
	Variable :space						
	lower_limit	upper_limit					
			1	2	3	4	5
0	1	0	0	0	0	0	2*row -1
1	1	1	1	1	1 + 1 - 1	2*1 - 1	2*row -1
2	1	3	2 + 1	2 + 2 - 1	2 + 2 - 1	2*2 - 1	2*row -1
3	1	5	3 + 2	3 + 3 - 1	3 + 3 - 1	2*3 - 1	2*row -1
4	1	7	4 + 3	4 + 4 - 1	4 + 4 - 1	2*4 - 1	2*row -1

For the uppler_limit of variable **space at row = 0,** the value will be (2*0 – 1) or -1**.**The lower_limit value of variable **space at row = 0 is also 0,** so the loop-condition (0 < -1) will fail and no space will get printed. This holds true even for **loop-condidtion** (1 < -1).

Hence, we are in a position to get generalised expression for lower_limit and upper_limit.

loop-condition	lower_limit <=space<= upper_limit
Mathematical expression	1<=space<=(2* row - 1)
`Syntax`	`for( space = 1 ; space <= ( 2* row – 1 ) ; space++)`

This time somehow we are able to generalise the expressions otherwise we need to follow if-else condition as we did for the variable **numDec** using temporary variable **ul _ numDec.**

Alternate solution

Row level Repetitive Action				
row	**Column level Repetitive Action**			
	Variable :col	Variable :col	Variable :col	
	Output	spaces		**Output**
1	1, 2 ,3 ,4	Print 5(no space)	6, 7, 8, 9	4, 3, 2, 1
2	1, 2, 3, 4	5	6, 7, 8, 9	4, 3, 2, 1
3	1, 2, 3	4,5,6	7, 8, 9	3, 2, 1
4	1, 2	3,4,5,6,7	8, 9	2, 1
5	1	2,3,4,5,6,7,8	9	1

Row level Repetitive Action							
row	**Column level Repetitive Action**						
	1 <=col<=9						
	Print numbers		**Print spaces**		**Print numbers**		
	lower_ limit	upper_limit	lower_ limit	upper_limit	lower_ limit	upper_limit	**Output**
1	1	4	Print 5		6	9	4, 3, 2, 1
2	1	4	5	5	6	9	4, 3, 2, 1
3	1	3	4	6	7	9	3, 2, 1
4	1	2	3	7	8	9	2, 1
5	1	1	2	8	9	9	1

For printing characters at the first row, means at row=1, this pattern doesn't required to print any space. So for the sake of initialization we can give it's upper_limit value as -1 and lower_limit value as 0.If we observe for printing spaces, the numeric pattern is such that the lower_limit value is less than upper_limit value for every row except the first row. This will help to allow, condition to be fail and we will be not able to print space in the first row. This what we need.

Moreover it's one of the common practice to initialize variable with invalid data in order to verify validity of the programming logic whenever that programming logic gets executed. The values of variable acts like a check point whether the programming logic executes as per expected or not and this our purpose of doing exercises on programming patterns to be able to visualize the flow of data.

If we take values of row variable starting from 0 instead of 1, then it will help us to generalise the **upper_limit .** Let try to generalise both of these using variable **row**.

If we observe, we try to generalise lower_limit & upper_limit of printing spaces column using **row** variable. With using simple maths we can alter lower_limit & upper_limit so that part of expression which is following a numeric patter may match with the numeric pattern of **row** variable.

Row level Repetitive Action							
row	**Column level Repetitive Action**						
	1 <=col<=9						
	Print numbers		Print spaces		Print numbers		
	lower_ limit	upper_limit	lower_limit	upper_limit	lower_ limit	upper_ limit	**Output**
1	1	totalRows - 1	0	-1	6	9	4, 3, 2, 1
2	1	totalRows - 1	totalRows - 0	totalRows + 0	6	9	4, 3, 2, 1
3	1	totalRows - 2	totalRows - 1	totalRows + 1	7	9	3, 2, 1
4	1	totalRows - 3	totalRows - 2	totalRows + 2	8	9	2, 1
5	1	totalRows - 4	totalRows - 3	totalRows + 3	9	9	1

Row level Repetitive Action							
row	**Column level Repetitive Action**						
	1 <=col<=9						
	Print numbers		Print spaces		Print numbers		
	lower_ limit	upper_limit	lower_limit	upper_limit	lower_ limit	upper_ limit	**Output**
0	1	totalRows - 1			6	9	4, 3, 2, 1
1	1	totalRows - 1	totalRows - 0	totalRows + 0	6	9	4, 3, 2, 1
2	1	totalRows - 2	totalRows - 1	totalRows + 1	7	9	3, 2, 1
3	1	totalRows - 3	totalRows - 2	totalRows + 2	8	9	2, 1
4	1	totalRows - 4	totalRows - 3	totalRows + 3	9	9	1

Row level Repetitive Action						
row	**Column level Repetitive Action**					
	1 <=col<=9					
	Print numbers		Print spaces		Print numbers	
	lower_ limit	upper_limit	lower_limit	upper_limit	lower_ limit	upper_ limit
0	1	totalRows - 1	0	-1	6	9
1	1	totalRows - row	totalRows – 1 + 1	totalRows + 1 - 1	6	9
2	1	totalRows - row	totalRows – 2 + 1	totalRows + 2 - 1	7	9
3	1	totalRows - row	totalRows – 3 + 1	totalRows + 3 - 1	8	9
4	1	totalRows - row	totalRows – 4 + 1	totalRows + 4 - 1	9	9

Row level Repetitive Action						
row	**Column level Repetitive Action**					
	1 <=col<=9					
	Print numbers(set 1)		Print spaces		Print numbers(set 2)	
	lower_ limit	upper_limit	lower_limit	upper_limit	lower_ limit	upper_ limit
0	1	totalRows - 1	0	-1	6	9
1	1	totalRows - row	totalRows – row + 1	totalRows + row - 1	6	9
2	1	totalRows - row	totalRows – row + 1	totalRows + row - 1	7	9
3	1	totalRows - row	totalRows – row + 1	totalRows + row - 1	8	9
4	1	totalRows - row	totalRows – row + 1	totalRows + row - 1	9	9

If we build our programming logic giving conditions around set1 of printing numbers and printing spaces, then in the last else-part will take of set2 of printing numbers.

By putting a condition we need to switch the expression of **lower_limit** or **upper_limit** depends upon the value of row variable.

Print numbers(set 1)

lower_limit variable : **lw_lt_num**

```
if(row == 0){
  lw_lt_num = ( totalRows - 1 )
}else{
 lw_lt_num = ( totalRows - row )
}
```

Print spaces

lower_limit variable : **lw_lt_spaces**

```
if(row == 0){
  lw_lt_spaces = 0;
}else{
 lw_lt_spaces =( totalRows - row + 1 );
}
```

Print spaces

upper_limit variable : **up_lt_spaces**

```
if(row == 0){
  up_lt_spaces = -1;
}else{
  up_lt_spaces =( totalRows + row - 1 )
}
```

We know that total number of rows is given by the variable **totalRows**=5 and col variable takes the values 1.. 9 and let to generalize the uppler_limit.

Col = 1, 2, 3… 9

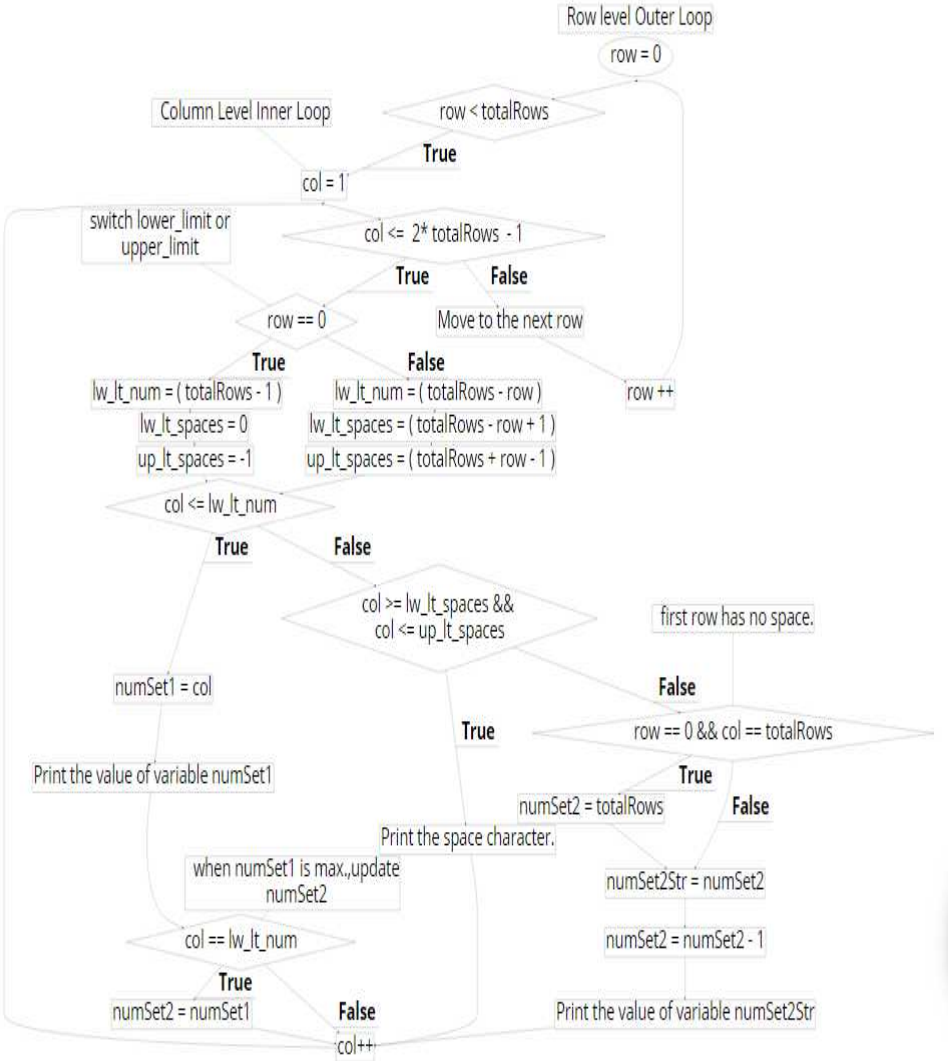
Col = 1, 2, 3… (10 -1)

Col = 1, 2, 3…, (2*5 -1)

Col = 1, 2, 3…, (2* **totalRows** -1)

Hence we expressed upper_limit of col variable as (2* **totalRows** -1) in a generalised form.

We can summarise all our observation in a flow-chart given as follows.



NumberPatternFF.c

```c
#include <stdio.h>
int main() {

	int totalRows = 5; //number of rows to display
	int lw_lt_num, lw_lt_spaces, up_lt_spaces;
	int numSet1;
	int numSet2 = 0;

	// Row level Repetitive Action :
	for ( int row = 0; row < totalRows; row++ ) {
		// Column level Repetitive Action :
		for ( int col = 1; col <= ( 2 * totalRows - 1 ); col++ ) {
			// switching to lower_limit or upper_limit
			// logical expressions
			if ( row == 0 ) {
				lw_lt_num = ( totalRows - 1 );
				lw_lt_spaces = 0;
				up_lt_spaces = -1;
			} else {
				lw_lt_num = ( totalRows - row );
				lw_lt_spaces = ( totalRows - row + 1 );
				up_lt_spaces = ( totalRows + row - 1 );
			}

			if ( col <= lw_lt_num ) {
				numSet1 = col;
				printf( "%d ", numSet1 );
				// updating numSet2 when numSet1 reached
				// it's maximum value
				if ( col == lw_lt_num ) {
					numSet2 = numSet1;
				}
			} else if ( col >= lw_lt_spaces &&
					col <= up_lt_spaces ) {
				printf( "  " );// 2 spaces
			} else {
				// first row where no space is required to print.
				if ( row == 0 &&
				     col == totalRows ) {
					numSet2 = totalRows;
				}
				printf( "%d ", numSet2 );
				numSet2 = numSet2 - 1;
			}
		}// inner loop

		printf( "\n" );// Move cursor to the next row

	}// outer loop

	return 0;
}
```

```
Output
1 2 3 4 5 4 3 2 1
1 2 3 4   4 3 2 1
1 2 3       3 2 1
1 2           2 1
1               1
```

If-else Shortcut

Short-cut in any matter is a special case to be applied in a specific situation. Not only in subject of programming people unknowingly confuse themselves simply by the fact that they choose a special case and tried to apply it commonly. Even in choice of words, two words may give almost similar meaning and when we actually apply those in different situations then only we will be in a position to explore it's true potential meaning.

The shortcut to if-else option is there in almost every programming language which is also known as **ternary operator** and is given by the symbol (:).

When to apply if-else shortcut

The ternary operator is a sort of syntactic sugar for if-else statements usually applied in a situation where both **if-block** and **else-block** assign different value to the same result variable. For example, let's suppose that if-else statement is something like the following;

If-else statement	**If-else shortcut or ternary operator (:)**

If(**condition**){
result = *one_value*;
}else{
result = *other_value*;
}
	result = (**condition**)**?** *one_value* **:** *other_value* ;

Simple if-else can be easily replaced by the use of if-else shortcut represented by the one liner **ternary operator.** In such situations only it makes true sense to use ternary **operator** to increase code readability.	• Two outcome result values are possible which are separated by a symbol colon (:). • If **condition** is true, then *one_value* will get assigned to the variable result. • If **condition** is false, then *one_value* will get assigned to the variable result.

In the program **NumberPatternFF.c** we can use ternary operator to improve that part of source code readability.

Separate if-else

```
if(row == 0){
  lw_lt_num = ( totalRows - 1 );
}else{
 lw_lt_num = ( totalRows - row );
}
```

```
if(row == 0){
  lw_lt_spaces = 0;
}else{
 lw_lt_spaces =( totalRows - row + 1 );
}
```

```
if(row == 0){
  up_lt_spaces = -1;
}else{
  up_lt_spaces =(totalRows + row - 1 );
}
```

Many If-else combined in single if-else

```
if(row == 0){
   lw_lt_num = ( totalRows - 1 );
   lw_lt_spaces = 0;
   up_lt_spaces = -1;
}else{
   lw_lt_num = ( totalRows - row );
   lw_lt_spaces = ( totalRows - row + 1 );
   up_lt_spaces = ( totalRows + row - 1 );
}
```

Equivalent If-else shortcut or ternary operator (:) expressions

```
lw_lt_num = (row == 0)? ( totalRows - 1 ) : ( totalRows - row );
```

```
lw_lt_spaces = (row == 0)? 0 : ( totalRows - row + 1 );
```

```
up_lt_spaces = (row == 0)? -1 : (totalRows + row - 1 );
```

I am not really a great fan of exceptions but just to share information, even **nested ternary operator is possible** like nested if-else. In most of the cases it makes hard to read the conditions and associated evaluation expressions. In some cases, a proper formatting of nested ternary operators may help but still code implemented will not be very easy read for junior level programmers atleast. In my opinion It is better to avoid nested ternary operators.

The updated program demonstrating the use of ternary operator (:).This is done in order to replace many simpler **if-else statements** to gain readability regarding programming code.

Let's check from the implementation.

NumberPatternFF2.c

```
#include <stdio.h>
int main() {

	int totalRows = 5; //number of rows to display
	int lw_lt_num, lw_lt_spaces, up_lt_spaces;
	int numSet1;
	int numSet2 = 0;
	// Row level Repetitive Action :
	for ( int row = 0; row < totalRows; row++ ) {
		// Column level Repetitive Action :
		for ( int col = 1; col <= ( 2 * totalRows - 1 ); col++ ) {
			// use of ternary operator, an simpler if-else shortcut
			lw_lt_num = ( row == 0 )?( totalRows - 1 ):(totalRows - row );
			lw_lt_spaces = ( row == 0 ) ? 0 : ( totalRows - row + 1 );
			up_lt_spaces = ( row == 0 ) ? -1 : ( totalRows + row - 1 );

			if ( col <= lw_lt_num ) {
				numSet1 = col;
				printf( "%d ", numSet1 );
				// updating numSet2 when numSet1 reach it's maximum value
				if ( col == lw_lt_num ) {
					numSet2 = numSet1;
				}
			} else if ( col >= lw_lt_spaces &&
						col <= up_lt_spaces ) {
				printf( "  ");// 2 spaces
			} else {
				// first row where no space is required to print.
				if ( row == 0 &&
					 col == totalRows ) {
					numSet2 = totalRows;
				}
				printf( "%d ", numSet2 );
				numSet2 = numSet2 - 1;
			}

		}// inner loop

		printf( "\n" );// Move cursor to the next row

	}// outer loop

	return 0;
}
```

A B C D E D C B A A B C D D C B A A B C C B A A B B A A A	a b c d e d c b a a b c d d c b a a b c c b a a b b a a a

NumberPatternGG

Tabular representation

	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1	1	2	3	4	5	4	3	2	1
Row=2	1	2	3	4	space	4	3	2	1
Row=3	1	2	3	space	space	space	3	2	1
Row=4	1	2	space	space	space	space	space	2	1
Row=5	1	space	space	space	space	space	space	space	1

(Row, Col) value pairs representation.

	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1	(1,1) 1	(1,2) 2	(1,3) 3	(1,4) 4	(1,5) 5	(1,6) 4	(1,7) 3	(1,8) 2	(1,9) 1
Row=2	(2,1) 1	(2,2) 2	(2,3) 3	(2,4) 4	**2,5 space**	(2,6) 4	(2,7) 3	(2,8) 2	(2,9) 1
Row=3	(3,1) 1	(3,2) 2	(3,3) 3	**3,4 space**	**3,5 space**	**3,6 space**	(3,7) 3	(3,8) 2	(3,9) 1
Row=4	(4,1) 1	(4,2) 2	**4,3 space**	**4,4 space**	**4,5 space**	**4,6 space**	**4,7 space**	(4,8) 2	(4,9) 1
Row=5	(5,1) 1	**5,2 space**	**5,3 space**	**5,4 space**	**5,5 space**	**5,6 space**	**5,7 space**	**5,8 space**	(5,9) 1

This pattern is a special case of **NumberPatternFF** and is about printing of alphabets instead of numbers. Every character what we see in keyboard whether it is alphabets, numbers or other special character has a unique code which is basically a number. So every character even the numbers also internally represented by a numeric value called **Unicode**. We can use in-built **typecasting** feature of C to get the alphabets from Unicode values. It is used to convert variable of one data-type to another. For example;

```
// typecasting - converting integer data-type to char data-type
int unicode = 65;
char alpahabet = (char)unicode;
```

Type casting operator

Analysis of output

In general, three repetitive action takes place.

a) Printing of alphabets starting from A in incrementing order.

b) Printing of spaces

c) Printing of alphabets in decrementing order and it's upper limit will start from last updated incremented value.

So in an increment value order of Unicode we can get alphabets. We can verify this from following program which prints all 26 alphabets, capital or small along with Unicode.

Capital Alphabets	Unicode	Small Alphabets	Unicode
A	**65**	**a**	**97**
B	**66**	**b**	**98**
…….	……	…..	…..
Z	**90**	**z**	**122**

UniCodeToAlphabets.c

```c
#include <stdio.h>
int main() {

	printf( "Alphabets and it's equivalent unicode.\n" );
	printf( "--------------------------------------\n" );
	for ( int cnt = 0; cnt < 26; cnt++ ) {
		int unicodeCap = ( 65 + cnt );
		int unicodeSmall = ( 97 + cnt );
		printf( "%c = %d  |  ", unicodeCap,unicodeCap);
		printf( "%c = %d\n", unicodeSmall,unicodeSmall);

	}

	return 0;
}
```

Output

```
Alphabets and it's equivalent unicode.
--------------------------------------
A=65  |  a=97
B=66  |  b=98
C=67  |  c=99
D=68  |  d=100
E=69  |  e=101
……..    ……..
……..    ……..
V=86  |  v=118
W=87  |  w=119
X=88  |  x=120
Y=89  |  y=121
Z=90  |  z=122
```

NumberPatternGG.c

```c
#include <stdio.h>
int main() {

	int totalRows = 5; //number of rows to display
	int lw_lt_num, lw_lt_spaces, up_lt_spaces;
	int numSet1;
	int numSet2 = 0;

	// Row level Repetitive Action :
	for ( int row = 0; row < totalRows; row++ ) {

		// Column level Repetitive Action :
		for ( int col = 1; col <= ( 2 * totalRows - 1 ); col++ ) {

		    // switching the lower_limit or upper_limit logical expressions
		      lw_lt_num =( row==0 )?( totalRows - 1 ):( totalRows - row );
		      lw_lt_spaces = ( row == 0 ) ? 0 : ( totalRows - row + 1 );
		      up_lt_spaces = ( row == 0 ) ? -1 : ( totalRows + row - 1 );

		      if ( col <= lw_lt_num ) {
		            numSet1 = col + ( 65 - 1 );
		            printf( "%c ", numSet1 );
		           // updating numSet2 when numSet1 reach it's maximum value
		            if ( col == lw_lt_num ) {
		                  numSet2 = numSet1;
		            }

		      } else if ( col >= lw_lt_spaces &&
		                  col <= up_lt_spaces ) {
		            printf( "  " );// two space characters
		      } else {
		            // first row where no space is required to print.
		            if ( row == 0 && col == totalRows ) {
		                  numSet2 = totalRows + ( 65 - 1 );
		            }
		            printf( "%c ", numSet2 );
		            numSet2 = numSet2 - 1;

		      }
		}// inner loop

		printf( "\n" );// Move cursor to the next row

	}// outer loop

	return 0;
}
```

Output

```
A B C D E D C B A
A B C D   D C B A
A B C       C B A
A B           B A
A               A
```

With little bit modification to the program **NumberPatternGG.c,** if we try we can generate the similar pattern in small alphabets. Instead of using the following for capital letters

```
numSet1 = col + ( 65 - 1); // Unicode for capital alphabet 'A' is 65
```

We can use the following

```
numSet1 = col + ( 97 - 1);  // Unicode for small alphabet 'a' is 97
```

Although with minor change we can get output in small alphabets but real point is to understand is that the most of the code will get repeated. We can avoid such repetition of programming logic by the concept of **functions** or methods. We can create our own function and use it as on demand in the same way as we are using in-built method/function like `printf("\n").`

Writing our own functions

Till now we have used many times the concept of generalised expression based on variable(s) to represent hard-coded values. We can still go ahead to generalise programming logic too. One of major advantage of function with input parameter(s) is that it helps to achieve different types of output by reusing the same programming logic.

Writing functions are a very sensible idea that suggests how we can encapsulate different programming logics differently so that repetition of same piece of programming logic should not happen in other parts of the programs. Before we apply this concept we need to understand the commonly used syntax of a function in a class with **main method** which is as follows:

return_type methodName(*parameter1, parameter2*,…){

    //programming logic

}

Where *parameter1, parameter2*… are the input(s) that a function may require and passing parameters in function is optional. The **return_type** is void if it is not returning any value or may return (int,char,..etc) using the return keyword. Since our main method is **static**, so just to keep technical stuff easy our method **printPattern** too using **static** keyword and moreover it is a part of object-oriented programming ,which is out of scope of this book**.** For the following three different types of output we can use the same **printPattern method**, encapsulating a common programming logic. The **printPattern** will print different pattern due to the differences in input parameters.

A B C D E D C B A A B C D D C B A A B C C B A A B B A A A	a b c d e d c b a a b c d d c b a a b c c b a a b b a a a	1 2 3 4 5 4 3 2 1 1 2 3 4 4 3 2 1 1 2 3 3 2 1 1 2 2 1 1 1

PatternByFunctionCall.c

```c
#include <stdio.h>

void printPattern(int totalRows, int unicode );// function declaration

/*
 * Main program showing the code reusability through functional calls.
 * It also suggests DRY(Don't Repeat Yourself) principle.
 */

int main() {

	int totalRows = 5; // number of rows to display
	int unicode_A = 65;// unicode for letter A
	int unicode_a = 97;// unicode for letter a
	int unicode_1 = 49;// unicode for digit 1.

	printf( "Print Capital Letter Pattern\n" );
	printf( "-----------------------------------\n" );
	printPattern( totalRows, unicode_A );// function call

	printf( "Print Small Letter Pattern\n" );
	printf( "-----------------------------------\n" );
	printPattern( totalRows, unicode_a );// function call

	printf( "Print Number Pattern FF\n" );
	printf( "-----------------------------------\n" );
	printPattern( totalRows, unicode_1 );// function call

	return 0;

}// method main() ends here

// method or function definition
void printPattern(int totalRows, int unicode ) {

	int lw_lt_num;
	int lw_lt_spaces;
	int up_lt_spaces;
	int numSet1 = 0 , numSet2 = 0;

	// Row level Repetitive Action :
	for ( int row = 0; row < totalRows; row++ ) {

		// Column level Repetitive Action:
		// Move cursor in the same row & prints a character.
		for ( int col = 1; col <= ( 2 * totalRows - 1 ); col++ ) {

			// switching the lower_limit or upper_limit
			// logical expressions
			lw_lt_num =(row==0)?(totalRows - 1 ) : ( totalRows - row );
			lw_lt_spaces = ( row == 0 ) ? 0 : ( totalRows - row + 1 );
			up_lt_spaces = ( row == 0 ) ? -1 : ( totalRows + row - 1 );
```

```
                        if ( col <= lw_lt_num ) {
                              numSet1 = col + ( unicode - 1 );
                              printf( "%c ", numSet1 );

                              // updating numSet2 when numSet1 reach
                              // it's maximum value
                              if ( col == lw_lt_num ) {
                                    numSet2 = numSet1;
                              }
                        //spaces are use for formatting the output
                        } else if ( col >= lw_lt_spaces &&
                                          col <= up_lt_spaces ) {
                              printf( "  " );
                        } else {
                              // first row where no space is required to print.
                              if ( row == 0 && col == totalRows ) {
                                    numSet2 = totalRows + ( unicode - 1 );
                              }
                              printf( "%c ", numSet2 );
                              numSet2 = numSet2 - 1;

                        }
                  } // inner loop

                  printf( "\n" );// Move cursor to the next row

            } // outer loop

}// method ends here
```

Output

```
Print Capital Letter Pattern
-----------------------------------
A B C D E D C B A
A B C D   D C B A
A B C       C B A
A B           B A
A               A
Print Small Letter Pattern
-----------------------------------
a b c d e d c b a
a b c d   d c b a
a b c       c b a
a b           b a
a               a
Print Number Pattern FF
-----------------------------------
1 2 3 4 5 4 3 2 1
1 2 3 4   4 3 2 1
1 2 3       3 2 1
1 2           2 1
1               1
```

A A A B B A A B C C B A A B C D D C B A A B C D E D C B A	a a a b b a a b c c b a a b c d d c b a a b c d e d c b a	1 1 1 2 2 1 1 2 3 3 2 1 1 2 3 4 4 3 2 1 1 2 3 4 5 4 3 2 1

NumberPatternHH

(Row, Col) value pairs representation.

	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1	(1,1) **A**	(1,2) space	(1,3) space	(1,4) space	(1,5) space	(1,6) space	(1,7) space	(1,8) space	(1,9) **A**
Row=2	(2,1) **A**	(2,2) **B**	(2,3) space	(2,4) space	2,5 space	(2,6) space	(2,7) space	(2,8) **B**	(2,9) **A**
Row=3	(3,1) **A**	(3,2) **B**	(3,3) **C**	3,4 space	3,5 space	3,6 space	(3,7) **C**	(3,8) **B**	(3,9) **A**
Row=4	(4,1) **A**	(4,2) **B**	(4,3) **C**	(4,4) **D**	(4,5 space	(4,6) **D**	(4,7) **C**	(4,8) **B**	(4,9) **A**
Row=5	(5,1) **A**	(5,2) **B**	(5,3) **C**	(5,4) **D**	(5,5) **E**	(5,6) **D**	(5,7) **C**	(5,8) **B**	(5,9) **A**

Analysis of output

This pattern is just reverse of pattern **NumberPatternGG.** What we have printed in first row in the pattern **NumberPatternGG**, same we need to print in the last row. So, by following the same trick we can generate this pattern. We can conclude two things about this pattern;

a) The outer for-loop which indicates the movement of cursor row-wise, need to reverse it.It means we need to follow decrementing for-loop using row variable.

b) The column wise the it will print in the same manner as it was done in **NumberPatternGG.**

ReversePatternByFunctionCall.c

```c
#include <stdio.h>
void printPattern(int totalRows, int unicode );// function declaration
/*
 * Main program showing the code re-usability through functional calls.
 * It also suggests DRY(Don't Repeat Yourself) principle.
 */
int main() {
	int totalRows = 5; // number of rows to display
	int unicode_A = 65;// unicode for letter A
	int unicode_a = 97;// unicode for letter a
	int unicode_1 = 49;// unicode for digit 1.

	printf( "Print Capital Letter Pattern\n" );
	printf( "-----------------------------------\n" );
	printPattern( totalRows, unicode_A );// function call

	printf( "\nPrint Small Letter Pattern\n" );
	printf( "-----------------------------------\n" );
	printPattern( totalRows, unicode_a );// function call

	printf( "\nPrint Number Pattern FF\n" );
	printf( "-----------------------------------\n" );
	printPattern( totalRows, unicode_1 );// function call

	return 0;
}

// method or function definition
void printPattern(int totalRows, int unicode ) {

	int lw_lt_num;
	int lw_lt_spaces;
	int up_lt_spaces;
	int numSet1 = 0, numSet2 = 0;


	// Row level Repetitive Action :
	for ( int row = (totalRows-1) ; row >= 0 ; row -- ) {
		// Column level Repetitive Action :
		// 1)Move cursor in the same row. 2)print character
		for ( int col = 1; col <= ( 2 * totalRows - 1 ); col++ ) {

			// switching the lower_limit or upper_limit
			// logical expressions
			lw_lt_num =(row==0)?( totalRows - 1 ):( totalRows - row );
			lw_lt_spaces = ( row == 0 ) ? 0 : ( totalRows - row + 1 );
			up_lt_spaces = ( row == 0 ) ? -1 : ( totalRows + row - 1 );

			if ( col <= lw_lt_num ) {
				numSet1 = col + ( unicode - 1 );
				printf( "%c ", numSet1 );

				// updating numSet2 when numSet1 reach
				// it's maximum value
				if ( col == lw_lt_num ) {
```

```
                        numSet2 = numSet1;
                    }
                //spaces are use for formatting the output
                } else if ( col >= lw_lt_spaces &&
                                col <= up_lt_spaces ) {
                    printf( "  " );
                } else {
                    // first row where no space is required to print.
                    if ( row == 0 && col == totalRows ) {
                        numSet2 = totalRows + ( 65 - 1 );
                    }
                    printf( "%c ", numSet2 );
                    numSet2 = numSet2 - 1;

                }
            } // inner loop

            printf( "\n" );// Move cursor to the next row

        } // outer loop

}// method ends here
```

Output

```
Print Capital Letter Pattern
-----------------------------------
A               A
A B           B A
A B C       C B A
A B C D   D C B A
A B C D E D C B A

Print Small Letter Pattern
-----------------------------------
a               a
a b           b a
a b c       c b a
a b c d   d c b a
a b c d E D C B A

Print Number Pattern FF
-----------------------------------
1               1
1 2           2 1
1 2 3       3 2 1
1 2 3 4   4 3 2 1
1 2 3 4 E D C B A
```

There are errors in the output related to small letter pattern and number pattern. Please correct the program to give the appropriate output.

ProblemGBEE: **Write logic trace table and a program for the following output.**

Output

```
        5
      4 5 6
    3 4 5 6 7
  2 3 4 5 6 7 8
1 2 3 4 5 6 7 8 9
```

Problem: Generate the following pattern using the concept of function.

A B C D E D C B A A B C D D C B A A B C C B A A B B A A A A B B A A B C C B A A B C D D C B A A B C D E D C B A	a b c d e d c b a a b c d d c b a a b c c b a a b b a a a a b b a a b c c b a a b c d d c b a a b c d e d c b a	1 2 3 4 5 4 3 2 1 1 2 3 4 4 3 2 1 1 2 3 3 2 1 1 2 2 1 1 1 1 2 2 1 1 2 3 3 2 1 1 2 3 4 4 3 2 1 1 2 3 4 5 4 3 2 1

Level– 3: Number pattern.

1

2 3

3 4 5

4 5 6 7

5 6 7 8 9

NumberPatternAAA

Tabular representation.

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	1				
Row=2	2	3			
Row=3	3	4	5		
Row=4	4	5	6	7	
Row=5	5	6	7	8	9

(Row, Col) value pairs representation.

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	1,1				
Row=2	2 ,1	2,2			
Row=3	3,1	3,2	3,3		
Row=4	4,1	4,2	4,3	4,3	
Row=5	5,1	5,2	5,3	5,4	5,5

Analysis of output.

It can also be interpreted in a tabular form.

row	1				
Col	1				
Output	1				

For row =1, 1<=col<=1

row	2				
Col	1	2			
Output	2	3			

For row =2, 1<=col<=2

row	3				
Col	1	2	3		
Output	3	4	5		

For row =3, 1<=col<=3

row	4				
Col	1	2	3	4	
Output	4	5	6	7	

For row =4, 1<=col<=4

row	5				
Col	1	2	3	4	5
Output	5	6	7	8	9

For row =5, 1<=col<=5

For all the rows, we need to generalise the lower_limit & upper_limit so it should be suitable for any number of rows.

Observations

3) There is repetition required at row level vertically, i.e., Row = 1 to 5.
4) There is repetition required at col level horizontally. , i.e., Col = 1 to 5.
   So it's a 2-way repetition process means require 2 for-loops(nested for – loops).

For Loop	**Lower limit**	**Upper limit**
Outer loop	row=1	row<=5
Inner Loop	col=(Unknown) Need to find a relation how exactly it's varying	(Unknown) Need to find a relation how exactly it's varying.

Column Level Repetitive Action

Row level Repetitive Action							**Expected Output**
row	**Column level Repetitive Action**						
	loop-condition(lower_limit <=col<= upper_limit)						
	lower_limit		upper_limit				
	default	1	default	1	2	3	
1	1	row	1	1	1+1 - 1	row + row - 1	1
2	2	row	3	2 + 1	2+2 - 1	row + row - 1	2 3
3	3	row	5	3 + 2	3+3 - 1	row + row - 1	3 4 5
4	4	row	7	4 + 3	4+4 - 1	row + row - 1	4 5 6 7
5	5	row	9	5 + 4	5+5 - 1	row + row - 1	5 6 7 8 9

The lower_limit matches the row value, so that it can be generalise to variable row.

Column level Repetitive Action						**Expected Output**
row	loop-condition(lower_limit <=col<= upper_limit)					
	lower_limit		upper_limit			
	default	1	default	3	4	
1	1	row	1	row + row - 1	2*row - 1	1
2	2	row	3	row + row - 1	2*row - 1	2 3
3	3	row	5	row + row - 1	2*row - 1	3 4 5
4	4	row	7	row + row - 1	2*row - 1	4 5 6 7
5	5	row	9	row + row - 1	2*row - 1	5 6 7 8 9

Column level Repetitive Action			**Expected Output**
row	loop-condition(lower_limit <=col<= upper_limit)		
	lower_limit	upper_limit	
	1	4	
1	row	2*row - 1	1
2	row	2*row - 1	2 3
3	row	2*row - 1	3 4 5
4	row	2*row - 1	4 5 6 7
5	row	2*row - 1	5 6 7 8 9

The lower_limit & upper_limit are become consistent for all the rows. The lower _limit is generalised to **row** and upper_limit is generalised to (**2*row – 1**).These logical expressions are such that on commonly applying for all rows and columns helps to give matching output.

So our Conditioning becomes;

loop-condition	lower_limit <=col<= upper_limit
Mathematical expression	row <= col <= (2*row – 1)
Syntax	`for( col = row ; col <= 2*row - 1 ; col++ )`

NumberPatternAAA.c

```c
#include <stdio.h>
int main() {

	int totalRows = 5; //number of rows to display

	// Row level Repetitive Action :
	for ( int row = 1; row <= totalRows; row++ ) {

		// Column level Repetitive Action:
		// 1)Move cursor in the same row. 2)print character
		for ( int col = row; col <= ( 2 * row - 1 ); col++ ) {

			// added extra space for clarity in output.
			printf( "%d ", col );
		}

		// move control to the next line
		printf( "\n" );
	}

	return 0;
}
```

Output

```
1
2 3
3 4 5
4 5 6 7
5 6 7 8 9
```

ProblemGBAAA: **Modify program NumberPatternAAA.c so that it should show following output using switch-case for values 1 and 2 respectively.**

Output

```
1
  3
3   5
  5   7
5   7   9
```

Output

```
2
  4
4   6
  6   8
```

```
        1
      3 2
    5 4 3
  7 6 5 4
9 8 7 6 5
```

NumberPatternBBB

Tabular representation

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	space	space	space	space	1
Row=2	space	space	space	3	2
Row=3	space	space	5	4	3
Row=4	space	7	6	5	4
Row=5	9	8	7	6	5

(Row, Col) value pairs representation.

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1					1,5
Row=2				2,4	2,5
Row=3			3,3	3,4	3,5
Row=4		4,2	4,3	4,4	4,5
Row=5	5,1	5,2	5,3	5,4	5,5

Analysis of output

In this case we need to take care of printing two different types of characters at a particular column in a particular row:

1) Space character

2) Numeric character

row	1				
Col	1	2	3	4	5
Output	space	space	space	space	1

For row =1, 1 <=spaces<=4, 1>=num>=1

row	2				
Col	1	2	3	4	5
Output	space	space	space	3	2

For row =2, 1 <=spaces<=3, 3>=num>=2

row	3				
Col	1	2	3	4	5
Output	space	space	5	4	3

For row =3, 1 <=spaces<=2, 5>=num>=3

row	4				
Col	1	2	3	4	5
Output	space	7	6	5	4

For row =4, 1 <=spaces<=1, 7>=num>=4

row	5				
Col	1	2	3	4	5
Output	9	8	7	6	5

For row =5, 9>=num>=5

Observations

1) There is repetition at row level vertically, i.e., Row = 1 to 5.
2) There is repetition at col level horizontally. , i.e., Col = 1 to 5.
   This gets divided into two parts .One for repetitively printing spaces and remaining column for printing numeric characters.

For Loop	**Lower limit**	**Upper limit**
Outer loop	row=1	row<=5
Inner Loop - spaces	col=1	col <= ?(Unkown)
Inner Loop - Number	num = ?(Unkown)	num = 1

Column Level Repetitive Action

Row level Repetitive Action					**Expected Output**
row	**Column Level Repetitive Action**				
	lower_limit <= col <= upper_limit		upper_limit >=num >= lower_limit		
	lower_limit	upper_limit	upper_limit	lower_limit	
1	1	4	1	1	1
2	1	3	3	2	3 2
3	1	2	5	3	5 4 3
4	1	1	7	4	7 6 5 4
5	1	0	9	5	9 8 7 6 5

We need to come out with a consistent expression for upper_limit for which it should suit any number of rows. We can refer previous pattern problem **PatternAAA's** result for the num variable. The other variables are row & totalRows=5. Let's try to generalise using these variables.

Column Level Repetitive Action							
row	*lower_limit* <= col <= *upper_limit*				*upper_limit >=num >= lower_limit*		
	lower_limit	upper_limit			upper_limit		lower_limit
			1	2	default	1	
1	1	4	totalRows - 1	totalRows - row	1	row	row
2	1	3	totalRows - 2	totalRows - row	3	row + 1	row
3	1	2	totalRows - 3	totalRows - row	5	row + 2	row
4	1	1	totalRows - 4	totalRows - row	7	row + 3	row
5	1	0	totalRows - 5	totalRows - row	9	row + 4	row

Again trying to express the hard coded numeric values of upper_limit related to variable num in terms of variable row.

Column Level Repetitive Action						
row	*lower_limit* <= col <= *upper_limit*		*upper_limit >=num >= lower_limit*			
	lower_limit	upper_limit	upper_limit			lower_limit
				1	2	
1	1	totalRows - row	1	row	row + 1 - 1	row
2	1	totalRows - row	3	row + 1	row + row - 1	row
3	1	totalRows - row	5	row + 2	row + row - 1	row
4	1	totalRows - row	7	row + 3	row + row - 1	row
5	1	totalRows - row	9	row + 4	row + row - 1	row

Column Level Repetitive Action						
row	*lower_limit* <= col <= *upper_limit*		*upper_limit >=num >= lower_limit*			
	lower_limit	upper_limit	upper_limit			lower_limit
			default	2	3	
1	1	totalRows - row	1	row + row - 1	2*row - 1	row
2	1	totalRows - row	3	row + row - 1	2*row - 1	row
3	1	totalRows - row	5	row + row - 1	2*row - 1	row
4	1	totalRows - row	7	row + row - 1	2*row - 1	row
5	1	totalRows - row	9	row + row - 1	2*row - 1	row

Column Level Repetitive Action				
row	*lower_limit* <= col <= *upper_limit*		*upper_limit* >=num >= *lower_limit*	
	lower_limit	upper_limit	upper_limit	lower_limit
1	1	totalRows - row	2*row - 1	row
2	1	totalRows - row	2*row - 1	row
3	1	totalRows - row	2*row - 1	row
4	1	totalRows - row	2*row - 1	row
5	1	totalRows - row	2*row - 1	row

So the upper_limit of col & num variables are generalised. The lower_limit & upper_limit are consistent for all the rows. It's now generalised for any number of rows.

So our loop-condition for space count on each row using variable col becomes;

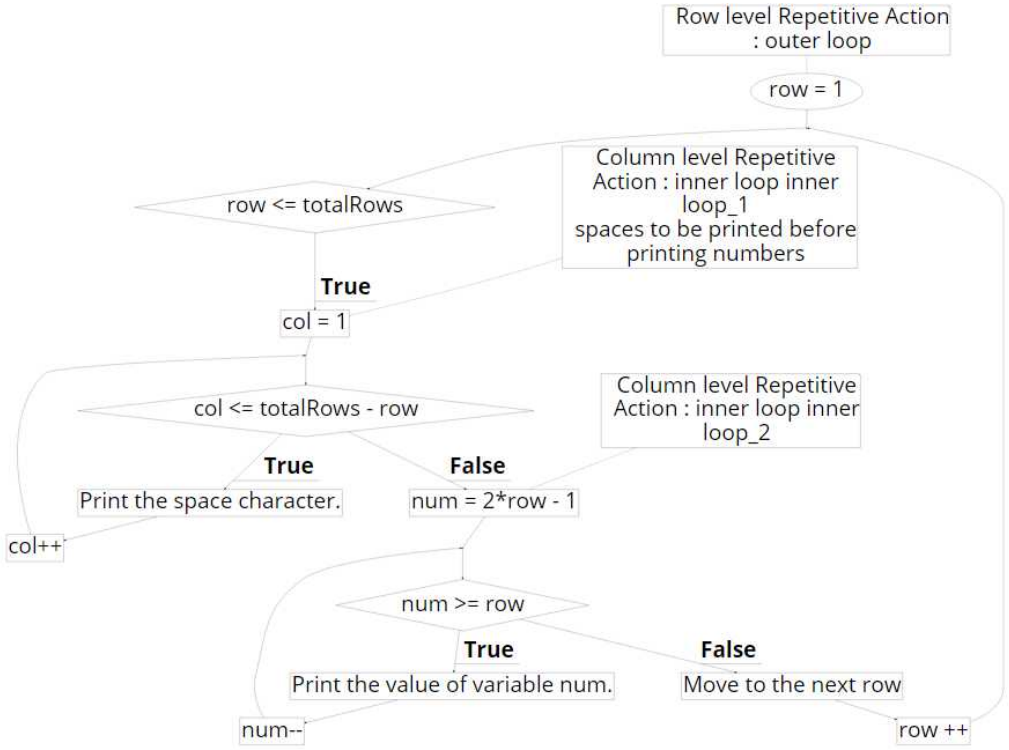
loop-condition	lower_limit <=col<= upper_limit
Mathematical expression	1 <= col <= (totalRows – row)
`Syntax`	`for( col = 1 ; col <= (totalRows - row) ; col++)`

So our loop-condition for printing numbers on each row using variable num becomes;

loop-condition	upper_limit >=num >= lower_limit
Mathematical expression	1 <= col <= (totalRows – row)
`Syntax`	`for( num = 2*row - 1  ; num >= row  ; num--)`

So it has two repetition process one for printing spaces and second for printing numeric values. It means require 2 inner for-loops (nested for – loops).

The flow-chart can be given as follows to captured the details.



NumberPatternBBB1.c

```c
#include <stdio.h>
int main() {

            int  totalRows = 5;  //number of rows to display

            // Row level Repetitive Action :
            for( int row = 1 ; row <= totalRows ; row ++ ) {

                  // Column level Repetitive Action :
                  // 1)Move cursor in the same row. 2)print character

                  // spaces to be printed before printing numbers
                  for( int col = 1 ; col <= (totalRows - row) ; col++ ) {
                        printf( "  " );
                  }

                  for( int num = ( 2*row - 1 ) ; num >= row  ; num-- ) {
                        // added extra space for clarity in output.
                        printf( "%d ", num );
                  }

                  printf("\n");// moves control to the next line
            }


      return 0;
}
```

Output

```
        1
      3 2
    5 4 3
  7 6 5 4
9 8 7 6 5
```

Alternative solution

Let's device a solution by merging two inner for-loops into single by introducing the if-else construct.

We can directly get the idea from for-loop condition of the both the inner for-loops. It's either printing space or number. Number starts with the formula (**2*row - 1)** and decreases till it reaches the row value. The lower_limit is an initialization part and need to do it only once.

If we take care of initialisation and step-decrementing part then the loop-condition can be converted into the **if-condition.**

NumberPatternBBB2.c

```
#include <stdio.h>
int main() {

	int totalRows = 5; //number of rows to display
	// Row level Repetitive Action :
	for ( int row = 1; row <= totalRows; row++ ) {

		// Column level Repetitive Action :
		// 1)Move cursor in the same row. 2)print character

		// spaces to be printed before printing numbers
		int num = 2 * row - 1;//  horizontal movements

		// spaces to be printed before printing numbers
		for ( int col = 1; col <= ( totalRows ); col++ ) {

			if ( col <= ( totalRows - row ) ) {
				printf( "  " );
			} else if ( num >= row ) {
				// added extra space for clarity in output.
				printf( "%d ", num );
				num = num - 1;
			}

		}

		// move control to the next line
		printf( "\n" );
	}

	return 0;
}
```

Output

```
        1
      3 2
    5 4 3
  7 6 5 4
9 8 7 6 5
```

5 6 7 8 9

4 5 6 7

3 4 5

2 3

1

NumberPatternCCC

Tabular representation

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=5	5	6	7	8	9
Row=4	4	5	6	7	
Row=3	3	4	5		
Row=2	2	3			
Row=1	1				

(Row, Col) value pairs representation.

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=5	5,1	5,2	5,3	5,4	5,5
Row=4	4,1	4,2	4,3	4,4	
Row=3	3,1	3,2	3,3		
Row=2	2,1	2,2			
Row=1	1,1				

Analysis of output

When Row = 1, Col = 1,2,3,4,5 (1 to 5) : repetition will be 5 times horizontally (1 <=Col <= 5)

When Row = 2, Col = 1,2,3,4 (1 to 4) : repetition will be 4 times horizontally (1 <=Col <= 4)

When Row = 3, Col = 1,2,3 (1 to 3) : repetition will be 3 times horizontally (1 <=Col <= 3)

When Row = 4, Col = 1,2 (1 to 2) : repetition will be 2 times horizontally (1 <=Col <=2)

When Row = 5, Col = 1 : will be 1 time (1 <=Col <=1)

We need to take care of logic of numeric output only and not to take care of logic required

to print spaces. The logic for spaces is to be considered only if spaces comes before numeric output.

So, it's irrelevant in this case.

row	5				
col	5	6	7	8	9
output	5	6	7	8	9

row =5 , (5 <=Col <= 9) (print till upper_limit=9)

row	4				
col	4	5	6	7	
output	4	5	6	7	

row = 4 , (4 <=Col <= 7) (print till upper_limit=7)

row	3				
col	3	4	5		
output	3	4	5		

row = 3 , (3 <=Col <= 5) (print till upper_limit=5)

row	2				
col	2	3			
output	2	3			

row = 2 , (2 <=Col <= 2) (print till upper_limit=3)

row	1				
col	1				
output	1				

row = 1 , (1 <=Col <= 1) (print till upper_limit=1)

The row value equals to the lower_limit of col variable. The upper_limit value of col variable varies in each row.

Let's device a solution by **decrementing the outer for-loop using variable row** (5 to 1) and determining the loop-condition(lower_limit <=col<= upper_limit) of col variable to give the desired numeric output.

Let's try to get generalised expression using the variable **row**.

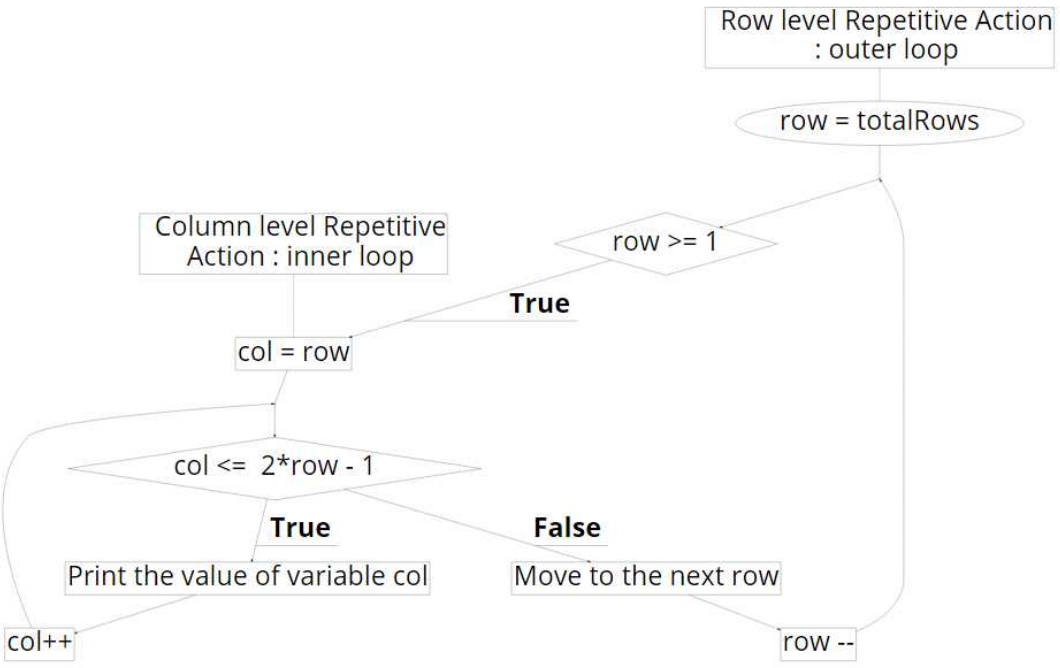
Column Level Repetitive Action

Row level Repetitive Action(decrementing loop)									**Expected Output**
row	**Column Level Repetitive Action**								
	loop-condition (lower_limit <=col<=upper_limit)								
	lower_limit		*upper_limit*						
		1		1	2	3	4	5	
5	5	row	9	5+4	row + 4	row + 5-1	row + row - 1	2* row - 1	5 6 7 8 9
4	4	row	7	4+3	row + 3	row + 4-1	row + row - 1	2* row - 1	4 5 6 7
3	3	row	5	3+2	row + 2	row + 3-1	row + row - 1	2* row - 1	3 4 5
2	2	row	3	2+1	row + 1	row + 2-1	row + row - 1	2* row - 1	2 3
1	1	row	1	1+0	row + 0	row + 1-1	row + row - 1	2* row - 1	1

So our inner loop-condition to give numeric output on each row using variable col becomes;

loop-condition	lower_limit <=col<= upper_limit
Mathematical expression	row <= col <= (2* row - 1)
`Syntax`	`for( col = row ; col <= ( 2*row - 1 ) ; col++)`

The flow-chart can be given as follows.



Let execute the following program to verify the output.

NumberPatternCCC1.c

```c
#include <stdio.h>
int main() {

            int totalRows = 5;  //number of rows to display

            // Row level Repetitive Action :
            for( int row = totalRows ; row >= 1 ; row -- ) {

                  // Column level Repetitive Action :
                  // 1)Move cursor in the same row. 2)print character
                  for( int col = row ; col <= ( 2*row - 1 ) ; col++ ) {
                        printf( "%d ", col );

                  }// inner for-loop

                  printf("\n");// Move cursor to the next row

            }// outer for-loop

      return 0;
}
```

Output

```
5 6 7 8 9
4 5 6 7
3 4 5
2 3
1
```

ProblemGBCCC1: **Modify program `NumberPatternCCC1.c` so that it should give following output.**

Output

```
        9
      7
    5
  3
1
```

Alternative solution

Let's try to device a solution using incrementing for-loop for row variable (1 to 5) and determining the loop-condition for col variable.

The row value in this case doesn't matches the lower_limit of col variable on each row. So row variable we can't use it directly to express lower_limit of col variable.

We shall try other variable totalRows=5 (default number of rows) to generalise it.

Row level Repetitive Action(incrementing loop)						**Expected Output**
row	**Column Level Repetitive Action**					
	loop-condition*(lower_limit <=col<= upper_limit)*					
	lower_limit			upper_limit		
	default	1	2	default	1	
1	5	totalRows	totalRows +1 - 1	9	2* totalRows - 1	5 6 7 8 9
2	4	totalRows -1	totalRows +1 - 2	7	2* totalRows - 3	4 5 6 7
3	3	totalRows -2	totalRows +1 - 3	5	2* totalRows - 5	3 4 5
4	2	totalRows -3	totalRows +1 - 4	3	2* totalRows - 7	2 3
5	1	totalRows -4	totalRows +1 - 5	1	2* totalRows - 9	1

Part of lower_limit's logical expression matches with the row value. Since all numeric values are matching with value of row variable in each row.

We should use row variable to remove numeric values occurring in the lower_limit's logical expression.

Column Level Repetitive Action					**Expected Output**
row	***loop-condition****(lower_limit <=col<= upper_limit)*				
	lower_limit		upper_limit		
	default	3	default	1	
1	5	totalRows +1 - row	9	2* totalRows - 1	5 6 7 8 9
2	4	totalRows +1 - row	7	2* totalRows - 3	4 5 6 7
3	3	totalRows +1 - row	5	2* totalRows - 5	3 4 5
4	2	totalRows +1 - row	3	2* totalRows - 7	2 3
5	1	totalRows +1 - row	1	2* totalRows - 9	1

Now it's turn to generalise the upper_limit of col variable.

Column Level Repetitive Action					**Expected Output**
row	***loop-condition****(lower_limit <=col<= upper_limit)*				
	lower_limit	upper_limit			
	3	default	1	2	
1	totalRows +1 - row	9	2* totalRows - 1	2* totalRows - 1	5 6 7 8 9
2	totalRows +1 - row	7	2* totalRows - 3	2* totalRows - 2 -1	4 5 6 7
3	totalRows +1 - row	5	2* totalRows - 5	2* totalRows - 3 -2	3 4 5
4	totalRows +1 - row	3	2* totalRows - 7	2* totalRows - 4 -3	2 3
5	totalRows +1 - row	1	2* totalRows - 9	2* totalRows - 5 -4	1

let's try to generalise part of the upper_limit using row variable and can be given as;

Column Level Repetitive Action				**Expected Output**
row	***loop-condition****(lower_limit <=col<= upper_limit)*			
	lower_limit	upper_limit		
	3	4	5	
1	totalRows +1 - row	2* totalRows - row	2* totalRows - row -1 +1	5 6 7 8 9
2	totalRows +1 - row	2* totalRows - row - 1	2* totalRows - row -2+1	4 5 6 7
3	totalRows +1 - row	2* totalRows - row - 2	2* totalRows - row - 3+1	3 4 5
4	totalRows +1 - row	2* totalRows - row - 3	2* totalRows - row – 4+1	2 3
5	totalRows +1 - row	2* totalRows - row - 4	2* totalRows - row – 5+1	1

Again, we can use row variable in the upper_limit to get rid of hard-coded numeric values.

Column Level Repetitive Action				**Expected Output**
row	***loop-condition****(lower_limit <=col<=upper_limit)*			
	lower_limit	upper_limit		
	3	6	7	
1	totalRows +1 - row	2* totalRows - row -row +1	2* (totalRows – row) + 1	5 6 7 8 9
2	totalRows +1 - row	2* totalRows - row -row +1	2* (totalRows – row) + 1	4 5 6 7
3	totalRows +1 - row	2* totalRows - row -row +1	2* (totalRows – row) + 1	3 4 5
4	totalRows +1 - row	2* totalRows - row -row +1	2* (totalRows – row) + 1	2 3
5	totalRows +1 - row	2* totalRows - row -row +1	2* (totalRows – row) + 1	1

The lower_limit and upper_limit expressions both have +1.Moreover, If we run outer loop through [1 <=row<= totalRows] or [0 <=row< totalRows] ,in either case it is the same. By deducting 1 from row , lower_limit and upper_limit expressions to make it more simpler.

Column Level Repetitive Action			**Expected Output**
row	***loop -condition****(lower_limit <=col< upper_limit)*		
	lower_limit	upper_limit	
	expression	**expression**	
1 -1	totalRows +1 – row - 1	2* (totalRows – row) + 1 - 1	5 6 7 8 9
2 -1	totalRows +1 – row - 1	2* (totalRows – row) + 1 - 1	4 5 6 7
3 -1	totalRows +1 – row - 1	2* (totalRows – row) + 1- 1	3 4 5
4 -1	totalRows +1 – row - 1	2* (totalRows – row) + 1- 1	2 3
5-1	totalRows +1 – row - 1	2* (totalRows – row) + 1- 1	1

Column Level Repetitive Action			**Expected Output**
row	***loop -condition****(lower_limit <=col< upper_limit)*		
	lower_limit	upper_limit	
	expression	**expression**	
0	totalRows – row	2* (totalRows – row)	5 6 7 8 9
1	totalRows – row	2* (totalRows – row)	4 5 6 7
2	totalRows – row	2* (totalRows – row)	3 4 5
3	totalRows – row	2* (totalRows – row)	2 3
4	totalRows – row	2* (totalRows – row)	1

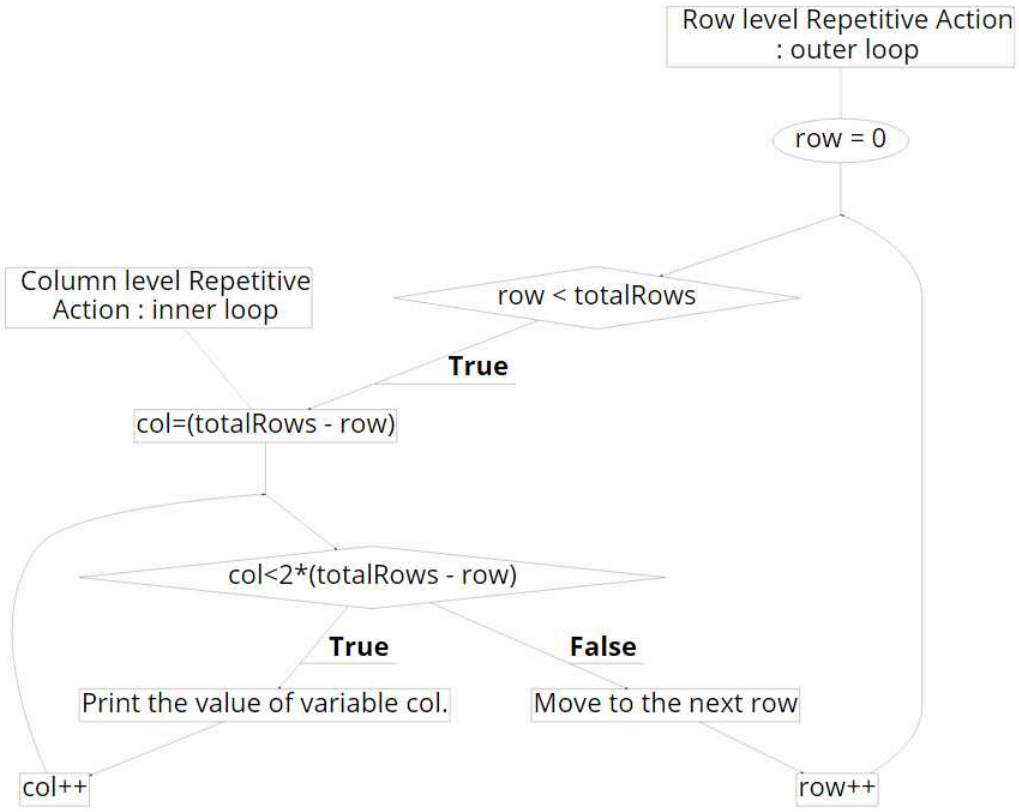
So our outer for-loop condition becomes ;

loop-condition	*lower_limit <=row< upper_limit*
Mathematical expression	0 <= row < `totalRows`
`Syntax`	`for( row = 0; row < totalRows; row++)`

So our inner for-loop condition becomes ;

loop-condition	*lower_limit <=col< upper_limit*
Mathematical expression	(totalRows – row) <=col < 2* (totalRows – row)
`Syntax`	`for(col=(totalRows - row);col<2*(totalRows - row);col++ )`

The flow-chart can be given as follows.



NumberPatternCCC2.c

```c
#include <stdio.h>
int main() {

	int totalRows = 5; //number of rows to display

	// Row level Repetitive Action :
	for ( int row = 0; row < totalRows; row++ ) {

		// Column level Repetitive Action :
		// 1)Move cursor in the same row. 2)print character
		int lower_limit = ( totalRows - row );
		int upper_limit = 2 * ( totalRows - row );
		for ( int col = lower_limit ; col < upper_limit ; col++ ) {
			printf( "%d ", col );
		}// inner loop

		printf( "\n" );// Move cursor to the next row

	}// outer loop

	return 0;
}
```

Output

```
5 6 7 8 9
4 5 6 7
3 4 5
2 3
1
```

```
9 8 7 6 5
  7 6 5 4
    5 4 3
      3 2
        1
```

NumberPatternDDD

Tabular representation

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	9	8	7	6	5
Row=2	space	7	6	5	4
Row=3	space	space	5	4	3
Row=4	space	space	space	3	2
Row=5	space	space	space	space	1

(Row, Col) value pairs representation.

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	1,1	1,2	1,3	1,4	1,5
Row=2		2,2	2,3	2,4	2,5
Row=3			3,3	3,4	3,5
Row=4				4,4	4,5
Row=5					5,5

Analysis of output

In this case we need to take care of printing two different types of characters at a particular column in a particular row:

1) Space character

2) Numeric character

row	5				
Col	1	2	3	4	5
Output	9	8	7	6	5

For row =1, 0 <=spaces<=0, 9>=num>=5

row	4				
Col	1	2	3	4	5
Output	space	7	6	5	4

For row =2, 1 <=spaces<=1, 9>=num>=5

row	3				
Col	1	2	3	4	5
Output	space	space	5	4	3

For row =3, 1 <=spaces<=2, 5>=num>=3

row	2				
Col	1	2	3	4	5
Output	space	space	space	3	2

For row =4, 1 <=spaces<=3, 3>=num>=2

row	1				
Col	1	2	3	4	5
Output	space	space	space	space	1

For row =5, 1 <=spaces<=4, 1>=num>=1

Observations

1) There is repetition at row level vertically, i.e., Row = 1 to 5.
2) There is repetition at col level horizontally. , i.e., Col = 1 to 5.
   This gets divided into two parts .One for repetitively printing spaces and remaining column for printing numeric characters.

For Loop	**Lower limit**	**Upper limit**
Outer loop(decrementing)	row=1	Row>=5
Inner Loop - spaces	col=1	col<= ?(Unkown)
Inner Loop – Number (decrementing)	num = row	num =?(Unkown)

Column Level Repetitive Action

Row level Repetitive Action					Expected Output
row	Column level Repetitive Action				
	lower_limit <=col<= upper_limit		*upper_limit >=num >= lower_limit*		
	lower_limit	upper_limit	upper_limit	lower_limit	
5	1	0	9	5	9 8 7 6 5
4	1	1	7	4	7 6 5 4
3	1	2	5	3	5 4 3
2	1	3	3	2	3 2
1	1	4	1	1	1

The lower_limit for spaces & num variable has the same numeric value expression for all the rows.

It's consistent for all the rows. Likewise we need to come out with such a consistent expression for upper_limit for which it should suit any number of rows.

The other remaining variables are row & totalRows=5. Let's try to generalise using these variables.

Column level Repetitive Action						
row	*lower_limit <=spaces<= upper_limit*			*upper_limit >=num >= lower_limit*		
	lower_limit	upper_limit		upper_limit	lower_limit	
		default	1		default	1
5	1	0	totalRows - row	9	5	row
4	1	1	totalRows - row	7	4	row
3	1	2	totalRows - row	5	3	row
2	1	3	totalRows - row	3	2	row
1	1	4	totalRows - row	1	1	row

The uppler_limit of row variable related inner for-loop is generalised, consistent expression on all the rows.
The uppler_limit of num variable related inner for-loop is not generalised as not having expression on all the rows. From previous cases we know it can follow the generalised expression 2*row – 1.

Column level Repetitive Action					
row	*lower_limit <=spaces<= upper_limit*			*upper_limit >=num >= lower_limit*	
	lower_limit	upper_limit		upper_limit	lower_limit
		default	1	1	1
5	1	0	totalRows - row	2*row - 1	row
4	1	1	totalRows - row	2*row - 1	row
3	1	2	totalRows - row	2*row - 1	row
2	1	3	totalRows - row	2*row - 1	row
1	1	4	totalRows - row	2*row - 1	row

So the upper_limit of col & num variables are generalised.

The lower_limit & upper_limit are consistent for all the rows. It's now generalised for any number of rows.

So it has two repetition process one for printing spaces and second for printing numeric values.

It means require 2 inner for-loops and 1 outer loop for iterating rows.

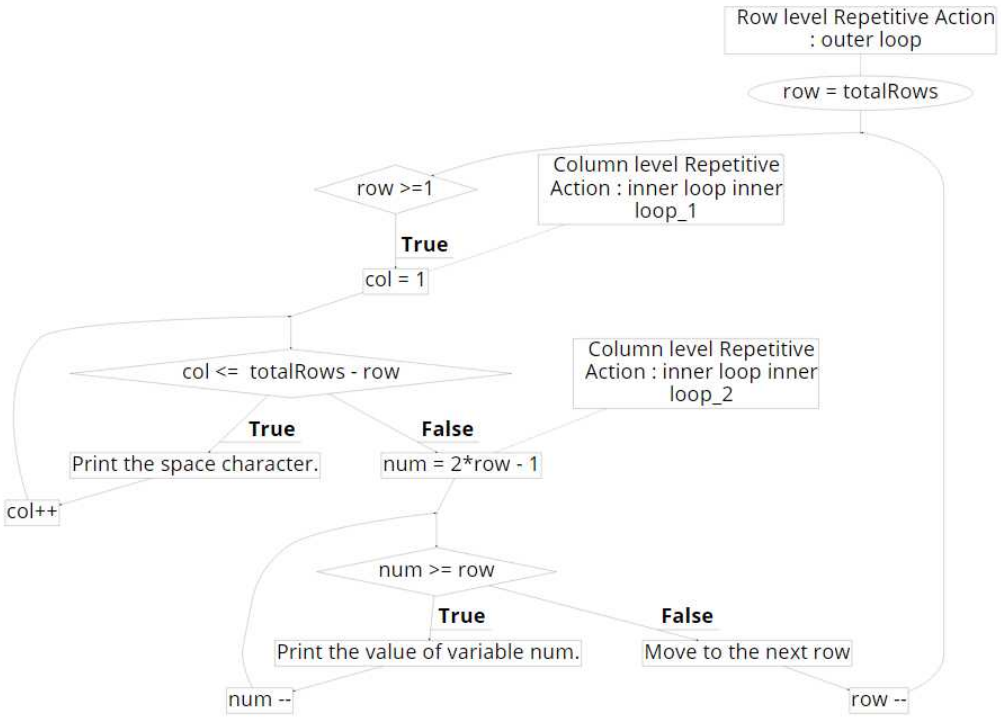
So our inner for-loop condition for printing spaces becomes;

loop-condition	lower_limit <=col<= upper_limit
Mathematical expression	1 <= col <= (totalRows – row)
Syntax	`for( col = 1; col <= ( totalRows - row ); col++)`

So our inner for-loop condition for printing numbers becomes;

loop-condition	upper_limit >=num>= lower_limit
Mathematical expression	(2*row – 1) >= num>= row
Syntax	`for( num = 2*row - 1 ; num >= row ; num -- )`

We can summarise all the related information in a flow-chart as given following.



Let's execute the following program.

NumberPatternDDD1.c

```
#include <stdio.h>
int main() {

	int totalRows = 5; //number of rows to display
	int num;

	// Row level Repetitive Action :
	for ( int row = totalRows; row >= 1; row-- ) {

		// Column level Repetitive Action :
		// 1)Move cursor in the same row.
		// 2)print space characters before number.
		for ( int col = 1; col <= ( totalRows - row ); col++ ) {
			printf( "  " );// 2 spaces
		}// inner for-loop_1

		// num value to be printed
		for ( num = 2 * row - 1; num >= row; num-- ) {
			printf( "%d ", num );
		}// inner for-loop_2

		printf( "\n" );// Move cursor to the next row

	}// outer for-loop

	return 0;
}
```

Output

```
9 8 7 6 5
  7 6 5 4
    5 4 3
      3 2
        1
```

Alternative solution

Since the column values get clear split between printing spaces and numbers. We can try to reproduce the same result using if-else condition construct. Using the **loop-condition** for spaces as an if-condition and printing number in **else-condition.**

NumberPatternDDD2.c

```
#include <stdio.h>
int main() {

	int totalRows = 5; // number of rows to display

	// Row level Repetitive Action :
	for ( int row = totalRows; row >= 1; row-- ) {

		int num = 2*row - 1 ;

		// Column level Repetitive Action :
		for ( int col = 1; col <= ( totalRows ); col++ ) {

			// when spaces are to be printed
			if ( col <= ( totalRows - row ) ) {
				printf( "  " );
			} else {
				printf( "%d ", num );
				num--;
			}
		} // inner for-loop

		printf( "\n" );// Move cursor to the next row

	} // outer for-loop

	return 0;
}
```

Output

```
9 8 7 6 5
  7 6 5 4
    5 4 3
      3 2
        1
```

```
        1
      3 2 3
    5 4 3 4 5
  7 6 5 4 5 6 7
9 8 7 6 5 6 7 8 9
```

NumberPatternEEE

Tabular representation

	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1	space	space	space	space	1	space	space	space	space
Row=2	space	space	space	3	2	3	space	space	space
Row=3	space	space	5	4	3	4	5	space	space
Row=4	space	7	6	5	4	5	6	7	space
Row=5	9	8	7	6	5	6	7	8	9

(Row, Col) value pairs representation.

	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1					1,5				
Row=2				2,4	2,5	2,6			
Row=3			3,3	3,4	3,5	3,6	3,7		
Row=4		4,2	4,3	4,4	4,5	4,6	4,7	4,8	
Row=5	5,1	5,2	5,3	5,4	5,5	5,6	5,7	5,8	5,9

Analysis of output

row	1								
col	1	2	3	4	5	6	7	8	9
output					1				

For row = 1, 5 <=Col <= 5

row	2								
col	1	2	3	4	5	6	7	8	9
output				3	2	3			

For row = 2, 4 <=Col <= 6

row	3								
col	1	2	3	4	5	6	7	8	9
output			5	4	3	4	5		

For row = 3 , 3 <=Col <= 7,

row	4								
col	1	2	3	4	5	6	7	8	9
output		7	6	5	4	5	6	7	

For row = 4, 2 <=Col <= 8,

row	5								
col	1	2	3	4	5	6	7	8	9
output	9	8	7	6	5	6	7	8	9

For row = 5, 1 <=Col <= 9

Column Level Repetitive Action

Row level Repetitive Action			**Expected Output**
row	**Column level Repetitive Action**		
	If-condition *(lower_limit <= col <= upper_limit)*		
	lower_limit	upper_limit	
1	5	5	1
2	4	6	3 2 3
3	3	7	5 4 3 4 5
4	2	8	7 6 5 4 5 6 7
5	1	9	9 8 7 6 5 6 7 8 9

The lower_limit & upper_limit ,both are not consistent for all the rows. We need to come out with consistent expressions for both upper_limit and lower_limit which should suit for n number of rows. The other remaining variables are row,totalRows=5.Let's try to generalised using these variables.

Column level Repetitive Action						
row	***If-condition*** *(lower_limit <= col <= upper_limit)*					
	lower_limit			upper_limit		
	default	1	2	default	1	
1	5	totalRows	totalRows +1 - 1	5	totalRows	totalRows +1 - 1
2	4	totalRows -1	totalRows +1 - 2	6	totalRows + 1	totalRows + 2 - 1
3	3	totalRows -2	totalRows +1 - 3	7	totalRows + 2	totalRows + 3 - 1
4	2	totalRows -3	totalRows +1 - 4	8	totalRows + 3	totalRows + 4 - 1
5	1	totalRows -4	totalRows +1 - 5	9	totalRows + 4	totalRows + 5 - 1

Column level Repetitive Action				
row	***If-condition****(lower_limit <= col <= upper_limit)*			
	lower_limit		upper_limit	
	default	3	default	3
1	5	totalRows +1 - row	5	totalRows + row - 1
2	4	totalRows +1 - row	6	totalRows + row - 1
3	3	totalRows +1 - row	7	totalRows + row - 1
4	2	totalRows +1 - row	8	totalRows + row - 1
5	1	totalRows +1 - row	9	totalRows + row - 1

The lower_limit & upper_limit are consistent for all the rows. It's now generalised for any number of rows, which gives the exact position where to print numeric value. `So our If-condition becomes;

if-condition	lower_limit <=col<= upper_limit
Mathematical expression	(totalRows +1 - row) <= col <= (totalRows + row - 1)
Syntax	`if( col >= ( (totalRows – row) + 1 ) &&` `col <= ( ( totalRows + row ) – 1 ) )`

When the if-condition satisfied, it will print the numeric value at that position at a particular row or else will print the space character. We can observe two things from the given output which can be given as;

a) The values at each row starts with the (2*row -1) value and goes on decreases till row value and further increases till it reaches the (2*row -1) value. The value basically oscillating like a pendulum of a wall clock. Let's choose a variable **num** which represents a (2*row -1) value, every time a new row starts. The value of variable num then oscillates to give output in a single row. It can be shown by following table.

Table for **num** variable starting at (2*row -1) value.

row	**num**	**col<totalRows**	**col=totalRows**	**col>totalRows**
1	1		1	
2	2	3	2	3
3	3	5 4	3	4 5
4	4	7 6 5	4	5 6 7
5	5	9 8 7 6	5	6 7 8 9

if(col<totalRows){
num =(2*row -1) value gets printed.
num value get decreased by 1.
}
	if(col>=totalRows){
	num value gets printed.
	num value get increased by 1 till (2*row -1)
	}

b) The column counts are more than the **totalRows** count other than the first row. The **totalRows** value is 5 .The column count is different on each row. The upper_limit of inner ***for–loop*** need to be generalised.

loop-condition	lower_limit <=col<= upper_limit
Mathematical expression	1 <= col <= 9
	1 <=col <= **totalRows + 4**
	1 <=col <= **totalRows + (**totalRows – 1)
	1<=col<=2* totalRows - 1
`Syntax`	`for( col = 1 ; col <= ( 2* totalRows  - 1 ) ; col++)`

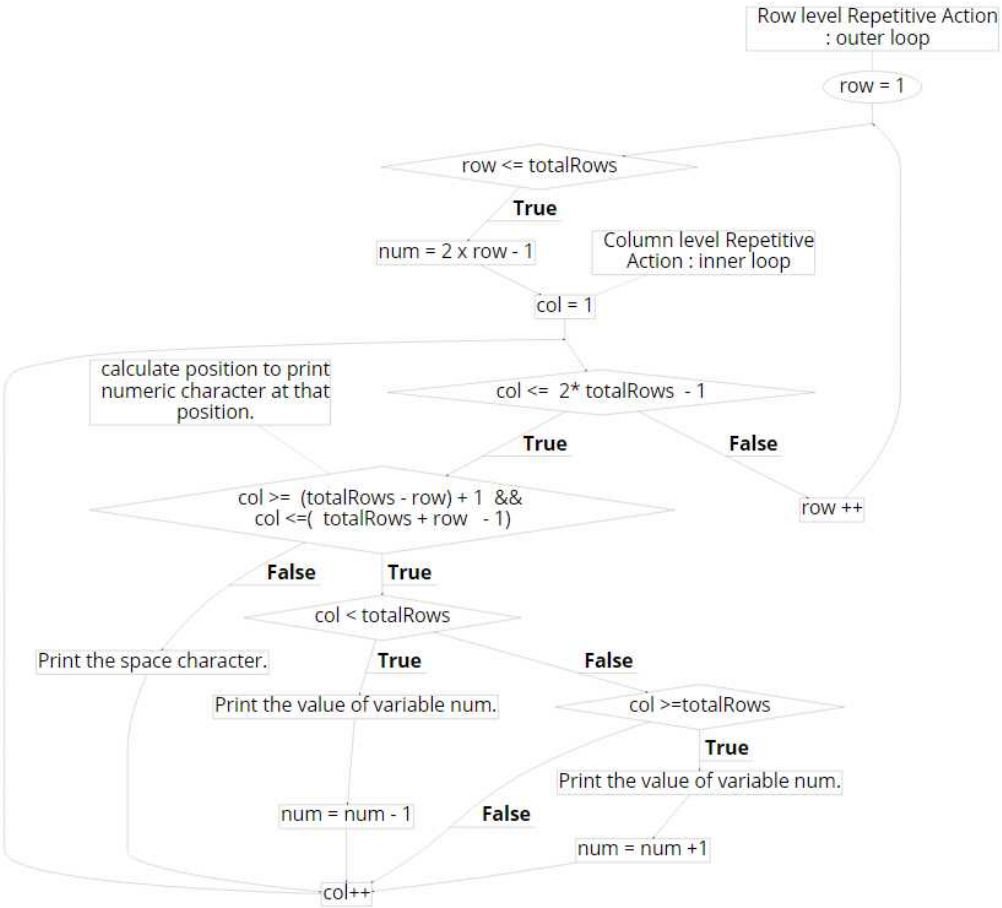
Let's verify the upper limit for col variables or total columns which will used to display each row

for totalRows = 5 ,the col <= 9 equals (2*5 – 1)

for totalRows = 4 ,the col <= 7 equals (2*4 – 1)

So this generalisation to calculate total columns for a given row value is satisfactory.

The flow-chart can be given as follows.



Let write the program based on our findings and flow-chart.

NumberPatternEEE.c

```c
#include <stdio.h>
int main() {

	int totalRows = 5; //number of rows to display
	int num;

	// Row level Repetitive Action :
	for ( int row = 1; row <= totalRows; row++ ) {
		num = 2 * row - 1;

		// Column level Repetitive Action :
		for ( int col = 1; col <= ( 2 * totalRows - 1 ); col++ ) {

			// calculates position to print numeric character
			if ( col >= ( ( totalRows - row ) + 1 ) &&
			     col <= ( ( totalRows + row ) - 1 ) ) {
				// print a number with a single space
				if ( col < totalRows ) {
					printf( "%d ", num );
					num = num - 1;
				} else if ( col >= totalRows ) {
					printf( "%d ", num );
					num = num + 1;
				}

			} else {
				printf( "  " );// print 2 spaces
			}
		}// inner loop

		// Move to next row
		printf( "\n" );

	}// outer loop


	return 0;
}
```

Output

```
        1
      3 2 3
    5 4 3 4 5
  7 6 5 4 5 6 7
9 8 7 6 5 6 7 8 9
```

ProblemGBBBB2: **Modify program NumberPatternBBB2.c so that it should able to show the following output.**

Output

```
        1
      2
    3
  4
5
```

ProblemGBDD1: **Modify program NumberPatternDDD1.c so that it should able to show the following output.**

Output

```
9
  7
    5
      3
        1
```

ProblemGBEEE: **Modify program NumberPatternEEE.c so that it should able to show the following output.**

Output

```
        1
      3   3
    5       5
  7           7
9               9
```

ProblemGBEEE1: **Modify program PatternEEE.c so that it should show the following output.**

Output

```
        1
      3 2 3
    5   3   5
  7     4     7
9       5       9
```

ProblemGB. *W*rite programs to generate the following outputs.

Output

```
1>=1

2>=1 2>=2

3>=1 3>=2 3>=3

4>=1 4>=2 4>=3 4>=4

5>=1 5>=2 5>=3 5>=4 5>=5
```

Output

```
  1<2 1<3 1<4 1<5 1<6 1<7 1<8 1<9 1<10

    2<3 2<4 2<5 2<6 2<7 2<8 2<9 2<10

      3<4 3<5 3<6 3<7 3<8 3<9 3<10

        4<5 4<6 4<7 4<8 4<9 4<10

          5<6 5<7 5<8 5<9 5<10

            6<7 6<8 6<9 6<10

              7<8 7<9 7<10

                8<9 8<10

                  9<10
```

Output

```
<<~>><<~>><<~>><<~>><<~>><<~>><<~>><<~>><<~>><<~>>

  <<~>><<~>><<~>><<~>><<~>><<~>><<~>><<~>><<~>>

    <<~>><<~>><<~>><<~>><<~>><<~>><<~>><<~>>

      <<~>><<~>><<~>><<~>><<~>><<~>><<~>>

        <<~>><<~>><<~>><<~>><<~>><<~>>
```

Output

```
2

3 4

4 5 6

5 6 7 8

6 7 8 9 10

7 8 9 10 11 12

8 9 10 11 12 13 14
```

Output

```
1

2 1

3 2 1

4 3 2 1

5 4 3 2 1

6 5 4 3 2 1

7 6 5 4 3 2 1
```

Output

```
0

1 0

2 1 0

3 2 1 0

4 3 2 1 0

5 4 3 2 1 0
```

Arrays

Till now we had deal with variables which deals only with single data. In short;

Relationship between variable and data		
Variable	**one – to – one**	**Data**

It is also possible to have relationship between variable and data as **one-to-many**, which means one variable can able to hold many data or group of data. Such type of variables specially known as array variables.

Relationship between array variable and data		
Array Variable	**one – to – many**	**Data1**
		Data2
		Data3
		Data4

We too can initialize array variable like we used to initialize a normal variable by considering it's data-type. For example, 5 different numeric scores can be given as;

```
int student_scores[5] = { 70, 50, 40, 56, 72 };
```

student_scores → Represents an array variable of size 5 and gets initialized with 5 different score values separated by commas. Each value is an owner of it's position value commonly known as index value/position. So the group of same data type values can be represented by an array variable. In general, values of an array are also referred as elements of an array.

Array name and **index value combination is used** to access specific data among the group.

Array name along with different **index values** are like different variables and each containing data.

Array name →	**student_scores**	
First index →	0	70
	1	50
	2	40
	3	56
Last Index →	4	72

1st element →	**student_scores**[0] →	70
2nd element →	**student_scores**[1] →	50
3rd element →	**student_scores**[2] →	40
4th element →	**student_scores**[3] →	56
5thelement →	**student_scores**[4] →	72

Access the values of an array

In order to refer to any stored value in an array, we need to use array's name along with index of that array. Let's try to understand bit more from the given array decelerations, how values are stored in arrays.

```
char bookTitle[20];

float bookWeight[20]; // in Kg

int bookPages[20];
```

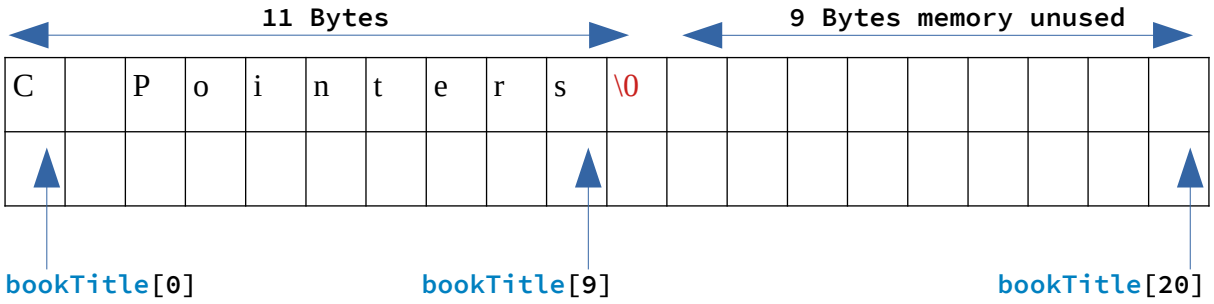
C	bookTitle[0]	.5	bookWeight[0]	370	bookPages[0]
P	bookTitle[1]	.73	bookWeight[1]	421	bookPages[1]
……. ……. …….		……. ……. …….		……. ……. …….	
	bookTitle[18]	.54	bookWeight[18]	355	bookPages[18]
	bookTitle[19]	.64	bookWeight[19]	423	bookPages[19]
Each 1 byte sized data		Each 4 bytes sized data		Each 2 bytes sized data	

The character string constants are given by enclosing it in quotes as shown by the following given example.

```
// Case1
char bookTitle[20] = "C pointers";

// Case2
char bookTitle[20] = {'C',' ','p','0','i',' n','t','e','r','s','\0'}
```

Case1: C automatically adds the NULL(ASCII 0 or '\0') character after the last character in a string.
Case2: C doesn't automatically adds null-terminated character('\0') at the end. The coder has to explicitly place it as it's last element.



We can directly access any value in an array. For example;

bookTitle[0] → It is the base memory address of an array.

bookTitle[9] => bookTitle[0] + 9(index value) * 1(data-type-size)

Hence using such internal calculation on memory address makes it possible to reach directly to 10th element means bookTitle[9] in an array.

Accessing array elements using array's index value

ArrayWithLooping.c

```c
#include <stdio.h>
int main() {
	// array initialization
	char name[20] = "Mr.Always Right";
	int ages[5] = { 10, 20, 30, 40, 50};
	float weight[5] = {23.27, 32.14, 37.45, 60.74, 47.25};
	long distance[5] = {1234567L, 4335454L, 98767567L, 4354321L,555588889L};
	int size = - 1;// size of an array

	printf(" Accessing char array element using array's index value.\n");
	for( int index = 0 ; name[index] != '\0'; index ++){
		printf(" %c |", name[index]);
	}

	printf("\n\n Accessing int array element using array's index value.\n");
	// finding size of array using C language's sizeof() operator.
	size = sizeof( ages )/ sizeof( int );
	for( int index = 0 ; index < size ; index++){
		printf(" %d | ", ages[index]);
	}

	printf("\n\n Accessing float array element using array's index value.\n");
	// finding size of array using C language's sizeof() operator.
	size = sizeof( weight )/ sizeof( float );
	for( int index = 0 ; index < size ; index++){
	// This means total 5 digits will be used to print this float number
	// and 2 digits after decimal point.
		printf(" %5.2f | ", weight[index]);
	}

	printf("\n\n Accessing long array element using array's index value.\n");
	// finding size of array using C language's sizeof() operator.
	size = sizeof( distance )/ sizeof( long );
	for( int index = 0 ; index < size ; index++){
	// print long digit number by using format specifier ld.
		printf(" %ld | ", distance[index]);
	}
	return 0;
}
```

Output

```
 Accessing char array element using array's index value.
 M | r | . | A | l | w | a | y | s |   | R | i | g | h | t |

 Accessing int array element using array's index value.
 10 |  20 |  30 |  40 |  50 |

 Accessing float array element using array's index value.
 23.27 |  32.14 |  37.45 |  60.74 |  47.25 |

 Accessing long array element using array's index value.
 1234567 |  4335454 |  98767567 |  4354321 |  555588889 |
```

PART 3

POINTERS AND TOPICS

Pointers And Memory Matters

Pointers are famously known as **address variables** as their only purpose is to store memory address of a data **not the data itself**. Pointer variable's name starts with asterisk(*) character. For example;

```
data_type  *dataPointer ;
int  *intPtr ; // pointer variable of type integer data-type
```

CONCEPTUAL LAYOUT OF A POINTER VARIABLE			
data-type	*pointer_variable_name	=	*Memory Address of a variable matching* data-type
			Memory Address of pointer_variable_name

Let, the 4 digit number depicts the memory locations of our main memory.

<table>
<tr><td colspan="6">Memory (RAM)</td></tr>
<tr><td rowspan="5">Heap</td><td colspan="2"></td><td></td><td colspan="2" rowspan="4"></td></tr>
<tr><td colspan="2"></td><td></td></tr>
<tr><td colspan="2">5264</td><td>5866</td><td>8014</td></tr>
<tr><td colspan="3"></td></tr>
<tr><td colspan="3">7085</td></tr>
<tr><td rowspan="4">Stack</td><td></td><td></td><td></td><td>7088</td><td>9244</td></tr>
<tr><td colspan="4">main()</td><td></td></tr>
<tr><td>0</td><td>1</td><td>5</td><td>……..</td><td>NULL</td></tr>
<tr><td>1024
row</td><td>1026
col</td><td>1028
totalRows</td><td>…….</td><td>2048
intPtr</td></tr>
<tr><td rowspan="2">Code</td><td colspan="5">Program will get converted into machine code after compiler complies the programming code. This machine code then gets loaded into this area.
Actual code is shown for understanding purpose as machine code is not easy to grasp.</td></tr>
<tr><td colspan="5">

```
#include <stdio.h>
int main() {
	int row = 0;
	int col = 1;
	int totalRows = 5;
	int *intPtr = NULL; // pointer variable
	return 0;
}
```

</td></tr>
</table>

We know that a variable represents a memory location. The variables r**ow, col and totalRows** are local to the main() method. Whenever program gets executing, it initialize these local variables and they are pushed into the **stack memory area**. These local variables which are on the **stack** are readily available to the machine code and are directly accessible. This machine code can't directly access the memory locations of a **heap memory area.** This short-coming can be overcome by using pointer variable which basically contains only one type of data and that is memory location. Pointer variable also resides in the stack memory area like other variables. The following programming instruction suggests that the pointer variable `intPtr` is not pointing to any memory location and a simple way to initialize any pointer variable before it's actual usage.

```
int *intPtr = NULL; // pointer variable initialized to NULL.
```

<table>
<tr><td colspan="6">Memory (RAM)</td></tr>
<tr><td rowspan="7">Heap</td><td colspan="3"></td><td rowspan="7"></td><td></td></tr>
<tr><td colspan="3"></td><td>6014</td></tr>
<tr><td colspan="3"></td><td></td></tr>
<tr><td colspan="3"></td><td></td></tr>
<tr><td colspan="3">4264 4866</td><td></td></tr>
<tr><td colspan="3"></td><td></td></tr>
<tr><td colspan="3">6085</td><td></td></tr>
<tr><td rowspan="3">Stack</td><td></td><td></td><td></td><td>7088</td><td>9244</td></tr>
<tr><td colspan="4">main()</td><td></td></tr>
<tr><td>0</td><td>1</td><td>5</td><td>……..</td><td>6014</td></tr>
<tr><td>1024
row</td><td>1026
col</td><td>1028
totalRows</td><td>…….</td><td>2048
intPtr</td></tr>
<tr><td></td><td></td><td></td><td></td><td></td></tr>
<tr><td>Code</td><td colspan="5">Program will get converted into machine code after compiler complies the programming code. This machine code then gets loaded into this area.
Actual code is shown for understanding purpose as machine code is not easy to grasp.</td></tr>
</table>

```
#include <stdio.h>
#include <stdlib.h>

int main() {
      int row = 0;
      int col = 1;
      int totalRows = 5;
      int *intPtr = NULL; // pointer variable

      // allocate a continuous block of 50 ints on the heap:
      intPtr = (int*) malloc( sizeof(int) * 50);
      if( intPtr == NULL ) {
         printf("*** malloc failed :");
```

```
        }
        // deallocate memory on the heap.
        free(intPtr);
        return 0;
}
```

If `malloc function call is successful` then pointer variable `intPtr` will hold the memory address location from the heap memory area. As an example, let us consider in this case where pointer variable `intPtr` contains memory address location **6014** and this memory address location exists on heap memory area where continuous block for **50 integers** will be get allocated. Similarly, `free(intPtr)` method call deallocates the memory.

So, pointer variable can be made to point to memory location and in this case it is pointing to heap memory area to avail the facility of getting extra memory. The demand for extra memory could be sufficient big and that might throw an **stackoverflow** error if requested from stack memory area. Some program may require a large piece of memory to hold a large set of data and this large data may be required to be available for a longer period of time during the execution of program. In such situations usually a continuous block of memory is allocated dynamically and also get released when it is not required. This freeing of memory gives chance to reuse the memory when requested by running program. The serious part in all these activities of allocation and freeing of memory is that it has to be done manually through programming. In case of local variables it is automated. As soon as function call ends, all local variables and function call are pop-out of the stack memory and get freed up the memory automatically.

Stack and function calls

Stack literally means arranging items on top of another. A pile of books as shown in the picture is an example of stack. If we want a book from this stack, we need to start removing book from the top and then get the actual one. This is called **last-in-first-out** or in short **LIFO** method. Stack is a memory area in a RAM where data are added and removed in **LIFO** style. When a method is called, data stored in a variable(s) push into the top of the stack and data pop-out or get removed when the execution of called function ends.



Stack of books

Suppose our **main** method calls the function **swap.** Then the stack-frames get developed into the stack memory area. Each stack-frame represents a functional call whose execution is yet to be completed.

Assumptions: The size of **int** data type is 4 bytes.

The program control is represented by →

Code	Stack

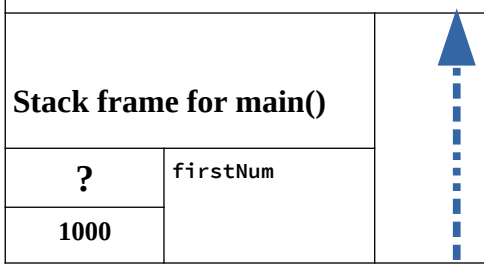
```
#include <stdio.h>

// function prototype

void swap(int, int );

int main() {
        // local variables:
         firstNum, secondNum
        int firstNum = 9;
        int secondNum = 10;

        // actual arguments:
         firstNum, secondNum
        swap( firstNum, secondNum );
        return 0;
}
// formal parameters :
// firstNo and secondNo
void swap(int firstNo, int secondNo) {
       int emptyVar = 0 ;
            ..................
            ..................
}
```



UPWARD GROWING STAACK

When program starts executing the stack frame for the method **main()** gets created on to the stack memory area.
When program control reaches variable declaration as;

```
int firstNum
```

The variable `firstNum` is pushed into the stack represented by memory address **1000** and data can be any garbage value(**?**).

In the assignment operation given as follows;

```
int firstNum = 9;
```

The left side of = represent a memory location.
The right side of = represents data on that memory location

Code

```c
#include <stdio.h>

// function prototype

void swap(int, int );

int main() {
        // local variables:
         firstNum, secondNum
        int firstNum = 9;
        int secondNum = 10;

        // actual arguments:
         // firstNum, secondNum
        swap( firstNum, secondNum );
        return 0;
}
// formal parameters :
// firstNo and secondNo
void swap(int firstNo, int secondNo) {
       int emptyVar = 0 ;
             ……
             ……
}
```

Stack

Stack frame for **main()**

?	secondNum
1004	
9	firstNum
1000	

UPWARD GROWING STACK

When the following statement/instruction gets executed;

```c
int firstNum = 9;
```

The data of the variable `firstNum` represented by memory address **1000** is gets updated with the value 9.
When program control reaches variable declaration as;

```c
int  secondNum
```

The variable `secondNum` is pushed into the stack represented by memory address **1004** and where data can be any garbage value(**?**).

In the assignment operation given as follows;

```c
int secondNum = 10;
```

The left side of = represent a **memory location (1004)**
The right side of = represents **data** on that memory location **(10)**

Code

```
#include <stdio.h>

// function prototype

void swap(int, int );

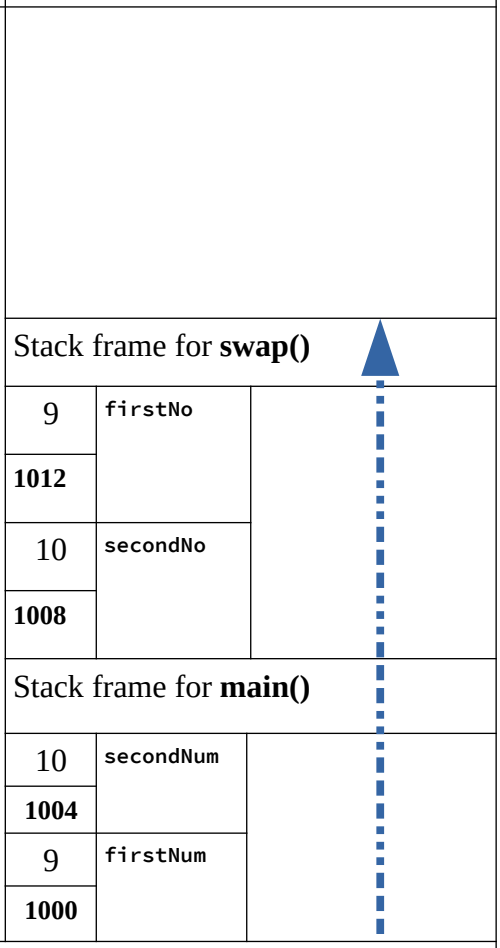
int main() {
        // local variables:
         firstNum, secondNum
        int firstNum = 9;
        int secondNum = 10;

        // actual arguments:
         // firstNum, secondNum
        swap( firstNum, secondNum );
        return 0;
}

                    DATA COPIED

void swap(int firstNo, int secondNo) {
       int emptyVar = 0 ;
        emptyVar = firstNo;
        firstNo = secondNo;
        secondNo = emptyVar;
}
```

Stack



UPWARD GROWING STAACK

When the following statement/instruction gets executed;
`int secondNum = 10;`

The data of the variable `secondNum` represented by memory address **1004** is gets updated with the value 10.

When program control reaches **function call swap** as;
`swap( firstNum, secondNum );`

a)The stack-frame of method **swap()** is then pushed into the stack.
b)The data of the variable `secondNum` represented by memory address **1004** is gets copied to the memory location **1008** with the value 10 represented by the variable `secondNo`.
The variable `secondNo` is a method parameter of the function **swap().**
c)The data of the variable `firstNum` represented by memory address **1000** is gets copied to the memory location **1012** with the value 10 represented by the variable `firstNo`.
The variable `firstNo` is a method parameter of the function **swap().**

The side-arrow shows the growth in stack.

Code

```
#include <stdio.h>

// function prototype

void swap(int, int );

int main() {
        // local variables:
         firstNum, secondNum
        int firstNum = 9;
        int secondNum = 10;

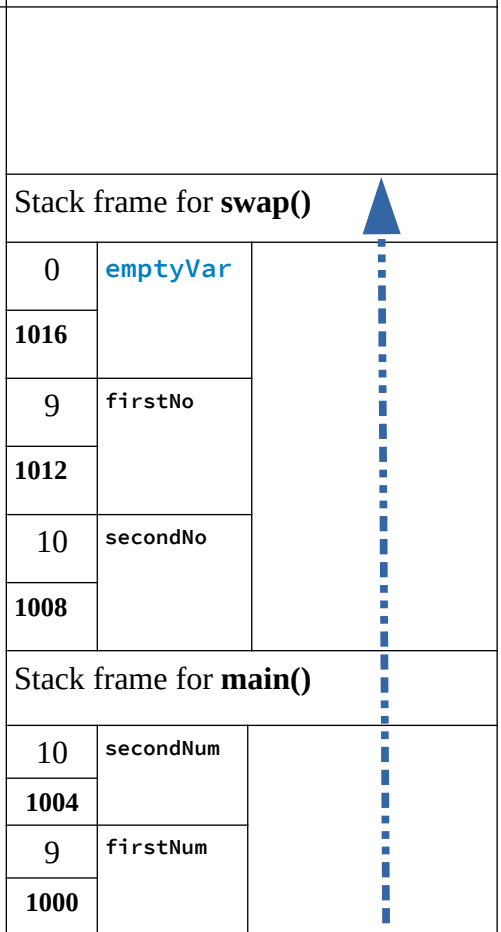
        // actual arguments:
         // firstNum, secondNum
        swap( firstNum, secondNum );
        return 0;
}
```

DATA COPIED

```
void swap(int firstNo, int secondNo) {

      // local variable: emptyVar
        int emptyVar = 0 ;
        emptyVar = firstNo;
        firstNo = secondNo;
        secondNo = emptyVar;
}
```

Stack



UPWARD GROWING STAACK

After function call **swap()**, the program control tries to execute the programming instructions of the function **swap().**

```
void swap(int firstNo, int secondNo) {
        // local variable: emptyVar
              ..................
              ..................
}
```

When program control reaches the following variable declaration given as shown by the pointed arrow;

```
int emptyVar;
```

The variable `emptyVar` is pushed into the stack represented by memory address **1016** and data can be any garbage value(**?**). The variable `emptyVar` is now at the top of the stack.

When the following statement/instruction gets executed;

```
int emptyVar = 0 ;
```

The data of the variable `emptyVar` represented by memory address **1016** is gets updated with the value 0.
As per logic implemented in swap() method, the data between variables `firstNo and secondNo` should get exchanged or swapped.

Code

```
#include <stdio.h>

// function prototype

void swap(int, int );

int main() {
        // local variables:
         firstNum, secondNum
        int firstNum = 9;
        int secondNum = 10;

        // actual arguments:
         // firstNum, secondNum
        swap( firstNum, secondNum );
        return 0;
}

                    DATA COPIED

void swap(int firstNo, int secondNo) {

       // local variable: emptyVar
        int emptyVar = 0 ;
        emptyVar = firstNo;
        firstNo = secondNo;
        secondNo = emptyVar;
}
```

Stack

Stack frame for **main()**

10	secondNum	
1004		
9	firstNum	
1000		

UPWARD GROWING STACK

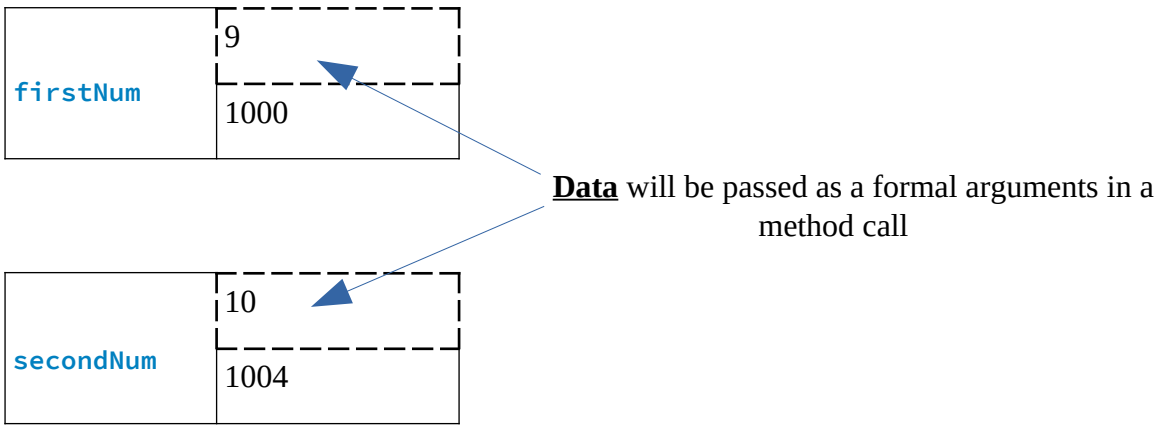
When **swap()** method's execution gets completed, the **stack-frame of swap()** method will **pop-out** or get removed from the stack memory area.

Finally, when **main()** method's execution gets completed, the **stack-frame of main()** method will also **pop-out** or get removed from the stack memory area. Now, stack memory will have no stack-frames available for their execution. The stack memory area is emptied and ready to reuse for next program's execution.

In a very loose term, the variable is basically made-up of three attributes. It has a name, data and memory address.
Let `firstNum` and `secondNum` are the two variables declared in the **main()** method.
Let the variables `firstNum` and `secondNum` are having data values 9 and 10 respectively.
Let the memory address of variables `firstNum` and `secondNum` are 1000 and 1004 respectively.

Call by Value

In this type of method call, the data value of a variable will be passed as an argument.



Whenever there is a method call, every method will have it's own individual stack frame. Hence, when we execute the following program, each method **main()** and **swap()** will maintain a separate stack frame.

CallByValue.c

```
#include <stdio.h>

void swap(int, int );// function prototype

int main() {

	// local variables: firstNum, secondNum

	int firstNum = 9, secondNum = 10;

	printf( "*** Before swap() method call ***\n");

	printf( "[firstNum = %d | secondNum = %d ]\n", firstNum, secondNum );

	printf( "-----------------------------------------\n");

	// actual arguments:firstNum, secondNum

	swap( firstNum, secondNum );

	printf( "-----------------------------------------\n");

	printf( "*** After swap() method call ***\n");

	printf( "[firstNum = %d | secondNum = %d] ", firstNum, secondNum );

	return 0;
}
```

```
// formal parameters : firstNo and secondNo
void swap(int firstNo, int secondNo) {
      printf( "Start swap() : ");
      printf( "firstNo = %d | secondNo = %d \n", firstNo, secondNo );
      // local variable: emptyVar
      int emptyVar = 0 ;
      emptyVar = firstNo;
      firstNo = secondNo;
      secondNo = emptyVar;
      printf( "End   swap() : " );
      printf( "firstNo = %d | secondNo = %d\n", firstNo, secondNo );
}
```

Output

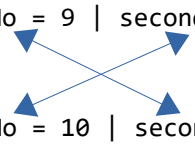
```
 *** Before swap() method call ***
[firstNum = 9 | secondNum = 10 ]
-----------------------------------------
Start swap() : firstNo = 9 | secondNo = 10
End   swap() : firstNo = 10 | secondNo = 9
-----------------------------------------
*** After swap() method call ***
[firstNum = 9 | secondNum = 10]
```

Points to Remember

- Local variables can easily transfer or manipulate data among themselves within their individual method's stack frame.

  The method **swap()** exchanges the data between it's first method parameter and it's second method parameter. It is shown in the output of program which is as follows;

  ```
  Start swap() : firstNo = 9 | secondNo = 10
  End   swap() : firstNo = 10 | secondNo = 9
  ```

- Local variables of a particular method has scope/visibility limited to their individual method's stack frame only. The local variables(`firstNum, secondNum`) of **main()** method are completely unknown to swap() method's stack frame. Similarly, the local variables(`firstNo, secondNo`) of **swap()** method are completely unknown to **main()** method's stack frame. The **swap()** method is called by the **main()** method. The actual parameters as a data are passed in the swap() method call. This means that the data of actual parameters(`firstNum, secondNum`) are get copied to the equivalent individual formal parameters(`firstNo, secondNo`).

The values of formal parameters (`firstNo, secondNo`)which got exchanged within the **swap()** method call by using temporary variable, has no impact on the local variables(`firstNum, secondNum`) of the **main()** method. This can be confirmed by observing the specific part of output of this program which is as follows;

```
*** Before swap() method call ***
```
[firstNum = 9 | secondNum = 10]

```
*** After swap() method call ***
```
[firstNum = 9 | secondNum = 10]

Conclusion:

One method's stack frame is completely blind about local variables which are present on another method's stack frame.

Problem: Give the stack-frame diagram for the **CallByValue.c** program. This program contains many **printf()** method calls and a separate stack-frame will also get created for the **printf()** method call in the stack memory area. Show all the push-pop operation of all the method calls.

Understanding the Concept of Pointers

The concept of pointers can be easily understood by using analogies. The method's stack frame is like a home. So, stack frames are very much like homes where family members of one home doesn't know anything about the family residing within another home, maintaining a complete secrecy with each other. Hence, the family members are like local variables.

Analogies to understand the concepts: -

Concept	**Analogy**
Method's stack frame	Home
Local variables are those variables which are available in their respective method's stack frame.	Family members (M1, M2, M3, M4). **Suppose family members (M1, M2,M3,M4) are available in their home.**
These local variables holds data.	**On a very broader level, the information which each family member is holding regarding anything is similar to what data means to a variable.**
Local variables can transfer or manipulate data/information among each other.	Family members **can share information among each other.**
Every local variable has a **unique memory address location.**	Every family member has a **unique mobile number.**
We write programming instructions using local variables in order to achieve a well-defined targeted output.	They can help each other in order to complete a well-defined task. This overlaps with the intention with which any method or function is written.

Let's suppose that one of the family member **M4** is interested to talk to his/her friend whose is living at a distance far away from his/her place. One of the commonest way with which they can connect to each other is through mobile phones. The mobile number uniquely identifies a person and helps in connecting each other through mobile phones.

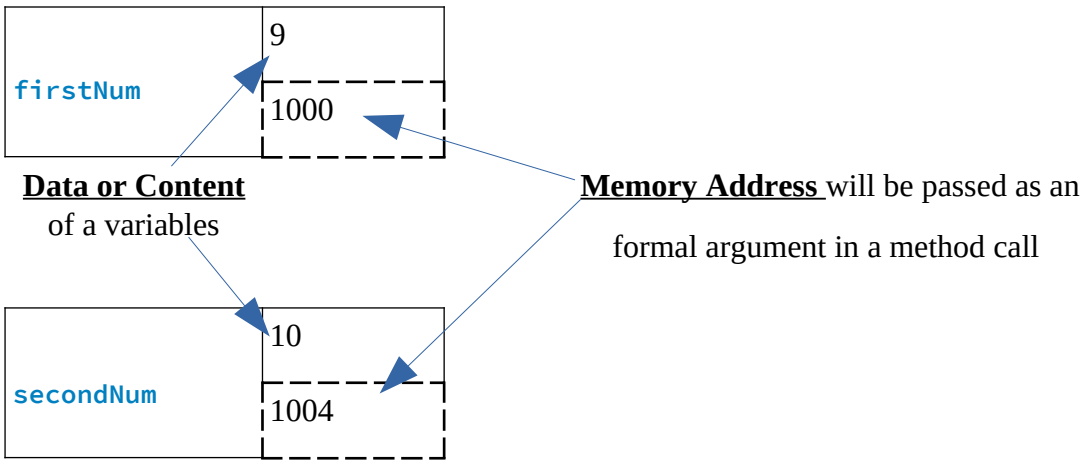
Every byte in a memory has a unique address value associated with it. So this makes sense that every variable declared has a unique memory address associated with it. This idea of uniqueness of memory address location matches with the idea of uniqueness of a mobile number. In this way, the concept of memory addresses resembles very much with the concept of mobile numbers. Consider an activity of taking an appointment of a doctor using a contact number remotely and meeting a doctor personally. In both the cases accessing a doctor differs. The doctor or medical staff has to maintain the state of each appointment The contact number or any other reference value like national identification number or unique appointment number can be used instead.

Call by Reference

In certain situations of programming requirement accessing data using memory address using pointer variable is favoured than accessing data directly using local variable. Let's assume there are many methods in a program. Many methods in a program can access the latest content/data of a particular variable by referring to the memory address of a variable with the help of a pointer variable. This particular variable is local to the **main()** method and whose data may get regularly updated in many methods. In case, suppose many methods on different occasions needed a latest updated value of this particular **main()** method's local variable. We know that in call by value functional call strategy, the called method works on the copy of the variable's data passed via formal arguments. When the execution of called method ends it has no impact on the values of variables which are passed as actual arguments.

How to update the values of actual arguments whose values got changed in a called method?

If there is a single variable, a called method may return a value and callee method will update it with the returned value. If more than one variable is to be updated, we may return many values using an array variable. Now, without returning any value(s) from calling method, how to update callee method's local variables values. So the trick will be, a pointer variable should be used for each method parameter instead of normal variable. We need to pass the memory location of a variable instead of passing the copy of the value of a variable as already done in the **call by value** functional call strategy. This way many methods can get access to the most updated data of a particular variable which might got changed many times by different methods in a program.



CallByReference.c

```c
#include <stdio.h>

// function prototype
void swap(int*, int* );


int main() {
	// local variables: firstNum, secondNum
	int firstNum = 9;
	int secondNum = 10;
	printf( "*** Before swap() method call ***\n");
	printf( "[firstNum = %d | secondNum = %d ]\n", firstNum, secondNum );
	printf( "-----------------------------------------\n");
	// memory address value of local variables are passed.
	swap( &firstNum, &secondNum );
	printf( "-----------------------------------------\n");
	printf( "*** After swap() method call ***\n");
	printf( "[firstNum = %d | secondNum = %d] ", firstNum, secondNum );
	return 0;
}
// formal parameters : firstNo and secondNo
void swap(int* firstNo, int* secondNo) {
	printf( "Start swap() : ");
	printf( "firstNo = %d | secondNo = %d \n", *firstNo, *secondNo );
	// local variable: emptyVar
	int emptyVar = 0 ;
	emptyVar = *firstNo;
	*firstNo = *secondNo;
	*secondNo = emptyVar;
	printf( "End   swap() : " );
	printf( "firstNo = %d | secondNo = %d \n", *firstNo, *secondNo );
}
```

Output

```
*** Before swap() method call ***
[firstNum = 9 | secondNum = 10 ]

-------------------------------------------

Start swap() : firstNo = 9 | secondNo = 10

End   swap() : firstNo = 10 | secondNo = 9

-------------------------------------------

*** After swap() method call ***

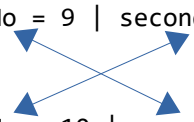
[firstNum = 10 | secondNum = 9]
```

Points to Remember

- Pointer variables can easily manipulate data directly, just by knowing address of a particular variable and so method's stack frame will not be any dependencies. The method **swap()** exchanges data between it's first method parameter and it's second method parameter using pointer variables. The output of program is as follows;

```
Start swap() : firstNo = 9 | secondNo = 10

End   swap() : firstNo = 10 | secondNo = 9
```



- The actual parameters are passed this time in the **swap()** method call is as a memory address location. This means that the memory address values of actual parameters(`&firstNum, &secondNum`) are get stored into the equivalent individual formal pointer variable parameters(`int* firstNo, int* secondNo`). Moreover, the data type of pointer variables(`int* firstNo, int* secondNo`) of **swap()** method are compatible with (`firstNum, secondNum`), the data type of **main()** method's local variables. ***The data type compatibility is unavoidable constraint when a memory address of a particular variable is assigned to the particular pointer variable.***

- The pointer variables(`int* firstNo, int* secondNo`) exchanged the data within the **swap()** method by using temporary variable. The pointer variable can update another variable's content by accessing that variable's memory location. This way pointer variables(`int* firstNo, int* secondNo`) of **swap()** method made a direct impact on the values of local variables(`firstNum, secondNum`) which belong to the **main()** method. This can be confirmed by observing the specific part of output of this program which is as follows;

```
*** Before swap() method call ***
```
[firstNum = 9 | secondNum = 10]

```
*** After swap() method call ***
```
[firstNum = 9 | secondNum = 10]

Conclusions:

- The address operator ampersand (&) is a must to determine address of a variable which can have basic data-type like (int, float, long). So the pointer variable which is made to point the memory address of another variable must have the same data type.
- Use asterisk(*) indirection operator to get the value stored in the particular memory location pointed to by a specific pointer variable.
- The memory address of an array data-type is the memory address of a first element in an array.
- The memory address of an array can be determined by it's name and which is same as the memory address of a 1st element in an array.

Let's verify these facts by implementing an example.

PointerAddressValues.c

```c
#include <stdio.h>
int main() {
	int *intPtr = NULL;// pointer variable of type int.
	int intVal = 10;
	int intArr[5] = { 10, 20, 30, 40, 50 };

	// address operator & is a must for a pointer
	// to point to basic data-type.
	intPtr = &intVal; // assign intVal's address to intPtr.
	printf( " Address of &intVal = %p \n", &intVal );
	printf( " Address of intPtr  = %p \n", intPtr );
	printf( " Address of *intPtr = %d | intVal = %d\n\n", *intPtr, intVal );

	printf( " Address of &intArr   = %p \n", &intArr );
	printf( " Address of intArr[0] = %p \n", &intArr[0] );
	printf( " Address of intArr    = %p \n", intArr );

	// Variable's name of array data-type for a pointer
	// to point to array data-type.
	intPtr = intArr;
	printf( " Address of intPtr after pointing to array   = %p \n", intPtr );

	return 0;
}
```

Output

```
 Address of &intVal = 0061FF28
 Address of intPtr  = 0061FF28
 Address of *intPtr = 10 | intVal = 10

 Address of &intArr   = 0061FF14
 Address of intArr[0] = 0061FF14
 Address of intArr    = 0061FF14
 Address of intPtr after pointing to array   = 0061FF14
```

Accessing array elements using pointers

Array is treated as a pointer to it's first element in memory. Hence, we can use pointer to access elements of an array. The pointer and the array should be of same data-type. The following program is a re-implementation of the program **WithLooping.c** using pointer variables.

PointerInLooping.c

```
#include <stdio.h>

int main() {

	// array initialization
	char name[20] = "Mr.Always Right";
	int ages[5] = { 10, 20, 30, 40, 50};
	float weight[5] = {23.27, 32.14, 37.45, 60.74, 47.25};
	long distance[5] = {1234567L, 4335454L, 98767567L, 4354321L,555588889L};
	int size = - 1;// size of an array

	// pointer declarations
	char *charPtr;
	int *intPtr;
	float *floatPtr;
	long *longPtr;

	printf(" Accessing char array element using char pointer.\n");

	// assigns the starting address of char array to the char pointer.
	charPtr = name ;

	for( int index = 0 ; name[index] != '\0'; index ++){
	// using asterisk(*) indirection operator to print the array's element.
		printf(" %c |", *charPtr);
		charPtr ++ ;
	}

	printf("\n Accessing int array element using int pointer.\n");

	// assigns the starting address of int array to the int pointer.
	intPtr = ages;

	// finding size of array using C language's sizeof() operator.
	size = sizeof( ages )/ sizeof( int );

	for( int index = 0 ; index < size ; index++){

	// using asterisk(*) indirection operator to print the array's element.
		printf(" %d | ", *intPtr);
		intPtr ++ ;
	}
```

```
        printf("\n Accessing float array element using float pointer.\n");

        // assigns the starting address of float array to the float pointer.
        floatPtr = weight;


        // finding size of array using C language's sizeof() operator.
        size = sizeof( weight )/ sizeof( float );

        for( int index = 0 ; index < size ; index++){

        // using asterisk(*) indirection operator to print the array's element.
        // total 5 digits to print float number and 2 digits after decimal point.

              printf(" %5.2f | ", *floatPtr);
              floatPtr ++ ;
        }

        printf("\n Accessing long array element using long pointer.\n");

        // assigns the starting address of long array to the long pointer.
        longPtr = distance;

        // finding size of array using C language's sizeof() operator.
        size = sizeof( distance )/ sizeof( long );

        for( int index = 0 ; index < size ; index++){

        // using asterisk(*) indirection operator to print the array's element.
        // print long digit number.
              printf(" %ld | ", *longPtr);
              longPtr ++ ;
        }
        return 0;
}
```

Output

```
 Accessing char array element using char pointer.
M | r | . | A | l | w | a | y | s |   | R | i | g | h | t |

 Accessing int array element using int pointer.

 10 |  20 |  30 |  40 |  50 |

 Accessing float array element using float pointer.

 23.27 |  32.14 |  37.45 |  60.74 |  47.25 |

 Accessing long array element using long pointer.

 1234567 |  4335454 |  98767567 |  4354321 |  555588889 |
```

Strings

It is common programming practice to modify or manipulate text in some way. Some of the examples are dynamically creating web-page url addresses or file paths, extracting specific information from different kinds of file like text files/PDFs/HTML, developing web-scrapping tools that extract text from web-pages. Every programming language provides in-built string libraries for string manipulation. We will experiment with some of commonly used in-built string methods or functions.

String concatenation

We can output a resultant string by joining one string with another using in-built **strcat** method or function and this process is known as ***concatenating*** strings. The string library to use is;

```
#include <string.h>

char *strcat( char target, const char *source );
This method appends source string to the target string.
```

StringConcat.c

```
#include <stdio.h>
#include <string.h>

int main() {

	char name[20]  = "Steve" ; // target string

	// appends target string to source string

	strcat( name , " Jobs");

	printf("%s \n", name);

	return 0;

}
```

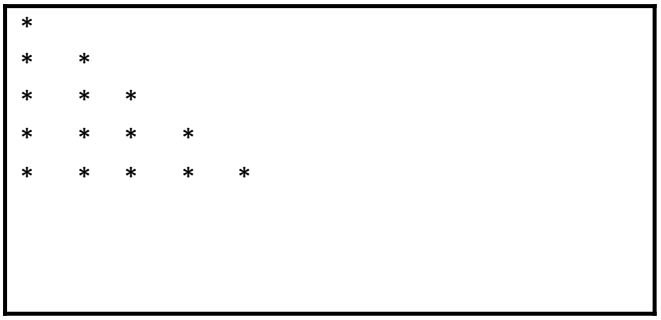
Output

```
Steve Jobs
```

Programming Patterns Using String functions and Arrays

We will now re-implement some of the patterns just using **single *for-loop*** and ***we know that*** these were already implemented using nested loop (one for-loop within another for-loop). It can be done simply by using in-built string methods which are included in the string functions library header given as follows;

```
#include <string.h>.
```



Pattern A1

StringPatternA1.c

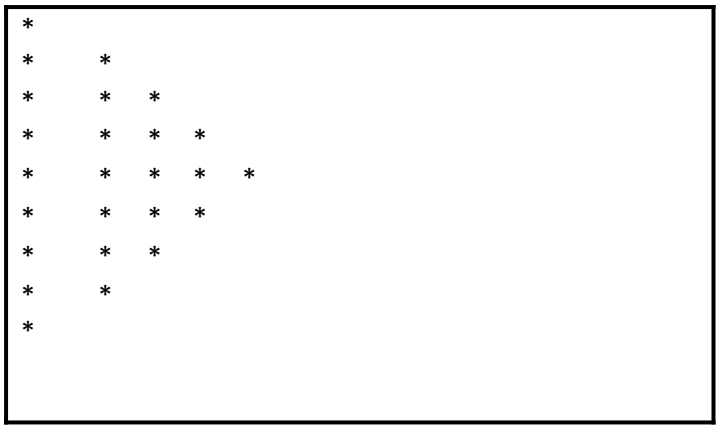
```
#include <stdio.h>
#include <string.h>
int main() {

	int totalRows = 5;  // number of rows to display
	char line[44]={""}; // initialize with blank string.

	// Column level Repetitive Action :
	for( int row = 1 ; row <= totalRows ; row ++ ) {
		strcat( line , "* "); // append strings
		// Print string and move cursor to the next row
		printf("%s\n",line);
	}
	return 0;
}
```

Output

```
*
* *
* * *
* * * *
* * * * *
```



Pattern G1

StringPatternG1.c

```c
#include <stdio.h>
int main() {
	int totalRows = 5;  // number of rows to display
	int index = -1;   // array's index to increment by 1.
	char line[44]={""}; // initialize with blank string.

	for( int row = 1; row <= ( 2 * totalRows - 1 ); row++ ) {
		// Column level Repetitive Action :
		// expanding array by filling it up using '*' char.
		if ( row <= totalRows ) {
			line[++index] = '*';
		} else {
			// replacing '*' char from the end of the array by space.
			// decrement index only when row > totalRows
			line[index--] = ' ';
		}
		// Print string and move cursor to the next row
		printf("%s\n",line);
	}
	return 0;
}
```

Output

```
*
**
***
****
*****
****
***
**
*
```

ProblemSPG1: In the program **`StringPatternG1.c`**, the star **"*"** characters are printed quite closed to each other. In order to get better formatted output suppose an extra space is added as shown in the following program by **arrow mark,** the output doesn't came as per our expectation.

What modifications in the following program should be applied so that we should able to get the expected output.

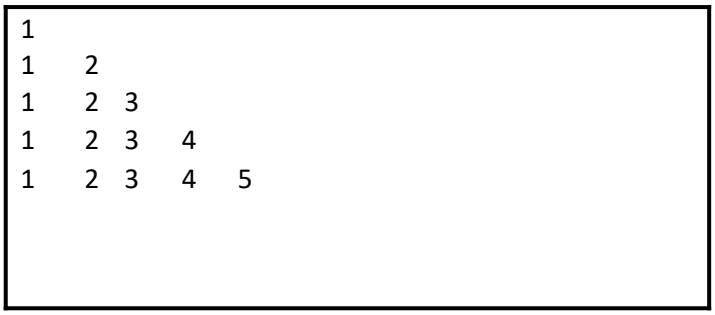
StringPatternG1.c

```
#include <stdio.h>
int main() {

	int totalRows = 5;  // number of rows to display
	int index = -1;   // array's index to be incremented by 1.
	char line[44]={""}; // initialize with blank string.

	for( int row = 1; row <= ( 2 * totalRows - 1 ); row++ ) {
		// Column level Repetitive Action :
		// expanding array by filling it up using '*' char.
		if ( row <= totalRows ) {
			line[++index] = '*';
			// Extra space added so to get better formatted output.
			line[++index] = ' ';
		} else {
			// replacing '*' char from the end of the array by space.
			// decrement index only when row > totalRows
			line[index--] = ' ';
		}
		// Print string and move cursor to the next row
		printf("%s\n",line);
	}
	return 0;
}
```

Output by this program	Expected Output
*	*
* *	* *
* * *	* * *
* * * *	* * * *
* * * * *	* * * * *
* * * * *	* * * *
* * * *	* * *
* * * *	* *
* * *	*



NumberPattern AA1

StringPatternAA1.c

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
int main() {

	int totalRows = 5;  // number of rows to display
	char line1[44]={""}; // initialize with blank string.
	char line2[5]={""}; // initialize with blank string.

	// Column level Repetitive Action :
	for( int row =  1 ; row <= totalRows ; row ++ ) {

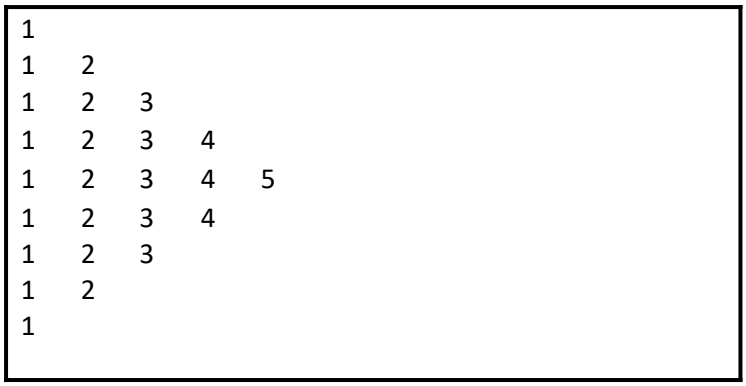
		// convert num to string
		itoa(row, line2, 10);

		strcat( line1 , line2 ); // append strings

		// Print string and move cursor to the next row
		printf("%s\n", line1);
	}
	return 0;
}
```

Output

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```



NumberPatternGG1

StringPatternGG1.c

```
#include <string.h>
#include <stdlib.h>
#include <stdio.h>
int main() {
	int totalRows = 5;  // number of rows to display
	int index = -1;   // array's index to increment by 1.
	char line1[44]={""}; // initialize with blank string.
	char line2[5]={""}; // initialize with blank string.

	for( int row = 1; row <= ( 2 * totalRows - 1 ); row++ ) {
		// Column level Repetitive Action :
		// expanding array by filling it up using '*' char.
		if ( row <= totalRows ) {
			++index; // will track record of line1 array elements
			// convert num to string
			itoa(row, line2, 10);
			strcat( line1 , line2 ); // append strings
		} else {
			// replacing numeric character from the end
			// of the array a space character.

			// decrement index only when row > totalRows
			line1[index--] = ' ';
		}
		// Print string and move cursor to the next row
		printf("%s\n",line1);
	}
	return 0;
}
```

Output

```
1
12
123
1234
12345
1234
123
12
1
```

Exercise: Write a program to output for the following staircase look alike simple picture.

```
P_|_P
r__|__r
o___|___o
g____|____g
r_____|_____r
a______|______a
m_______|_______m
m________|________m
i_________|_________i
n__________|__________n
g___________|___________g
```

Staircase.c

```
#include <stdio.h>
#include <string.h>
int main() {

	char string[44] = "Programming";
	char spaces[20] = {""};

	for( int index = 0 ; string[index]!= '\0' ; index++) {
		strcat( spaces , "_" ); // append strings
		char ch = string[index] ;
		printf("%c",ch);
		printf("%s|%s",spaces, spaces);
		// Print last char and move cursor to the next row
		printf("%c\n",ch);
	}
	return 0;
}
```

Exercise1: Rewrite the program **Staircase.c** without using **strcat**() method.

The first step will be to write the logic trace table with appropriate row & columns.

From the analysis of logic trace table, a programmer can be able to find out generalised way to get the solution to print series of "_"characters. A different character could also be taken like "*" ,"=" or "#" in case you didn't get the count.

Exercise2: Generate the following output using sub-strings of the string "Programming".

```
P
Pr
Pro
Prog
Progr
Progra
Program
Programm
Programmi
Programmin
Programming
```

Exercise3: *Generate the following output using* sub-strings of the string *"Programming".*

```
P
Pr
Pro
Prog
Progr
Progra
Program
Programm
Programmi
Programmin
Programming
Programmin
Programmi
Programm
Program
Progra
Progr
Prog
Pro
Pr
P
```

PatternBySubString.c

```c
#include <stdio.h>
#include <stdbool.h> //C99
int main() {

	char string[44] = "Programming";
	char line[44]={""}; // initialize with blank string.
	int chars = -1 ;
	int index = -1;
	bool isLast = false;
	for( index = 0 ; ( chars > 0 || string[index] != '\0'); index++ ) {
		// Column level Repetitive Action :

		// expanding array by filling it up using array char.
		line[index] = string[index];
		if(string[index] == '\0'){
			isLast = true;
		}
		if ( isLast ) {
			line[chars--] = ' ';
		}else{
			chars ++ ;
		}
		// Print string and move cursor to the next row
		printf("%s\n",line);

	}
	return 0;
}
```

Pattern1H

	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1					*(1,5)				
Row=2				*(2,4)	*(2 ,5)	*(2,6)			
Row=3			*(3,3)	*(3,4)	*(3,5)	*(3,6)	*(3,7)		
Row=4		*(4,2)	*(4,3)	*(4,4)	*(4,5)	*(4,6)	*(4,7)	*(4,8)	
Row=5	*(5,1)	*(5,2)	*(5,3)	*(5,4)	*(5,5)	*(5,6)	*(5,7)	*(5,8)	*(5,9)
Row=6		*(6,2)	*(6,3)	*(6,4)	*(6,5)	*(6,6)	*(6,7)	*(6,8)	
Row=7			*(7,3)	*(7,4)	*(7,5)	*(7,6)	*(7,7)		
Row=8				*(8,4)	*(8,5)	*(8,6)			
Row=9					*(9,5)				

It's equivalent logic trace table can be given as follows

For totalRows = 5			**Expected Output**
Variable: row	**Column level Repetitive Action**		
	Variable : spaces	Variable: col	
1	1,2,3,4	5	*
2	1,2,3	4, 5,6	* * *
3	1,2	3, 4, 5,6,7	* * * * *
4	1	2, 3, 4, 5,6,7,8,	* * * * * * *
5	0 (no space)	1, 2, 3, 4, 5,6,7,8,9	* * * * * * * * *
6	1	2, 3, 4, 5,6,7,8,	* * * * * * *
7	1,2	3, 4, 5,6,7	* * * * *
8	1,2,3	4, 5,6	* * *
9	1,2,3,4	5	*

Let's try to solve this pattern using a 1-dimensional array. We know that array is a continuous memory block which needs to declare earlier before it's use. For example our array will be;

`strings[44]`											
position	1	2	3	4	5	6	7	8	9	…...	44
Character										…..	….

```
char strings[44] = { "" };// continuous block of memory allocated.
Moreover, the following printing statement prints horizontally all the
content of an array in a single chance.
printf( "%s", strings );
```

Hence for every row we need to print the array but with modification in the contents of this array. Basically we will re-use the same array and print it's contents.

Two main observations are as follows:

a) If we initialize this array with space, then logic to dynamically print spaces will not be required.

b) Only we need to track of putting the star(*) characters at an appropriate places. The count goes on increases till row=5 and goes on decrease. Initially we need to fill the array with star(*) characters to complete the first half triangle and then need to decrease star(*) characters to generate the reverse triangle.

For totalRows = 5 (mid position)				**Expected Output**
Variable: row	**Column level Repetitive Action**			
	`Characters to be filled in array positions`			
	array position towards beginning	Filled positions	array position towards end	
1		5		*
2	4 = (**5** - 1)	**5**	6 = (**5** + 1)	* * *
3	3 = (**5** - 2)	**4**, 5,**6**	7 = (**5** + 2)	* * * * *
4	2 = (5 - 3)	3,4, 5,6,7	8 = (5 + 3)	* * * * * * *
5	1 = (5 - 4)	2, 3, 4, 5,6,7,8	9 = (5 + 4)	* * * * * * * * *
6	space filled in position 1	**2**, 3, 4, 5,6,7,**8,**	space filled in position 9	* * * * * * *
7	space filled in position 2	3, 4, 5,6,7	space filled in position 8	* * * * *
8	space filled in position 3	4, 5,6	space filled in position 7	* * *
9	space filled in position 4	5	space filled in position 6	*

For totalRows = 5 (mid position)					**Expected Output**
row	**Column level Repetitive Action**				
	`Characters to be filled in array positions`				
	character assigned	array position	Filled positions	array position	
1			5		*
2	*	4 = (**5** - 1)	**5**	6 = (**5** + 1)	* * *
3	*	3 = (**5** - 2)	**4**, 5,**6**	7 = (**5** + 2)	* * * * *
4	*	2 = (5 - 3)	3,4, 5,6,7	8 = (5 + 3)	* * * * * * *
5	*	1 = (5 - 4)	2, 3, 4, 5,6,7,8	9 = (5 + 4)	* * * * * * * * *
6	space	1 = (5 - 4)	**2**, 3, 4, 5,6,7,**8,**	9 = (5 + 4)	* * * * * * *
7	space	2 = (5 - 3)	3, 4, 5,6,7	8 = (5 + 3)	* * * * *
8	space	3 = (**5** - 2)	4, 5,6	7 = (**5** + 2)	* * *
9	space	4 = (**5** - 1)	5	6 = (**5** + 1)	*

Let's consider a **variable: count** to represent increment and decrement values to used to calculate the borders of this pattern. Every time a new border values in a row are calculated in either of sides and other contents of an array are re-used as it is for new updates in the array. The updated array will represent a new row in the considered pattern problem to be solve.

For totalRows = 5 (mid position)						**Expected Output**
row	count	**Column level Repetitive Action**				
		`Characters to be filled in array positions`				
		char	array position	Filled positions	array position	
1	0		5 = 5 - 0	5	5 = 5 + 0	*
2	1	*	4 = (**5 –** count)	**5**	6 = (**5** + count)	* * *
3	2	*	3 = (**5 –** count)	**4**, 5,**6**	7 = (**5** + count)	* * * * *
4	3	*	2 = (5 **–** count)	3,4, 5,6,7	8 = (5 + count)	* * * * * * *
5	4	*	1 = (5 **–** count)	2, 3, 4, 5,6,7,8	9 = (5 + count)	* * * * * * * * *
6	4	space	1 = (5 **–** count)	**2**, 3, 4, 5,6,7,**8**	9 = (5 + count)	* * * * * * *
7	3	space	2 = (5 **–** count)	3, 4, 5,6,7	8 = (5 + count)	* * * * *
8	2	space	3 = (**5 –** count)	4, 5,6	7 = (**5** + count)	* * *
9	1	space	4 = (**5 –** count)	5	6 = (**5** + count)	*

Let's try to implement program based on our fact-findings.

Pattern1HByArray.c

```c
#include <stdio.h>
#include <stdlib.h>
int main() {

	char strings[44] = { "" };
	int totalRows = 5; // number of rows to display
	int MAX_COLUMNS = 2 * totalRows - 1;
	int midIndex = totalRows - 1; // since array index start at 0;
	int count = 0;

	// initializing array with spaces
	for ( int col = 0; col < MAX_COLUMNS; col++ ) {
		strings[col] = ' ';
	}

	// Column level Repetitive Action :
	for ( int row = 1; row <= MAX_COLUMNS; row++ ) {

		// filling array using '*' char at specific positions.
		if ( row == 1 ) {
			strings[midIndex] = '*';
		} else if ( row <= totalRows ) {
			count = count + 1;
			strings[midIndex - count] = '*';
			strings[midIndex + count] = '*';
		} else {
			// for second-half portion
			strings[midIndex - count] = ' ';
			strings[midIndex + count] = ' ';
			count = count - 1;
		}
		// Print string and move cursor to the next row
		printf( "%s\n", strings );
	}
	return 0;
}
```

Output

```
    *
   ***
  *****
 *******
*********
 *******
  *****
   ***
    *
```

Programming Pattern Using Pointer

In most of the case related to arrays, memory remains unused. The C library does provides a facility to allocate memory as per our exact memory allocation demands. Such dynamic memory allocation functions are declared in the <stdlib.h> header. For example;

```
int totalRows = 5;

char *charPtr;

charPtr = (char *)malloc( sizeof(char) * (totalRows + 1) );
```

malloc or memory allocation method is used to dynamically allocate a single large block of memory with the given specified element count and data-type. In this example data-type is char and elements are 5 and require 1 more extra byte of memory for string termination by **NULL**. So in total it will be (totalRows + 1) and this suggests (1 * 6 or 6) bytes of memory block.

6 bytes	byte	byte	byte	byte	byte	byte
Index	0	1	2	3	4	5
Address	1024	1025	1026	1027	1028	1029



charPtr

charPtr will hold the address of the first element in the allocated memory block.

```
// string is made null-terminated at 6th byte after
// allocating characters in first 5 bytes.
*( charPtr + 5 ) = '\0';
```

In general;

```
dataType  *dataTypePtr = (dataType *) malloc ( sizeof(dataType) * count );
```

If the memory is insufficient, then allocation will fail and will returns a NULL pointer.

```
        // checking if memory block allocated or not.
        if ( charPtr == NULL) {
                printf("malloc failed.\n");
                exit(0); // terminate the program.
         }
```

In order to de-allocate the allocated memory, we can use method **free** from <stdlib.h> header.

```
// freeing the allocated memory
free( charPtr );
```

Let's implement a program to print the **pattern1H(diamond pattern)** using dynamic memory allocation and pointer.

Pattern1HByPointer.c

```c
#include <stdio.h>
#include <stdlib.h>
/* function declaration */
void printPattern(int rows);

int main() {

	// number of rows
	int totalRows = 5;

	printPattern( totalRows );// function call to print pattern

	totalRows = 7;
	printPattern( totalRows );// function call to print pattern

	return 0;
}

/**
 * function printing diamond pattern by dynamically allocating memory.
 */
void printPattern(int totalRows) {

	/* local variable declaration */
	int MAX_COLUMNS = 0;
	int midIndex = 0;
	int count = 0;
	// This pointer will holds the memory address of the first element
	// in the allocated memory block.
	char *charPtr;

	// To dynamically allocate memory.
	printf( "Enter number of rows : %d\n", totalRows );
	MAX_COLUMNS = 2 * totalRows - 1;
	midIndex = totalRows - 1; // since array index start at 0;
	charPtr = (char*) malloc( sizeof(char) * MAX_COLUMNS + 1 );

	// checking if memory block allocated or not.
	if ( charPtr == NULL ) {
		printf( "malloc failed.\n" );
		exit( 0 ); // terminate the program.
	}
	// initializing array with spaces
	for ( int spaces = 0; spaces < MAX_COLUMNS; spaces++ ) {
		* ( charPtr + spaces ) = ' ';
	}

	// string is made null-terminated.
	* ( charPtr + MAX_COLUMNS ) = '\0';
```

```
        // Column level Repetitive Action :
        for ( int col = 0; * ( charPtr + col ) != '\0'; col++ ) {
                // filling array using '*' char at specific positions.

                if ( col == 0 ) {
                        * ( charPtr + midIndex ) = '*';
                } else if ( col < totalRows ) {
                        count = count + 1;
                        * ( charPtr + midIndex - count ) = '*';
                        * ( charPtr + midIndex + count ) = '*';
                } else {
                        // for second-half portion
                        * ( charPtr + midIndex - count ) = ' ';
                        * ( charPtr + midIndex + count ) = ' ';
                        count = count - 1;
                }

                // Print string and move cursor to the next row
                printf( "%s\n", charPtr );
        }

        // freeing the allocated memory
        free( charPtr );
        printf( "Memory freed successfully.\n\n" );
}
```

Output

```
Enter number of rows : 5
    *
   ***
  *****
 *******
*********
 *******
  *****
   ***
    *
Memory freed successfully.

Enter number of rows : 7
      *
     ***
    *****
   *******
  *********
 ***********
*************
 ***********
  *********
   *******
    *****
     ***
      *
Memory freed successfully.
```

Problem1. Print the following output using C in-built functions.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1											P										
2										r	P	r									
3									o	r	P	r	o								
4								g	o	r	P	r	o	g							
5							r	g	o	r	P	r	o	g	r						
6						a	r	g	o	r	P	r	o	g	r	a					
7					m	a	r	g	o	r	P	r	o	g	r	a	m				
8				m	m	a	r	g	o	r	P	r	o	g	r	a	m	m			
9			i	m	m	a	r	g	o	r	P	r	o	g	r	a	m	m	i		
10		n	i	m	m	a	r	g	o	r	P	r	o	g	r	a	m	m	i	n	
11	g	n	i	m	m	a	r	g	o	r	P	r	o	g	r	a	m	m	i	n	g

Problem2. Print the following output using string in-built functions.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1											P										
2										r		r									
3									o				o								
4								g						g							
5							r								r						
6						a										a					
7					m												m				
8				m														m			
9			i																i		
10		n																		n	
11	g																				g
12		n																		n	
13			i																i		
14				m														m			
15					n												n				
16						a										a					
17							r								r						
18								g						g							
19									o				o								
20										r		r									
21											P										

Problem3. Please verify wavelength of your logical thinking capabilities. Write a generalised program, so that for different size=1,2,3 inputs, the size of the wave should get doubled or triple. You can write separate program for each different type of wave.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
1	*	*	*	*			*	*	*	*			*	*	*	*						
2	*			*			*			*			*			*						
3	*			*			*			*			*			*						
4	*			*			*			*			*			*			*			
5				*			*			*			*			*			*			
6				*			*			*			*			*			*			
7				*	*	*	*			*	*	*	*			*	*	*	*			
SIZE = 1																						
1	*						*	*	*	*	*	*	*	*								
2	*						*							*								
3	*						*							*								
4	*						*							*								
5	*						*							*								
6	*						*							*								
7	*						*							*								
8	*						*							*								
9	*						*							*								
10	*						*							*								
11	*						*							*								
12	*						*							*								
13	*	*	*	*	*	*	*							*	*	*	*	*	*	*		
SIZE = 2																						
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
1				*												*						
2			*		*										*		*					
3		*				*								*				*				
4	*						*						*						*			
5								*				*								*		
6									*		*										*	
7										*												*
SIZE = 1																						

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
1							*						*						*			
2						*	*					*	*					*	*			
3					*		*				*		*				*		*			
4				*			*			*			*			*			*			*
5			*				*		*				*		*				*		*	
6		*					*	*					*	*					*	*		
7	*						*						*						*			
SIZE = 1																						
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
1	*						*						*									
2	*	*					*	*					*	*								
3	*		*				*		*				*		*							
4	*			*			*			*			*			*						
5	*				*		*				*		*				*					
6	*					*	*					*	*					*				
7	*						*						*						*			
SIZE = 1																						
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
1	1	1	1	1			2	2	2	2			3	3	3	3						
2	1			1			2			2			3			3						
3	1			1			2			2			3			3						
4	1			1			2			2			3			3			3			
5				1			1			2			2			3			3			
6				1			1			2			2			3			3			
7				1	1	1	1			2	2	2	2			3	3	3	3			
SIZE = 1																						
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
1	1	1	1	1			0	0	0	0			1	1	1	1						
2	1			1			0			0			1			1						
3	1			1			0			0			1			1						
4	1			1			0			0			1			1			1			
5				1			1			0			0			1			1			
6				1			1			0			0			1			1			
7				1	1	1	1			0	0	0	0			1	1	1	1			
SIZE = 1																						

Problem3. Write a program for each of the following different wave design.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
1		*	*					*	*					*	*							
2	*			*			*			*			*			*						
3	*			*			*			*			*			*						
4	*			*			*			*			*			*			*			
5				*			*			*			*			*			*			
6				*			*			*			*			*			*			
7					*	*					*	*					*	*				

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
1		<	>					<	>					<	>							
2	<			>			<			>			<			>						
3	<			>			<			>			<			>						
4	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-			
5				<			>			<			>			<			>			
6				<			>			<			>			<			>			
7					<	>					<	>					<	>				

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
1		<	>					<	>					<	>							
2	<			>			<			>			<			>						
3	<			>			<			>			<			>						
4	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-			
5				[			]			[			]			[			]			
6				[			]			[			]			[			]			
7					[	]					[	]					[	]				

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
1		<	>					[	]					<	>							
2	<			>			[			]			<			>						
3	<			>			[			]			<			>						
4	<			>			[			]			<			>						
5				<			>			[			]			<			>			
6				<			>			[			]			<			>			
7					<	>					[	]					<	>				

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
1		3	4					15	16					27	28							
2	2			5			14			17			26			29						
3	1			6			13			18			25			30						
4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
5				7			12			19			24			31			36			
6				8			11			20			23			32			35			
7					9	10					21	22					33	34				

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
1		1	2					5	6					9	10							
2	1			2			5			6			9			10						
3	1			2			5			6			9			10						
4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
5				3			4			7			8			11			12			
6				3			4			7			8			11			12			
7					3	4					7	8					11	12				

Recursion: Repeat without looping

In ***for-loop,*** the same piece of code which is enclosed in for-loop's code block gets repeated until the condition to repeat it becomes false. We can able to get similar kind of affect using function and function calls. We know that a function can call other function(s) as not to repeat the same piece of code or set of programming instructions again and again. In case, if we do the opposite and repeat the same piece of code again and again by calling the same function then this process is known as **recursion** and function is known as **recursive function**. Moreover, like in most of repetitive processing we tried to maintain a **terminating condition** so that after which it should stop working. Similarly, we are required to give a terminating condition in order to end the motion of recursive function calls. You need to maintain a simple format to write recursive function which is given as follows;

recursiveFunction(***startValue***, parm1,param2,…){

1.W*rite terminating condition* which is just opposite to how you write loop-condition /update-condition in a for-loop

2. Write your programming logic.

Pass all the required variables as parameters (parm1, param2,…) in this functional call which are used in writing programming logic.

3.**recursiveFunction**(***nextValue***, parm1,param2,…);

}

Let's implement a recursive function call program to print number in ascending order.

RecursiveLoopIncrement.c

```c
#include <stdio.h>

/* function declaration */
void loopInc(int counter, int totalRows);

int main() {

	// number of rows
	int totalRows = 10;
	// recursive function call to print number in an incremental order
	loopInc( 1, totalRows );

	return 0;
}

void loopInc(int counter, int totalRows) {
	// terminating condition
	if ( counter > totalRows ) {
		return;
	}
	// programming logic
	if ( counter <= totalRows ) {
		printf( "%d\n", counter );//print to console
	}

	// tail recursive call
	loopInc( counter + 1, totalRows );
}
```

```
Output
```

1
2
3
4
5
6
7
8
9
10

Note: - Practically program with iterative logic (for-loop based) can be converted into a program based on recursive function calls and vice-versa. One of the important questions related to this topic generally asked in interview.

Let's implement a recursive function call based program to print a diamond pattern.

RecursiveDiamondPattern.c

```
#include <stdio.h>

/* function declaration */
void outerLoop( int row, int totalRows );
void nestedLoopLowerHalf( int col, int row, int totalRows );
void nestedLoopUpperHalf(int col, int row, int totalRows );

int main() {

	int totalRows = 7;
		outerLoop(1, totalRows );
	return 0;
}

	void outerLoop( int row, int totalRows ) {
	// for every row , col = 1,... and hence 1 value as a parameter.
		int col = 1;

		// Row level Repetitive Action : printing single row.
		if ( row <= (2 * totalRows - 1) ) {

			// Action1.Execute's Column level Repetitive Action

			// 1st half
			if ( row <= totalRows ) {
				nestedLoopUpperHalf(col, row, totalRows );
			}
			// 2nd half
			if ( row > totalRows ) {
				nestedLoopLowerHalf(col, row, totalRows );
			}
			printf("\n"); // Action2.Move cursor to next row.
		} else {
			return;// recursive call terminating condition
		}
		// next recursive call for the next row.
		outerLoop(row + 1, totalRows );
	}

	void nestedLoopUpperHalf( int col, int row, int totalRows ) {
		// recursive call terminating condition
		if ( !(col <= (2 * totalRows - 1)) ) {
			return;
		}

		if ( (col <= (2 * totalRows - 1)) ) {
			if ( col >= (totalRows - row + 1) &&
			    col <= (totalRows + row - 1) ) {
				printf("* " );
			} else {
				printf("  " );
			}
		}
```

```
            // next recursive call to print it in the next column.
            nestedLoopUpperHalf(col + 1, row, totalRows );

      }// end function

      void nestedLoopLowerHalf( int col, int row, int totalRows ) {
            if ( (col <= (2 * totalRows - 1)) ) {
                  if ( col >= (row - totalRows + 1) &&
                       col <= (3 * totalRows - (row + 1)) ) {
                        printf("* " );
                  } else {
                        printf("  " );
                  }
            } else {
                  return; // recursive call terminating condition
            }
            // next recursive call for the next column.
            nestedLoopLowerHalf(col + 1, row, totalRows );

      }// end function
```

Output

```
            *
          * * *
        * * * * *
      * * * * * * *
    * * * * * * * * *
  * * * * * * * * * * *
* * * * * * * * * * * * *
  * * * * * * * * * * *
    * * * * * * * * *
      * * * * * * *
        * * * * *
          * * *
            *
```

Appendix-A: Installation of eclipse IDE and project set-up

https://www3.ntu.edu.sg/home/ehchua/programming/howto/EclipseCpp_HowTo.html

References:

1. https://www.analyticsvidhya.com/blog/2013/06/art-structured-thinking-analyzing/

3.Object Oriented Programming with C++ and C [2004] D. Samanta – PHI

4.https://en.wikipedia.org/wiki/Kaizen

5.https://en.wikipedia.org/wiki/Stack-based_memory_allocation

Acknowledgments

I personally wish to thank everyone that helped me to make this, my book possible.

My family who supported me throughout it's completion.

My youngest brother, Sadique S. Naikwadi helped me in contributing technical reviews.

He is a freelancer developer and always been famous for his programming skills since college times. He worked on many different languages and never hesitate in trying programming stuffs.

He is always been reached by college mates and beginner programmers for improving their programming skills. His suggestions helped me a lot to put technical stuffs in an easy way and he discussed in detail, the issues of a struggling programmers. There was a time when programming is alien to him and the set of exercises which are covered in this book was once a test-bed for him and he was able to pick programming very nicely. With all these important factors, he is the best person around me to give me an honest technical reviews. Moreover, whenever he gets chance, he runs test-bed based on this book's exercises to help programmatically week learners.

I like to thank Mrs. Shabana Mursal and her family. She teaches in a school and she made me understand how different teaching style is important to teach students of different mindset.

Her inputs compelled me to put a completely new section which I would have never thought of including it.

Thank you all so much for taking interest to help me.

Being a learner it is not advised to be left alone without testing their skills.

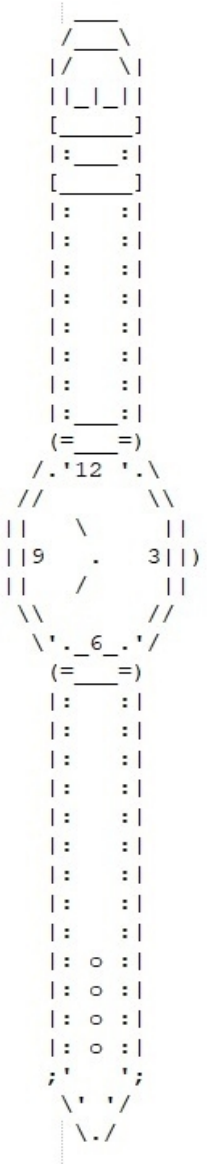
About Final Exam

Give exam whenever you can spend at-least 2 to 3 hours.

There is no upper limit for time and it is open-book exam.

Your target should be to execute the program correctly in a single chance, giving the matching output.

FINAL EXAM – PROVE YOUR WORTH



Write a generalized logic so that it should able to satisfy at least three different inputs which will basically either ZOOM-IN(print smaller-sized watch) or ZOOM-OUT(print bigger-sized watch).

Question A → **Solve it using nested-loop.**

Question B → **Solve it using functions. Write functions to reuse the logic so that no repeated logic should be possible.**

Question C → **Solve it using in-built C library String functions and array(s).**

Question D → **Solve it using C pointers.**

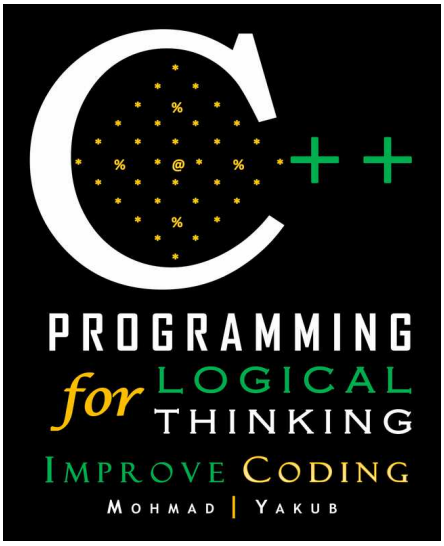
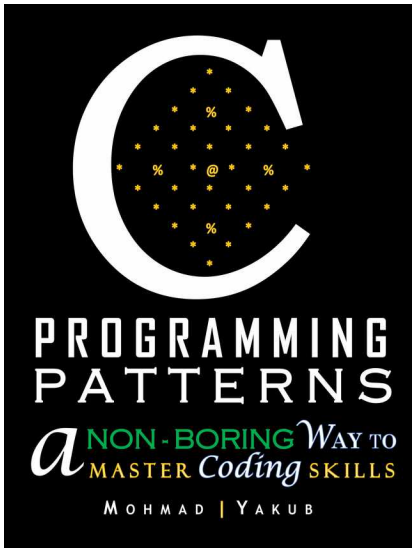
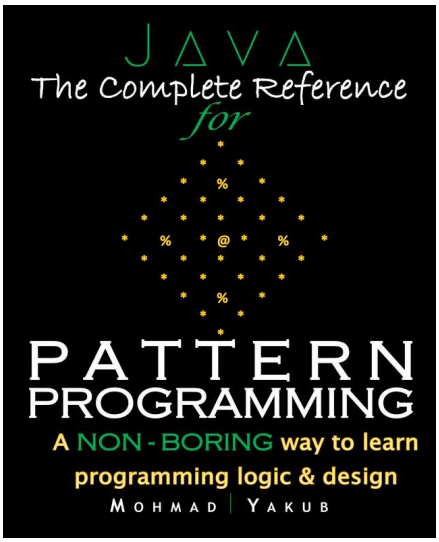
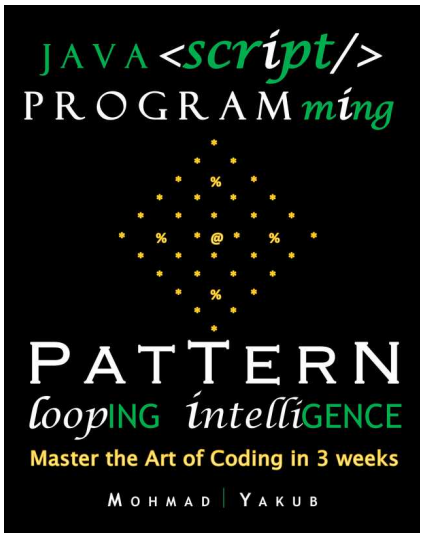
Question E → **Solve it using recursive function calls.**

I am not teaching you anything.

I just help you to explore yourself.

- B r u c e L e e

Expect more books by the end of this year 2019.

Available in all amazon book-store.	
	
	
Not Available	
